

STUDIO 심화 과정

(26.0.186 STS)

3 일 과정



목차

목차.....	2
SESSION1. 예외 처리	9
학습 목표.....	9
1. 예외 처리 개념.....	9
1.1. 예외 처리 정의	9
1.2. 예외 처리가 필요한 이유	10
1) 프로세스 중단 방지.....	10
2) 오류 원인 파악.....	10
3) 재시도 및 대체 처리 가능	10
4) 안정적인 운영 환경 구축.....	11
1.3. 오류와 예외의 개념	11
1) 오류 (Error)	11
2) 예외 (Exception)	11
1.4. RPA에서 자주 발생하는 예외 상황	12
1) 파일 관련 예외.....	12
2) 웹/앱 관련 예외.....	12
3) 데이터 관련 예외.....	12
4) 업무 규칙 관련 예외	12
2. 디버깅(Debugging)	13
2.1. 디버깅 개요	13
2.2. 디버그 실행 방법	13
2.3. 중단점 (Breakpoint)	14
3. 예외 유형	15
3.1. 예외 유형을 구분하는 이유	15
3.2. System Exception	15
3.3. Business Exception.....	16

4. Try-Catch.....	18
4.1. Try-Catch 개념	18
4.2. Try-Catch 구조	18
Try.....	19
Catch.....	19
Finally	19
4.3. Try-Catch 실습	21
4.4. Try-Catch 사용 시 권장 사항.....	23
5. Throw	24
5.1. Throw 개념	24
5.2. Throw 동작 방식	24
5.3. Throw 실습	25
6. Rethrow	27
6.1. Rethrow 개념	27
6.2. Rethrow 가 필요한 이유	28
6.3. Rethrow 활용 상황.....	28
6.4. Rethrow 실습	29
7. Studio 파일 분석.....	34
7.1 파일 분석 기능 개요.....	34
7.2. 프로젝트 분석 (Analyze Project)	35
7.3. 워크플로우 분석기 설정	36
7.4. 파일 및 프로젝트 유효성 검사.....	37
SESSION2. 정보 관리(CONFIG).....	38
학습 목표.....	38
1. Config 소개.....	38

2. Dictionary 변수 이해하기	39
2.1. Dictionary 변수 정의	39
2.2. Dictionary 변수 만들기.....	39
3. Config.xlsx 단일 시트 읽기	42
3.1. Config 파일 데이터 추출.....	42
3.3. Config 변수 값 출력하기.....	43
3.2. 예외 사항 처리	44
4. Window 자격 증명 만들기.....	46
4.1. Window 자격 증명 정의.....	46
4.2. 일반 자격 증명 생성.....	46
5. UiPath Studio 에서 자격 증명 확인	48
5.1. UiPath Credential 패키지 설치	48
5.2. Window 자격 증명 값 추출	49
SESSION3. RPA 프로세스 관리.....	51
학습 목표.....	51
1. 기능 비교	51
2. Invoke Workflow File	52
2.1. Invoke Workflow File 정의	52
2.2. 인수.....	53
2.3. Invoke Workflow File 활용	54
3. 라이브러리(Library).....	67
3.1. Library 정의.....	67
3.2. Library 활용.....	67
4. 스니펫(Snippet).....	75

4.1. Snippet 정의	75
4.2. Snippet 활용	75
5. 개체 저장소(Object Repository)	78
5.1. Object Repository 정의	78
5.2. Object Repository 설정 및 팝업 설명	78
5.2.1. 활용 시나리오	78
5.2.2. 개체 저장소에 UI 자동 저장 옵션 설정/해제	79
5.2.3. 개체 저장소에 있는 UI 요소들로만 개발하는 옵션 설정/해제	80
5.2.4. 이미 개체 저장소에 UI 요소가 등록되어 있는 경우	81
5.2.5. 개체 저장소에 UI 요소 동기화 상태 확인	82
5.2.6. 개체 저장소 요소를 스니펫과 라이브러리로 변환	82
SESSION4. 오케스트레이터(ORCHESTRATOR)	86
학습 목표.....	86
1. 오케스트레이터 정의.....	86
2. Orchestrator 특징.....	87
2.1. 테넌트 기능	87
2.2. Robot Types	87
3. Orchestrator 기본 설정	89
3.1. Orchestrator 접근	89
3.2. Orchestrator Tenant 설정.....	90
3.2.1. 일반(General).....	91
3.2.2. 배포(Deployment).....	93
3.2.3. 휴무일 (Non-Working Days).....	94
4. Orchestrator Tenant 메뉴 구성.....	95
4.1. 로봇 (Robots).....	95
4.2. 폴더 (Folders)	95

4.3. 액세스 관리(Manage Access).....	97
4.4. 머신 (Machines).....	98
4.5. 패키지 (Package).....	98
4.6. 감사 (Audit)	98
5. Orchestrator 프로세스 배포하기.....	100
5.1. 진행 순서	100
5.2. 머신 생성	101
5.3. 액세스 관리 설정	103
5.4. 폴더 만들기	106
5.5. Robot 생성	107
5.6. UiPath Studio 프로세스 배포	110
5.7. 시간 트리거 생성	114
5.8. Orchestrator 프로세스 실행.....	114
5.9. Orchestrator 프로세스 업데이트	117
6. Orchestrator 정보 관리 (Asset)	119
6.1. 자산(Asset) 정의.....	119
6.2. 자산(Asset) 종류.....	119
6.3. 자산(Asset) – Single Value 생성	119
6.4. 자산(Asset) – Value Per Robot 생성.....	120
6.5. UiPath Studio 에서 Asset 확인.....	122
7. Orchestrator 데이터 관리 (Queue).....	123
7.1. Queue 정의.....	123
7.2. Queue Item 상태 종류	124
7.3. 트랜잭션(Transaction) 정의	126
7.4. 트랜잭션(Transaction)의 예외사항 처리	126
7.4.1. 비즈니스 예외(Business Exception).....	127
7.4.2. 애플리케이션 예외(Application Exception)	127

7.5. Orchestrator Queue 생성.....	128
7.6. UiPath 에서 Orchestrator 로 Queue 큐 전달	129
7.7. UiPath 에서 Transaction Item 호출.....	130
7.8. 큐 트리거 만들기	135
SESSION5. REFRAMEWORK	137
학습 목표.....	137
1. 비즈니스 프로세스 형태	137
1.1. State Machine 액티비티 살펴보기	138
1.2. State Machine 프로세스 만들어보기	138
2. Dispatcher 와 Performer	146
3. REFramework 의 정의.....	146
4. REFramework 의 주요 이점	147
5. REFramework 의 구조 및 작동 원리.....	147
5.1. REFramework State 구성.....	147
5.2. REFramework Structure Diagram.....	148
5.2. REFramework 폴더 트리	149
5.3. Config.xlsx 구성	150
5.4. 공통변수	150
6. REFramework 의 세부 구조	151
6.1. Initialization(Init).....	151
6.2. Get Transaction Data.....	156
6.3. Process Transaction	158
6.4. End Process	161
7. Orchestrator 와 데이터 주고받기	162

7.1. 초기 세팅	162
7.2. “Init” State.....	165
7.3. “Get Transaction Data” State	169
7.4. “Process Transaction” State	169
7.5. “End Process” State.....	171
8. Orchestrator 없이 사용하기.....	172
8.1. 초기 세팅	172
8.2. “Init” State.....	179
8.3. “Get Transaction Data” State	183
8.4. “Process Transaction” State	185
8.5. “End Process” State.....	186

SESSION1. 예외 처리

학습 목표

- 디버깅에 대해 이해하고 기능을 사용할 수 있다.
- 예외 처리 방식에 대해 이해하고 예외 처리를 할 수 있다.

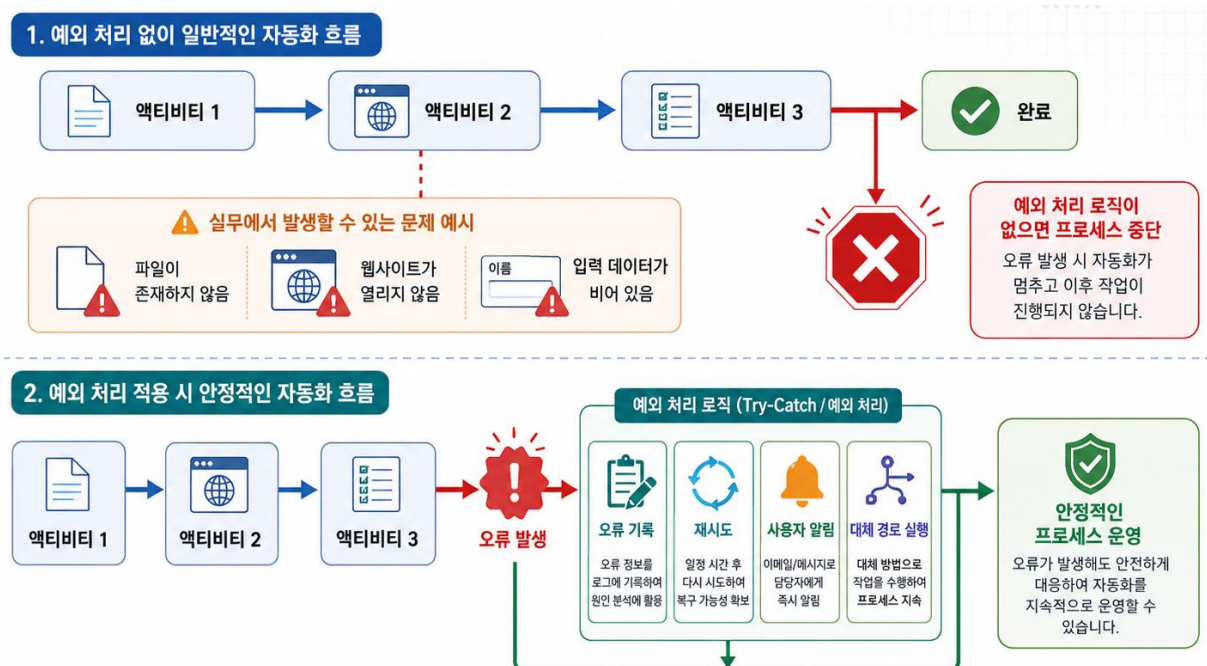
1. 예외 처리 개념

1.1. 예외 처리 정의

- 예외 처리(Exception Handling)란 프로세스 실행 중 발생할 수 있는 오류 상황을 감지하고, 해당 오류에 적절히 대응하여 프로세스가 비정상적으로 종료되지 않도록 제어하는 방법입니다.

일반적으로 자동화 프로세스는 정해진 순서에 따라 액티비티를 실행합니다.

하지만 실제 업무 환경에서는 항상 정상적인 상황만 발생하지 않습니다. 파일이 존재하지 않거나, 웹사이트가 열리지 않거나, 입력해야 하는 데이터가 비어 있는 경우처럼 다양한 문제가 발생할 수 있습니다. 이러한 상황에서 별도의 예외 처리 로직이 없다면 프로세스는 즉시 중단될 수 있습니다.



1.2. 예외 처리가 필요한 이유

- RPA는 사람이 직접 실시간으로 개입하지 않는 상태에서 실행되는 경우가 많습니다. 이 때문에 오류가 발생했을 때 사람이 즉시 판단하여 대응하기 어렵습니다. 만약 예외 처리가 설계되어 있지 않다면, 작은 문제 하나로 전체 업무가 중단되거나 잘못된 결과가 저장될 수 있습니다.

1) 프로세스 중단 방지

예외가 발생하더라도 적절한 처리 로직이 있으면 전체 프로세스가 즉시 종료되지 않고, 지정된 방식으로 대응할 수 있습니다.

예시:

1. 파일이 없을 경우 오류 메시지를 기록하고 다음 작업으로 이동
2. 특정 데이터가 잘못된 경우 해당 건만 제외하고 다음 건 처리

2) 오류 원인 파악

예외 처리 과정에서 로그를 남기면 어떤 단계에서 어떤 문제로 실패했는지 확인할 수 있습니다.

예시:

1. 로그인 단계 실패
2. Excel 파일 경로 오류
3. Queue Item 처리 중 데이터 검증 실패

3) 재시도 및 대체 처리 가능

일부 예외는 일시적인 문제일 수 있으므로 재시도로 해결할 수 있습니다.

또는 특정 업무 규칙에 따라 다른 처리 흐름으로 전환할 수도 있습니다.

예시:

1. 웹 페이지 응답 지연 → 재시도
2. 필수 입력값 누락 → 예외 발생 후 담당자 확인

4) 안정적인 운영 환경 구축

실무에서는 자동화가 하루에 여러 번 반복 수행되거나 대량의 데이터를 처리하는 경우가 많습니다. 이때 예외 처리가 설계되어 있어야 운영 중단 위험을 줄일 수 있습니다.

1.3. 오류와 예외의 개념

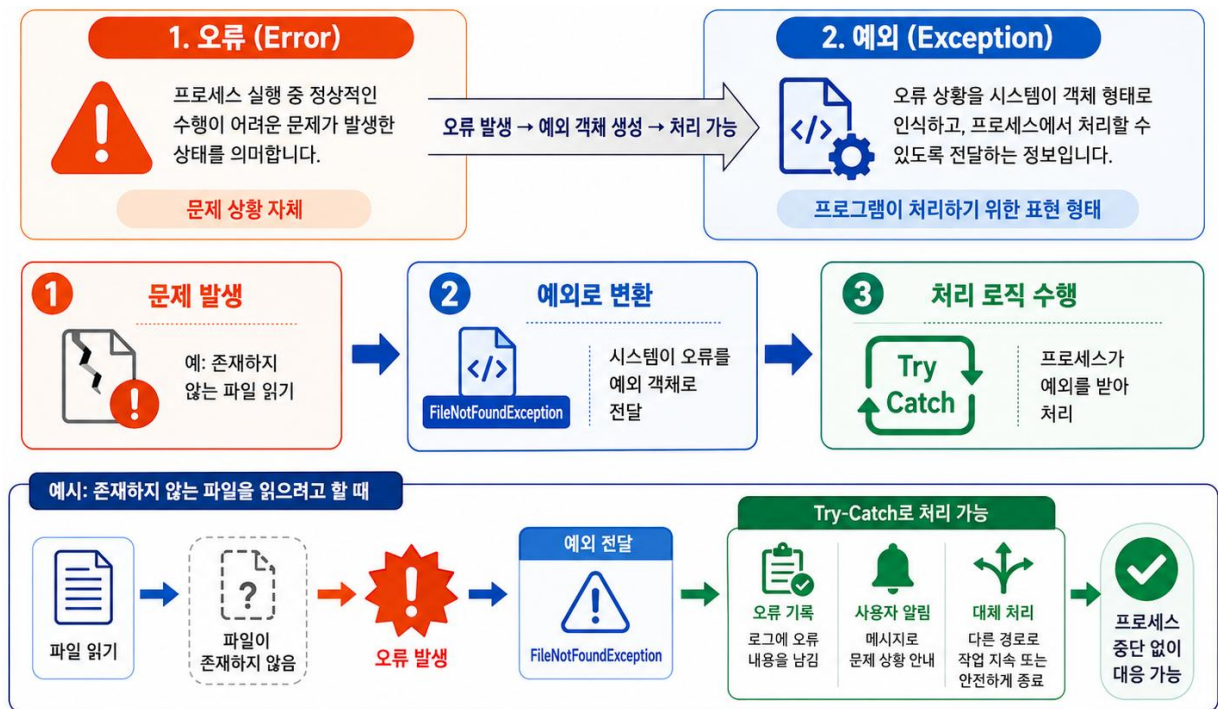
1) 오류 (Error)

- 프로세스 실행 중 정상적인 수행이 어려운 문제가 발생한 상태를 의미합니다.

2) 예외 (Exception)

- 그 오류 상황을 시스템이 객체 형태로 인식하고, 프로세스에서 처리할 수 있도록 전달하는 정보입니다. 즉, 오류는 문제 상황 자체이고, 예외는 그 문제를 프로그램이 처리하기 위해 표현한 형태라고 이해할 수 있습니다.

예를 들어, 존재하지 않는 파일을 읽으려고 하면 “파일이 없음”이라는 문제가 발생합니다. 이때 시스템은 해당 문제를 예외로 전달하고, Try-Catch 같은 구조를 사용하면 그 예외를 처리할 수 있습니다.



1.4. RPA 에서 자주 발생하는 예외 상황

예외 처리는 추상적으로 설명하면 이해가 잘 되지 않기 때문에, 반드시 RPA 실무 사례와 연결하는 것이 좋습니다. RPA 에서 자주 발생하는 예외 상황은 다음과 같습니다.

1) 파일 관련 예외

1. 지정된 경로에 파일이 존재하지 않음
2. 이미 열려 있는 파일에 접근 시도
3. 읽기/쓰기 권한 부족

예시: Read Range 실행 시 Excel 파일이 존재하지 않음

2) 웹/앱 관련 예외

1. 페이지 로딩 실패
2. 버튼 또는 입력창을 찾지 못함
3. 응답 지연 또는 타임아웃 발생

예시: Click 액티비티 수행 시 대상 요소를 찾을 수 없음

3) 데이터 관련 예외

1. 숫자 형식이어야 하는 값에 문자열이 들어 있음
2. Null 값 또는 빈 값 존재
3. 필수 컬럼 누락

예시: Age 컬럼 값이 비어 있는데 정수형으로 변환 시도

4) 업무 규칙 관련 예외

1. 승인 금액이 허용 범위를 초과함
2. 고객 ID 가 중복됨
3. 필수 입력값이 누락됨

예시: 주문번호가 없는 데이터는 업무 처리 대상이 될 수 없음

2. 디버깅(Debugging)

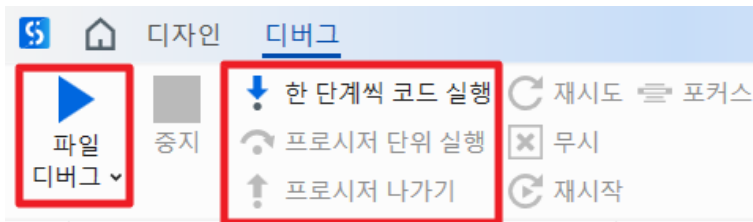
2.1. 디버깅 개요

- 디버깅(Debugging)은 자동화 프로세스를 실행하면서 워크플로우의 실행 과정을 단계별로 확인하고 오류 원인을 분석하는 기능입니다.

자동화 개발 과정에서는 예상과 다른 결과가 발생하거나 오류가 발생하는 경우가 있습니다. 이러한 문제를 해결하기 위해서는 프로세스가 어떤 순서로 실행되는지, 변수 값이 어떻게 변경되는지를 확인해야 합니다.

2.2. 디버그 실행 방법

- Debug 패널 또는 상단 리본 메뉴를 통해 디버깅을 실행할 수 있습니다.



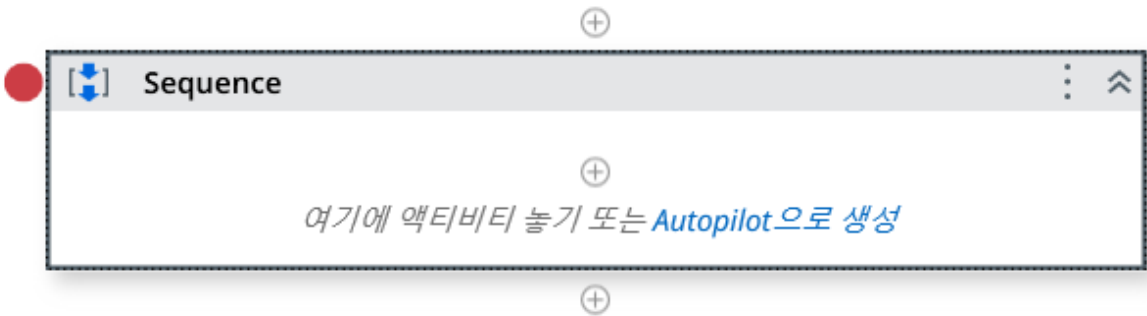
기능	설명
파일 디버깅	현재 워크플로우 디버깅 실행
한 단계씩 코드 실행	한 단계씩 실행
프로시저 단위 실행	내부 활동을 건너뛰고 다음 단계 실행
프로시저 나가기	현재 워크플로우 종료 후 상위 단계 이동
계속	다음 중단점까지 실행

2.3. 중단점 (Breakpoint)

중단점은 특정 액티비티에서 실행을 일시 중지하도록 설정하는 기능입니다. 디버깅 중 중단점에 도달하면 프로세스 실행이 멈추며, 이 상태에서 변수 값이나 실행 흐름을 확인할 수 있습니다.

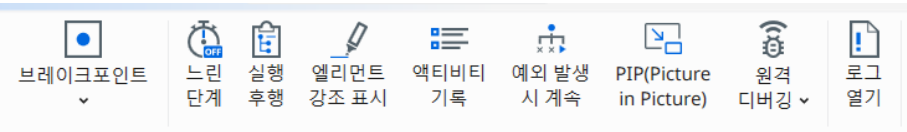
중단점은 다음 방법으로 설정할 수 있습니다.

- 액티비티 우클릭 → 중단점 토글
- 단축키 F9



중단점이 설정된 액티비티는 빨간색 표시가 나타납니다.

기타 다른 기능 정리



기능	설명	활용 목적
브레이크포인트	특정 액티비티에서 실행을 일시 중지하도록 설정하는 기능	특정 위치에서 실행을 멈추고 변수 값이나 흐름을 확인할 때 사용
느린 단계	워크플로우를 한 단계씩 천천히 실행하는 기능	실행 흐름을 눈으로 확인하면서 디버깅할 때 사용
실행 후행	현재 액티비티를 실행하고 다음 단계로 이동하는 기능	Invoke Workflow 등의 내부 로직을 들어가지 않고 다음 단계로 진행
엘리먼트 강조 표시	UI 자동화 실행 시 대상 UI 요소를 화면에서 강조 표시하는 기능	어떤 UI 요소가 선택되었는지 확인할 때 사용
액티비티 기록	실행 중 수행되는 액티비티를 기록하는 기능	워크플로우 실행 흐름을 추적할 때 사용
로그 열기	실행 중 생성된 로그 파일을 열어 확인하는 기능	오류 원인 분석 및 실행 기록 확인

3. 예외 유형

3.1. 예외 유형을 구분하는 이유

- 자동화 프로세스에서 발생하는 모든 오류를 동일한 방식으로 처리하면 비효율이 발생할 수 있습니다. 어떤 오류는 시스템 환경 문제로 인해 일시적으로 발생하기 때문에 다시 시도하면 정상적으로 처리될 수 있지만, 어떤 오류는 입력 데이터 자체가 잘못된 경우이기 때문에 동일한 작업을 반복하더라도 계속 실패하게 됩니다.

따라서 예외를 유형별로 구분하면 오류 원인을 보다 정확하게 파악할 수 있으며, 상황에 맞는 대응 방식을 설계할 수 있습니다. 또한 재시도 여부를 판단하거나, 로그를 체계적으로 기록하고 운영 중 장애 대응을 효율적으로 수행하는 데에도 도움이 됩니다.

3.2. System Exception

- System Exception은 **시스템 환경, 애플리케이션 상태, 네트워크, 파일 접근과 같은 외부 환경 요인으로 인해 발생하는 예외**를 의미합니다. 즉, 자동화 프로세스의 로직이나 데이터 자체에는 문제가 없지만, 실행 환경이 정상적이지 않아 작업이 수행되지 못하는 상황을 말합니다.

예를 들어 웹 페이지가 정상적으로 열리지 않거나, 파일 경로에 접근할 수 없거나, 네트워크 연결이 일시적으로 끊어진 경우와 같은 상황이 System Exception에 해당합니다. 이러한 문제는 자동화 로직이 잘못된 것이 아니라 **실행 환경의 상태에 영향을 받기 때문에** 발생합니다.

System Exception의 특징은 대부분 일시적인 문제일 가능성이 있다는 점입니다. 따라서 같은 작업을 다시 수행하면 정상적으로 처리되는 경우가 많습니다. 이러한 이유로 System Exception은 재시도 로직과 함께 설계되는 경우가 많습니다.

3.3. Business Exception

- Business Exception 은 업무 규칙, 입력 데이터, 처리 조건 등 비즈니스 로직과 관련된 문제로 인해 발생하는 예외를 의미합니다. 시스템이나 애플리케이션은 정상적으로 동작하고 있지만, 처리하려는 데이터가 업무 규칙에 맞지 않기 때문에 작업을 수행할 수 없는 상황을 말합니다.

예를 들어 필수 입력값이 누락된 경우, 데이터 형식이 맞지 않는 경우, 승인 조건을 만족하지 못하는 경우와 같은 상황이 Business Exception 에 해당합니다. 이러한 경우에는 자동화 로직을 다시 실행하더라도 동일한 데이터가 사용되는 한 같은 오류가 반복됩니다.

Business Exception 의 특징은 대부분 재시도로 해결되지 않는다는 점입니다.

문제의 원인이 시스템이 아니라 데이터 또는 업무 조건이기 때문입니다. 따라서 Business Exception 이 발생하면 해당 데이터를 실패 처리하거나 검토 대상에 포함시키고, 다음 작업을 계속 수행하는 방식으로 설계하는 것이 일반적입니다.

구분	System Exception 사례	Business Exception 사례
파일 처리	Excel 파일이 존재하지 않음	Excel 데이터에 필수 컬럼 값이 비어 있음
웹 자동화	웹 페이지 로딩 실패	입력 데이터가 업무 규칙에 맞지 않음
UI 자동화	버튼 또는 입력창을 찾지 못함	승인 조건을 만족하지 못한 데이터
네트워크	서버 또는 API 연결 실패	주문번호 또는 고객 ID 누락
애플리케이션	프로그램 응답 없음 또는 강제 종료	중복 데이터 입력
데이터 처리	Null 참조 오류 또는 타입 변환 실패	금액 또는 날짜 형식 오류
시스템 환경	파일 접근 권한 없음	처리 대상이 아닌 데이터
실행 환경	Timeout 발생	업무 규칙 위반 데이터

Quiz 1.

자동화 프로세스가 Excel 파일을 읽어 데이터를 처리하려고 합니다. 프로세스 실행 시 Excel 파일이 다른 사용자에 의해 열려 있어 접근할 수 없다는 오류가 발생했습니다. 몇 분 후 해당 파일이 닫힌 뒤 다시 실행했을 때 정상적으로 처리되었습니다.

이 상황에서 발생한 예외 유형은 무엇인가요?

- ① System Exception
- ② Business Exception

Quiz 2.

자동화 프로세스가 ERP 시스템에서 주문 데이터를 조회한 뒤, 각 주문을 처리하는 작업을 수행하고 있습니다. 프로세스는 정상적으로 로그인하고 주문 데이터를 조회했으며, 이후 각 주문을 순서대로 처리하는 단계에서 다음 상황이 발생했습니다.

특정 주문 데이터를 처리하는 과정에서 “고객 등급 코드” 값이 ERP 시스템에 존재하지 않는 코드로 입력되어 있어 시스템에서 해당 주문 처리를 거부했습니다. 프로세스를 다시 실행하더라도 동일한 주문 데이터에서는 계속 같은 오류가 발생합니다.

이 상황에서 발생한 예외 유형은 무엇인가요?

- ① System Exception
- ② Business Exception

정답: Quiz 1: System Exception, Quiz 2: Business Exception

4. Try-Catch

4.1. Try-Catch 개념

- 자동화 프로세스를 실행하는 과정에서 예외가 발생하면 해당 프로세스는 즉시 중단될 수 있습니다. 이러한 상황을 방지하기 위해 UiPath에서는 Try-Catch 액티비티를 사용하여 예외 처리를 구현할 수 있습니다.

Try-Catch는 프로세스 실행 중 발생할 수 있는 오류를 감지하고, 오류가 발생했을 때 미리 정의된 방식으로 대응할 수 있도록 하는 구조입니다. 이를 통해 프로세스가 비정상적으로 종료되는 것을 방지하고, 오류 상황에서도 안정적으로 작업을 관리할 수 있습니다.

Try-Catch 구조를 사용하면 예외가 발생했을 때 다음과 같은 처리가 가능합니다.

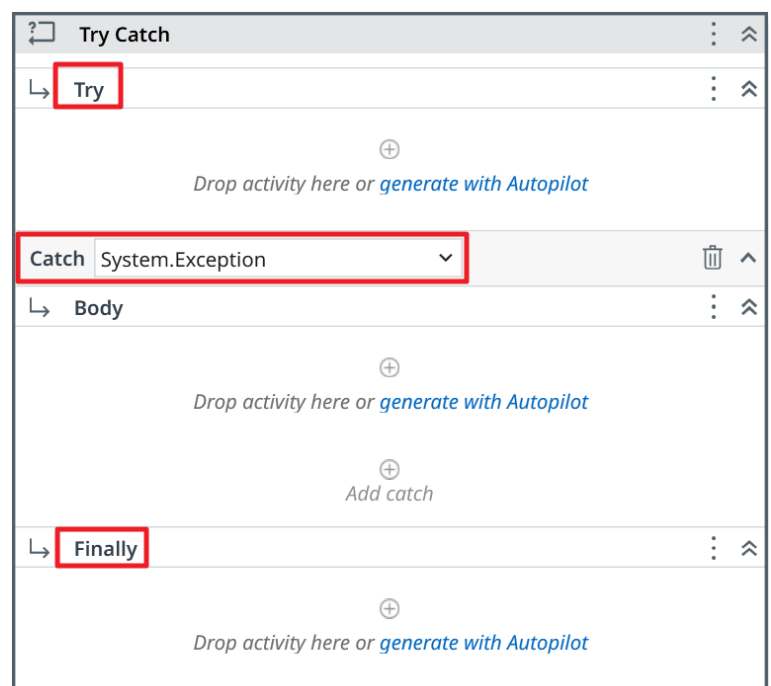
- 오류 메시지 기록
- 특정 작업 재시도
- 사용자에게 오류 알림
- 다음 작업으로 프로세스 계속 진행

따라서 Try-Catch는 자동화 프로세스의 안정성을 높이기 위해 가장 기본적으로 사용되는 예외 처리 구조입니다.

4.2. Try-Catch 구조

Try-Catch는 세 가지 영역으로 구성됩니다.

- Try
- Catch
- Finally



Try

Try 영역에는 예외가 발생할 가능성이 있는 작업을 작성합니다. 프로세스는 먼저 Try 영역의 작업을 실행하며, 이 과정에서 오류가 발생하면 Catch 영역으로 흐름이 이동합니다.

예를 들어 Excel 파일을 읽거나 웹 페이지에 접근하는 작업은 예외가 발생할 가능성이 있기 때문에 Try 영역에 배치하는 것이 일반적입니다.

Catch

Catch 영역은 Try 영역에서 예외가 발생했을 때 실행되는 부분입니다. 여기에서는 발생한 오류를 처리하기 위한 작업을 정의합니다.

Catch 영역에서는 오류 메시지 출력, 로그 기록, 스크린샷 저장, 재시도 수행, 프로세스 종료 또는 다음 단계 진행 등 여러 작업을 수행할 수 있습니다.

Catch에서는 발생한 예외 정보를 Exception 객체를 통해 확인할 수 있습니다.

속성	설명
Message	발생한 오류 메시지
Source	오류가 발생한 프로그램 또는 구성 요소
StackTrace	오류가 발생한 코드 위치
InnerException	내부에서 발생한 추가 예외

Finally

Finally 영역은 Try 영역에서 예외가 발생했는지 여부와 관계없이 항상 실행되는 영역입니다. 즉, Try 영역이 정상적으로 수행되더라도 Finally 영역은 실행되고, 예외가 발생하여 Catch가 실행된 경우에도 Finally 영역이 실행됩니다.

Finally 영역에는 일반적으로 브라우저 종료, 파일 닫기, 리소스 정리, 로그 기록 등과 같은 작업을 수행합니다. 이러한 작업은 프로세스의 정상 수행 여부와 관계없이 항상 실행되어야 하기 때문에 Finally 영역에 배치하는 것이 좋습니다.

Try-Catch 구조 및 동작 흐름

 Try	예외가 발생할 수 있는 코드를 포함하는 영역 • 오류가 없으면 Catch 블록은 건너뛰고 Finally 블록으로 이동
 Catch	Try 블록에서 예외가 발생하면 실행되는 영역 • 예외를 적절히 처리 • 여러 Catch를 사용해 다양한 예외를 구분하여 처리 가능
 Finally	Try가 정상 완료되거나, 예외가 Catch에서 정상적으로 처리된 경우에만 실행되는 영역 • 주로 자원 해제, 로그 기록, 연결 종료 등 정리(마무리) 작업 수행

**오류 (Error)**
프로세스 실행 중 정상적인 수행이 어려운 문제가 발생한 상태

**예외 (Exception)**
그 오류 상황을 시스템이 객체 형태로 인식하고, 프로세스에서 처리할 수 있도록 전달하는 정보

예시



존재하지 않는 파일을 읽으려고 하면 "파일이 없음"이라는 문제가 발생(오류)
→ 시스템은 해당 문제를 예외(Exception)로 전달
→ Try-Catch 구조를 사용하면 그 예외를 처리할 수 있음

실행 흐름 (UiPath Try Catch 동작 기준)



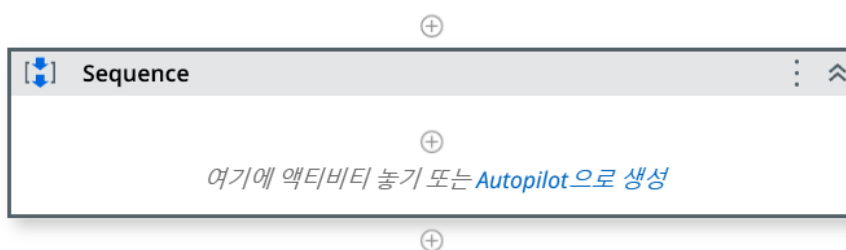
4.3. Try-Catch 실습

다음 실습을 통해 Try-Catch 를 사용한 예외 처리를 구현해 보겠습니다.

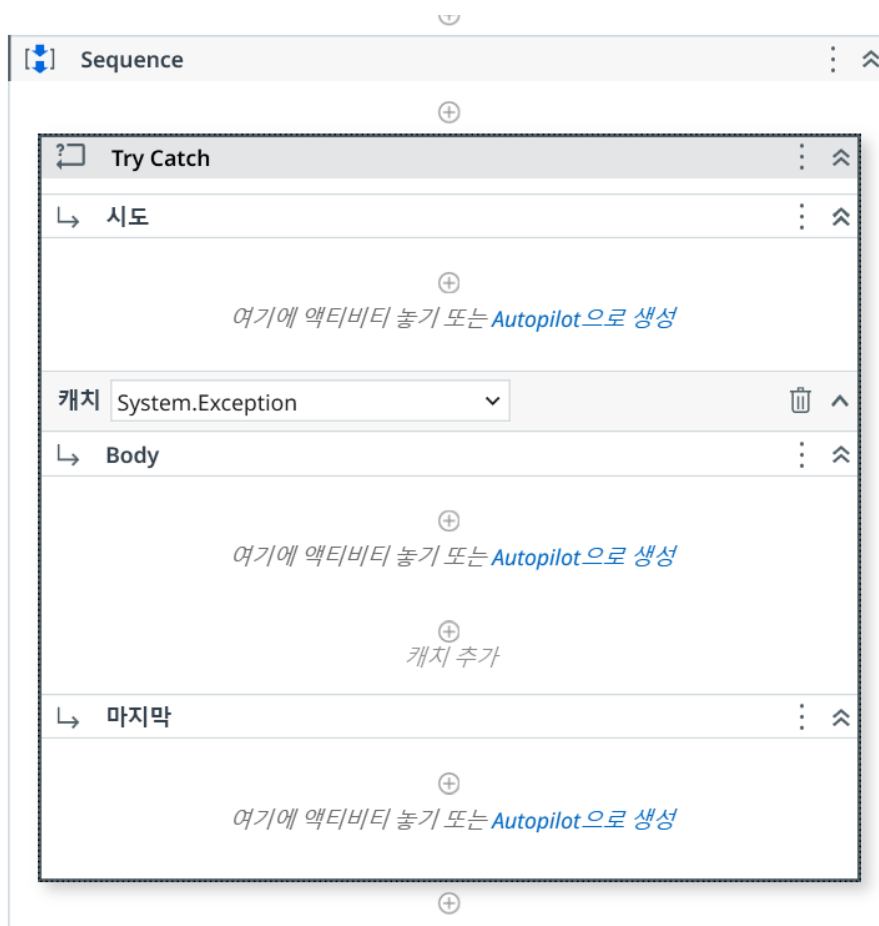
실습 목표

- 존재하지 않는 Excel 파일을 읽을 때 발생하는 오류를 Try-Catch 를 사용하여 처리합니다.

Step1. Sequence 를 생성합니다.

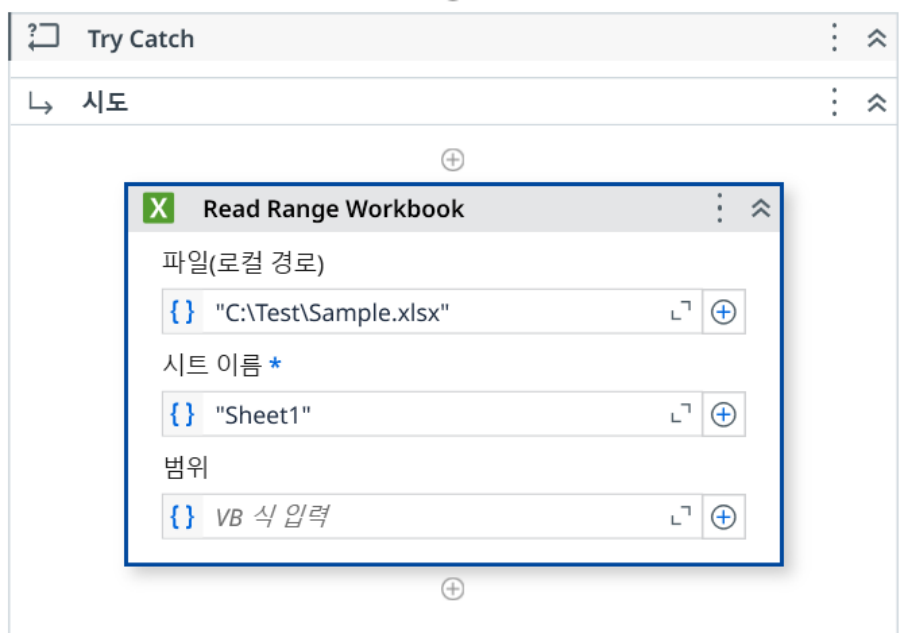


Step2. Try-Catch 액티비티를 추가합니다.



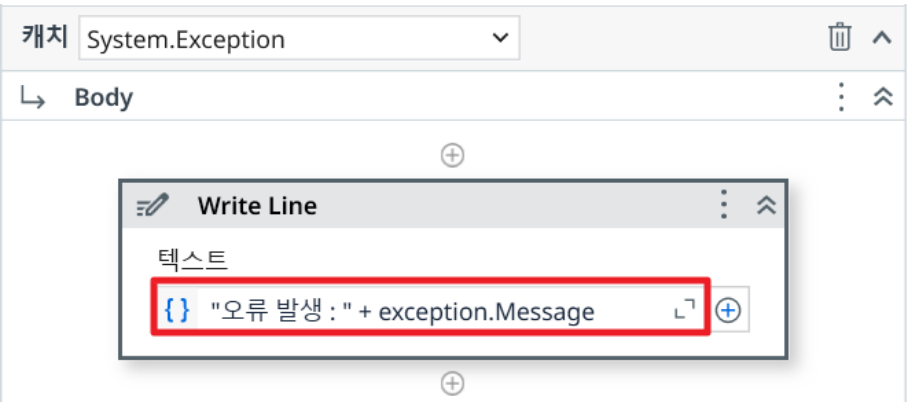
Step3. Try 영역에 Read Range Workbook 액티비티를 배치합니다.

- 읽을 파일 경로는 존재하지 않는 파일 경로로 설정합니다.

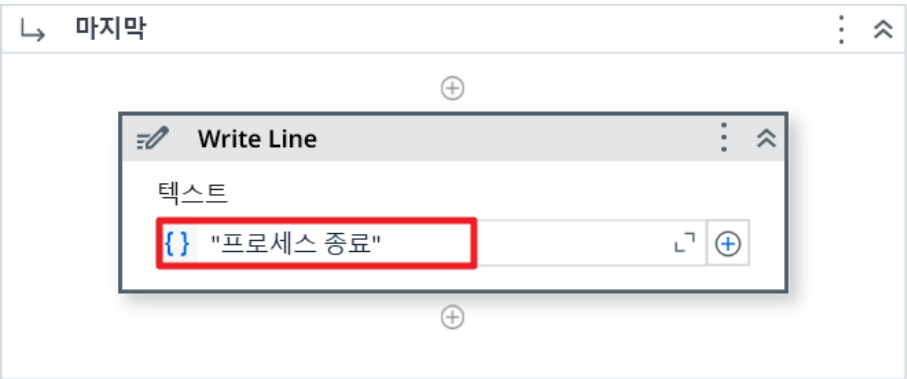


Step4. Catch 영역에 Write Line 액티비티를 추가합니다.

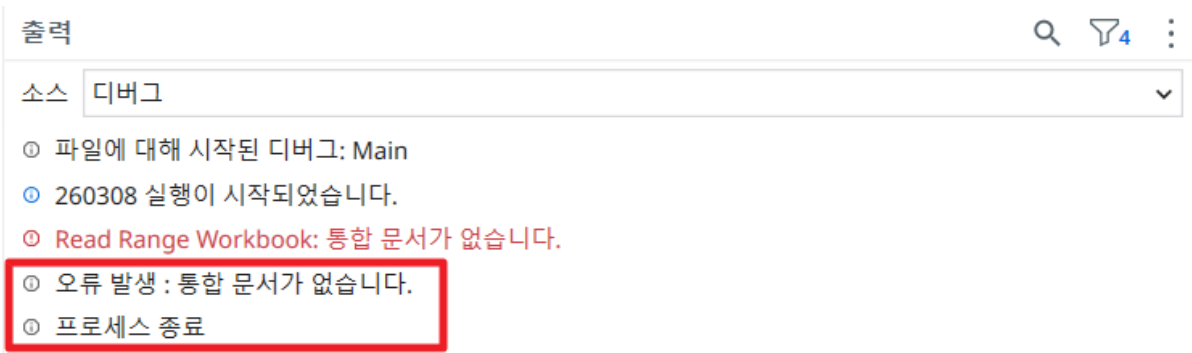
- 오류 메시지: "오류 발생 : " + exception.Message



Step5. Finally 영역에 Write Line 액티비티를 추가합니다.



Step6. 프로세스를 실행하고 결과를 확인합니다.



파일이 존재하지 않기 때문에 Try 영역에서 예외가 발생하고 Catch 영역이 실행됩니다.
이후 Finally 영역이 실행되는 것을 확인할 수 있습니다.

4.4. Try-Catch 사용 시 권장 사항

Try-Catch 를 사용할 때는 다음 사항을 고려하는 것이 좋습니다.

- 예외가 발생할 가능성이 높은 작업에 Try-Catch 를 적용합니다.
- 너무 넓은 범위에 Try-Catch 를 적용하면 오류 원인을 파악하기 어려울 수 있습니다.
- Catch 영역에서는 오류 메시지를 반드시 기록하는 것이 좋습니다.
- Finally 영역에는 리소스 정리 작업을 배치하는 것이 좋습니다.

이러한 방식으로 Try-Catch 구조를 설계하면 자동화 프로세스의 안정성과 유지보수성을 높일 수 있습니다.

5. Throw

5.1. Throw 개념

- 자동화 프로세스에서는 **시스템 오류뿐만 아니라 업무 규칙을 위반하는 상황도** 발생할 수 있습니다. 예를 들어 주문 금액이 허용 범위를 초과하거나, 필수 입력값이 누락된 경우와 같이 데이터 자체에 문제가 있는 상황입니다. 이러한 경우 단순히 오류 메시지를 출력하는 것만으로는 프로세스 흐름을 적절히 제어하기 어렵습니다.

이때 사용하는 것이 **Throw 액티비티**입니다. Throw 는 **의도적으로 예외를 발생시키는 기능**을 제공합니다. 즉, 자동화 프로세스에서 특정 조건이 만족되지 않을 때(데이터가 업무 규칙을 만족하지 않는 경우, 필수 값이 누락된 경우, 특정 조건을 충족하지 못한 경우, 비정상 데이터 발견 시 등) 예외를 발생시켜 Try-Catch 구조로 흐름을 전달할 수 있도록 합니다.

5.2. Throw 동작 방식

Throw 액티비티는 예외를 발생시키고 현재 실행 흐름을 중단한 뒤, 가장 가까운 Try-Catch 구조의 Catch 영역으로 흐름을 이동시킵니다.

Throw 가 실행되면 다음과 같은 순서로 동작합니다.

1. Throw 액티비티에서 예외가 발생합니다.
2. 현재 실행 중인 Try 영역이 중단됩니다.
3. Try-Catch 구조의 Catch 영역으로 흐름이 이동합니다.
4. Catch 영역에서 예외 처리를 수행합니다.

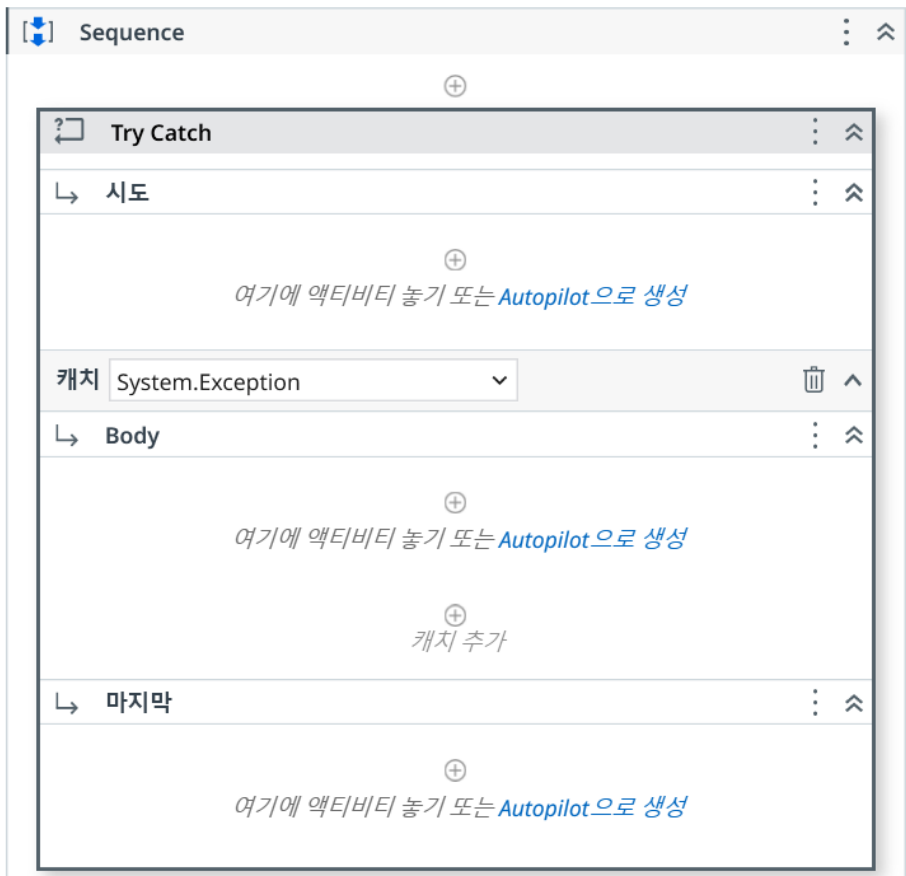
따라서 Throw 는 단순한 오류 메시지 출력이 아니라 **프로세스 흐름 자체를 제어하는 역할**을 합니다.

5.3. Throw 실습

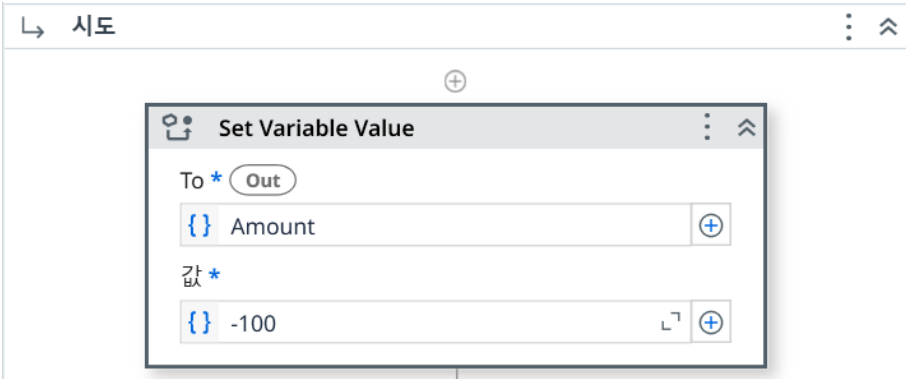
실습 목표

- 데이터 검증 과정에서 Throw 를 사용하여 예외를 발생시키고 Catch 영역에서 오류를 처리합니다.

Step 1. Try-Catch 구조를 생성합니다.

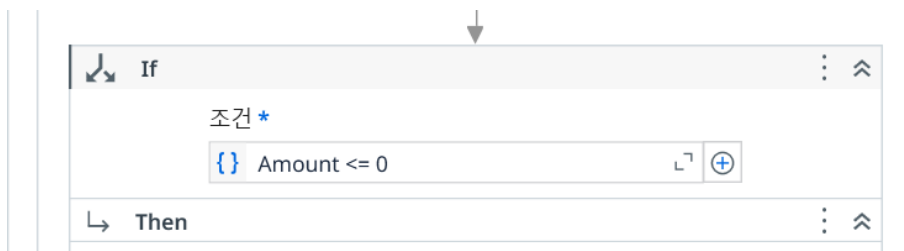


Step 2. 변수 Amount 를 만들고 정수 값을 입력합니다.



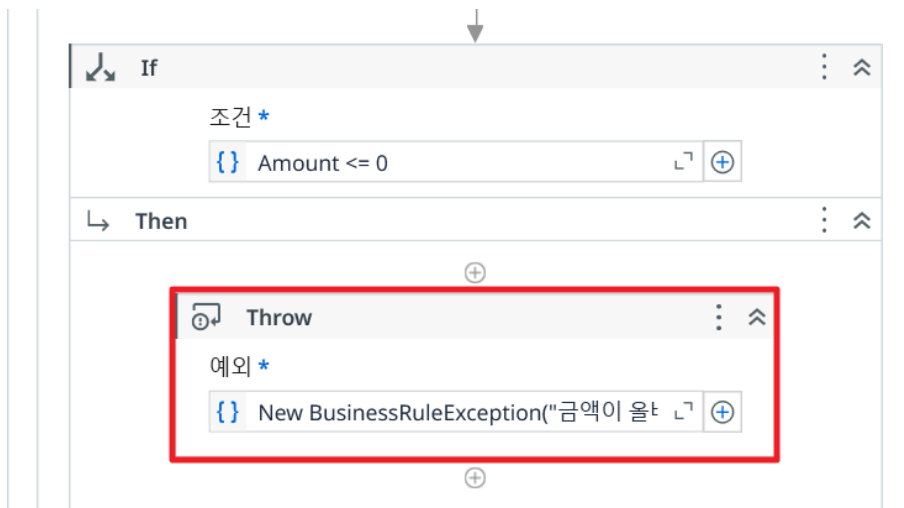
Step3. Try 영역에서 If 액티비티를 활용해 다음 조건을 검사합니다.

- 조건: Amount <= 0



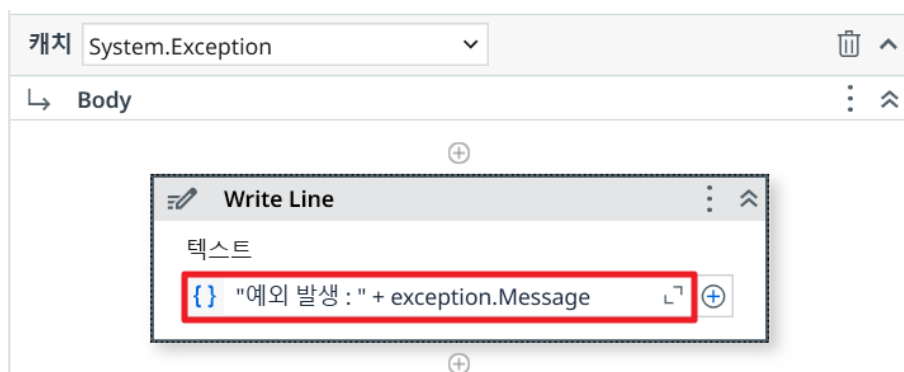
Step4. 조건이 만족되면 Throw 액티비티를 사용하여 예외를 발생시킵니다.

- New BusinessException("금액이 올바르지 않습니다.")



Step 5. Catch 영역에서 오류 메시지를 출력합니다.

- write Line: "예외 발생 : " + exception.Message



Step 6. 프로세스를 실행하여 Throw 로 발생한 예외가 Catch 영역에서 처리되는 것을 확인합니다.

6. Rethrow

6.1. Rethrow 개념

자동화 프로세스에서 예외가 발생하면 일반적으로 Catch 영역에서 해당 예외를 처리합니다. 하지만 모든 예외를 현재 단계에서 완전히 처리할 수 있는 것은 아닙니다. 어떤 경우에는 오류 내용을 기록한 뒤, 그 예외를 상위 단계로 다시 전달해야 하는 상황이 있습니다.

이때 사용하는 것이 **Rethrow 액티비티**입니다. Rethrow는 Catch 영역에서 현재 잡은 예외를 그대로 다시 발생시키는 기능입니다. 즉, 새로운 예외를 만드는 것이 아니라, 이미 발생한 예외를 상위 Try-Catch 구조나 호출한 프로세스로 다시 전달하는 역할을 합니다.

이 기능은 현재 단계에서 오류를 일부 기록하거나 정리 작업을 수행한 뒤, 최종적인 예외 처리는 더 상위 단계에서 하도록 설계할 때 유용하게 사용됩니다.



6.2. Rethrow 가 필요한 이유

실무 자동화에서는 하나의 프로세스가 여러 단계 또는 여러 워크플로우로 나누어져 있는 경우가 많습니다. 이때 하위 단계에서 발생한 예외를 단순히 Catch 에서 끝내버리면, 상위 프로세스는 오류가 발생했는지 알 수 없게 됩니다.

예를 들어 하위 워크플로우에서 오류가 발생했는데 Catch 에서 로그만 남기고 종료해버리면, 상위 프로세스에서는 해당 작업이 정상 종료된 것처럼 보일 수 있습니다.

이러한 문제를 방지하기 위해 Catch 에서 예외를 확인한 뒤 다시 상위로 전달해야 할 때 Rethrow 를 사용합니다. 즉, Rethrow 는 예외를 숨기지 않고 **상위 로직까지 전달하기 위한 목적으로** 사용됩니다.

구분	Throw	Rethrow
목적	새로운 예외를 발생시킴	현재 잡은 예외를 다시 전달함
사용 위치	조건문, 일반 로직, Try 내부 등	Catch 영역 내부
예외 정보	새로 생성됨	기존 예외 정보 유지
활용 상황	업무 규칙 위반, 검증 실패	로그 기록 후 상위 전달

6.3. Rethrow 활용 상황

Rethrow 는 다음과 같은 상황에서 주로 사용됩니다.

- 하위 워크플로우에서 발생한 예외를 상위 프로세스에 전달해야 하는 경우
- Catch 에서 로그를 남긴 뒤 예외를 숨기지 않아야 하는 경우
- 현재 단계에서는 복구가 어렵고 상위 단계 판단이 필요한 경우
- REFramework 와 같이 상위 구조에서 예외 유형을 판단해야 하는 경우

예를 들어 하위 프로세스에서 시스템 오류가 발생했을 때, 현재 단계에서는 단순히 오류 내용을 기록만 하고 실제 재시도 여부나 종료 여부는 상위 프로세스에서 판단하도록 만들 수 있습니다. 이때 Rethrow 가 적절합니다.

6.4. Rethrow 실습

실습 목표

하위 워크플로우에서 발생한 예외를 Catch 영역에서 확인한 뒤, **Rethrow** 를 사용하여 상위 워크플로우로 다시 전달하는 과정을 확인합니다.

실습 시나리오

두 개의 워크플로우를 사용합니다.

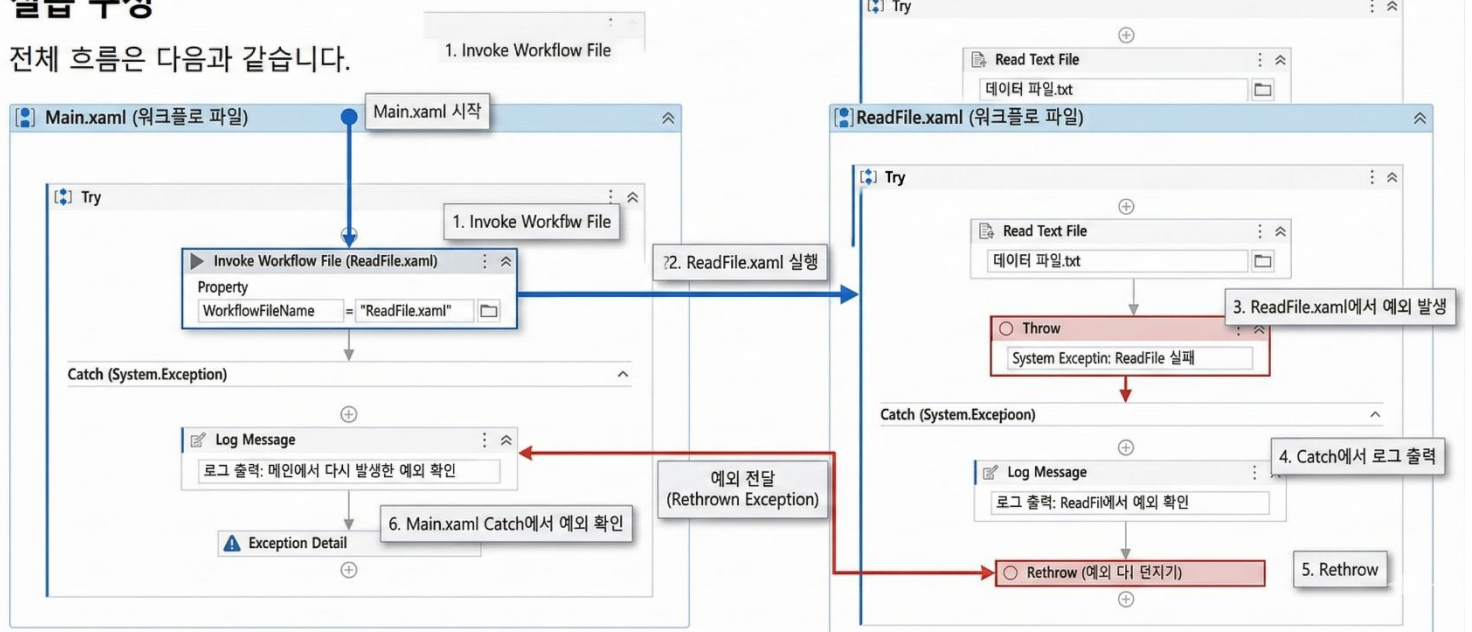
- **Main.xaml** : 상위 프로세스
- **ReadFile.xaml** : 하위 프로세스

하위 프로세스인 **ReadFile.xaml** 에서 존재하지 않는 Excel 파일을 읽도록 설정합니다.

이 과정에서 예외가 발생하면 Catch 영역에서 오류 메시지를 출력한 뒤 **Rethrow** 를 사용하여 상위 프로세스인 Main.xaml 으로 예외를 다시 전달합니다. 그 후 Main.xaml 의 Try-Catch 에서 해당 예외를 다시 받아 최종적으로 메시지를 출력합니다.

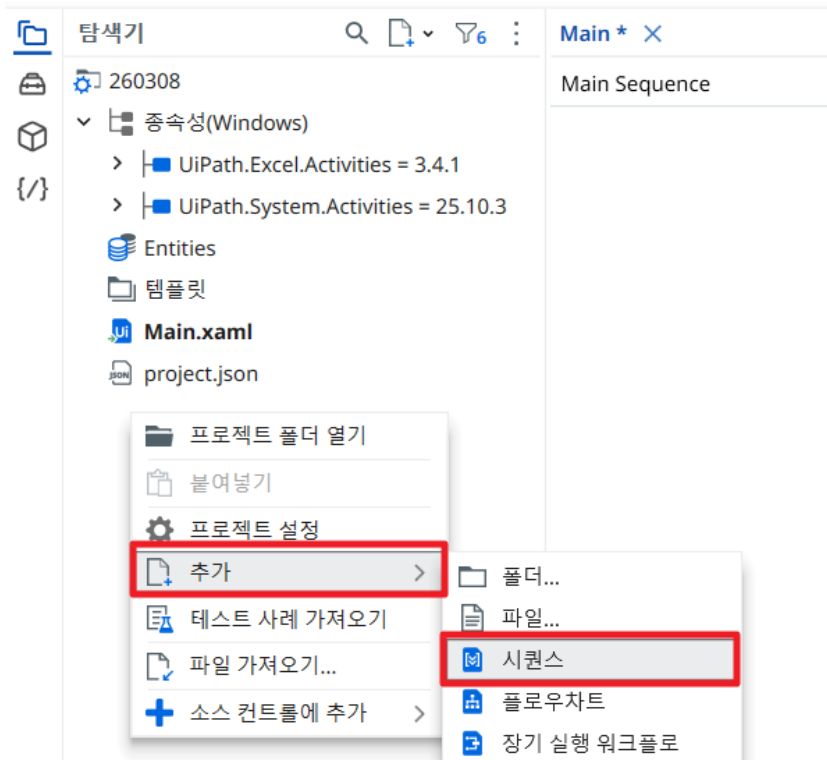
실습 구성

전체 흐름은 다음과 같습니다.

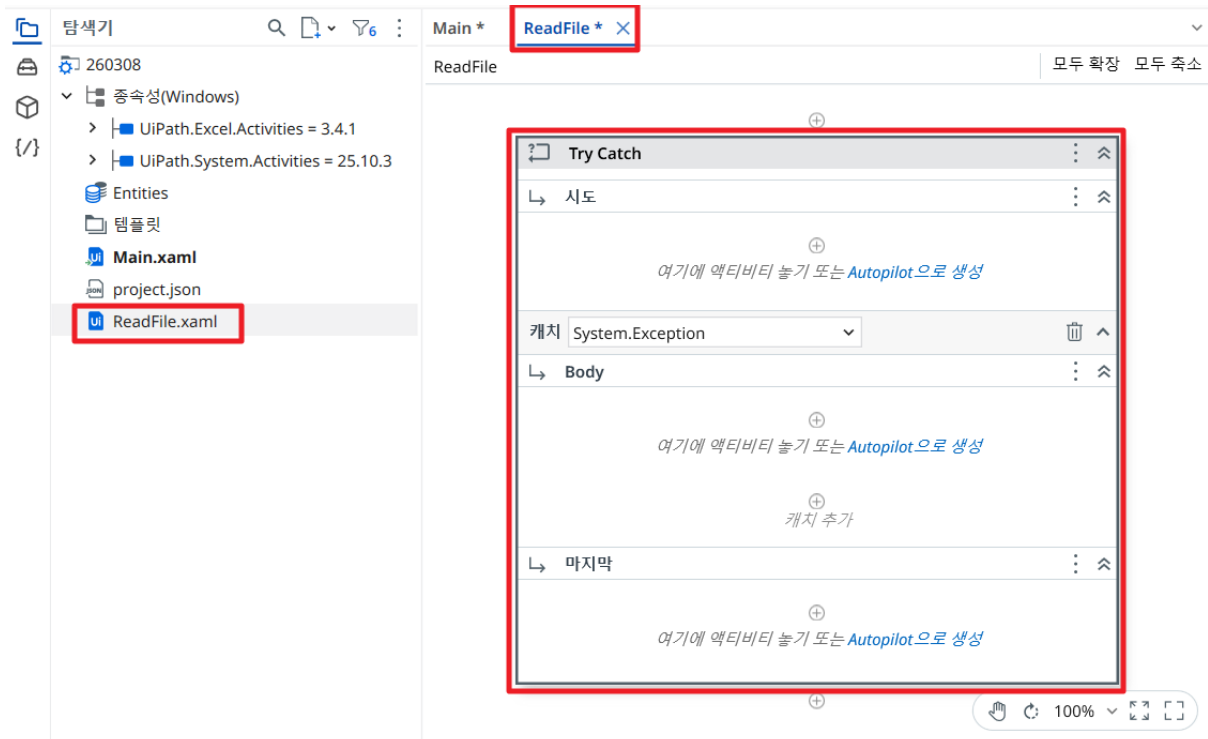


Step1. ReadFile.xaml 생성

- 프로젝트 패널에서 새로운 Sequence 워크플로를 생성하고 이름을 ReadFile.xaml 로 지정합니다.

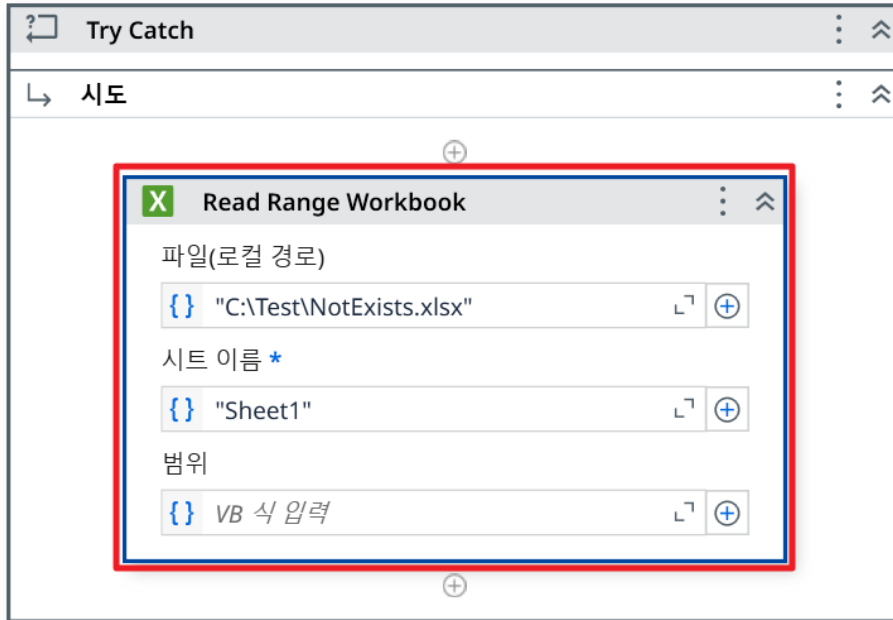


Step2. ReadFile.xaml 에 Try-Catch 추가



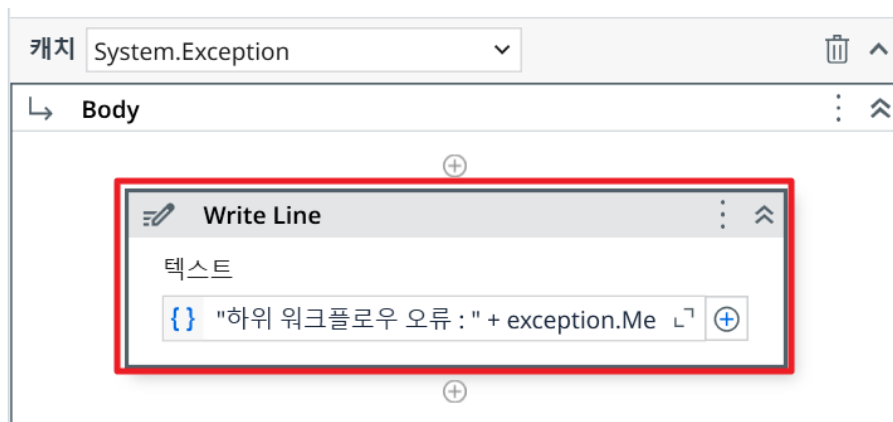
Step3. Try 영역에 존재하지 않는 Excel 파일을 읽는 액티비티를 배치합니다.

- Read Range Workbook 액티비티 드래그 앤 드롭
- 파일(로컬 경로): C:\Test\NotExists.xlsx

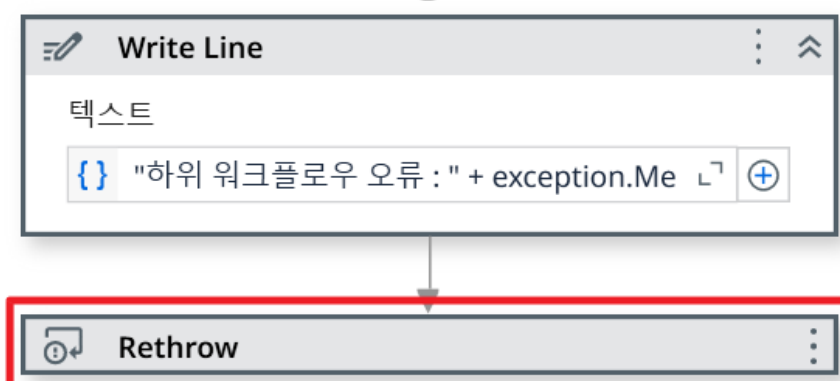


Step4. Catch 내부에 Write Line 액티비티를 배치하여 다음 메시지를 출력합니다..

- Write Line 텍스트: "하위 워크플로우 오류 : " + exception.Message

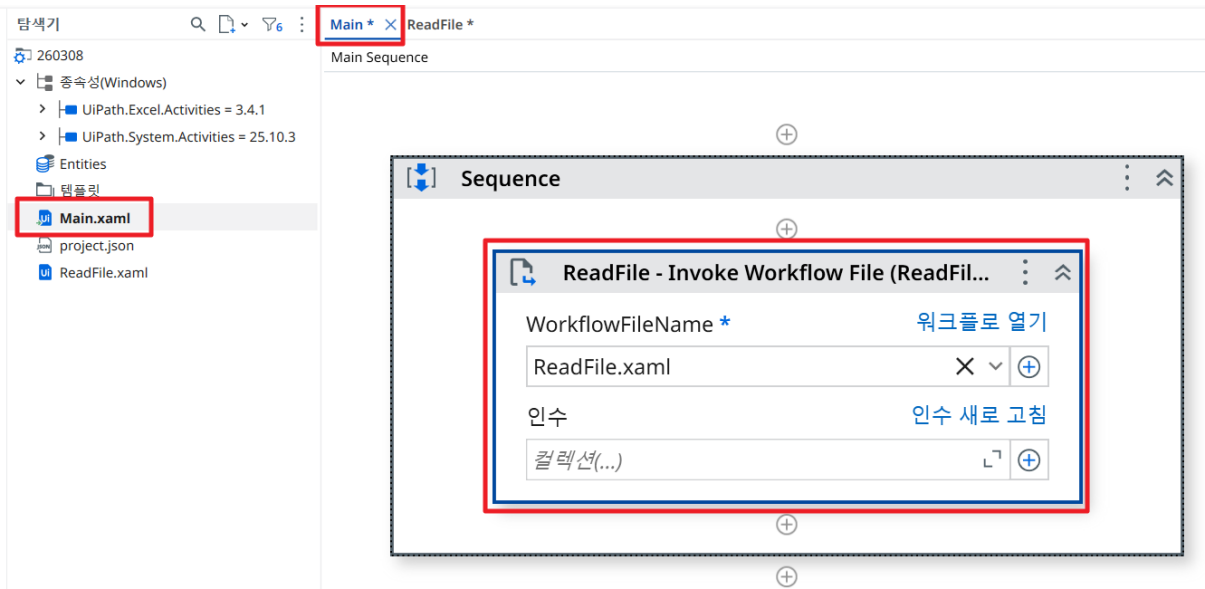


Step5. Catch 아래에 Rethrow 추가



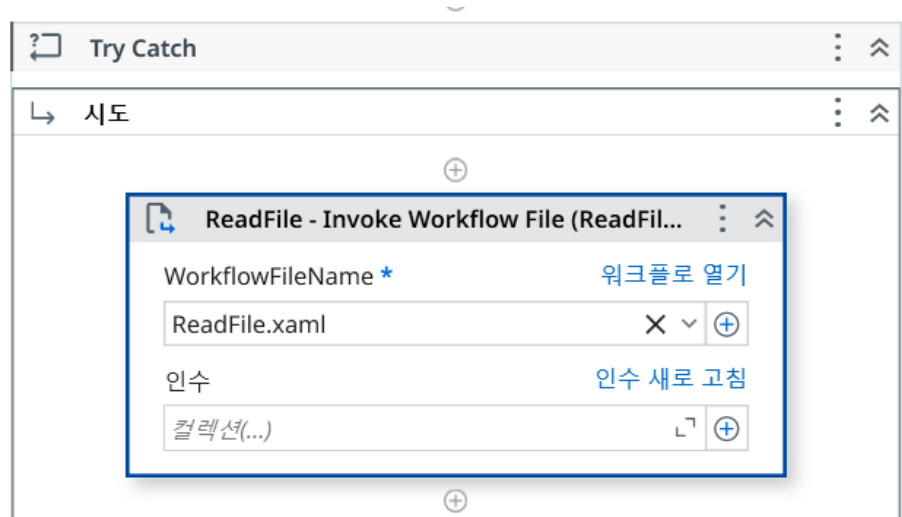
Step6. Main.xaml 에서 ReadFile.xaml 호출

- Invoke Workflow File 액티비티 드래그 앤 드롭 후 ReadFile.xaml 을 호출



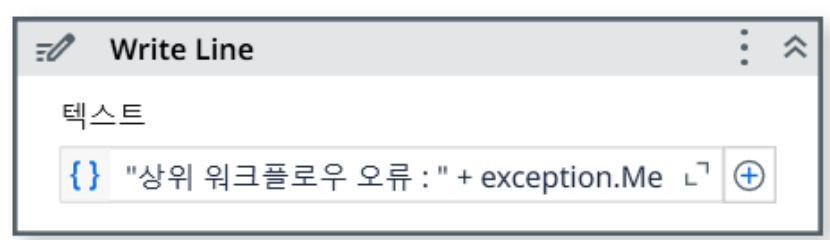
Step7. Main.xaml 의 Try-Catch 구성

- Try-Catch 액티비티 드래그 앤 드롭 후, Try 내부에 Invoke Workflow File 이동

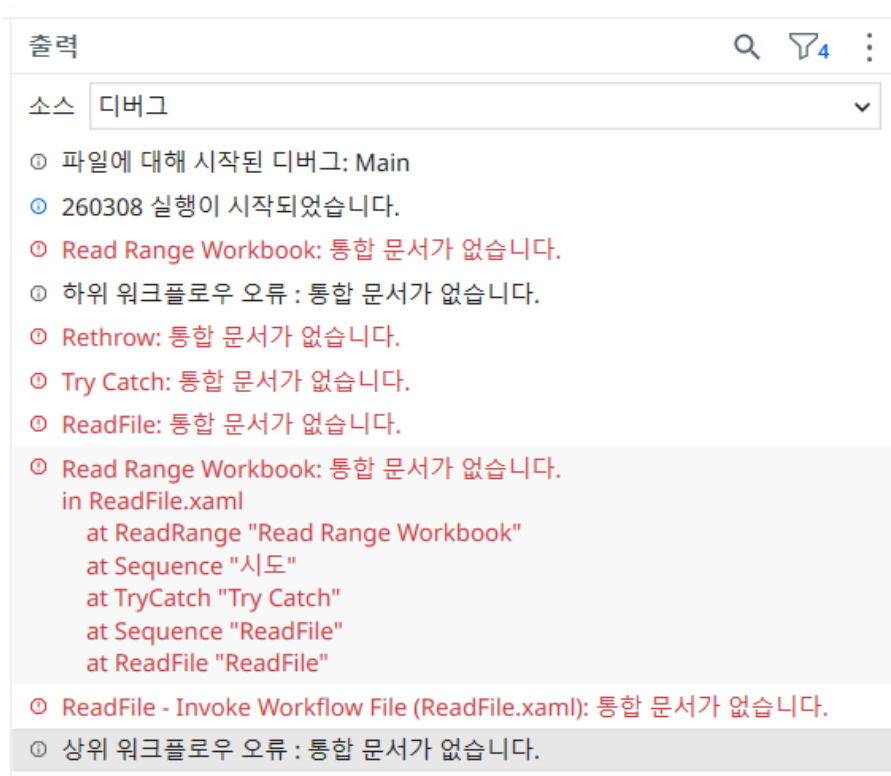


- Catch 내부에 Write Line 액티비티를 배치하여 다음 메시지를 출력

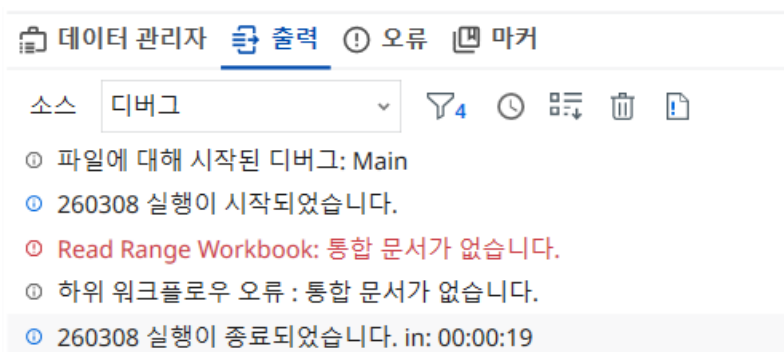
Write Line 텍스트: "상위 워크플로우 오류 : " + exception.Message



Step8. 실행 결과 확인



Step9. Rethrow 액티비티가 없는 경우도 실행 결과 확인



이 실행을 통해 확인할 수 있는 핵심은 다음과 같습니다.

- 하위 워크플로우에서 예외가 발생했다.
- 하위 Catch 에서 예외를 확인하고 로그를 남겼다.
- Rethrow 를 사용하여 예외를 상위로 다시 전달했다.
- 상위 워크플로우에서도 동일한 예외를 다시 처리할 수 있었다.

즉, Rethrow 는 Catch 에서 예외를 숨기지 않고 상위 단계까지 연결해주는 역할을 합니다.

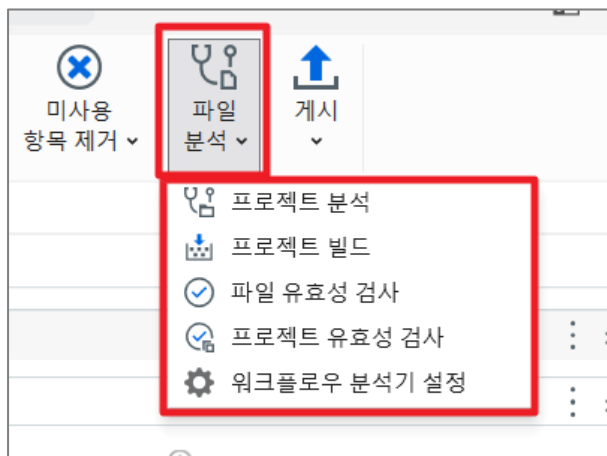
7. Studio 파일 분석

7.1 파일 분석 기능 개요

UiPath Studio에서는 워크플로우를 실행하기 전에 코드 품질과 오류 가능성을 확인할 수 있는 **파일 분석 기능(Workflow Analyzer)**을 제공합니다. 파일 분석 기능을 사용하면 프로젝트 내 워크플로우에서 발생할 수 있는 문제를 사전에 확인할 수 있습니다.

예를 들어 사용되지 않는 변수, 잘못된 구성, 권장되지 않는 설계 방식 등 다양한 항목을 자동으로 검사합니다. 이를 통해 자동화를 실행하기 전에 잠재적인 오류를 발견하고, 보다 안정적인 프로세스를 설계할 수 있습니다.

Studio 상단 리본 메뉴의 파일 분석 기능을 통해 분석을 실행할 수 있습니다.



메뉴	설명
프로젝트 분석	전체 프로젝트의 워크플로우를 분석
프로젝트 빌드	프로젝트를 패키지로 빌드
파일 유효성 검사	현재 워크플로우 파일의 오류 검사
프로젝트 유효성 검사	프로젝트 전체 구조 검사
워크플로우 분석기 설정	분석 규칙 설정

7.2. 프로젝트 분석 (Analyze Project)

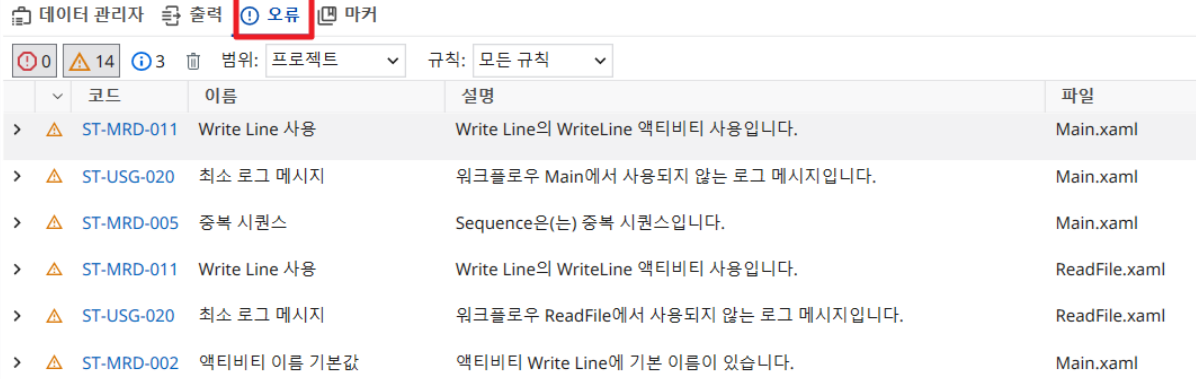
프로젝트 분석은 현재 프로젝트에 포함된 **전체 워크플로우 파일을 대상으로** 검사를 수행하는 기능입니다. 단일 파일만 확인하는 것이 아니라, 프로젝트 전반의 구조와 규칙 준수 여부를 한 번에 점검할 때 사용합니다.

프로젝트 분석을 실행하면 단순 문법 오류만 확인하는 것이 아니라, 변수·인수·표현식·Imports 구성의 유효성뿐 아니라 규칙 위반 여부도 함께 확인할 수 있습니다. 예를 들어 사용되지 않는 변수, 네이밍 규칙 위반, 과도하게 복잡한 구조, 권장되지 않는 설계 방식 등이 분석 대상이 될 수 있습니다.

분석 결과는 보통 다음 수준으로 구분되어 표시됩니다.

- **Error**: 반드시 수정해야 하는 항목
- **Warning**: 실행은 가능하지만 잠재적인 문제가 될 수 있는 항목
- **Message / Info**: 개선 권장 사항 또는 참고 정보

분석 결과는 Studio 하단의 **Error List 패널**에 표시됩니다.



코드	이름	설명	파일
> ST-MRD-011	Write Line 사용	Write Line의 WriteLine 액티비티 사용입니다.	Main.xaml
> ST-USG-020	최소 로그 메시지	워크플로우 Main에서 사용되지 않는 로그 메시지입니다.	Main.xaml
> ST-MRD-005	중복 시퀀스	Sequence은(는) 중복 시퀀스입니다.	Main.xaml
> ST-MRD-011	Write Line 사용	Write Line의 WriteLine 액티비티 사용입니다.	ReadFile.xaml
> ST-USG-020	최소 로그 메시지	워크플로우 ReadFile에서 사용되지 않는 로그 메시지입니다.	ReadFile.xaml
> ST-MRD-002	액티비티 이름 기본값	액티비티 Write Line에 기본 이름이 있습니다.	Main.xaml

Errors 패널에서는 결과를 **프로젝트 전체**, **현재 파일**, 또는 **특정 파일 범위**로 필터링할 수 있고, 규칙 ID 기준으로도 확인할 수 있습니다. 항목을 더블클릭하면 해당 .xaml 파일과 문제 위치로 이동할 수 있어 수정이 빠릅니다.

7.3. 워크플로우 분석기 설정

워크플로우 분석기 설정은 어떤 규칙을 사용할지, 각 규칙을 어느 수준으로 판단할지 정하는 기능입니다. 즉, 프로젝트 분석이 “검사 실행”이라면, 워크플로우 분석기 설정은 “검사 기준 정의”에 해당합니다. UiPath 에서는 각 분석 규칙이 고유한 Rule ID 를 가지며, 규칙마다 다음과 같은 항목을 설정할 수 있습니다.

- 규칙 사용 여부(IsEnabled)
- 기본 사용 여부(DefaultIsEnabled)
- 오류 수준(ErrorLevel / DefaultErrorLevel)
- 규칙별 파라미터(예: 이름 규칙, 길이 제한, 정규식 등)

즉, 사용자는 특정 규칙을 켜거나 끌 수 있고, 같은 규칙이라도 어떤 환경에서는 Warning 으로, 어떤 환경에서는 Error 로 더 엄격하게 적용할 수 있습니다. 예를 들어 변수 이름 규칙을 팀 표준에 맞게 강제하고 싶다면, 해당 규칙을 활성화한 뒤 정규식이나 길이 조건을 설정할 수 있습니다.

프로젝트 설정

일반

워크플로우 분석기

Test Manager 구성

액티비티 설정

시스템

Excel Modern

워크플로우 분석기

프로젝트가 자동화 모범 사례 및 제한사항을 따르는지 분석합니다. 자세히 알아보기

이름 또는 코드로 규칙 검색

범위: All 기본 액션: All

☐	코드	이름	범위	기본 액션
☒	ST-DBP-007	여러 플로우차트 레이어	워크플로우	경고
☒	ST-DBP-010	[워크플로] 또는 [테스트 사례]의 여러 인스턴스	Missing translation	오류
☒	ST-DBP-023	빈 워크플로우	워크플로우	경고
☒	ST-DBP-024	지속성 액티비티 확인	워크플로우	오류
☒	ST-DBP-025	변수 직렬화 전제조건	워크플로우	오류
☒	ST-DBP-027	Persistence 베스트 프랙티스	워크플로우	경고
☒	ST-DBP-028	인수 직렬화 전제조건	워크플로우	오류
☒	ST-MRD-002	액티비티 이름 기본값	액티비티	경고
☒	ST-MRD-004	연결할 수 없는 액티비티	워크플로우	경고
☒	ST-MRD-005	중복 시퀀스	워크플로우	경고
☒	ST-MRD-007	중첩된 If 절	워크플로우	경고
☒	ST-MRD-008	빈 시퀀스	워크플로우	경고

권장 사항

가독성 향상을 위해 중첩된 if 절을 사용하지 않습니다. 중첩되지 않은 If 절, 플로우차트 또는 스위치를 사용하여 다시 씁니다. 자세히 알아보기

임계값:

3

확인

취소

7.4. 파일 및 프로젝트 유효성 검사

UiPath Studio에서는 워크플로우 실행 전에 오류 여부를 확인할 수 있도록 **유효성 검사(Validation)** 기능을 제공합니다. 이 기능은 워크플로우의 구조나 설정에 문제가 없는지 확인하여 자동화 실행 중 발생할 수 있는 오류를 사전에 발견하는 데 목적이 있습니다. 유효성 검사는 검사 범위에 따라 **파일 유효성 검사**와 **프로젝트 유효성 검사**로 구분됩니다. **파일 유효성 검사**는 현재 열려 있는 단일 워크플로우 파일(.xaml)을 대상으로 검사합니다.

현재 작업 중인 워크플로우의 액티비티 설정이나 변수, 인수, 표현식 등의 구성이 올바른지 확인할 수 있으며, 개발 중 특정 파일의 오류를 빠르게 점검할 때 유용합니다.

반면 **프로젝트 유효성 검사**는 프로젝트에 포함된 모든 워크플로우 파일을 대상으로 검사를 수행합니다. 프로젝트 전체의 워크플로우 구조와 액티비티 설정, 변수 및 인수 구성 등을 확인하여 자동화가 정상적으로 실행 가능한 상태인지 점검합니다.

검사 결과는 Studio 하단의 **Errors** 패널에 표시되며, 오류 항목을 선택하면 해당 워크플로우 위치로 바로 이동하여 문제를 수정할 수 있습니다. 유효성 검사는 자동화가 정상적으로 실행 가능한지 확인하는 기능이라는 점에서, 코드 품질이나 설계 규칙을 검사하는 **프로젝트 분석(Workflow Analyzer)**과 차이가 있습니다.

구분	프로젝트 분석 (Analyze Project)	파일 유효성 검사 (Validate File)	프로젝트 유효성 검사 (Validate Project)
주요 목적	코드 품질 및 설계 규칙 검사	현재 워크플로우 오류 확인	프로젝트 전체 실행 가능 여부 확인
검사 범위	프로젝트 전체 워크플로우	현재 열려 있는 .xaml 파일	프로젝트 전체 워크플로우
검사 대상	코드 품질 규칙, 개발 표준, 권장 설계 방식	액티비티 설정, 변수, 표현식 등 구조 오류	워크플로우 구조, 액티비티 설정, 프로젝트 구성
검사 결과 유형	Error / Warning / Info	Error 중심	Error 중심
사용 시점	코드 리뷰, 품질 점검, 배포 전 품질 확인	개발 중 특정 파일 오류 확인	개발 완료 후 프로젝트 전체 점검
주요 특징	워크플로우 분석기 기반 분석	빠른 단일 파일 검사	전체 프로젝트 실행 가능 여부 확인

SESSION2. 정보 관리(CONFIG)

학습 목표

- Dictionary 변수에 대해 설명할 수 있다.
- Config 파일과 Window Credential 을 연결할 수 있다.

1. Config 소개

프로젝트 설정을 정의하는 데 사용되는 엑셀 파일입니다. 일반적으로 RPA 개발에서 사용되는 데이터를 통합하여 관리하는 목적으로 사용됩니다.

- Accounts: Window Credential 의 계정 정보를 호출합니다.

Name	Value	Description
System1_Login	System1_Account	Window Credential Manager System1_Account

- Path: 폴더 경로 및 브라우저 URL 을 설정합니다.

Name	Value	Description
Input	C:\Studio\Data\WInput	프로세스가 사용하는 입력 파일들이 저장
Output	C:\Studio\Data\WOutput	프로세스의 최종 산출물 파일들이 저장
Temp	C:\Studio\Data\WTemp	프로세스 수행 중에 생기는 임시파일들이 저장
System1_URL	https://acme-test.uipath.com	ACME System1 URL

- Code: 파일명, ERP 코드명들을 저장합니다.

Name	Value	Description
MonthlyReport	Report-{TaxID}-{Year}-{Month}	Download Monthly Report File Name
YearlyReport	Yearly-Report-{Year}-{TaxID}	Create Yearly Report File Name
KCCReport	KCC Vendor Information {Year} Yearly Report	KCC Vendor Information Result File Name

- Email: 이메일과 관련된 내용을 설정합니다.

Name	Value	Description
To		받는사람
Cc		참조자
From		보내는사람
Title	[RPA 종료 메일] {0}	제목
Body	{0} 과제가 정상적으로 종료되었습니다.	내용

2. Dictionary 변수 이해하기

2.1. Dictionary 변수 정의

- 키(Key) - 값(Value) 쌍으로 이루어져 있으며 특정 값을 가져올 때 유용한 변수로 이 구조는 “사전”에서 단어(키)를 찾으면 그 뜻(값)을 나오는 것처럼, Dictionary 변수도 키를 통해 값을 가져온다 해서 붙여졌습니다. 주의할 점은, 같은 키를 가진 두 개의 값은 존재할 수 없으며, 그렇게 사용할 경우에는 기존의 값이 새 값으로 덮어씌워집니다.

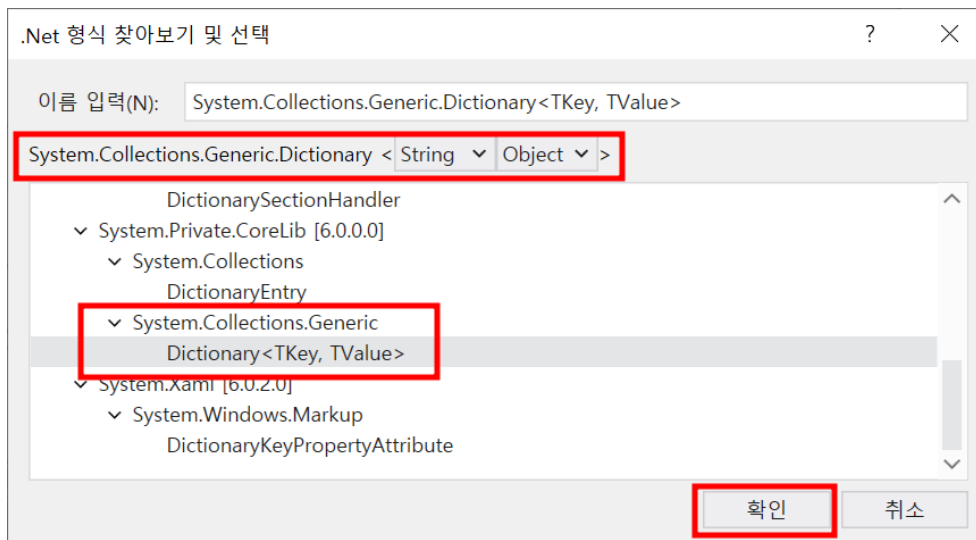
2.2. Dictionary 변수 만들기

Step1. 변수 패널에서 변수 선언하기

이름	변수 형식	범위
Dic_Name	String	Main Sequence
변수 만들기		
변수	인수	가져오기

Step2. 변수 형식 찾기

- 변수 형식 - ‘Browser for Types...’ 클릭 - 입력 창에 Dictionary 입력
- 변수 형식을 찾은 후 확인 클릭



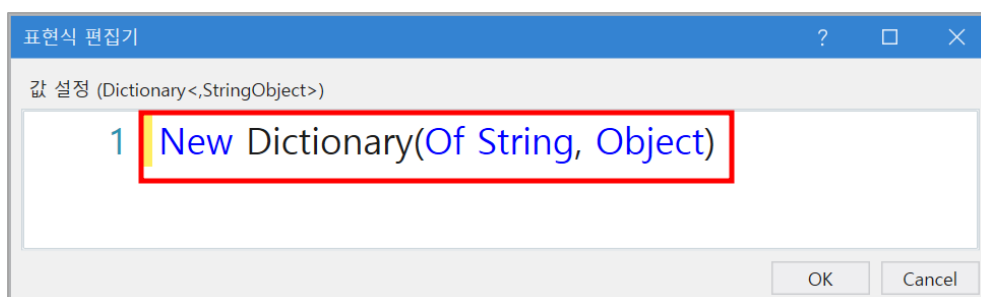
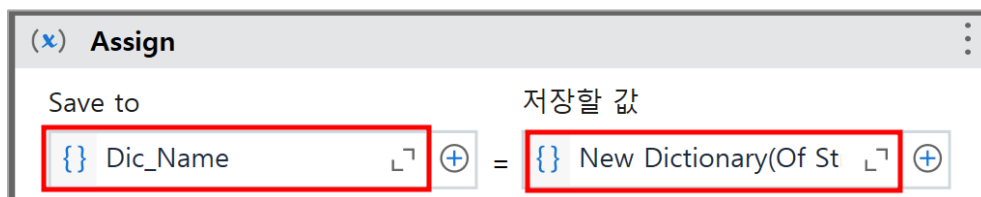
Step3. Dictionary 변수 초기화

- “변수 초기화”라는 것은 변수에 최초의 값을 할당하는 과정입니다. 이렇게 함으로써 변수는 저장공간을 가지게 되고, 그 저장공간에는 초기화 과정에서 할당한 값이 저장됩니다. 만약, 초기화 없이 변수를 사용하려고 하면, 해당 변수는 메모리를 할당 받지 못했으므로 어떤 값도 갖지 않으며, 이 상태에서 변수를 사용하려고 하면 Null 참조 오류가 발생합니다.

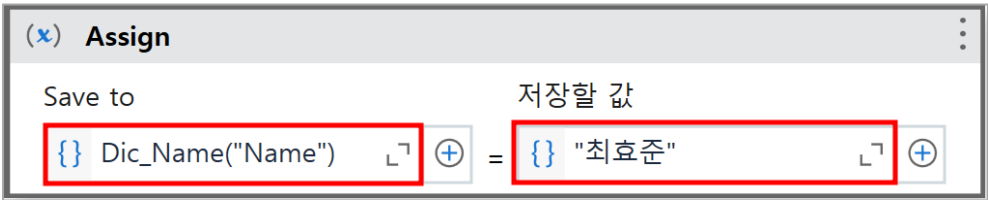
Q1. String 이나 Int32 같은 변수들은 그냥 변수 만들고 초기화 없이 사용할 수 있는데 왜 Dictionary 변수 유형은 초기화를 하나요?

A. String 이나 Int32 와 같은 기본형 변수들은 초기화하지 않고 사용할 수 있지만, 그럴 경우 그 변수는 해당 타입의 기본 값으로 설정됩니다. 예를 들어, 초기화하지 않은 Int32 (integer) 변수는 0 으로, String 변수는 null (또는 VB.NET 에서는 Nothing)로 설정됩니다.

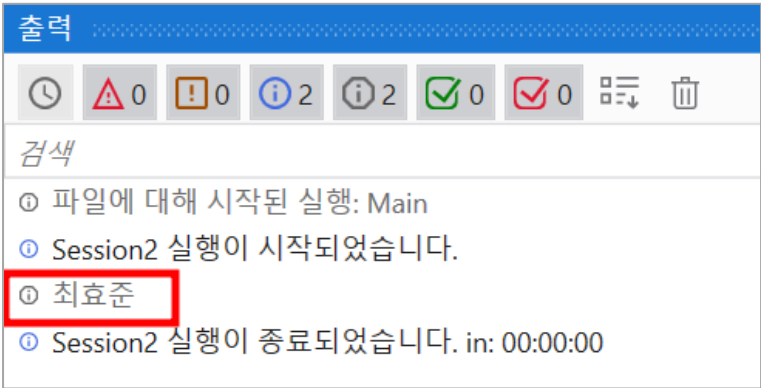
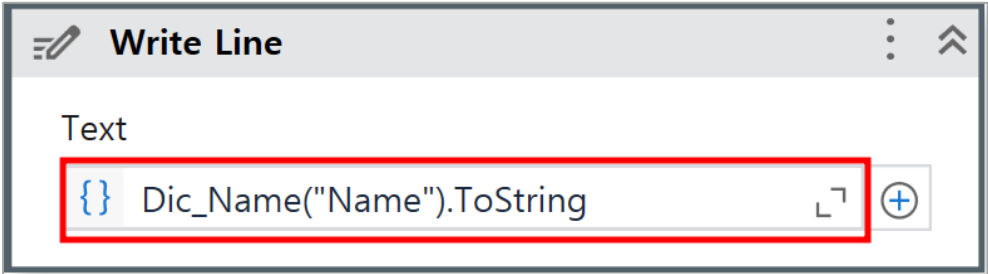
하지만, Dictionary 와 같은 복합 객체(컬렉션, 클래스 등)의 경우, 그들은 기본적으로 null 로 초기화됩니다. 따라서 이러한 타입의 변수에 접근하려면 반드시 먼저 초기화를 해야 합니다. 그렇지 않으면 null 참조 오류가 발생합니다.



Step4. Name(Key)에 “최효준”(Value) 저장



Step5. 출력 후 결과 확인



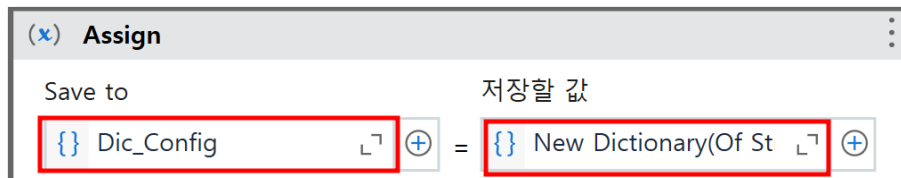
3. Config.xlsx 단일 시트 읽기

3.1. Config 파일 데이터 추출

- “Path” 시트에서 “System1_URL” 값을 출력하는 프로세스를 만들어 보시다.

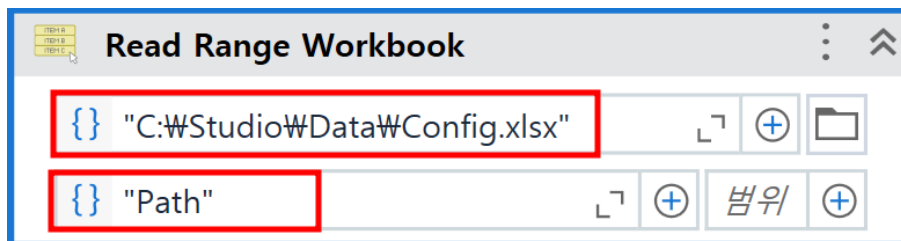
Step1. Dictionary 변수 선언 및 초기화

- 변수 이름: Dic_Config / - 변수 유형: Dictionary<String, Object>



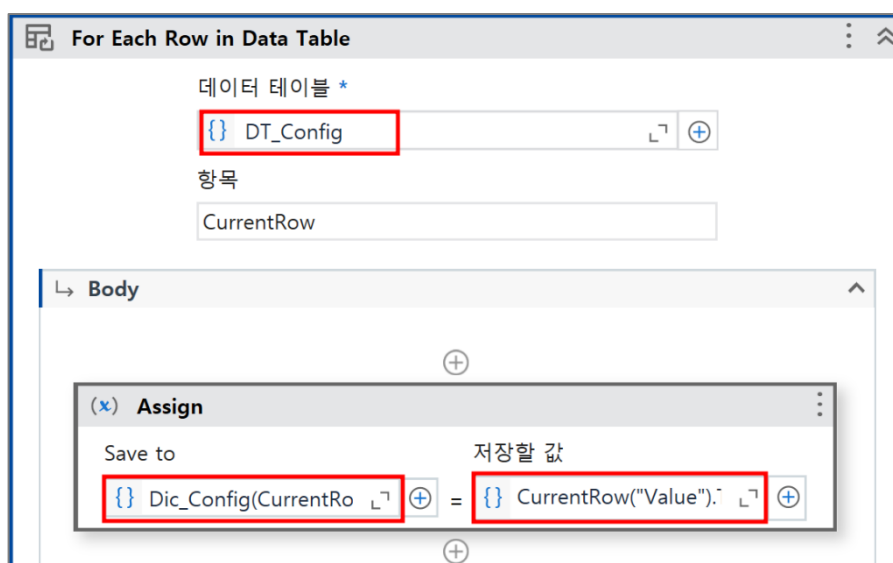
Step2. Config.xlsx 데이터 추출

- Read Range Workbook 액티비티 드래그 앤 드롭
- 경로: "C:\Studio\Data\Config.xlsx" / - 시트 이름: "Path"
- 시트 데이터 저장 변수 이름: DT_Config



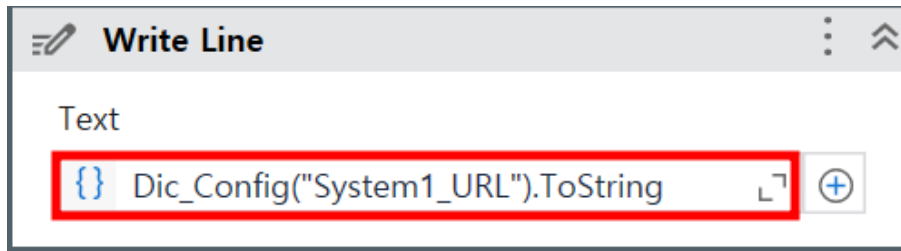
Step3. Dictionary 변수에 DataTable 값 순서대로 저장

- For Each Row in DataTable 액티비티 드래그 앤 드롭
- Save to: Dic_Config(CurrentRow("Name").ToString)
- 저장할 값: CurrentRow("Value").ToString

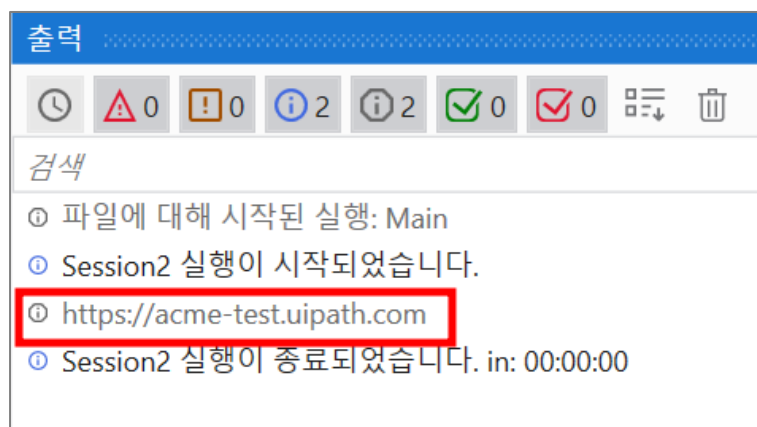


3.3. Config 변수 값 출력하기

- Write Line 액티비티 드래그 앤 드롭



- “System1_URL” 값 출력



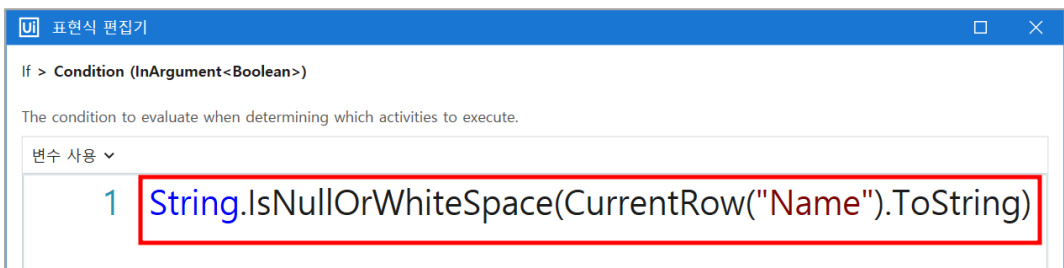
3.2. 예외 사항 처리

- Dic_Config("Name")의 자리가 비어있는 경우 잘못된 값이 들어가거나 수행 중 문제가 발생하는 것을 최소한으로 하기 위해 For Each 작업을 할 때는 'Name' 칼럼이 비어있는 경우 넘어갈 수 있도록 예외 처리가 필요합니다.

Name	Value	Description
Input	C:\Studio\Data\WInput	프로세스가 사용하는 입력 파일들이 저장
Output	C:\Studio\Data\WOutput	프로세스의 최종 산출물 파일들이 저장
Temp	C:\Studio\Data\WTemp	프로세스 수행 중에 생기는 임시파일들이 저장
System1_URL	https://acme-test.uipath.com	ACME System1 URL

Step1. 현재 접근하고 있는 값이 비어 있는지 확인

- If 액티비티 드래그 앤 드롭
- 조건: `String.IsNullOrEmpty(CurrentRow("Name").ToString)`



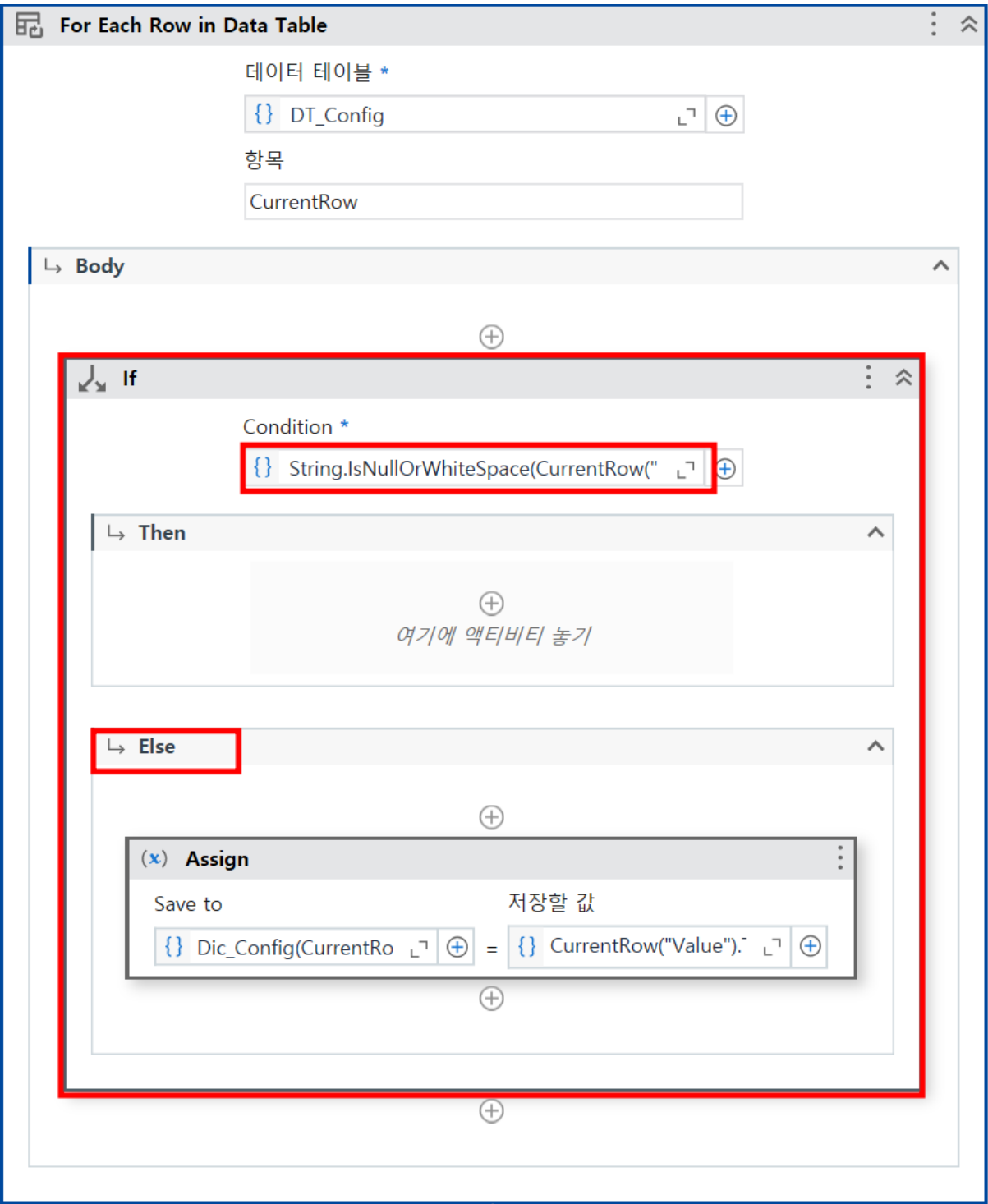
Q1. “String.IsNullOrEmpty”와 “String.IsNullOrEmpty”의 차이가 뭔가요?

A. 두 개의 메서드는 모두 문자열이 비어 있는지 또는 null 인지 확인하는 데 사용됩니다. 이 두 메서드의 주요 차이점은 공백 문자열(" ")에 대한 처리 방식입니다.

만약, 문자열이 (" ")인 경우, “String.IsNullOrEmpty” 메서드는 ‘True’ 즉, 비어있다. 라는 뜻이고, “String.IsNullOrEmpty” 메서드는 False 값을 반환하기 때문에 비어있지 않다 라는 뜻을 가지고 있습니다.

따라서, Excel Cell 값이 1) 비어 있거나, 2) Null 값 이거나, 3) (" ") 공백 문자열인 상황을 모두 체크하는 상황에서는 “String.IsNullOrEmpty” 메서드를 사용하는 것이 더 적합합니다.

Step2. 해당 조건이 거짓(False)일 때, Dictionary 변수에 값을 저장하기 위해 Else 쪽에
기존 프로세스 위치

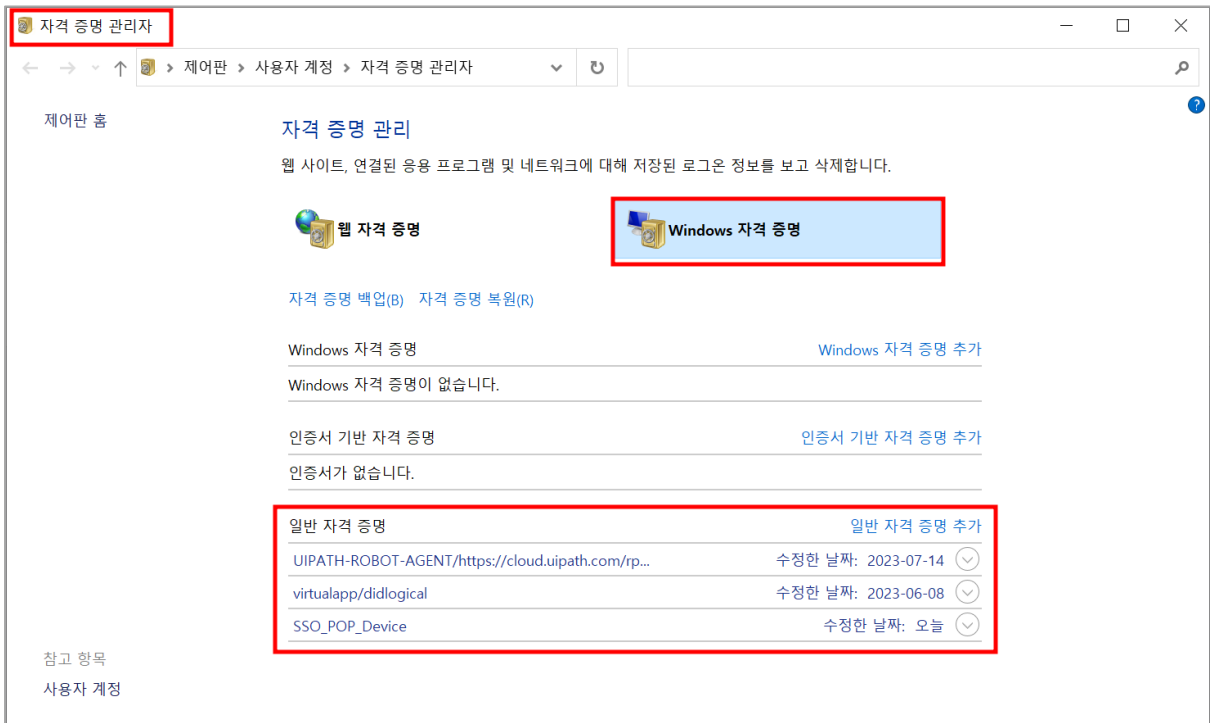


4. Window 자격 증명 만들기

4.1. Window 자격 증명 정의

- Microsoft Windows 에서 사용하는 인증 메커니즘이며, 사용자의 ID 와 비밀번호 같은 개인 정보를 안전하게 관리하는 방법을 제공합니다. 이 자격 증명은 사용자가 특정 자원 또는 서비스에 접근할 때 필요한 인증 정보를 저장하고 관리하는 데 사용됩니다.

특히, **일반 자격 증명**은 웹 또는 네트워크 리소스에 대한 사용자 이름과 비밀번호를 저장하는 데 사용됩니다.



4.2. 일반 자격 증명 생성

Step1. 일반 자격 증명 추가 클릭



Step2. ACME 사이트 로그인 정보 입력 후 확인 버튼 클릭

웹 사이트 주소 및 자격 증명 정보를 입력하십시오.

입력한 사용자 이름과 암호를 사용하여 해당 위치에 액세스할 수 있는지 확인하십시오.

인터넷 또는 네트워크 주소:

System1_Account

사용자 이름:

rpakcc3@gmail.com

암호:

●●●●●●●●

확인(O)

취소(N)

Q1. “System1_Account”가 뭔가요?

A. 여기서 “System1_Account” 는 우리가 Config.xlsx 파일에서 값을 호출할 때 어떤 이름으로 가지고 올 것인지 정의하는 부분입니다. Config.xlsx 파일 ‘Account’ 시트를 확인해보면 알 수 있습니다.

	A	B	C
1	Name	Value	Description
2	System1_Login	System1_Account	Window Credential Manager System1_Account

만약, 일반 자격 증명 – 인터넷 또는 네트워크 주소 값을 변경한다면, Config.xlsx 파일 값도 동일하게 변경해야 프로세스 수행 중에 접근할 수가 있습니다.

Step3. 생성된 자격 증명 확인

일반 자격 증명

일반 자격 증명 추가

System1_Account

수정한 날짜: 오늘

인터넷 또는 네트워크 주소: System1_Account

사용자 이름: rpakcc3@gmail.com

암호: ●●●●●●

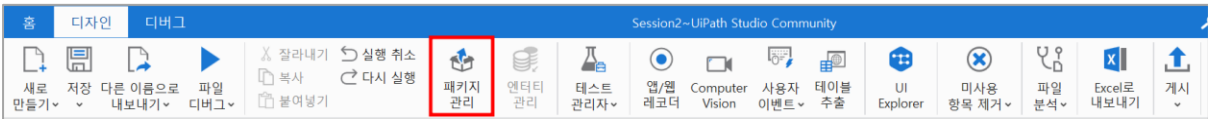
지속성: 엔터프라이즈

편집 제거

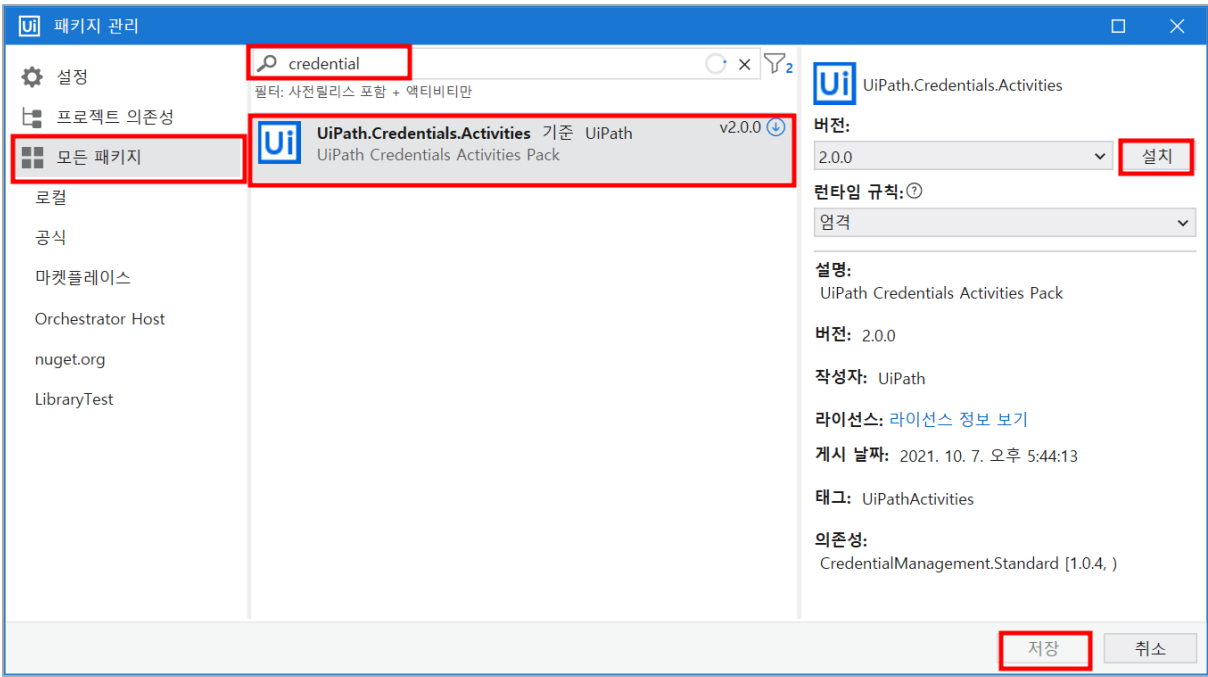
5. UiPath Studio 에서 자격 증명 확인

5.1. UiPath Credential 패키지 설치

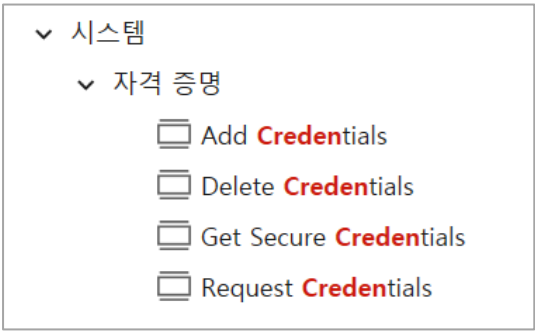
Step1. UiPath Studio – 패키지 관리 메뉴 클릭



Step2. 모든 패키지 – Credential 입력 후 패키지 설치



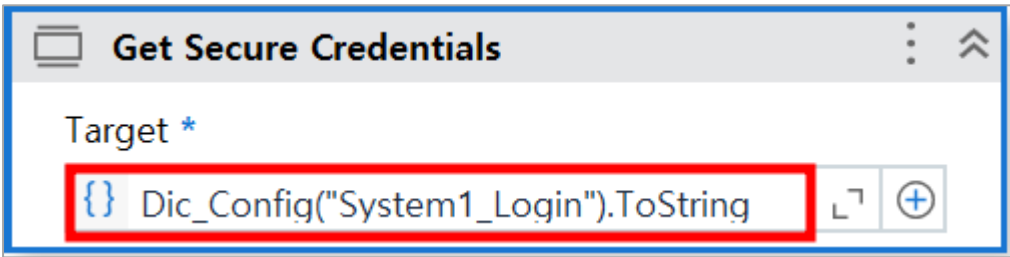
Step3. Credentials 패키지 확인



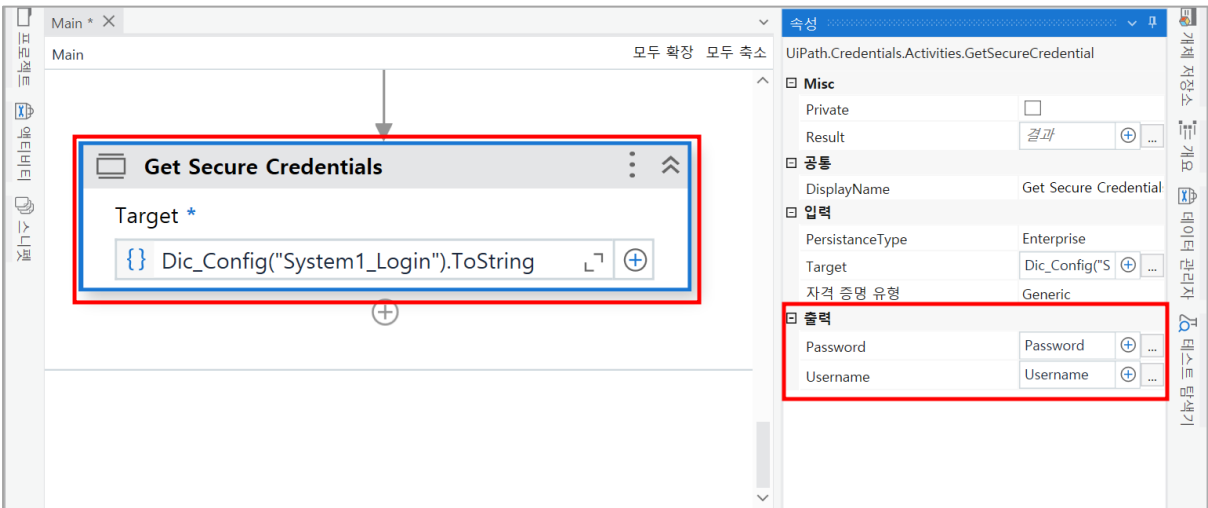
5.2. Window 자격 증명 값 추출

Step1. 프로세스 최하단에 'Get Secure Credentials' 액티비티 드래그 앤 드롭

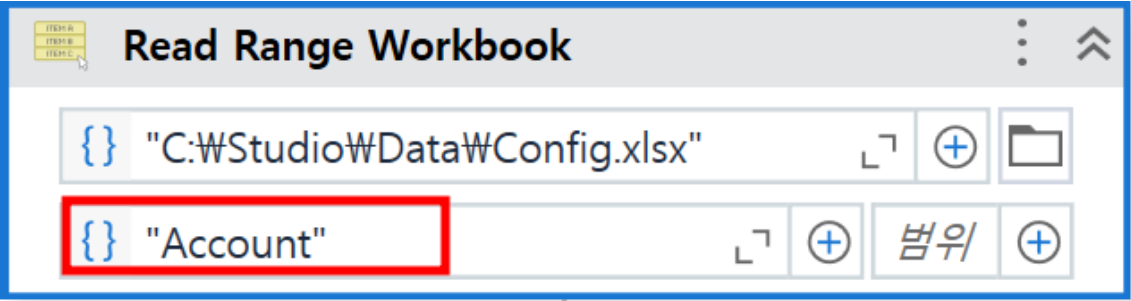
- Target: Dic_Config("System1_Login").ToString 값 입력



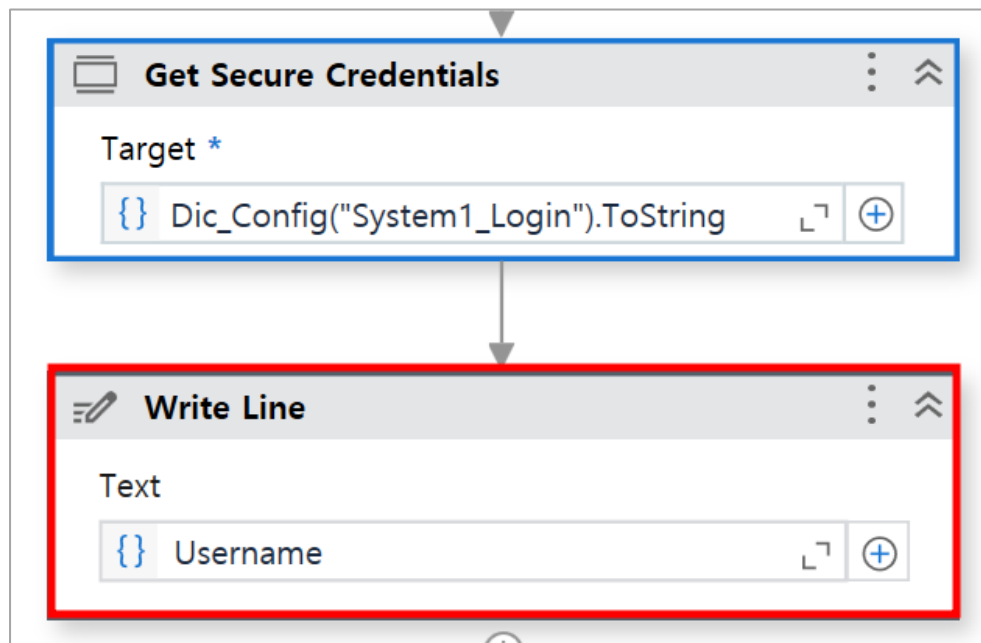
Step2. 속성 - 출력 옵션에서 Password, Username 변수 생성



Step3. Read Range 시트명 변경



Step4. Write Line 값 변경 후 출력 결과 확인



출력

⌚ ⚠️ 0 ⚠️ 0 ⓘ 2 ⓘ 2 ✅ 0 ✅ 0 ⌵ 🗑️

검색

- ⌚ 파일에 대해 시작된 실행: Main
- ⓘ Session2 실행이 시작되었습니다.
- ⓘ **rpakcc3@gmail.com**
- ⓘ Session2 실행이 종료되었습니다. in: 00:00:00

SESSION3. RPA 프로세스 관리

학습 목표

- Invoke Workflow File, Library, Snippet, Object Repository 를 구분할 수 있다.

1. 기능 비교

구분	특징	장점	단점
Invoke Workflow	- 동일 프로젝트 또는 참조된 XAML을 호출하는 활동 - 인수를 통해 데이터 전달	- 모듈화 및 가독성 향상 - 단위 테스트와 디버깅 용이	- 인수 설계 및 매핑 관리 비용 - 과도한 분할 시 구조 복잡
Library	- 재사용 가능한 워크플로우/활동을 NuGet 패키지로 배포 - Orchestrator 또는 로컬/공유 피드 활용 - 버전 관리 가능	- 여러 프로젝트 간 재사용 - 표준화와 일관성 확보 - 버전 고정 및 변경 이력 관리	- 패키지 버전 업 시 호환성 검증 필요 - 배포/피드 운영 관리 비용 - 변경 사항 자동 전파 아님
Snippet	- 자주 쓰는 코드 블록/패턴을 편집기에서 삽입하는 템플릿	- 반복 코드 입력 속도 향상 - 스타일 일관성 유지	- 삽입 후 원본 변경 소급 불가 - 버전 관리 체계 부재 - 팀 간 공유 한계
Object Repository	- UI 요소/셀렉터 중앙 관리 - 라이브러리 내보내기 가능	- 셀렉터 재사용 및 일괄 업데이트 - UI 변경 영향 범위 관리 용이	- 잘못된 변경 시 광범위 영향 - 동적 UI 대응 위한 튜닝 필요

UiPath 에서 재사용성을 높이는 방법은 크게 네 가지가 있습니다.

Invoke Workflow 는 동일 프로젝트 내에서 XAML 을 불러와 모듈화를 지원하지만, 인수 관리가 필요하고 프로젝트 간 재사용은 제한됩니다.

Library 는 워크플로우나 활동을 NuGet 패키지로 배포하여 여러 프로젝트에서 재사용할 수 있으며, 버전 관리가 가능하지만 배포와 호환성 검증 과정이 필요합니다.

Snippet 은 자주 쓰는 코드 조각을 빠르게 삽입하는 템플릿으로 개발 효율성을 높이지만, 삽입 후 원본 변경이 소급되지 않는 한계가 있습니다.

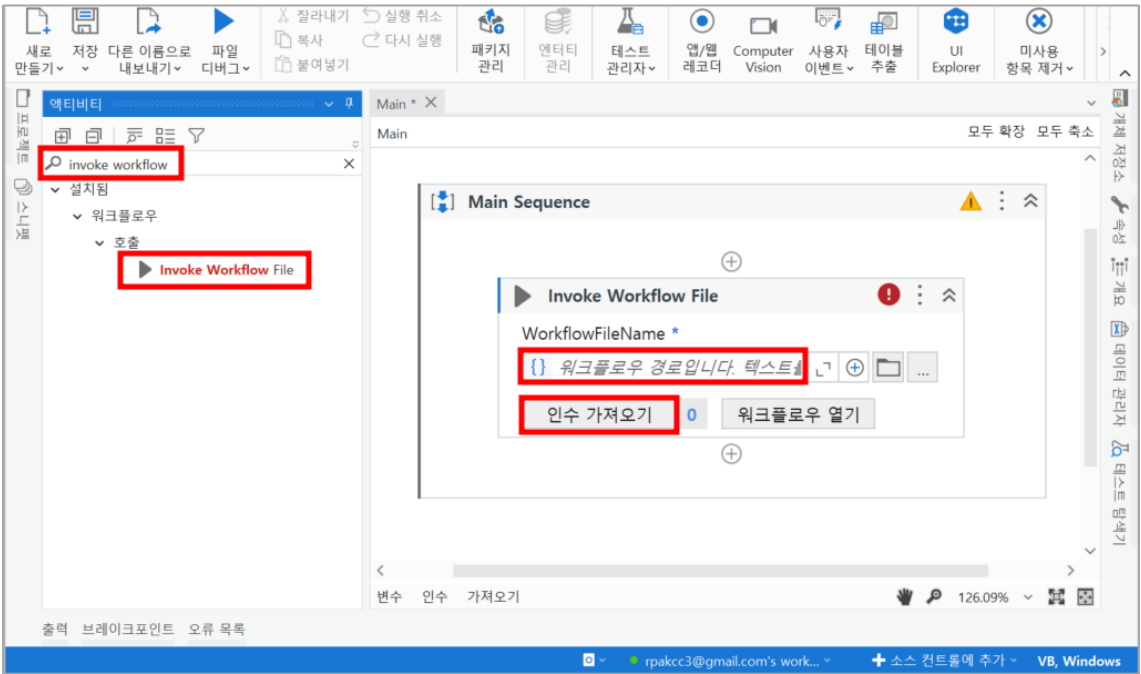
Object Repository 는 UI 요소와 셀렉터를 중앙에서 관리해 변경 시 일괄 반영할 수 있으나, 잘못된 변경 시 광범위에 영향을 줄 수 있습니다.

따라서 목적에 맞게 각 기능을 적절히 선택해 사용하는 것이 효율적인 자동화 개발의 핵심입니다.

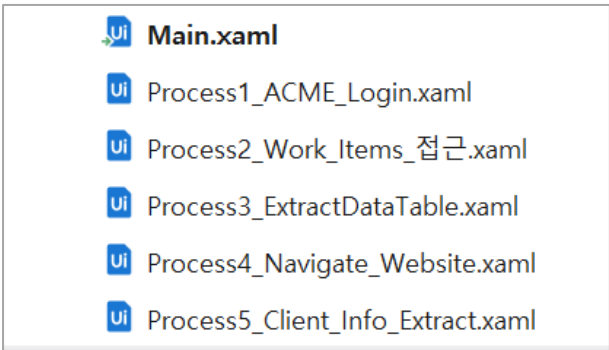
2. Invoke Workflow File

2.1. Invoke Workflow File 정의

- 이미 만들어진 워크플로우 파일(.xaml)을 다른 워크플로우 안에서 실행하는 액티비티입니다. 이를 통해 워크플로우를 모듈화와 코드 재사용성을 높일 수 있습니다.



- Project 폴더 및 기타 다른 폴더에 있는 워크플로우 파일과 상호작용을 할 수 있습니다.

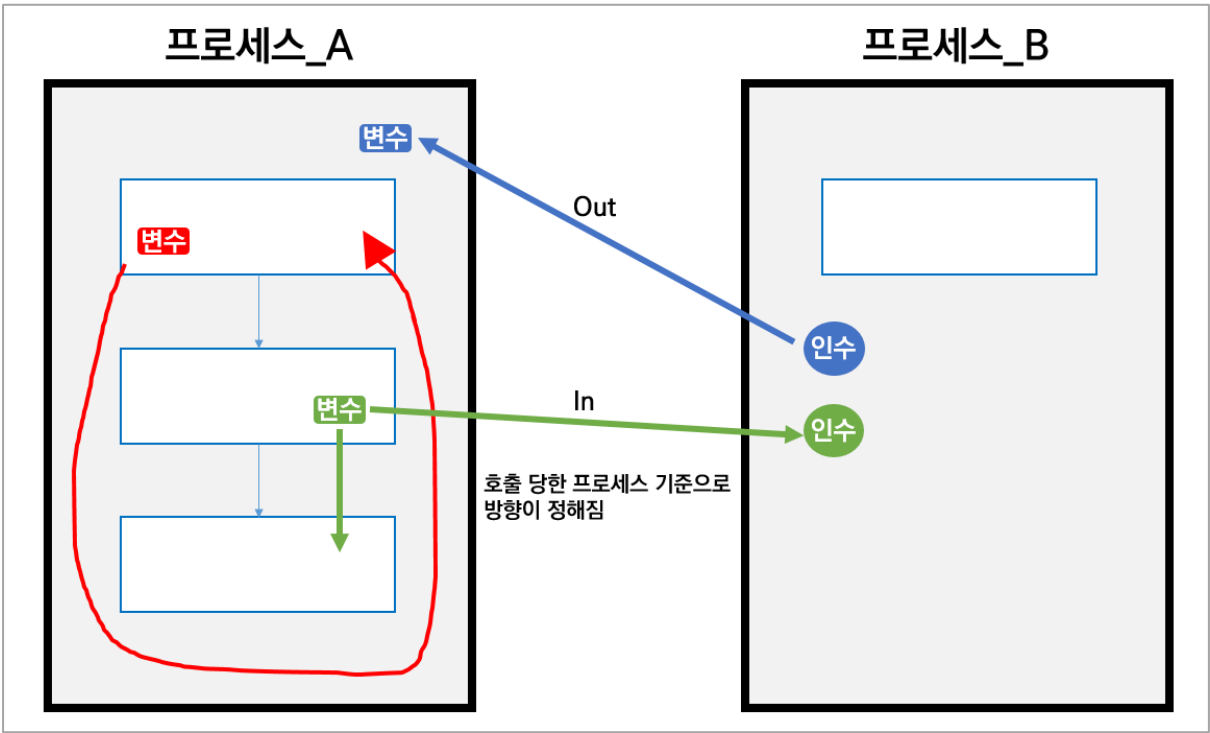


2.2. 인수

저희는 Studio 기본편에서 외부에서 데이터가 발생한 경우 직접 데이터를 처리하지 않고 ‘변수’라는 저장공간에 데이터를 담아 가공하고 출력한다고 배웠습니다.

마찬가지로 ‘인수’도 일종의 변수의 역할을 수행하지만 가장 대표적인 차이점이라고 하면, ‘변수’는 프로세스 내부에서만 데이터를 주고받을 수 있지만, ‘인수’는 프로세스 간 데이터를 주고받을 때 사용할 수 있습니다.

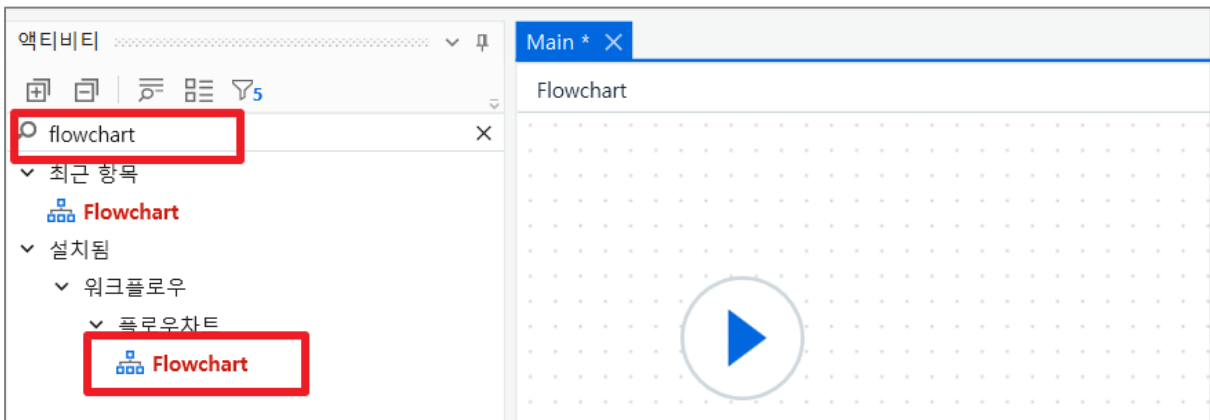
항목	인수(Argument)	변수(Variable)
공통점	데이터를 저장합니다.	
차이점	프로세스 - 프로세스 파일 간 데이터를 전달합니다.	프로세스 파일 안에서만 사용할 수 있습니다.
특징	방향(Direction) - In, Out, In/Out	범위(Scope) - Local, Global



2.3. Invoke Workflow File 활용

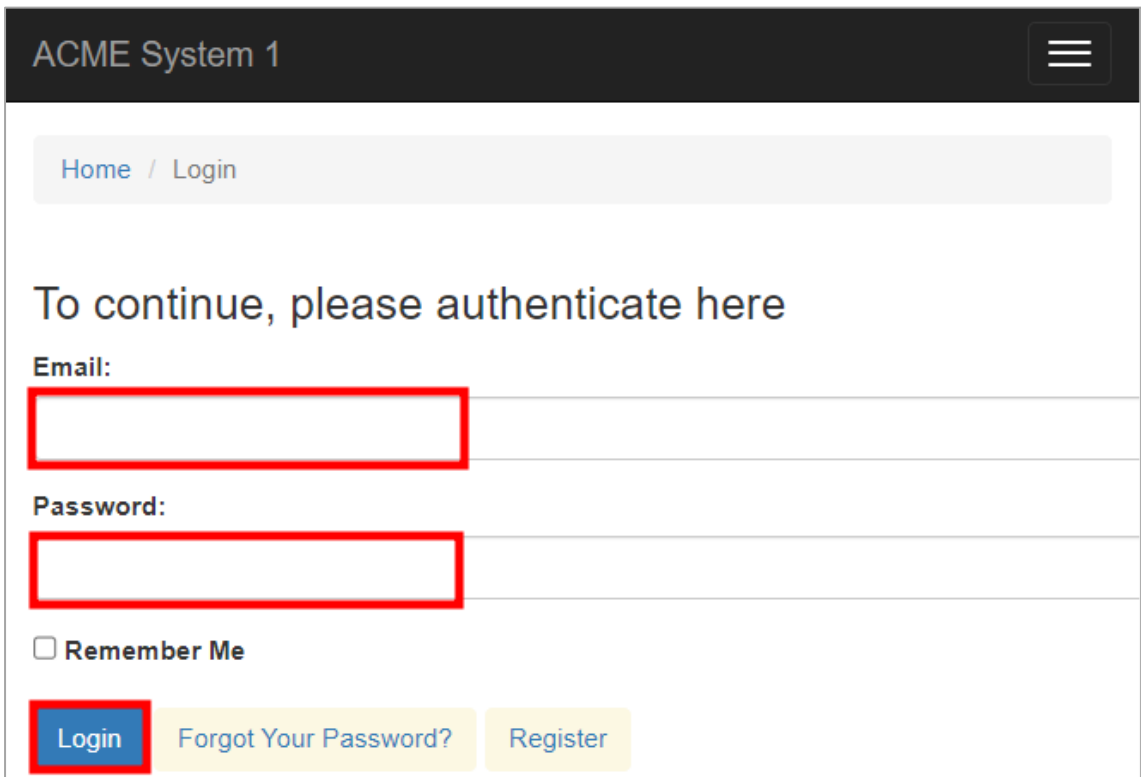
- ACME 사이트 - Work Items 데이터 전체를 읽고 Type 이 WI5 인 항목만 따로 추출해 각 항목의 Work Item Detail 에 접속하는 프로세스를 만들어 보시다.

Step0. Flowchart 드래그 앤 드롭

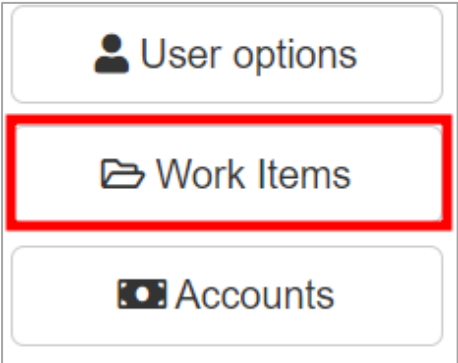


Step1. ACME 사이트 로그인

- Window 자격 증명을 활용해 로그인 진행



Step2. Work Items 메뉴 클릭 후 레코딩 종료



- Sequence 이름 지정



Step3. Work Items 전체 데이터 추출 후 테이블 추출 종료

ACME System 1HomeSwitch to new themeLog Out

Search Results

Please find below your work items. They need to be completed in the order specified by your manager.

Actions	WIID	Description	Type	Status	Date
	92439202	Process Vendor Invoice	WI3	Open	2018-12-08
	92439230	Research Client Check Copy	WI2	Open	2020-04-01
	92439201	Process Vendor Invoice	WI3	Open	2023-02-05
	92439181	Calculate Client Security Hash	WI5	Open	2017-05-31
	92439252	Verify Account Position	WI1	Open	2019-10-25
	92439238	Research Client Check Copy	WI2	Open	2020-11-19
	92439258	Verify Account Position	WI1	Open	2020-09-22
	92439187	Generate Yearly Report for Vendor	WI4	Open	2021-05-10
	92439179	Calculate Client Security Hash	WI5	Open	2021-01-17
	92439206	Process Vendor Invoice	WI3	Open	2019-06-19

1 2 3 4 5 6 7 8 9 10 >

- Sequence 이름 지정

필터 마법사

입력 데이터 테이블 {} ExtractDi L⁷ + 필터링된 데이터 테이블 {} ExtractDi L⁷ +

행 필터링 출력 열

일치하는 행 유지 또는 제거

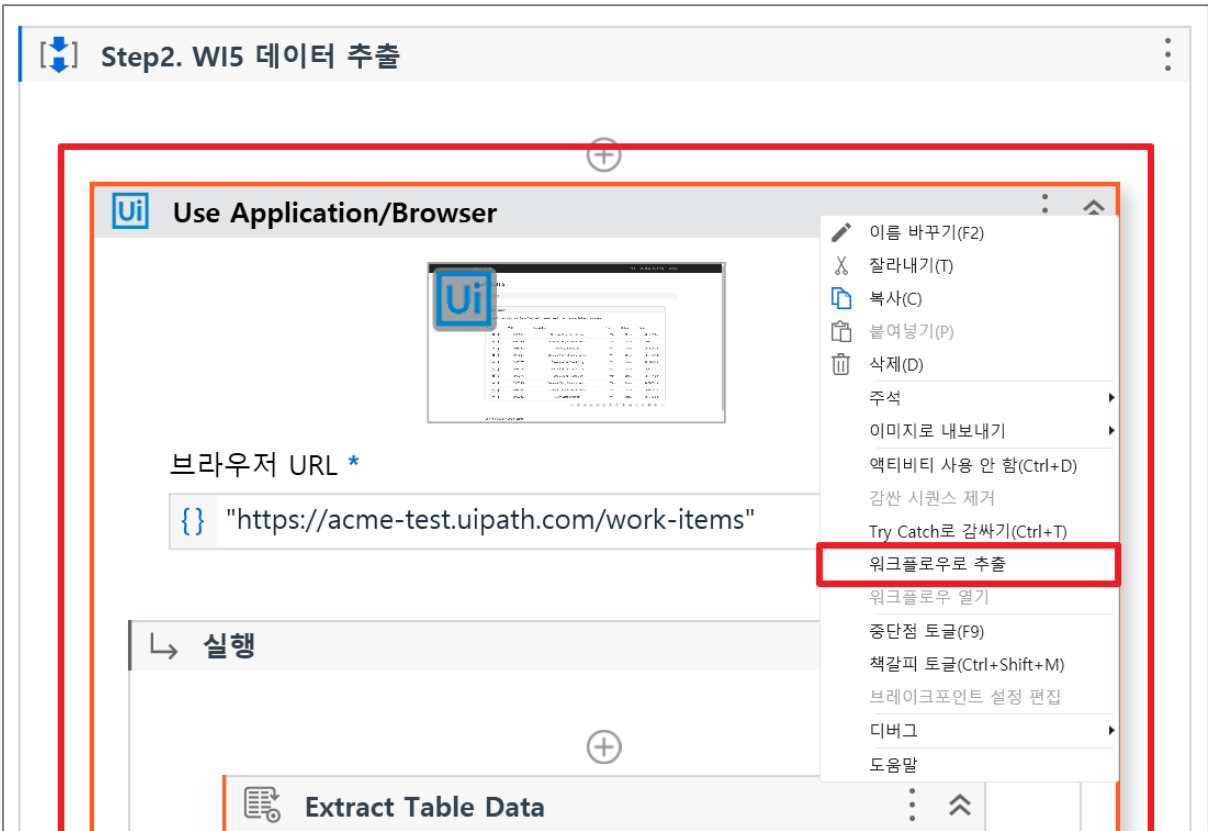
☒ 유지 ☐ 제거

열	동작	값
{ } "Type" L ⁷ +	=	{ } "W15" L ⁷ + × +

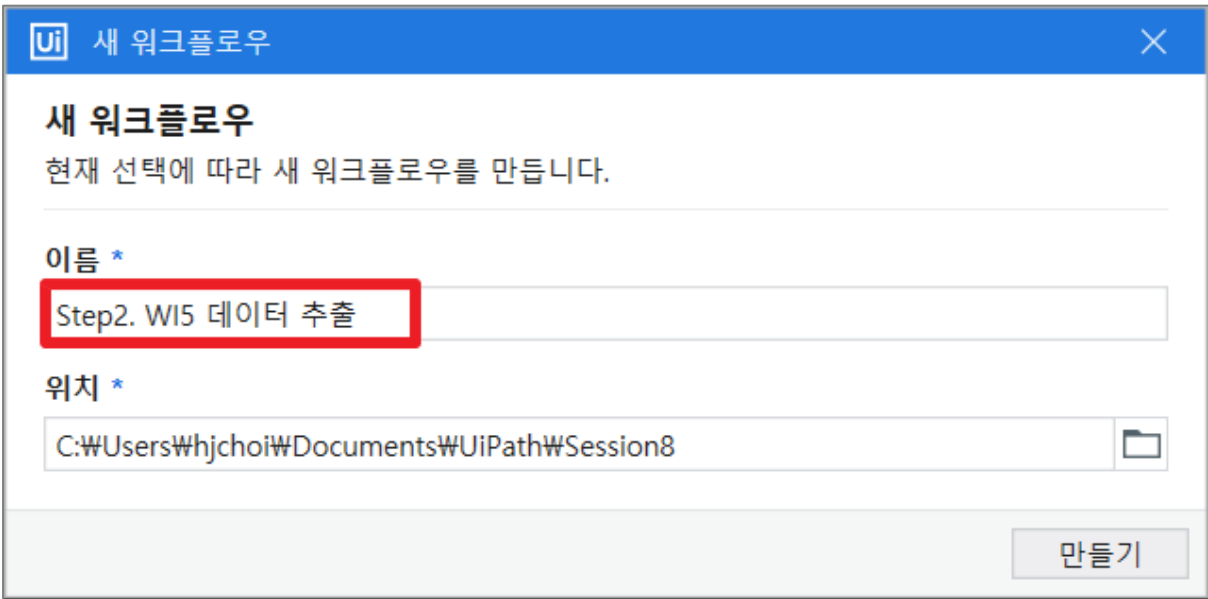
OK Cancel

Step5. 워크플로우로 추출

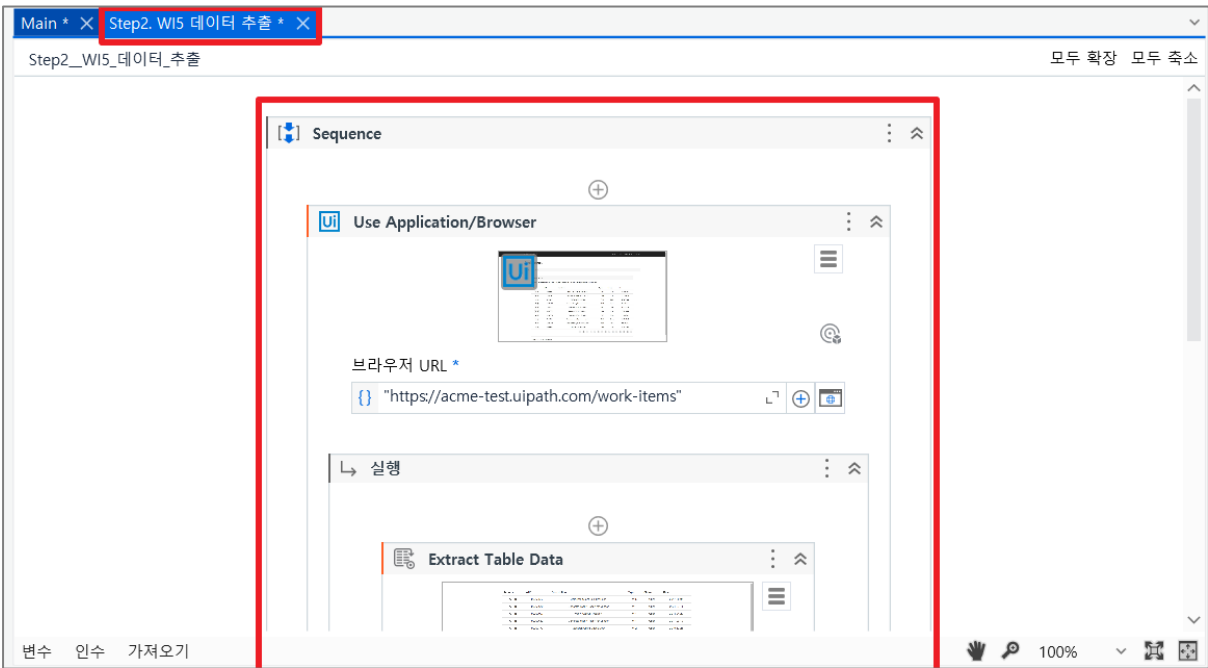
- Use Application/Browser 우클릭 - 워크플로우로 추출 클릭



- 새 워크플로우 이름 입력

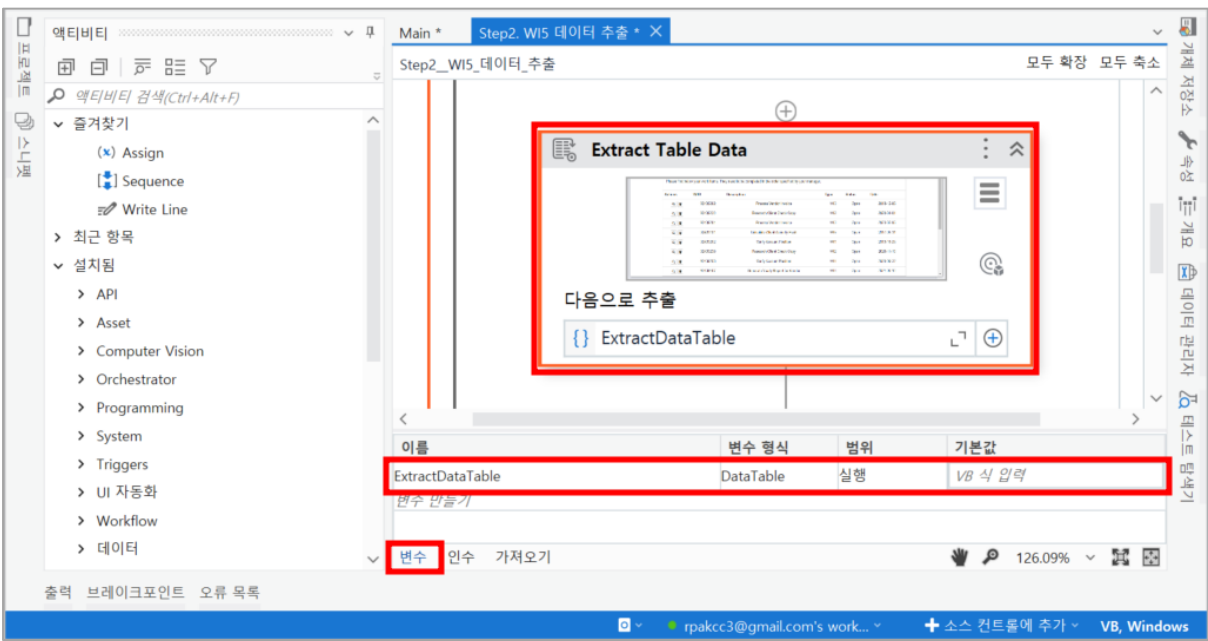


- 생성된 .xaml 파일 확인

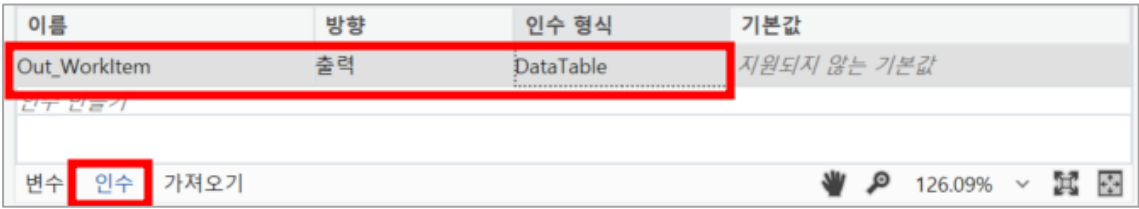


Step6. 인수에 데이터 저장하기

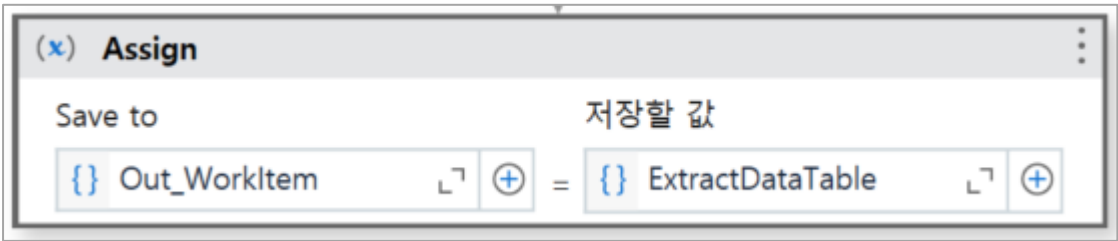
- Extract Table Data 변수 생성



- 기존에 있는 인수들은 모두 지우고 새로운 인수 생성



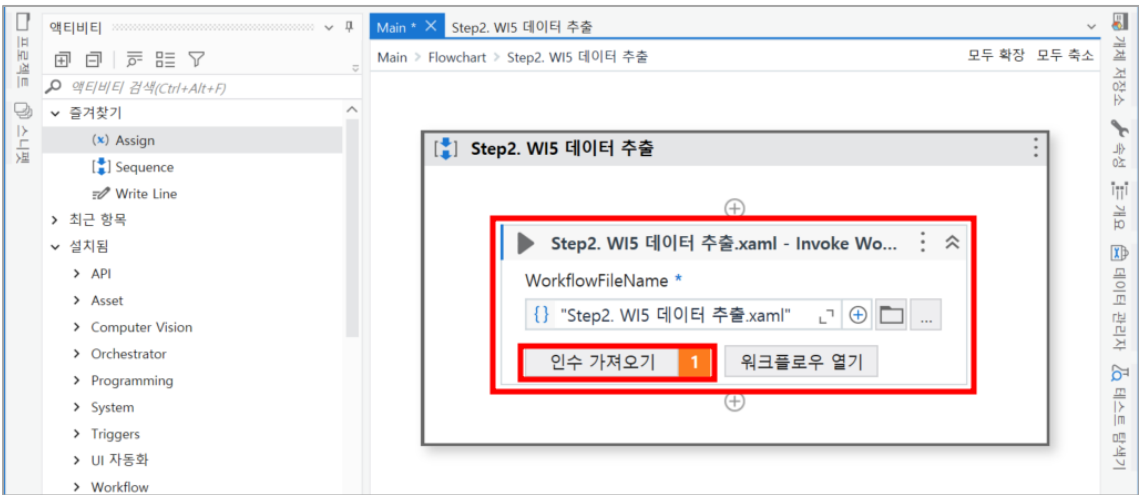
- Assign 액티비티 드래그 앤 드롭 후 인수 Out_WorkItem 에 변수 ExtractDataTable 저장



- 작업이 모두 끝났으면 프로세스 저장 후 Main.xaml 이동

Step7. 인수로 받아온 데이터를 변수에 저장

- Invoke Workflow File 액티비티 생성 확인 후 인수 가져오기 클릭



- 변수 생성(Ctrl+K) 후 입력한 다음, 변수의 범위 설정

호출된 워크플로우의 인수

Name	Direction	Type	Value
Out_WorkItem	출력	DataTable	변수 설정: WorkItem

Create Argument

OK Cancel

- 다음 프로세스에서 사용할 변수이기 때문에 범위를 Flowchart 까지 확장

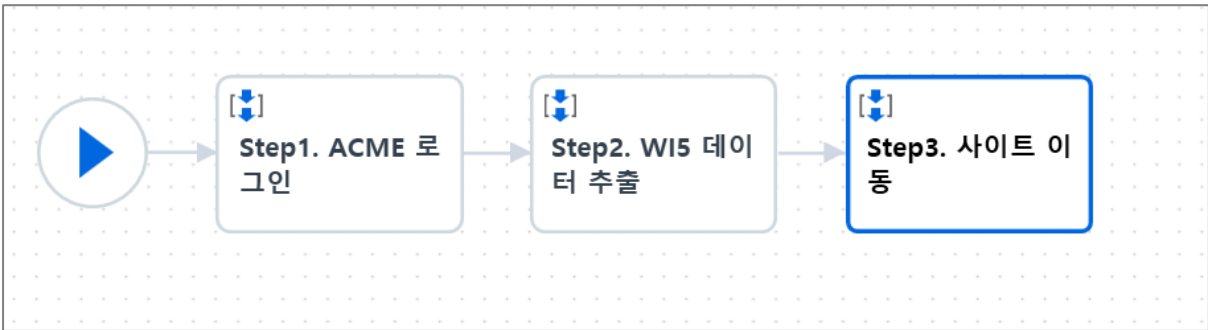
이름	변수 형식	범위	기본값
WorkItem	DataTable	Flowchart	VB 식 입력
		Step2. WIS 데이터 추출	
		Flowchart	

변수 만들기

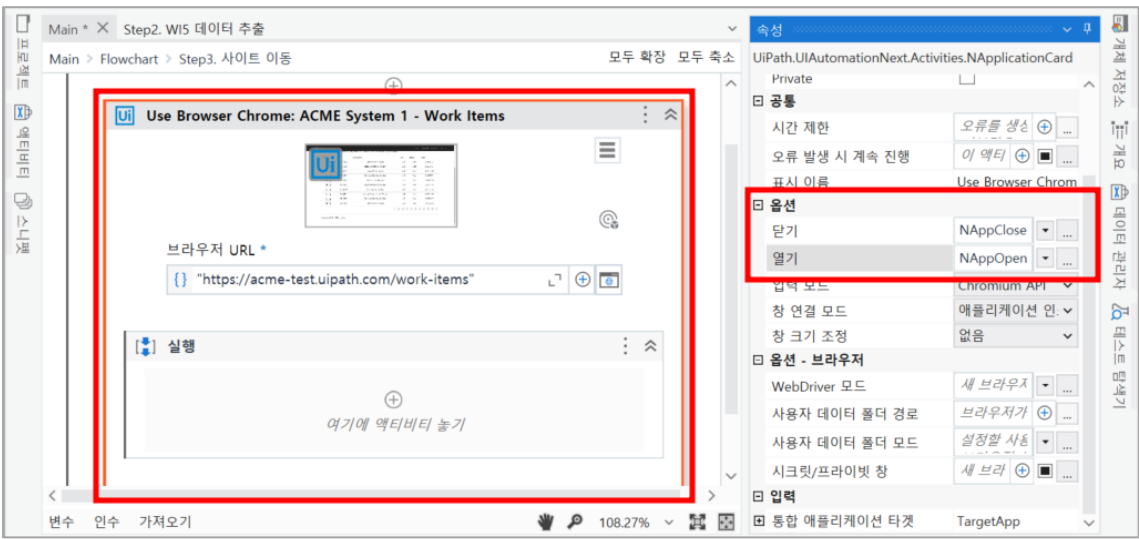
변수 인수 가져오기 126.09%

Step8. 사이트 이동 프로세스 생성

- Sequence 생성 후 이름 입력



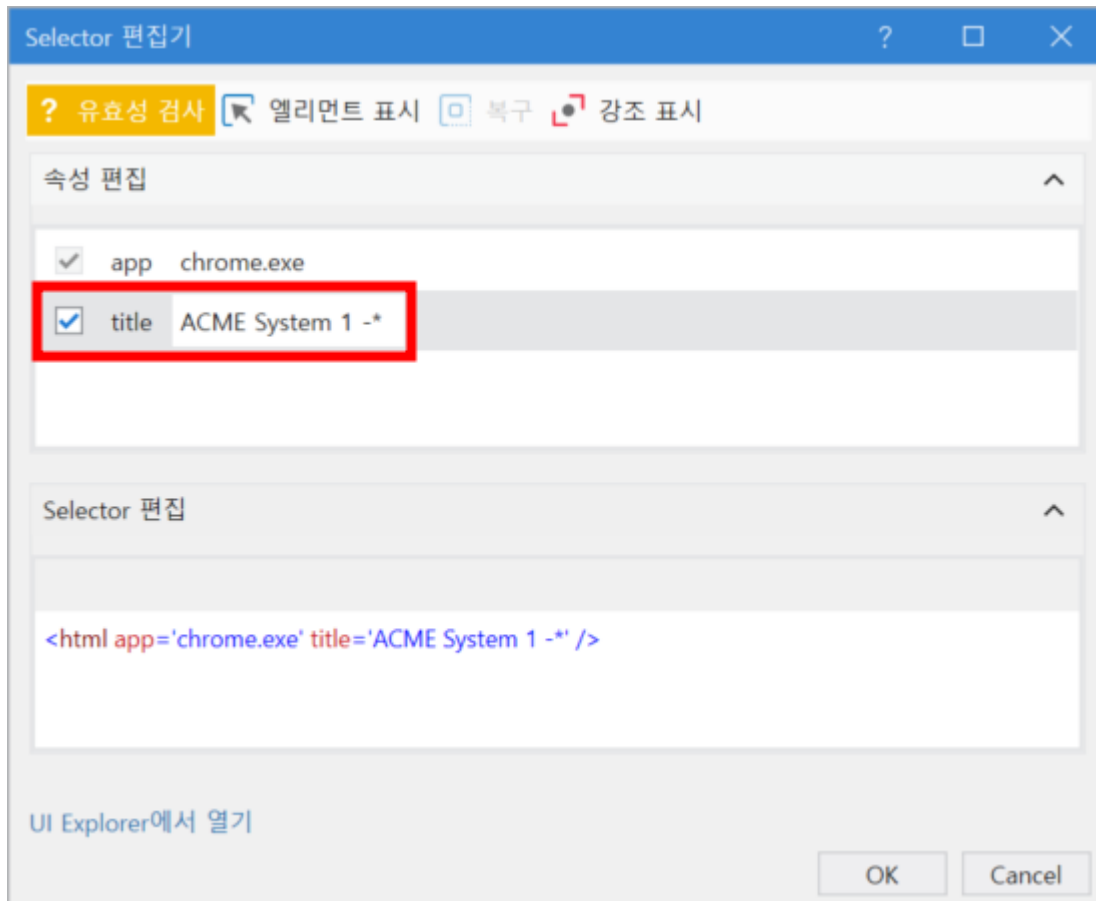
- Use Application/Browser 드래그 앤 드롭 후 ACME 사이트 클릭
- 닫기, 열기 옵션 Never 설정



- ACME 사이트 내 어디든 이동할 수 있도록 '통합 애플리케이션 타겟' 확장 후 선택기 클릭

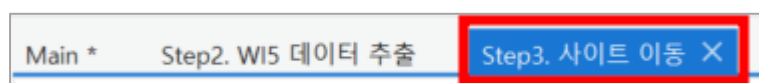
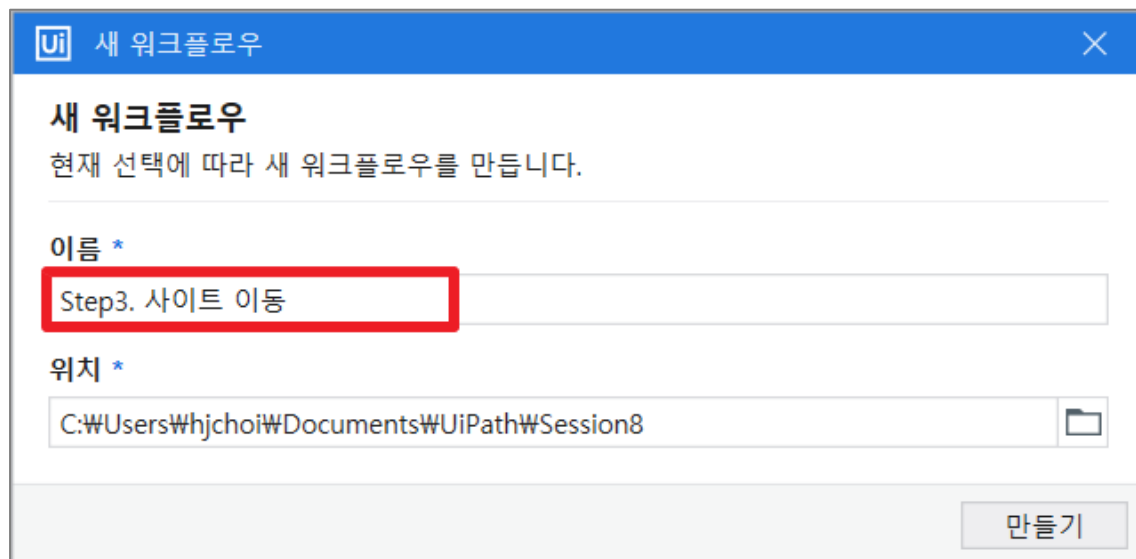


- 'Work Items'를 와일드카드(*)로 대체



Step9. 워크플로우로 추출 후 만들어진 파일 확인

- Use Application/Browser 우클릭 - 워크플로우로 추출 클릭



Step10. 인수 설정

- 이름: In_URL / 방향: 입력 / 인수 형식: String

이름	방향	인수 형식	기본값
in_URL	입력	String	VB 식 입력
인수 만들기			
변수 인수 가져오기			

- Go To URL 에 인수 입력 후 프로세스 저장

Go To URL

URL *

in_URL

Step11. 반복문 설정

- Main.xaml 에서 For Each Row in Data Table 액티비티 드래그 앤 드롭 후 Invoke 이동

For Each Row in Data Table

ForEach

CurrentRow

WorkItem

본문

Step3. 사이트 이동.xaml - Invoke Workflo...

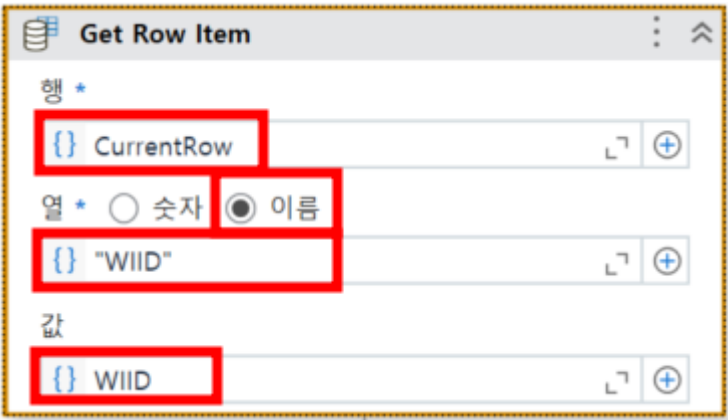
WorkflowFileName *

"Step3. 사이트 이동.xaml"

인수 가져오기 0 워크플로우 열기

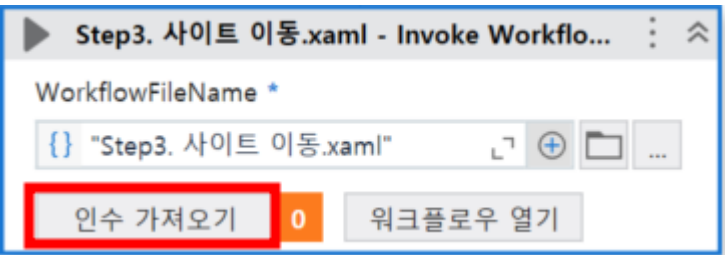
Step12. WIID 값 추출

- Get Row Item 액티비티 드래그 앤 드롭
- 행: CurrentRow / 열: 이름 - "WIID" / 값: WIID 변수 입력

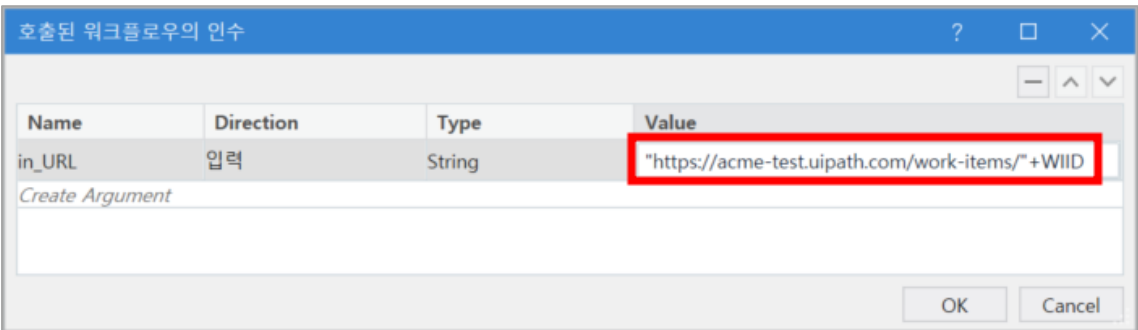


Step13. URL 입력

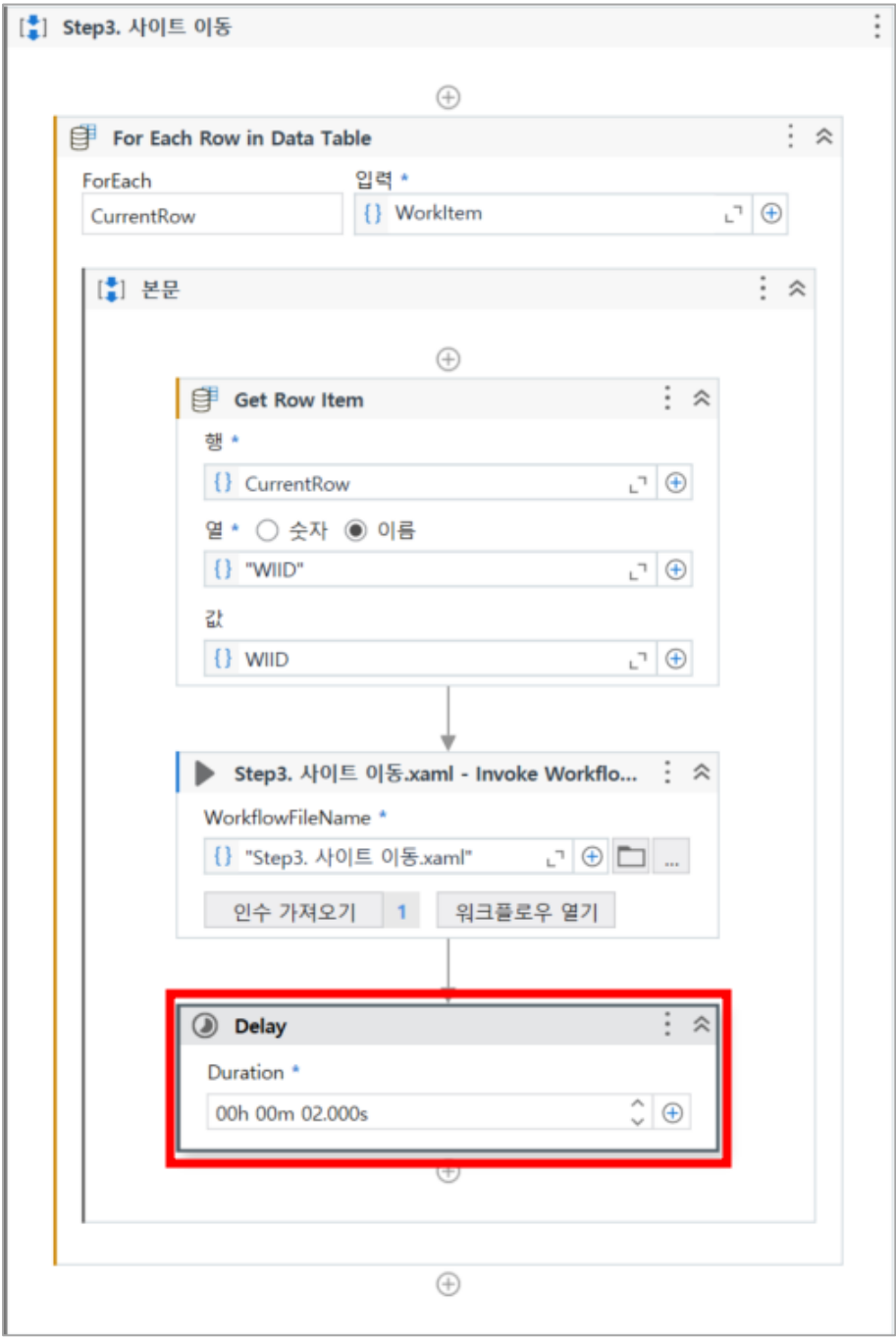
- Invoke Workflow File 인수 가져오기 클릭



- URL+변수 입력 후 OK 버튼 클릭



Step14. 사이트 간 이동을 확인하기 위해 Delay 액티비티 드래그 앤 드롭



Step15. 실행 후 결과 확인

3. 라이브러리(Library)

3.1. Library 정의

- 라이브러리는 사용자가 정의한 커스텀 액티비티들의 모음입니다.
- 다른 워크플로우에서 손쉽게 재사용이 가능하고, 코드의 유지보수를 용이하게 할 수 있습니다.

3.2. Library 활용

- ACME 사이트 - WI5 - Work Item Detail 에서 텍스트를 추출해 원하는 문자열만 분리해주는 문자열 처리 라이브러리를 만들어 보시다.

Calculate Client Security Hash

Client Information Details

Client ID: **AZ79829**

Client Name: **Marylou Kenworthy**

Client Country: **Italy**

Work Item Details

WIID: 92439181

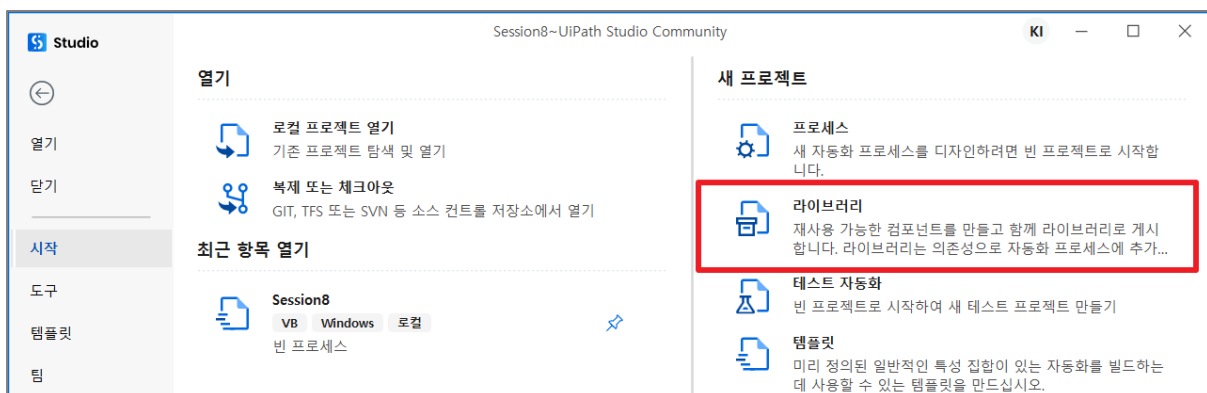
Type: Calculate Client Security Hash

Status: Open

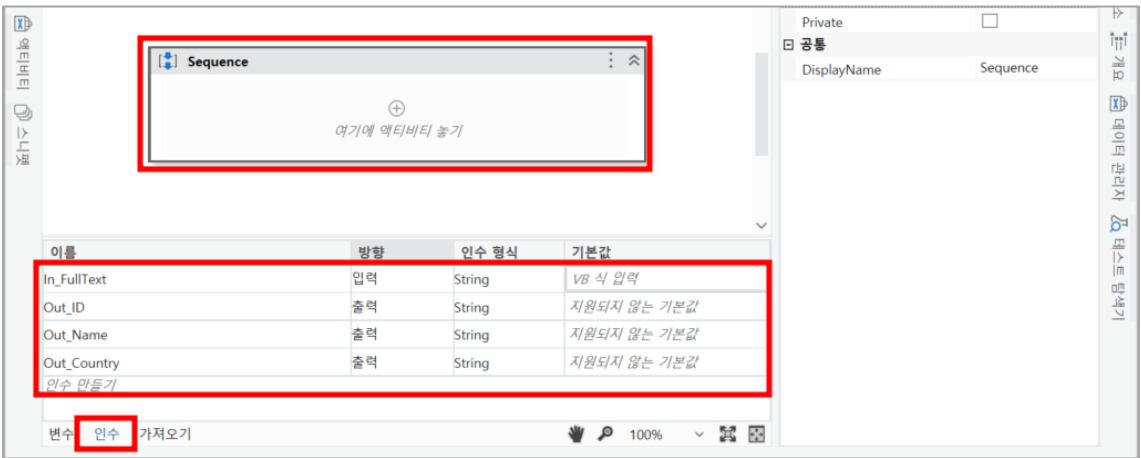
Date: 2017-05-31

Update Work Item

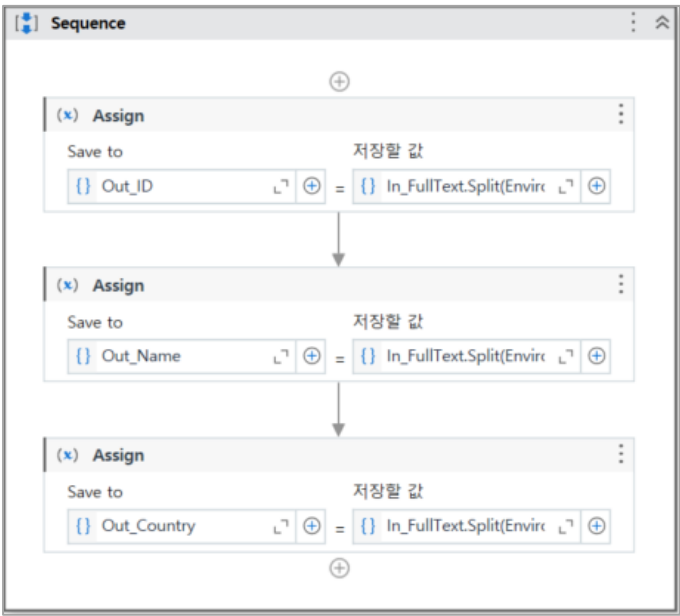
Step0. 새 라이브러리 생성



Step2. Sequence 와 인수 생성



Step3. 텍스트 분리 작업

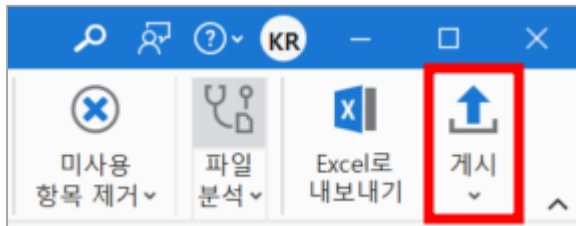


```
ID = In_FullText .Split(VBLf, StringSplitOptions.RemoveEmptyEntries)(0).Split(":\"c)(1).Trim()
Name = In_FullText .Split(VBLf, StringSplitOptions.RemoveEmptyEntries)(1).Split(":\"c)(1).Trim()
Country = In_FullText .Split(VBLf, StringSplitOptions.RemoveEmptyEntries)(2).Split(":\"c)(1).Trim()
```

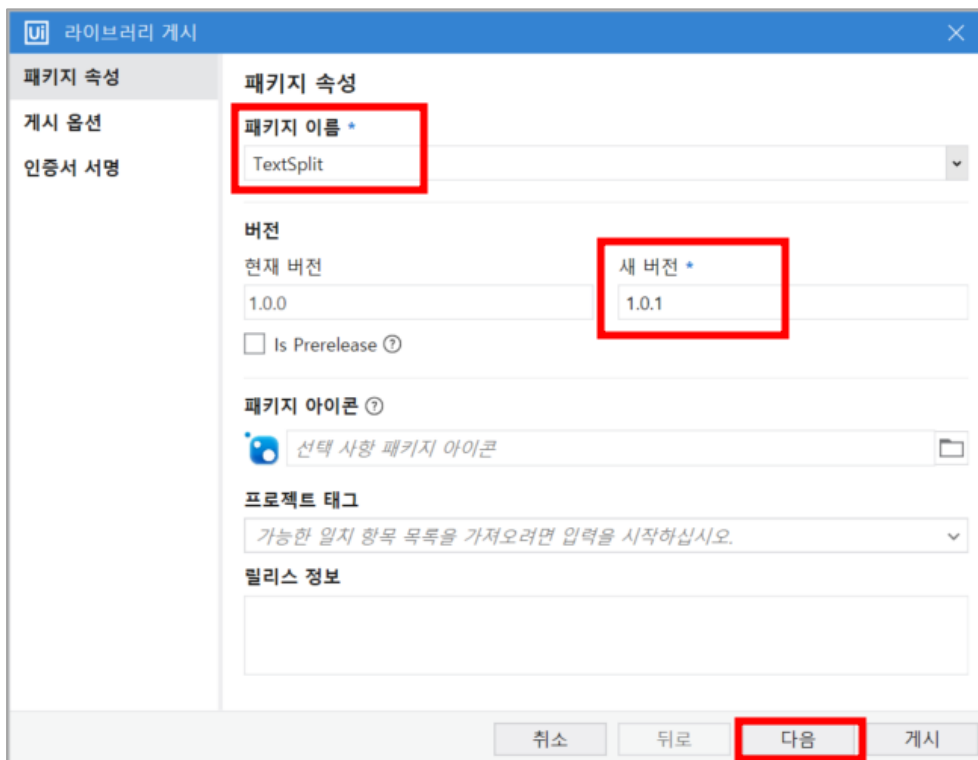
In_FullText	분할하려는 원본 문자
.Split	문자열을 주어진 구분자로 분할 하는 메서드
VBLf	시스템에서 사용하는 줄바꿈 문자
.StringSplitOptions.RemoveEmptyEntries	빈 문자열은 결과 배열에 포함되지 않음
.Trim	문자열 양 끝 공백을 제거해주는 문자열 메서드

Step4. 라이브러리 게시

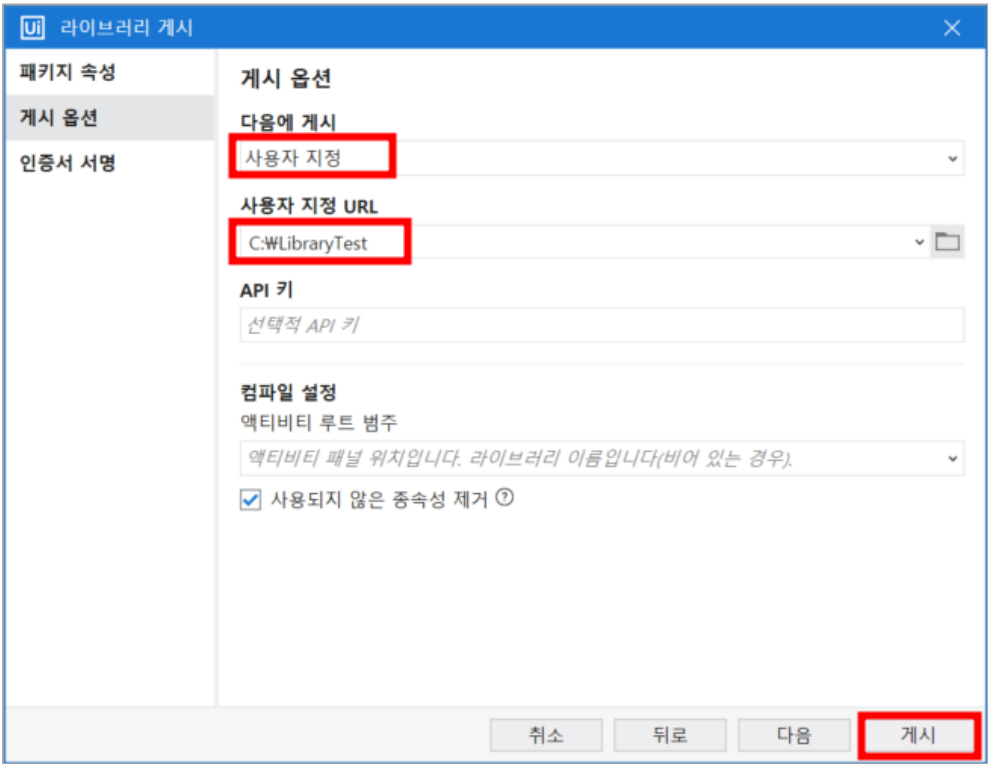
- 게시 버튼 클릭



- 패키지 이름 및 버전 확인 후 다음 버튼 클릭

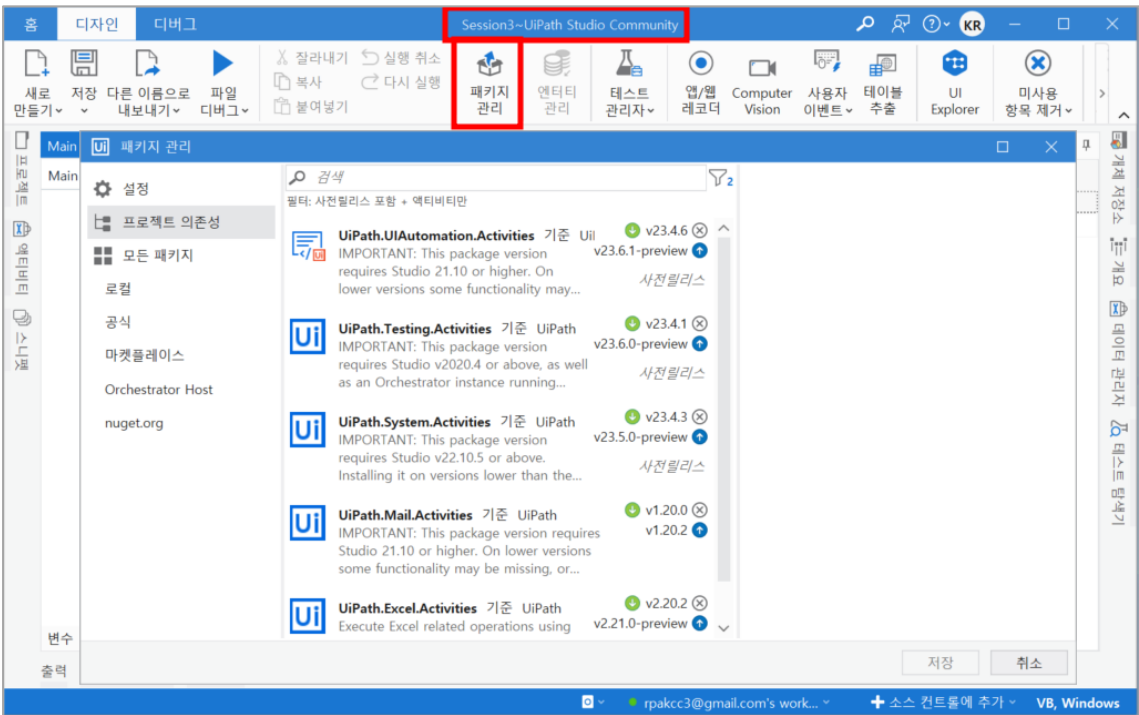


- 다음에 게시: 사용자 지정 선택 / 사용자 지정 URL: 자유 위치에 폴더 생성 후 게시

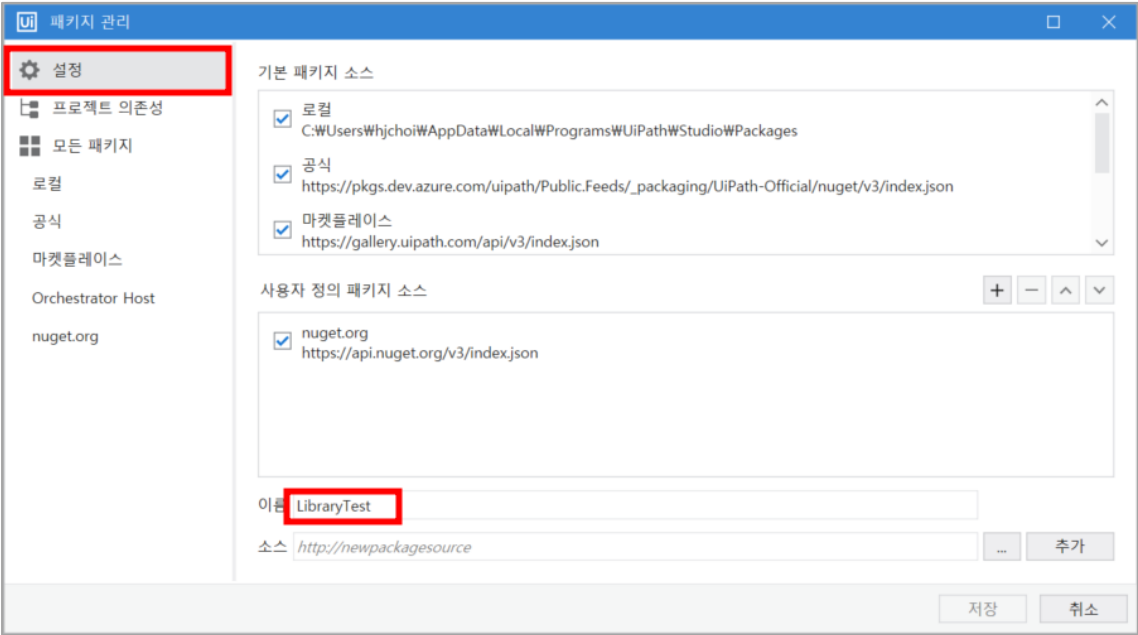


Step5. SESSION3' 프로세스 생성 후 라이브러리 설치

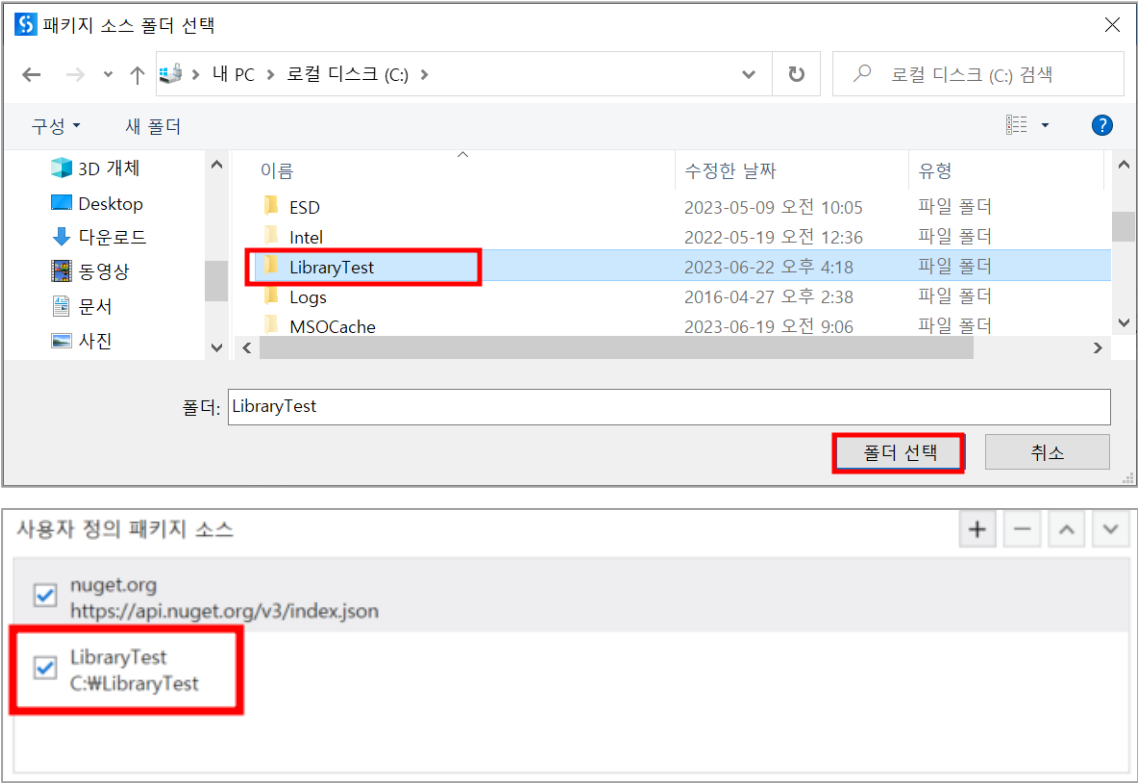
- 패키지 관리 클릭



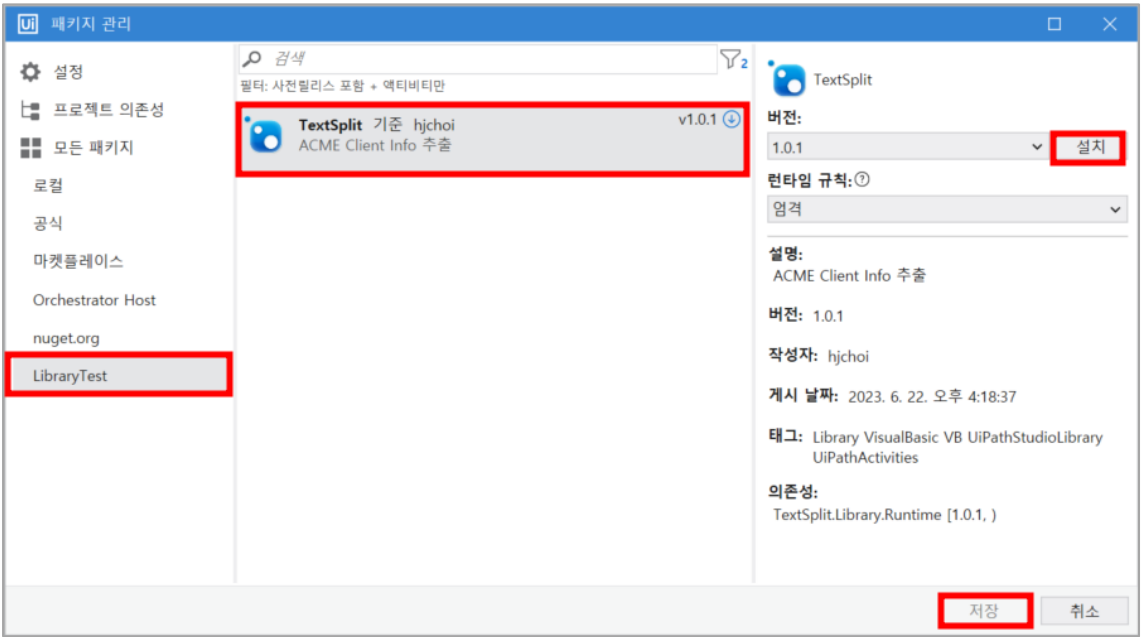
- 설정 클릭 후 추가할 패키지(라이브러리) 이름 추가



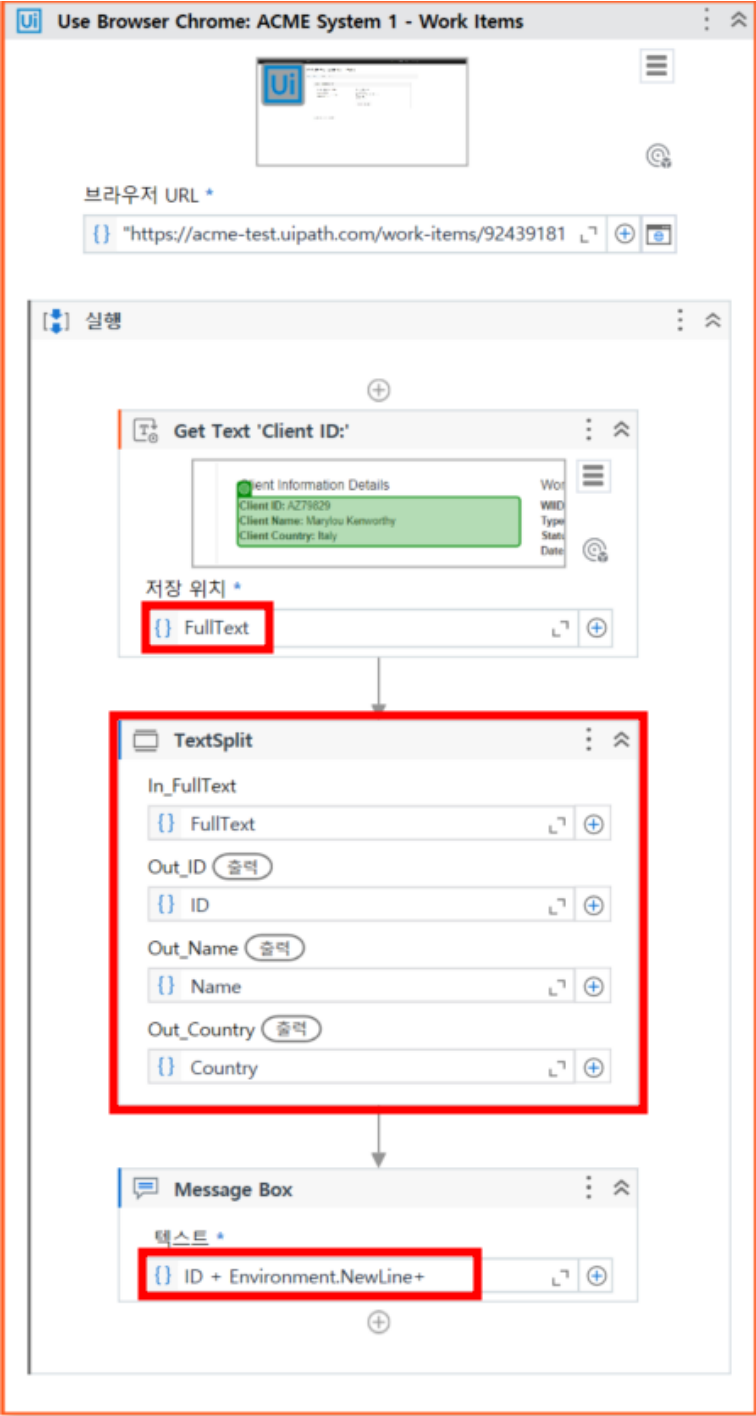
- '...' 버튼 클릭 후 LibraryTest 폴더 추가



- TextSplit 라이브러리 설치 후 저장



Step6. 텍스트 추출 프로세스 생성



- Step7. 실행 후 결과 확인

4. 스니펫(Snippet)

4.1. Snippet 정의

- 스니펫은 반복적으로 사용되는 워크플로우나 액티비티의 일부를 빠르게 재사용할 수 있는 코드 조각입니다.

4.2. Snippet 활용

- 로그인 프로세스를 스니펫에 담는 프로세스를 만들어 봅시다.

Step0. 스니펫 프로세스 생성

새 빈 프로세스

새 자동화 프로세스를 디자인하려면 빈 프로젝트로 시작합니다.

이름 * ACME 로그인

위치 * C:\Users\Whjchoi\Documents\UiPath

설명 빈 프로세스

호환성 ① Windows

언어 ☒ VB ☐ C#

만들기

Step1. ACME 로그인 프로세스 작성 후 저장

Home / Login

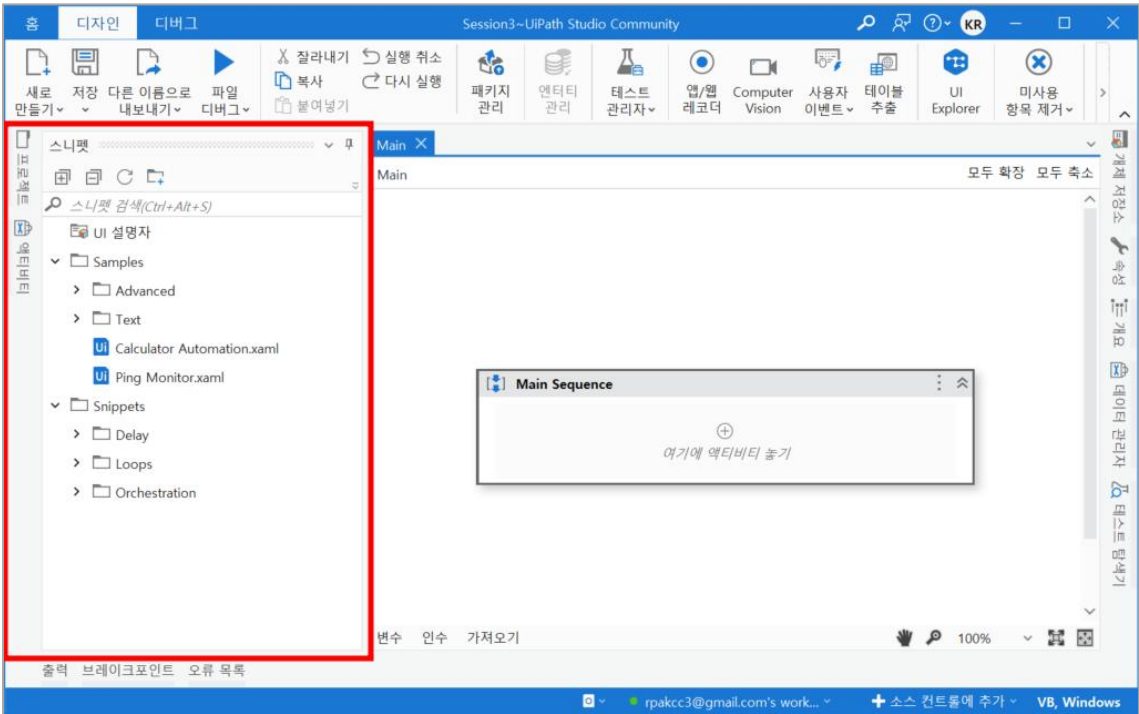
To continue, please authenticate here

Email:

Password:

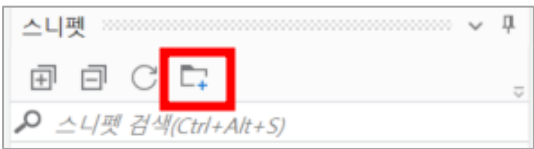
☐ Remember Me

Step2. SESSION3' 프로세스 열고 스니펫 패널 클릭

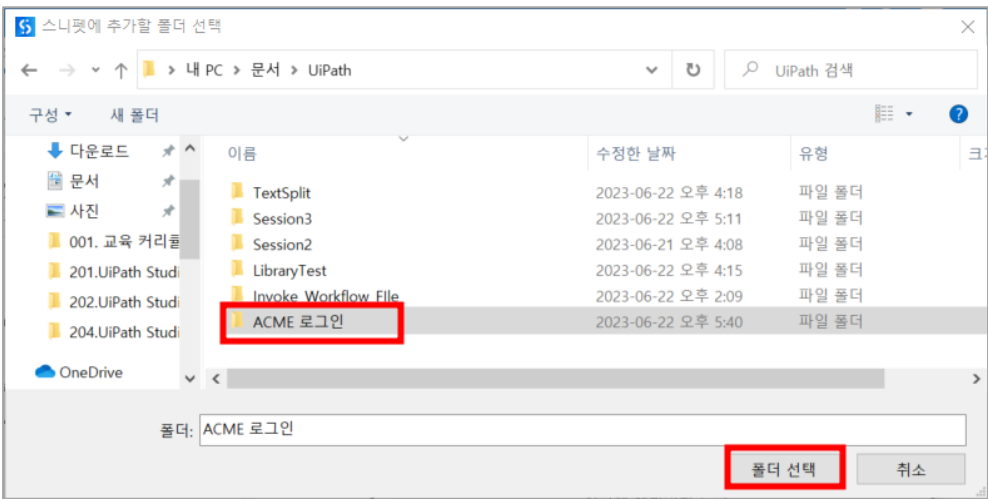


SESSION3. 스니펫 프로세스 등록

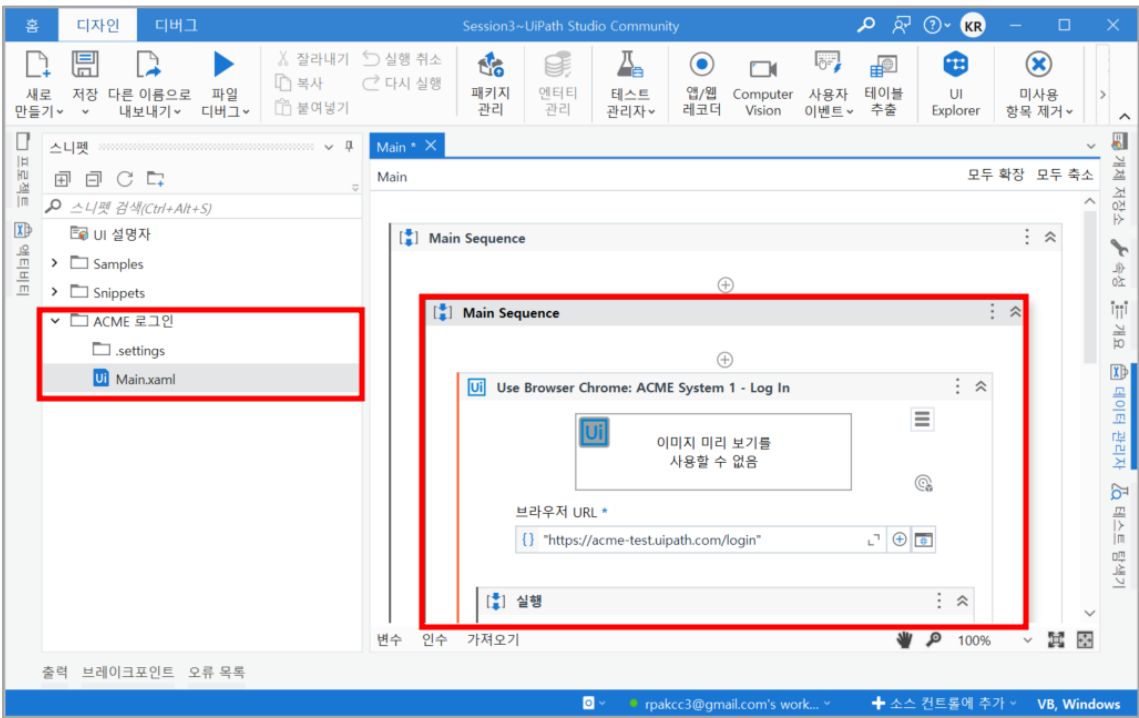
- 폴더 추가 클릭



- 폴더 위치 확인 후 폴더 선택 클릭



Step4. 스니펫 등록 확인 후 Main.xaml 드래그 앤 드롭

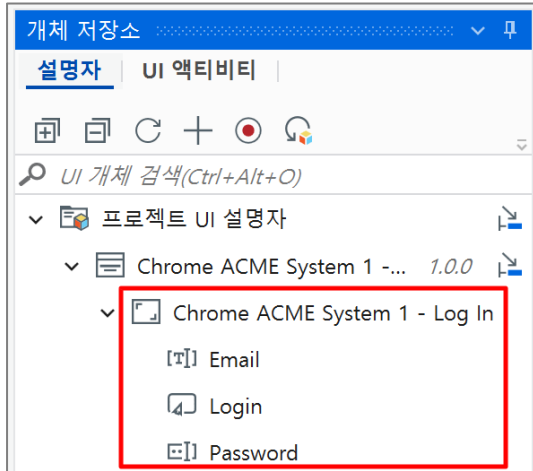


Step5. 실행 후 결과 확인

5. 개체 저장소(Object Repository)

5.1. Object Repository 정의

- UiPath Studio 에서 자주 사용되는 UI 요소들을 중앙 집중식으로 저장하고 관리할 수 있는 기능입니다.



5.2. Object Repository 설정 및 팝업 설명

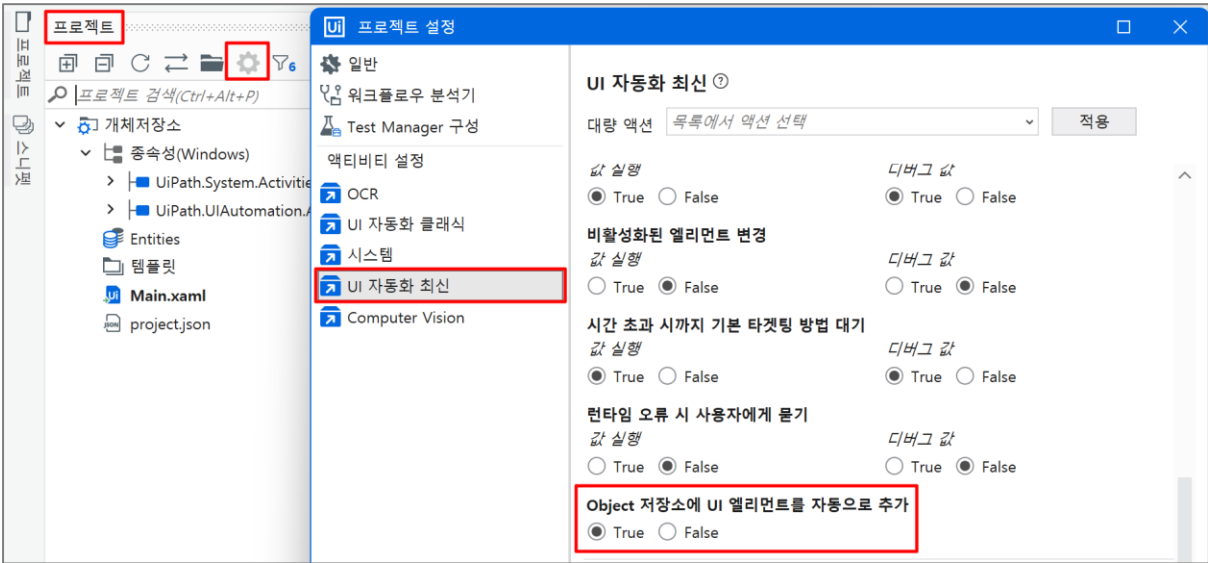
5.2.1. 활용 시나리오

- 기존 RPA 프로세스는 각 액티비티에서 직접 UI 요소에 접근했기 때문에, 웹사이트 고도화나 UI 변경 시 즉각 대응이 어려웠습니다. 반면 개체 저장소를 사용하면 모든 UI 요소를 중앙에서 관리하므로, UI 변경 시 해당 요소만 수정하면 이를 참조하는 모든 프로세스가 자동으로 업데이트되어 유지보수가 용이합니다.

5.2.2. 개체 저장소에 UI 자동 저장 옵션 설정/해제

- UiPath Studio 를 설치했을 때, 기본적으로 UI 를 인식했을 경우, 개체 저장소에 저장하도록 설정이 되어 있습니다. 하지만 해당 기능을 사용하지 않는 경우, 개체 저장소 관련 팝업이 거슬리는 경우도 있습니다. 이에 개체 저장소 자동 저장 설정을 해제할 수 있습니다.

Step1. 프로젝트 설정 > UI 자동화 최신 > Object 저장소에 UI 엘리먼트를 자동으로 추가 False 로 변경



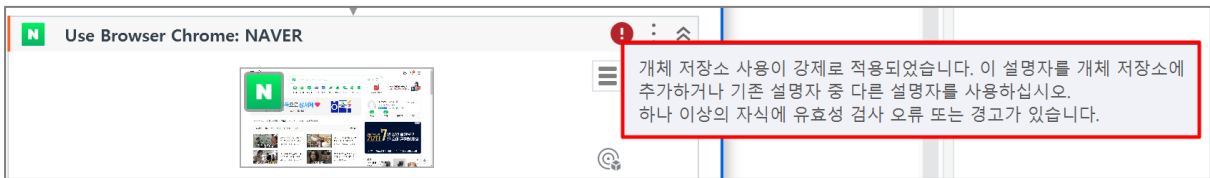
5.2.3. 개체 저장소에 있는 UI 요소들로만 개발하는 옵션 설정/해제

- 개체 저장소 적용 옵션을 활성화하면 프로젝트 내의 모든 UI 자동화 액티비티는 반드시 개체 저장소에 등록된 엘리먼트를 참조해야 하므로, 무분별한 UI 요소 선택기 사용을 방지하고 체계적인 UI 요소 관리가 가능합니다.

Step1. Studio 설정 – 디자인 – 개체 저장소 적용 활성화



Step2. 만약, 현재 프로세스에 개체 저장소에 등록되지 않은 UI 요소가 있는 상태에서 ‘개체 저장소 적용’ 옵션이 활성화 된 경우

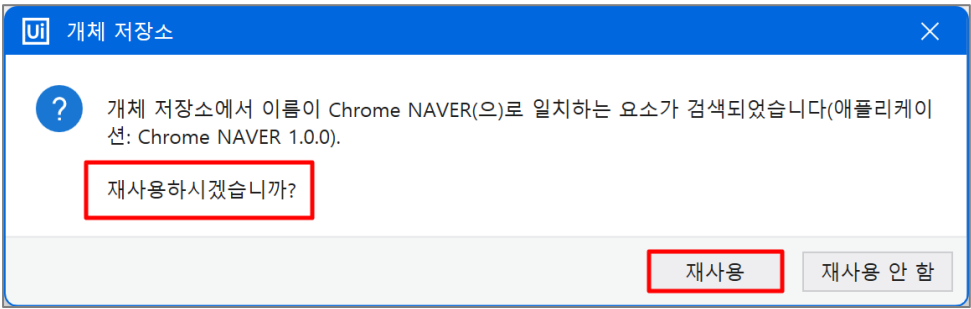


Step3. 등록이 안된 UI 요소를 개체 저장소에 빠르게 등록할 경우, 액티비티 내 클릭

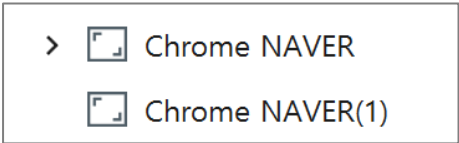


5.2.4. 이미 개체 저장소에 UI 요소가 등록되어 있는 경우

Step1. 재사용 클릭할 경우, 개체 저장소에 추가 등록 X

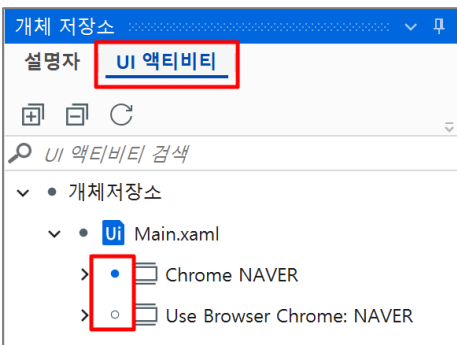


Step2. 재사용 안 함 클릭할 경우, 개체 저장소에 추가 등록 O

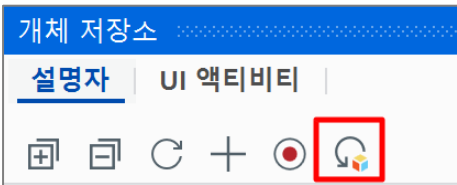


5.2.5. 개체 저장소에 UI 요소 동기화 상태 확인

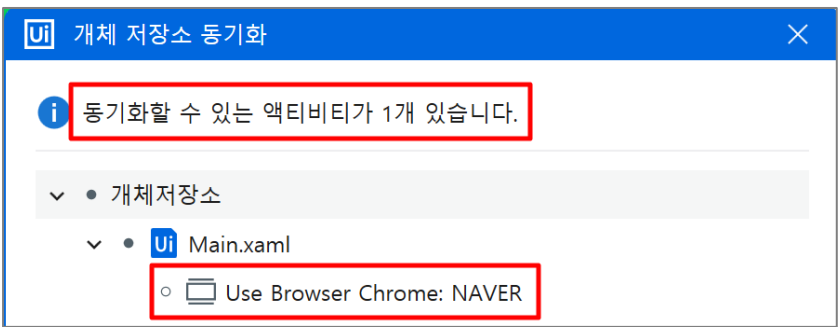
Step1. 개체 저장소 패널 > UI 액티비티 메뉴에서 확인 가능



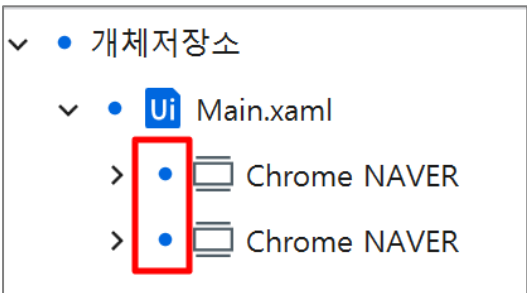
Step2. 설명자 메뉴에서 개체 저장소 동기화 버튼 클릭



Step3. 동기화가 안 된 액티비티 확인 후, 시작 버튼 클릭

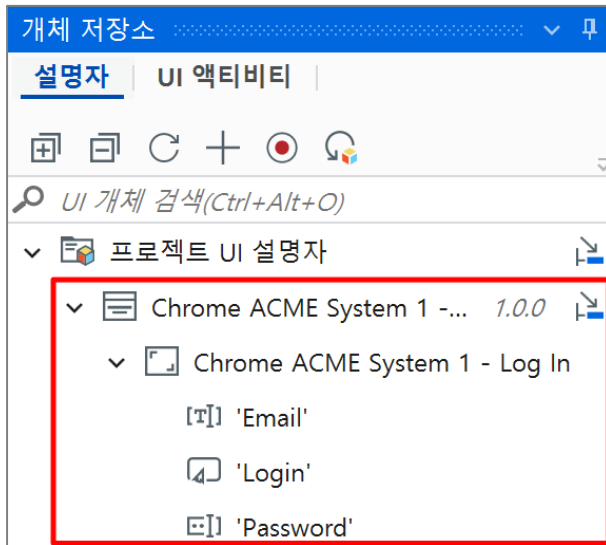


Step4. 개체 저장소 동기화 결과 확인

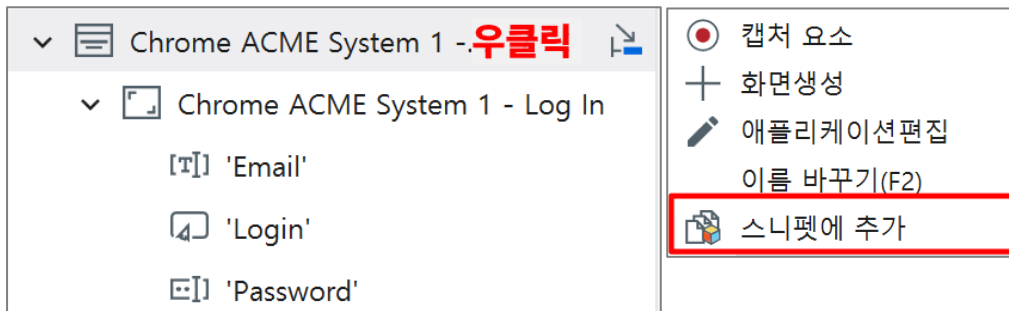


5.2.6. 개체 저장소 요소를 스니펫과 라이브러리로 변환

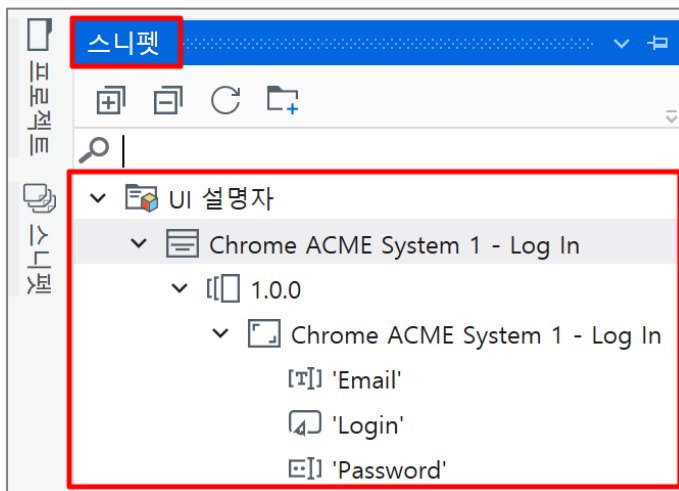
Step1. ACME 사이트 로그인 프로세스를 개체 저장소에 등록



Step2. 애플리케이션, 화면, 엘리먼트 단위로 스니펫에 등록 가능



Step3. 스니펫에 등록 확인



Step4. 애플리케이션에서 버튼 클릭

- 이름 및 설명, 위치 설정 후 만들기 버튼 클릭

Ui

새 UI 라이브러리

✕

새 UI 라이브러리

애플리케이션 버전, 화면, UI 엘리먼트 및 설명자와 같은 UI 개체를 포함하는 새 라이브러리를 만듭니다.

이름 *

ChromeACMESystem1Login.Descriptors

설명

Chrome ACME System 1 - Log In UI 라이브러리

고급 옵션 숨기기

⌵

위치 *

D:\W100. UiPath

📁

호환성 ?

언어

Windows

VB

만들기

Step5. 라이브러리 게시하기


게시
 ▼

 라이브러리 게시
✕

패키지 속성
 게시 옵션
 인증서 서명

게시 옵션

다음에 게시

사용자 지정 ▼

사용자 지정 URL *

D:\100. UiPath ▼ 

API 키

선택적 API 키

컴파일 설정

액티비티 루트 범주

액티비티 패 널 위치입니다. 라이브러리 이름입니다(비어 있는 경우). ▼

☒ 사용되지 않은 종속성 제거 ?

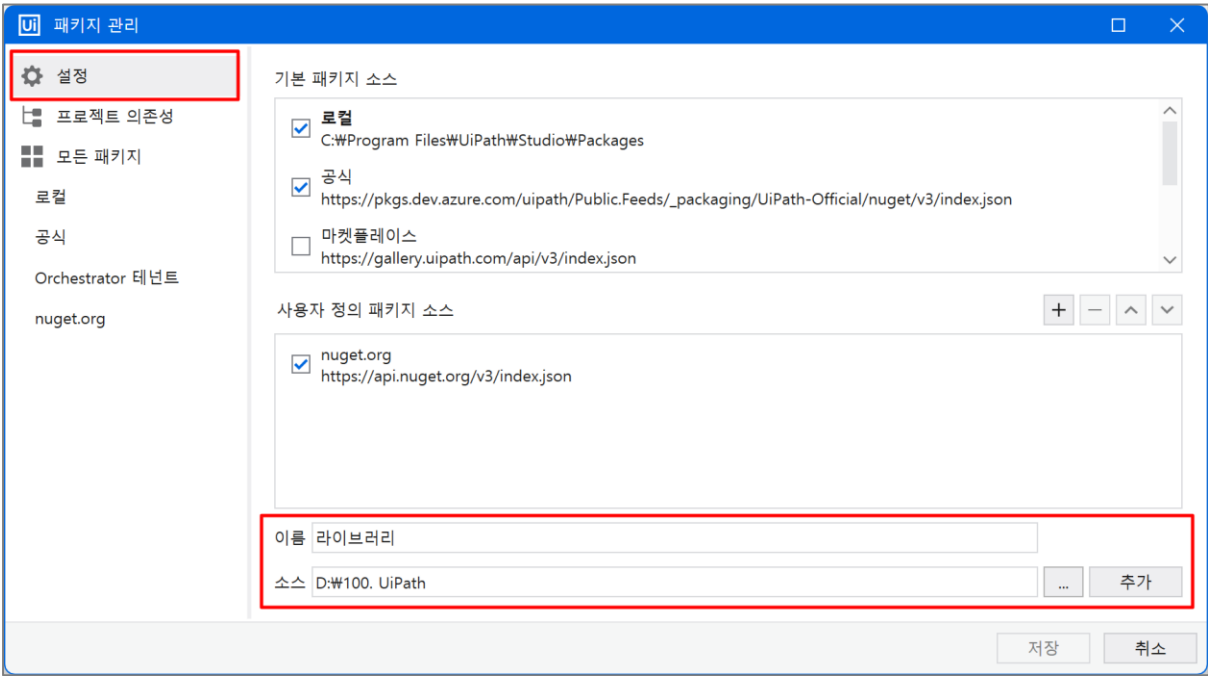
취소

뒤로

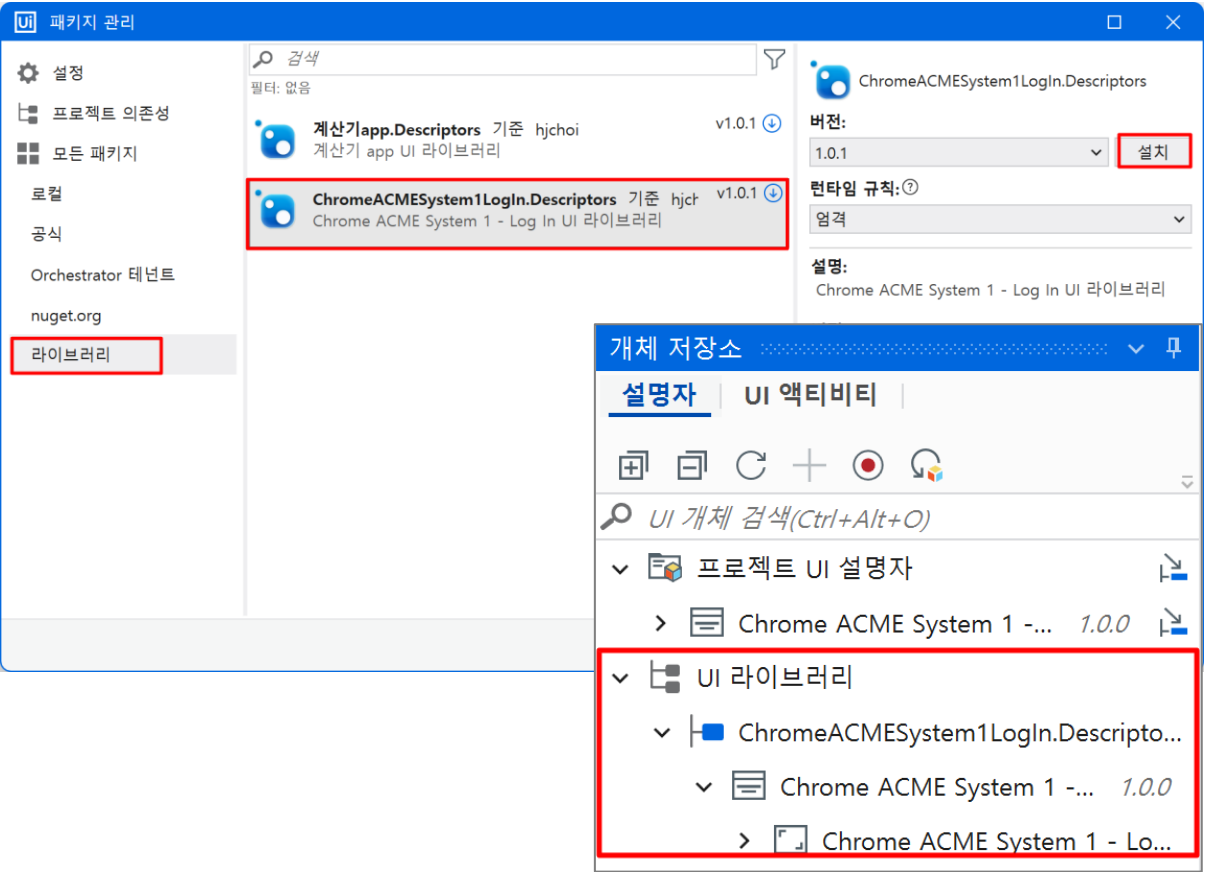
다음

게시

Step6. 개체저장소 프로젝트 이동 후, '패키지 관리' 설정에서 라이브러리 폴더 추가



Step7. 게시된 라이브러리 설치 후, UI 라이브러리 확인



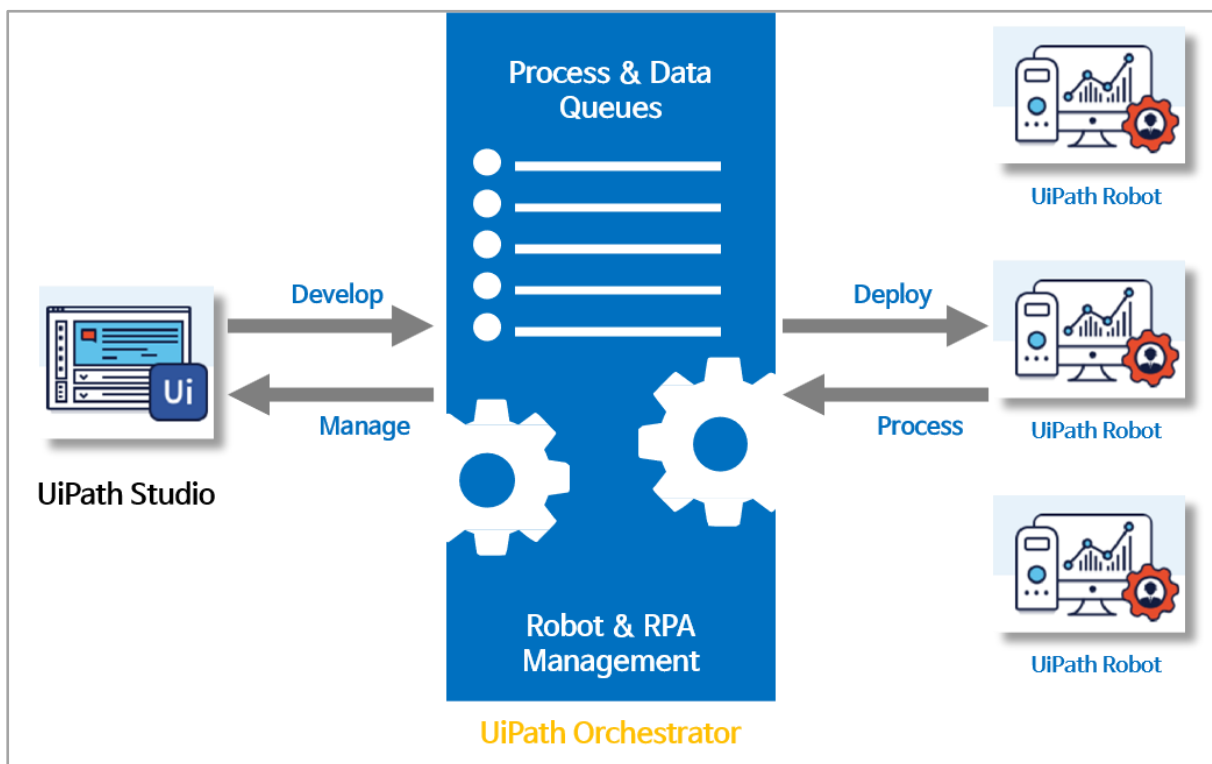
SESSION4. 오케스트레이터(ORCHESTRATOR)

학습 목표

- 오케스트레이터의 기능에 대해 이해할 수 있다.
- 오케스트레이터와 로봇을 연결할 수 있다.
- Asset 과 Queue 를 이용해 개발할 수 있다.

1. 오케스트레이터 정의

- UiPath 에서 개발한 프로세스와 로봇들을 모니터링하고 관리해주는 중앙 관리 대시보드입니다. 오케스트레이터 내에서 Processes, Assets, Queues, Triggers, Robots, Jobs, Transactions 등 오케스트레이터가 관제하는 현황에 대해 시각화하여 보여줄 수 있습니다.



2. Orchestrator 특징

2.1. 테넌트 기능

기능	기본 설명	업데이트된 주요 기능
Provisioning	로봇과 오케스트레이터 간의 연결을 설정 및 유지	• Modern Robot 자동 라이선스 관리 • 클라우드 네이티브 지원 • 자동 프로비저닝 기능
Deployment	로봇에 올바른 프로세스가 배포되었는지 확인	• Modern Folders 기반 배포 관리 • 자동화된 버전 관리 • 패키지 종속성 자동 확인
Configuration	로봇 환경과 프로세스 구성의 유지보수 및 전달 관리	• Active Directory 통합 강화 • 세분화된 권한 구성 • 환경별 설정 자동화
Queue	로봇 간에 작업량(작업 부하)을 자동 분산 관리	• 고급 우선순위 관리 • 실시간 부하 분산 • 트랜잭션 상세 추적
Monitoring	사용 권한 유지 / 관리 및 로봇의 작업 현황 모니터링	• Insights 실시간 대시보드 • 고급 분석 기능 • 커스텀 모니터링 뷰
Logging	SQL DB 로그값 저장 기능 및 Elastic Search DB 인덱싱	• Kibana 통합 • 고급 로그 분석 • 실시간 로그 모니터링
Inter-Connectivity	외부 솔루션 및 응용프로그램과의 커뮤니케이션	• 확장된 REST API • Webhook 지원 • 실시간 이벤트 처리

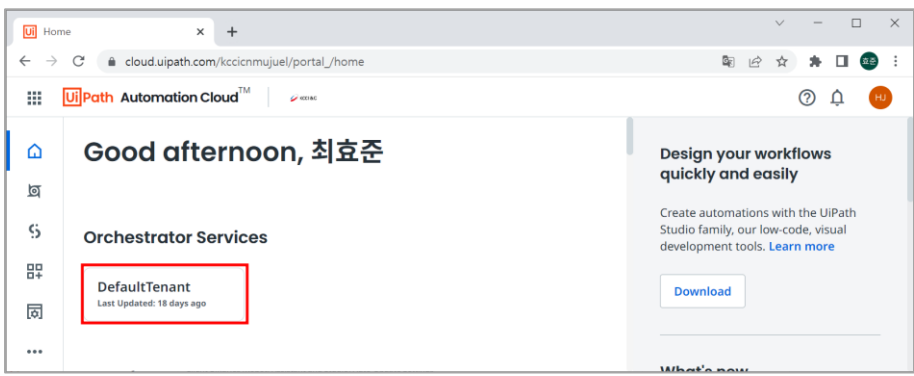
2.2. Robot Types

로봇 종류	설명	특징
Attended Robot	사용자의 개입과 함께 작동하며, 데스크톱에서 실행되는 작업을 보조하는 로봇.	- 사용자가 직접 실행 - Orchestrator를 통해 모니터링 가능 - 사용자 워크스테이션에 설치
Unattended Robot	사용자의 개입 없이 백그라운드에서 작업을 수행하는 로봇. 주로 대규모 자동화 및 서버 환경에서 사용됨.	- Orchestrator를 통해 트리거 및 스케줄링 가능 - 다중 프로세스 실행 가능 - 24/7 운영 가능
Testing Robot	테스트 자동화에 특화된 로봇으로, 테스트 케이스 실행에 사용됨.	- UiPath Test Suite와 통합 - 테스트 실행 및 보고 기능 지원 - 자동화된 테스트 수행

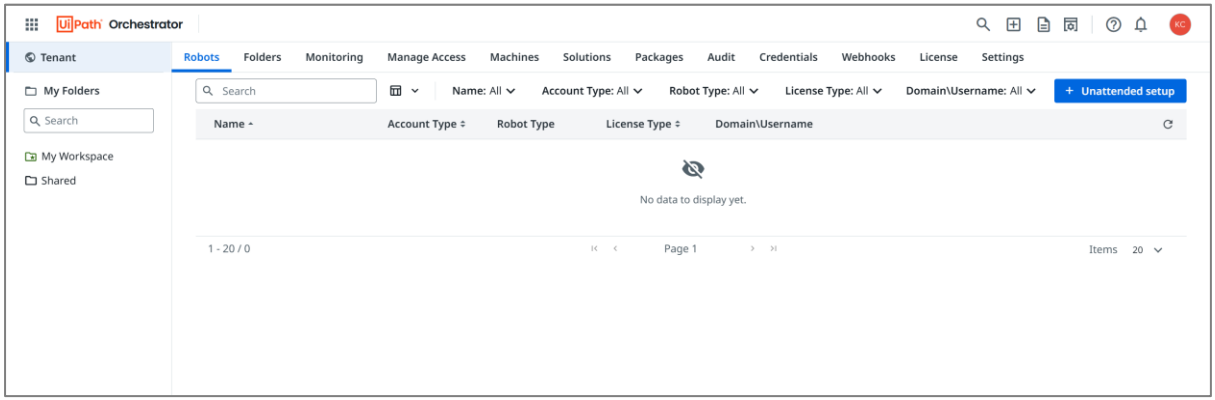
3. Orchestrator 기본 설정

3.1. Orchestrator 접근

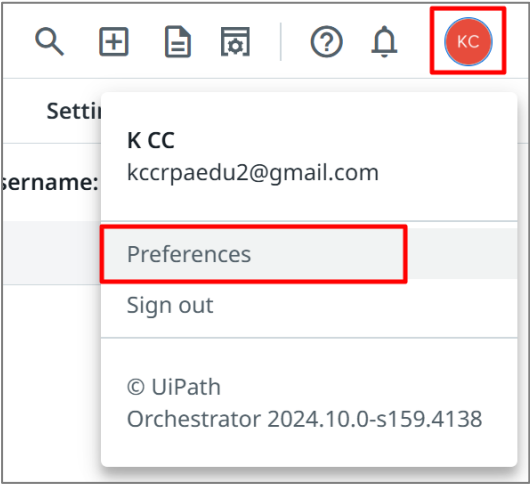
– URL: <https://cloud.uipath.com/> – DefaultTenant 클릭



– Orchestrator 첫 화면



– 언어 한글로 변경



3.2. Orchestrator Tenant 설정

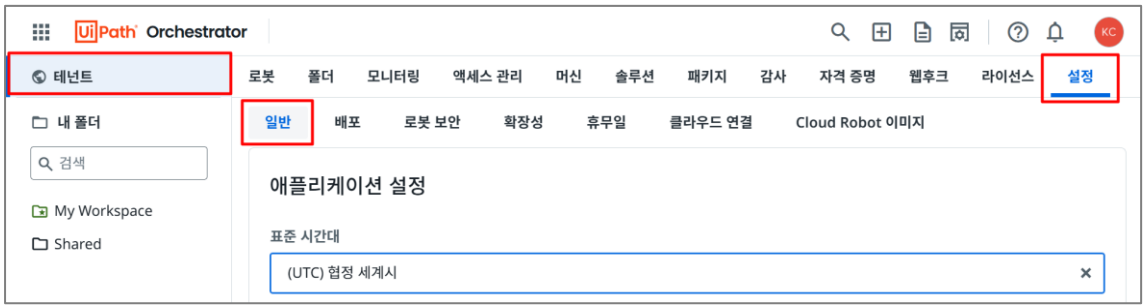


일반	• 시간대, 개인작업영역 활용 여부 등 테넌트의 기본 환경 설정
배포	• 프로세스와 패키지와 라이브러리 배포 관리
로봇 보안	• 로봇 접근 권한 및 보안 정책 설정
확장성	• 테넌트의 확장 기능 설정
휴무일	• 로봇 운영 일정 관리 • 공휴일 및 운영 중지 일정 설정
클라우드 연결	• 클라우드 서비스 연동 설정
Cloud Robot 이미지	• 클라우드 로봇용 이미지 관리

3.2.1. 일반(General)

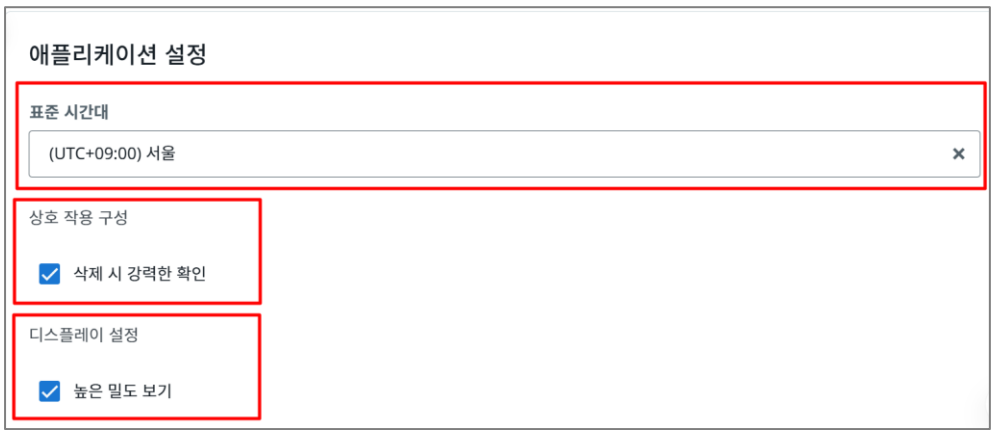
Step1. 일반 메뉴 이동

- 테넌트 - 설정 - 일반 접근

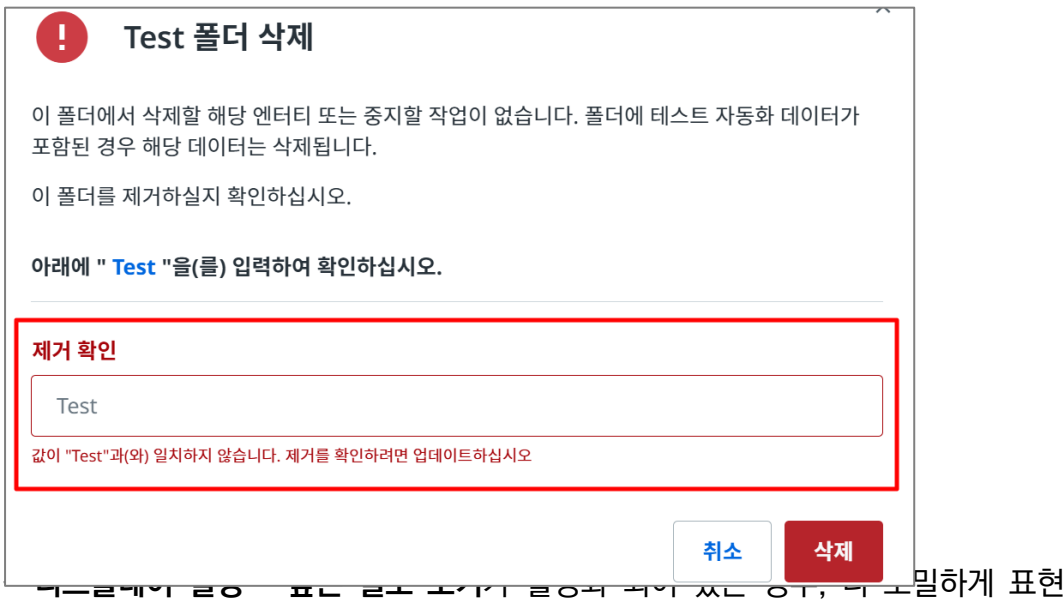


Step2. 애플리케이션 설정

- 표준 시간대: 로봇 스케줄 및 대시보드에 표시되는 시간대를 설정



- 상호 작용 구성 - 삭제 시 강력한 확인이 활성화 되어 있는 경우, 확인 절차 추가



- 활성화 안 된 경우

로봇

폴더

모니터링

엑세스 관리

머신

솔루션

패키지

감사

자격 증명

웹훅크

라이선스

설정

감사

테스트 자동화 감사

Q 검색

📄 열 (5) ▾

컴포넌트: 모두 ▾

사용자: 모두 ▾

액션: 모두 ▾

시간: 지난 1일 ▾

📄 내보내기

컴포넌트	사용자 ⇅	액션	동작	시간 ▾	🔄
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	51초 전	①
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	4분 전	①
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	5분 전	①
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 Test	5분 전	①
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 Test	5분 전	①

- 활성화 된 경우

로봇

폴더

모니터링

엑세스 관리

머신

솔루션

패키지

감사

자격 증명

웹훅크

라이선스

설정

감사

테스트 자동화 감사

Q 검색

📄

컴포넌트: 모두

사용자: 모두

액션: 모두

시간: 지난 1일

내보내기

컴포넌트	사용자 ⇅	액션	동작	시간 ⇅	🔄
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	15초 전	🕒
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	1분 전	🕒
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	5분 전	🕒
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	5분 전	🕒
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 Test	6분 전	🕒
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 Test	6분 전	🕒
폴더	kccrpaedu2@gm...	만들기	사용자 kccrpaedu2@gmail.com 생성됨 폴더 Test	9분 전	🕒
설정	kccrpaedu2@gm...	업데이트	사용자 kccrpaedu2@gmail.com이(가) 설정에 대해 업데이트됨을(를) 수행합니다.	9분 전	🕒
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 Test	9분 전	🕒
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 Test	9분 전	🕒

1 - 10 / 15

1/2페이지

항목 10

3.2.2. 배포(Deployment)

일반

배포

로봇 보안

확장성

휴무일

클라우드 연결

Cloud Robot 이미지

패키지

배포

☒ 내부

☐ 외부

보안

☒ 보안 배포

인증된 요청만 허용

☐ API 키

1167f95462e04d1db3312797e32a6efa

라이브러리

피드

☐ 호스트 피드만

☒ 테넌트 피드만

☐ 호스트 피드와 테넌트 피드 모두

배포

☒ 내부

☐ 외부

보안

☒ 보안 배포

인증된 요청만 허용

☐ API 키

종류	구분	옵션	설명
패키지	배포	내부	- UiPath 내부에서만 패키지를 관리 - 기본으로 설정되어 있는 값
		외부	- 외부 저장소를 사용해서 패키지 관리 - 외부 주소(URL) 필요
	보안	보안 배포	- 허가된 사용자만 패키지 다운로드 가능 - 인증된 요청만 허용
		API 키	- 패키지 업로드/삭제 등 쓰기 작업 보안 - 외부 클라이언트 사용 시 필요
라이브러리	피드	호스트 피드만	- 모든 테넌트가 공유하는 중앙 라이브러리 - Orchestrator에서 직접 업로드 불가
		테넌트 피드만	- 테넌트별 독립된 라이브러리 - 내부/외부 피드 선택 가능
		호스트 피드와 테넌트 피드 모두	- 두 피드 모두 접근 가능 - 테넌트별 독립성 유지
	배포	내부	- UiPath 안에서만 라이브러리 관리 - 가장 기본적이고 간단한 방식
		외부	- 외부 저장소에 라이브러리 보관 - URL, API 키, 인증정보 필요
	보안	보안 배포	- 허가된 사용자만 라이브러리 접근 가능 - 보안이 강화된 관리 방식
		API 키	- 라이브러리 관리용 API 키 - 라이브러리 수정/삭제 시 필요한 인증 수단

3.2.3. 휴무일 (Non-Working Days)

- 비영업일을 캘린더에서 하나씩 선택하는 방법과 CSV 파일로 업로드하는 방법 중 선택

일반배포로봇 보안확장성휴무일클라우드 연결Cloud Robot 이미지

휴무일 제한이 있는 트리거가 시작되지 않은 휴무일 캘린더를 구성하십시오.
[샘플 CSV 파일을 다운로드하십시오.](#)

일정

일정 이름
휴무일

+

아래 날짜를 수동으로 선택하거나

CSV 업로드

2월 2025

일	월	화	수	목	금	토
2월						1일
2일	3일	4일	5일	6일	7일	8일
9일	10일	11일	12일	13일	14일	15일
16일	17일	18일	19일	20일	21일	22일
23일	24일	25일	26일	27일	28일	

삭제

저장

- CSV 로 일괄 업로드

sample-excluded-dates.csv - Excel

	A	B	C	D	E	F
1	ExcludedDate					
2	2025-02-01					
3	2025-02-02					
4	2025-02-03					
5	2025-02-04					
6	2025-02-05					

94

4. Orchestrator Tenant 메뉴 구성

4.1. 로봇 (Robots)

- 오케스트레이터 내에서 수행되고 있는 로봇의 상태와 성능을 관리하고 로봇에 대한 정보를 모니터링하는데 초점이 맞춰진 메뉴입니다. Orchestrator에서는 직접 로봇을 생성, 삭제는 할 수 없습니다.

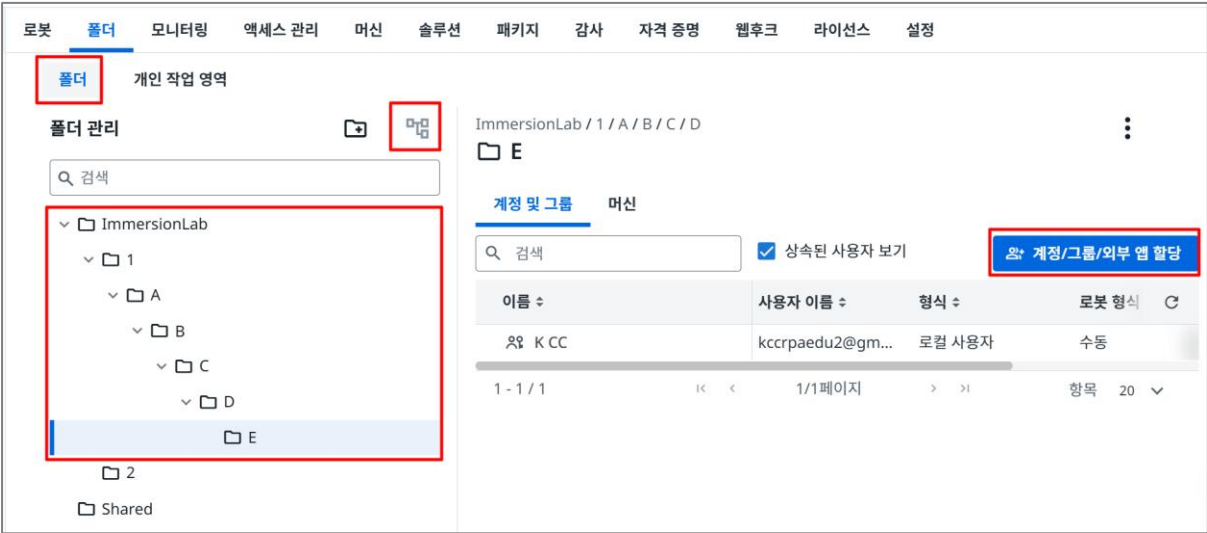


열 이름	설명
이름	로봇이 실행되는 계정의 이름
계정 유형	로봇이 실행되는 계정 유형
로봇 형식	오케스트레이터의 계정에 활성화 된 로봇 유형 (수동, 자동)
라이선스 형식	로봇이 작업을 실행하기 위해 사용하는 라이선스 유형
도메인/사용자 이름	로봇이 실행되는 사용자의 domain\username

4.2. 폴더 (Folders)

- 기존에는 Robot 과 Machine 이 1:1 로만 매칭되는 Machine 중심의 Classic 폴더를 많이 사용했다면, 최근에는 Folder 를 만들고 그 안에 Robot 과 Machine 과 이를 사용할 User 까지 추가해 로봇을 특정 사용자에게 직접 할당하고, 폴더 단위의 권한 관리, 자원 공유 등 사용자 중심의 자동화를 구현할 수 있습니다.

- 폴더는 1 개의 루트 폴더와 최대 6 개의 서브 폴더(최대 7 단계)로 구성할 수 있습니다.
- 상위 폴더의 권한이 하위 폴더로 자동 상속



- 테넌트와 폴더 권한으로 접근 가능한 리소스

Tenant Resources	Folder Resources
•Robots •Folders •Manage Access •Machines •Packages •Libraries •ML Skills •ML Logs •Audit •Credential Stores •Webhooks •Licenses •Alerts	•Subfolders •Processes •Jobs •Execution Media •Triggers •Logs •Monitoring •Queues •Transactions •Assets •Storage Buckets •Storage Files •Folder Packages •Actions •Test Case Execution Artifacts •Test Data Queue Items •Test Data Queues •Test Set Executions •Test Sets •Test Set Schedules

4.3. 액세스 관리(Manage Access)

- 사용자의 역할을 관리하는 메뉴입니다. 다양한 사용자와 그룹을 생성하고 관리하며, 각 사용자 및 그룹에 특정 역할을 부여할 수 있습니다. 대표적으로 Orchestrator 에 사용자가 어떻게 로그인할 것인지 정할 수 있고, 사용자나 그룹을 만들어 각각의 권한을 할당하며, 폴더의 액세스 권한도 설정할 수 있습니다.

- 사용자/그룹/로봇/외부 앱에 역할 설정

로봇 폴더 모니터링 액세스 관리 머신 솔루션 패키지 감사 자격 증명 웹훅 라이선스 설정

액세스 규칙 역할

모두 그룹 사용자 로봇 계정 외부 앱

Q 검색

형식: 모두 역할 할당: 모두 사용자 액세스 형식: 모두 상태: 모두 기본값으로 재설정

+ 액세스 할당

☐ 이름 ^	형식	역할	역할 할당	
☐ Administrators	로컬 그룹	Orchestrator Administrator,Solutions Administrator	직접 할당	:
☐ Automation Developers	로컬 그룹	Allow to be Automation Developer	직접 할당	:
☐ Automation Express	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Automation Users	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Citizen Developers	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Everyone	로컬 그룹		그룹에서 상속됨	:
☐ K CC	로컬 사용자		그룹에서 상속됨	:

1 - 7 / 7 1/1 페이지 항목 10

- 필요한 경우 직접 역할 생성 가능

로봇 폴더 모니터링 액세스 관리 머신 솔루션 패키지 감사 자격 증명 웹훅 라이선스 설정

액세스 규칙 역할

Q 검색 형식: 모두

+ 새 역할 추가

테넌트 역할 추가
폴더 역할 추가

☐ 이름 ^	형식 ^	
☐ Administrator	혼합됨	
☐ Allow to be Automation Developer	테넌트	:
☐ Allow to be Automation Publisher	테넌트	:
☐ Allow to be Automation User	테넌트	:

4.4. 머신 (Machines)

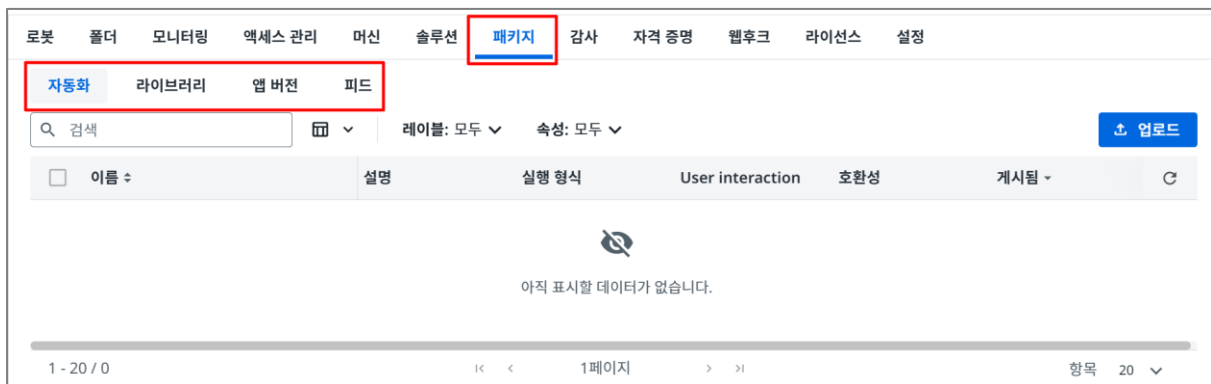
– 일반적으로 머신이란, Orchestrator 에서 무인(Unattended) 인프라 설정을 관리하기 위한 객체로, Orchestrator 와 연결되는 물리적 또는 가상의 컴퓨터를 의미하며, 머신 메뉴는 이러한 컴퓨터들을 관리하는 공간입니다.

각 머신은 Orchestrator 에 등록 시 고유한 이름과 Key 를 부여받으며, 이는 Robot 과 Orchestrator 간의 연결에 사용됩니다. 주목할 점은 Machines 메뉴가 머신을 직접 제어하는 것이 아닌, Orchestrator 와 해당 머신 간의 연결을 관리하는 데 중점을 둔다는 것입니다.



4.5. 패키지 (Package)

– UiPath Studio 에서 게시된 프로젝트와 수동으로 업로드한 프로젝트 및 라이브러리 패키지(.nupkg 파일) 그리고 Cloud 에서 게시된 애플리케이션들을 관리하는 공간입니다. 각 패키지의 버전과 릴리스 노트, 패키지 다운로드 및 업로드 작업을 진행할 수 있습니다.



4.6. 감사 (Audit)

– Orchestrator 의 감사 로그를 검토합니다. 시스템에서 발생한 이벤트에 대한 자세한 정보를 확인하며, 이를 통해 시스템의 상태와 이벤트의 원인을 파악할 수 있습니다.

로봇

폴더

모니터링

엑세스 관리

머신

솔루션

패키지

감사

자격 증명

웹후크

라이선스

설정

감사

테스트 자동화 감사

Q

검색

컴포넌트: 모두

사용자: 모두

액션: 모두

시간: 지난 1일

내보내기

컴포넌트	사용자	액션	동작	시간	
머신	kccrpaedu2@gm...	만들기	사용자 kccrpaedu2@gmail.com이(가) 머신 Test Machine에 대해 ...	1시간 전	
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 3	2시간 전	
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 3	2시간 전	
폴더	kccrpaedu2@gm...	만들기	사용자 kccrpaedu2@gmail.com 생성됨 폴더 3	2시간 전	
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 ImmersionLab/3	2시간 전	
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 Imm...	2시간 전	
폴더	kccrpaedu2@gm...	삭제	사용자 kccrpaedu2@gmail.com 삭제됨 폴더 ImmersionLab/1/A/...	2시간 전	
폴더	kccrpaedu2@gm...	삭제 시작	사용자 kccrpaedu2@gmail.com 다음에 대한 삭제 시작됨 폴더 Imm...	2시간 전	
폴더	kccrpaedu2@gm...	만들기	사용자 kccrpaedu2@gmail.com 생성됨 폴더 ImmersionLab/1/A/...	2시간 전	
폴더	kccrpaedu2@gm...	만들기	사용자 kccrpaedu2@gmail.com 생성됨 폴더 ImmersionLab/1/A/...	2시간 전	

1 - 10 / 52

<<

1/6페이지

>>

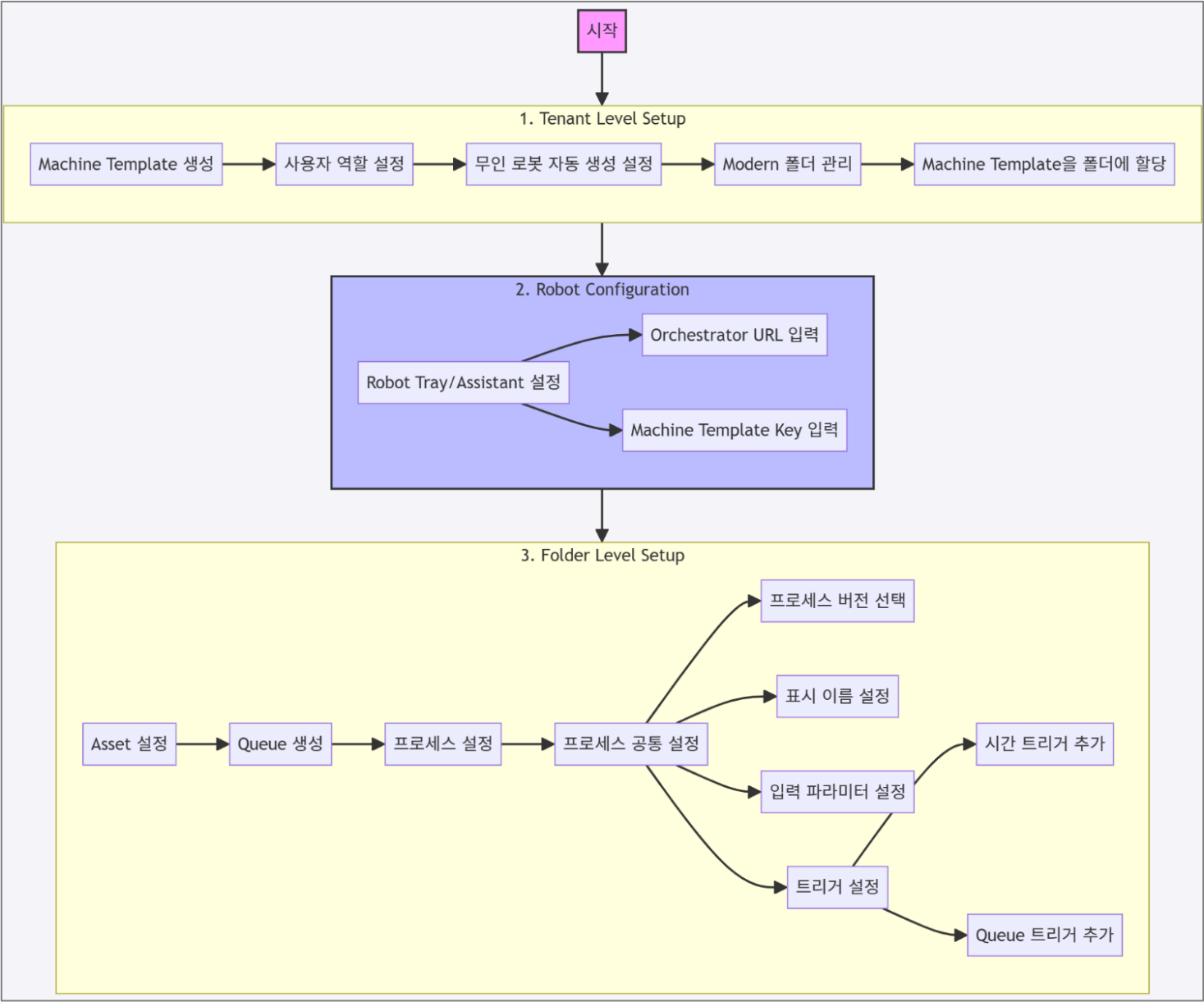
항목 10

구성요소	감시 가능한 동작	구성요소	감시 가능한 동작
로봇 (Robots)	• 생성 (Create) • 수정 (Update) • 삭제 (Delete) • 상태 변경 (Change Status)	테스트 세트	• 생성 (Create) • 수정 (Update) • 삭제 (Delete)
프로세스 (Processes)	• 생성 (Create) • 수정 (Update) • 삭제 (Delete)	테스트 일정	• 생성 (Create) • 수정 (Update) • 삭제 (Delete) • 활성화/비활성화
작업 (Jobs)	• 작업 시작 (Start Job) • 작업 중지 (Stop Job) • 작업 재개 (Resume Job)	테스트 실행	• 트리거 (Trigger) • 취소 (Cancel)
패키지 (Packages)	• 업로드 (Upload) • 삭제 (Delete)	테스트 데이터 큐	• 추가/대량 추가 • 삭제/전체 삭제 • 소비/소비 취소
자산 (Assets)	• 생성 (Create) • 수정 (Update) • 삭제 (Delete)		
큐 (Queues)	• 생성 (Create) • 수정 (Update) • 삭제 (Delete) • 대량 업로드 (Bulk Upload)		
사용자 (Users)	• 생성 (Create) • 수정 (Update) • 삭제 (Delete) • 비밀번호 변경 (Change Password) • 권한 연결 (Associate)		

5. Orchestrator 프로세스 배포하기

5.1. 진행 순서

- ① ~ ③번까지 차례대로 학습하시면 프로세스 배포/실행/관리를 구현할 수 있습니다.



5.2. 머신 생성

Step1. 테넌트 → 머신 메뉴 접근 → + 머신 추가 → 머신 템플릿 선택



- 각 머신별 특징 정리

구분	Machine Template	Elastic Robot Pool	Cloud Robot – Serverless
기본 정의	로봇 설치 컴퓨터의 기본 설정 템플릿	클라우드 기반 동적 로봇 리소스 관리	서버리스 환경의 클라우드 로봇
설치 환경	On-premise	클라우드 (Azure/AWS)	클라우드 (UiPath 관리)
주요 특징	- 표준화된 머신 구성 템플릿 - 재사용 가능한 설정 프로필 - 일관된 환경 구성 가능	- 동적 확장 가능한 로봇 풀 - AWS/Azure 클라우드 지원 - 자동 스케일링 기능	- 인프라 관리 불필요 - UiPath 관리 환경 - 즉시 실행 환경
적합한 업무 유형	정기적인 일상 업무	변동적인 대량 업무	간헐적/비정기 업무
처리량	고정적	탄력적	완전 유동적
보안 요구사항	높음 (내부 통제 가능)	중간	중간-낮음
모니터링 도구	상세함	중간	기본적
커스터마이징	높은 자유도	제한적	매우 제한적
주요 장점	- 완전한 통제 가능 - 높은 보안성 - 상세한 설정 가능	- 유연한 확장성 - 비용 최적화 - 자동화된 관리	- 빠른 구축 - 최소한의 관리 - 사용한 만큼 지불
실행 환경	물리적/가상 머신	클라우드 VM	서버리스 컨테이너
백업/복구	수동 관리 필요	자동화된 백업 지원	플랫폼에서 자동 관리

Step2. 내용 작성

- 템플릿 이름과 설명, 사용할 라이선스 개수를 적고 프로비전 버튼 클릭

테넌트 > 머신 > 머신 템플릿

1 머신

2 유지보수
선택 사항

일반 세부 정보 ⓘ

템플릿 이름 *

HJCHOI

설명

20250204_RPA심화교육

지원되는 사용

프로세스 유형*

모두

프로세스 호환성*

모두

태그

레이블

가능한 결과 목록을 가져오려면 입력을 시작하십시오.

속성(키-값 쌍)

고급 설정

이 개체에는 속성이 없습니다.

Runtime 세부 정보

Runtime 라이선스: 프로덕션, 테스트

아래에 지정된 라이선스 수는 이 템플릿에 연결된 각 머신에서 사용됩니다. 머신과의 연결이 끊어지면 바로 라이선스가 해제됩니다.

이 구성은 머신이 동시에 실행할 수 있는 최대 작업 수를 정의합니다.

Production (Unattended)*

1 /1

Testing*

0 /1

Runtime 형식: 프로덕션, 테스트

머신 템플릿의 경우 Runtime 라이선스는 작업 시작/트리거 생성 시 선택할 수 있는 Runtime 형식과 일치합니다.

취소

프로비전

Step3. 생성한 머신 확인

- 닫기 클릭

테넌트 > 머신 > 머신 템플릿

✓ 성공 머신이 생성되었습니다.

이제 아래 세부 정보를 사용하여 Assistant에서 로봇에 대해 이 머신을 인증할 수 있습니다.

머신 이름

HJCHOI

클라이언트 ID/머신 키

4e6d2880-919f-4fda-a9c0-ed7751122934

클라이언트 암호

KdqByWu6btH6BANX

위 ID와 함께 사용됩니다. 머신을 편집할 때 암호를 추가하거나 삭제할 수 있습니다.

1 암호를 안전하게 보관하십시오.

위 자격 증명을 안전한 저장소 위치에 복사하십시오. 암호화 또는 해싱을 사용하여 스토리지를 보호하십시오. 클라이언트 암호 값이 손실된 경우 머신 인증 세부 정보를 편집하여 새 값을 생성할 수 있습니다.

닫기

102

- 결과 확인

로봇 폴더 모니터링 액세스 관리 머신 솔루션 패키지 감사 자격 증명 웹후크 라이선스 설정						
Q 검색		형식: 모두	레이블: 모두	속성: 모두	+ Unattended 설정 + 머신 추가	
☐ 이름 ^	설명	형식	설치된 버전	버전 상태	레이블	속
☐ HJCHOI	20250204_RPA심...	템플릿	로봇 없음	① 해당 없음		
☐ kccrpaedu2@gmail.co...		템플릿	25.0.157-cloud.1...	🟢 준수		
1 - 2 / 2 < > 1/1 페이지 > > 항목 20						

5.3. 액세스 관리 설정

Step1. 테넌트 → 액세스 관리 메뉴에 접속 → 사용자 계정  클릭 → 편집 선택

테넌트

내 폴더

Q 검색

My Workspace

ImmersionLab

Shared

로봇 폴더 모니터링 액세스 관리 머신 솔루션 패키지 감사 자격 증명 웹후크 라이선스 설정

액세스 규칙 역할

모두

 그룹 사용자 로봇 계정 외부 앱

Q 검색

형식: 모두 역할 할당: 모두 사용자 액세스 형식: 모두 상태: 모두

기본값으로 재설정

+ 액세스 할당

☐ 이름 ^	형식	역할	역할 할당	
☐ Administrators	로컬 그룹	Orchestrator Administrator,Solutions Administrator	직접 할당	:
☐ Automation Developers	로컬 그룹	Allow to be Automation Developer	직접 할당	:
☐ Automation Express	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Automation Users	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Citizen Developers	로컬 그룹	Allow to be Automation User	직접 할당	:
☐ Everyone	로컬 그룹		그룹에서 상속됨	:
☐ K CC	로컬 사용자		그룹에서 상속됨	:

1 - 7 / 7 < > 1/1 페이지 > >

편집

액세스 확인

Step2. 설정해야 할 위치 확인

테넌트 > 액세스 관리 > 액세스 규칙 > 사용자 > 편집: K CC

엑세스 규칙 할당

사용자 계정

K CC (kccrpaedu2@gmail.com)

구성

아래에서 선택한 값은 상속된 값으로 대체될 수 있습니다. 유효 액세스 규칙에 대한 요약 카드를 확인하십시오.

추가 역할

역할을 선택하지 않았습니다.

+ 새 역할

설정

UI 프로필

없음

개인 자동화 설정에서 개인 작업 영역이 부여되지 않는 경우 개인 작업 영역을 선택할 수 없습니다

개인 자동화 옵션에 대해 자세히 알아보십시오.

개인 자동화 설정

없음

클라이언트 바이너리(Robot, Assistant 및 Studio) 자동 업데이트 정책*

최신 버전

Unattended 설정(사용자에게 권장되지 않음)

요약 카드

엑세스는 항상 부가적이며, 명시적으로 할당된 것과 그룹에서 상속된 것이 있습니다.

역할

권한 확인

Orchestrator Administrator (Administrators)

Solutions Administrator (Administrators)

Allow to be Automation Developer (Automation Developers)

설정

UI 프로필: 표준 인터페이스 (Administrators, Automation Developers)

개인 자동화 설정: 사용자가 자동화 + 개인 작업 영역을 실행하도록 활성화 (Automation Developers)

업데이트 정책 설정: 최신 버전 (직접 할당)

Unattended Robot 설정: 없음

고급 로봇 옵션: 없음

- 추가 역할 선택

추가 역할

역할을 선택하지 않았습니다.

+ 새 역할

☐ Allow to be Folder Administrator

☐ Orchestrator Administrator

☐ Solutions Administrator

☐ Solutions Contributor

☐ Administrator

혼합됨

입니다

- Unattended 설정 내용 입력

- 로봇 설정: 사용자 지정 Windows 자격 증명을 사용하는 Unattended Robot 선택
- 명령 프롬프트(CMD) 열고 'whoami' 명령어 입력
- 결과물 복사하여 Domain\Username 에 붙여 넣기
- 암호 칸에 PC 비밀번호 입력 (없는 경우 비워 두기)
- 모든 작업이 끝났으면 업데이트 클릭

Unattended 설정(사용자에게 권장되지 않음)

For running unattended automations, we recommend using a [Robot account](#) instead.

Unattended Robot 설정

사용자 지정 Windows 자격 증명을 사용하는 Unattended Robot

☐ 한 번에 하나의 작업만 실행

Domain\Username*

hjchoi\hjchoi

다양한 계정 유형과 자격 증명을 가져오는 방법에 대한 [자세한 내용](#)을 읽어보십시오.

자격 증명 저장소*

Orchestrator Database

암호

....

자격 증명 유형*

Windows 자격 증명

```
명령 프롬프트
Microsoft Windows [Version 10.0.17134.1]
(c) Microsoft Corporation. All rights reserved.

C:\Users\mrchu>whoami
hjchoi\hjchoi

C:\Users\mrchu>
```

5.4. 폴더 만들기

Step1. 테넌트 → 폴더 메뉴에 접속 → 새 폴더() 클릭

- 폴더 이름 입력 후 Create 클릭

Step2. 생성된 폴더 선택 후 머신 → 폴더의 머신 관리 클릭

Step3. 사용할 머신 선택 → 업데이트 클릭

1 행이 선택되었습니다. ✖

☐	이름 ⇅	형식 ⇅	Product... ⇅	NonPro... ⇅	Testing ⇅	레이블	속성	↻
☒	HJCHOI	템플릿	1	0	0			
☐	 kccrpaedu2@gmail.c...	템플릿	0	0	0			

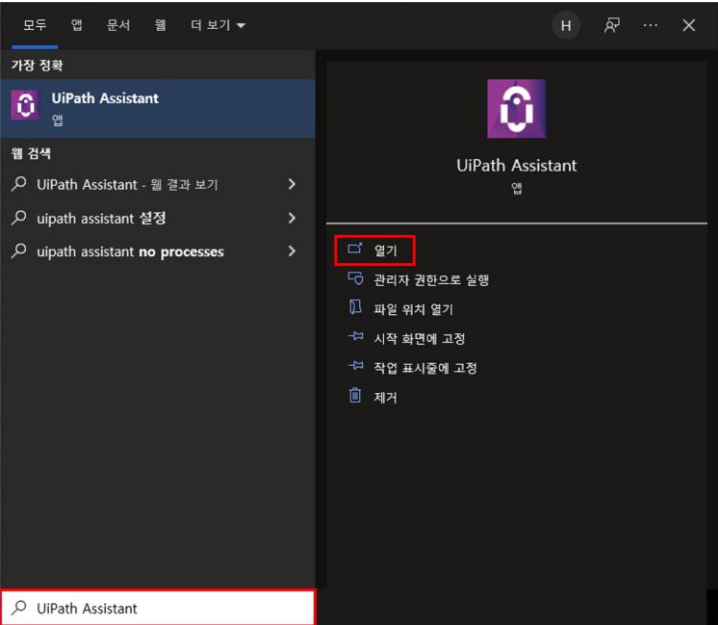
1 - 2 / 2 1/1페이지 항목 20 ▼

취소
업데이트

5.5. Robot 생성

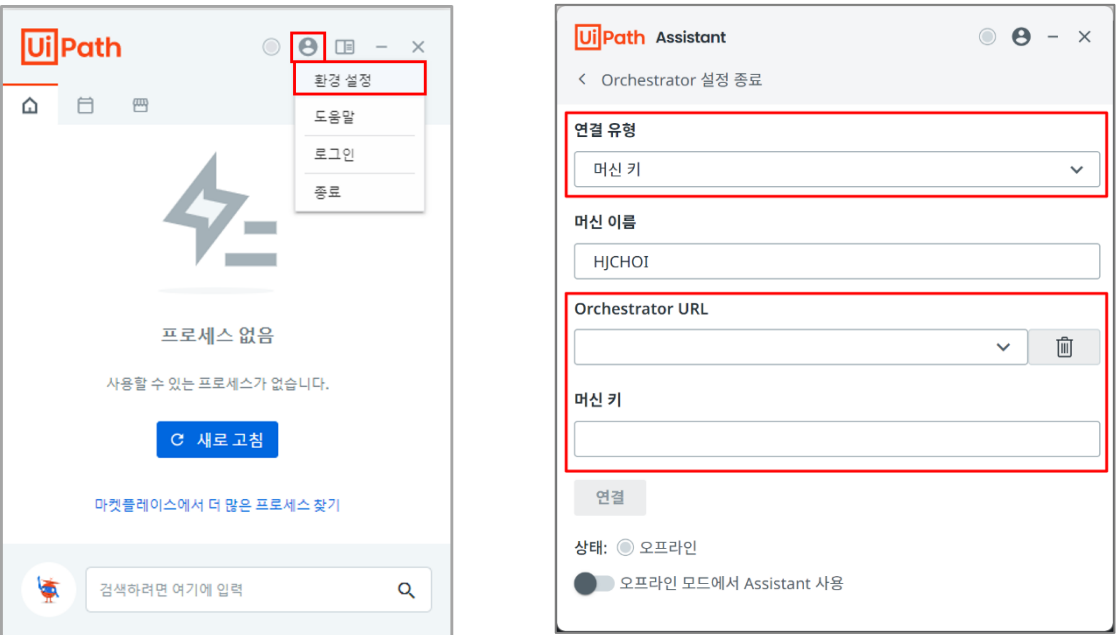
Step1. UiPath Assistant 등록

- 시작메뉴 - UiPath Assistant 검색 - 열기를 클릭합니다.
- UiPath Assistant 를 처음 실행할 경우, Robot 의 이름을 적어줍니다

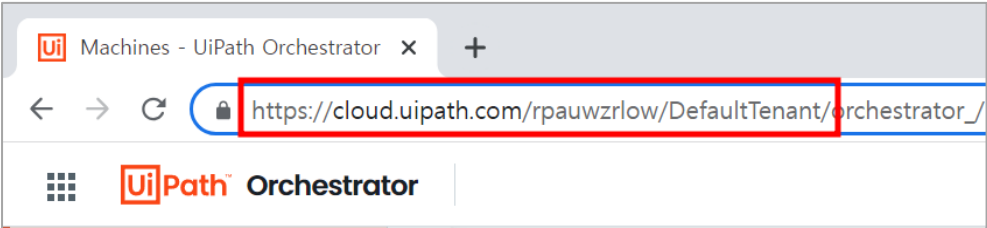


Step2. Orchestrator 연동

- 환경 설정 클릭 → Orchestrator 설정 클릭 → 로그아웃 후 연결 유형 - '머신키' 선택

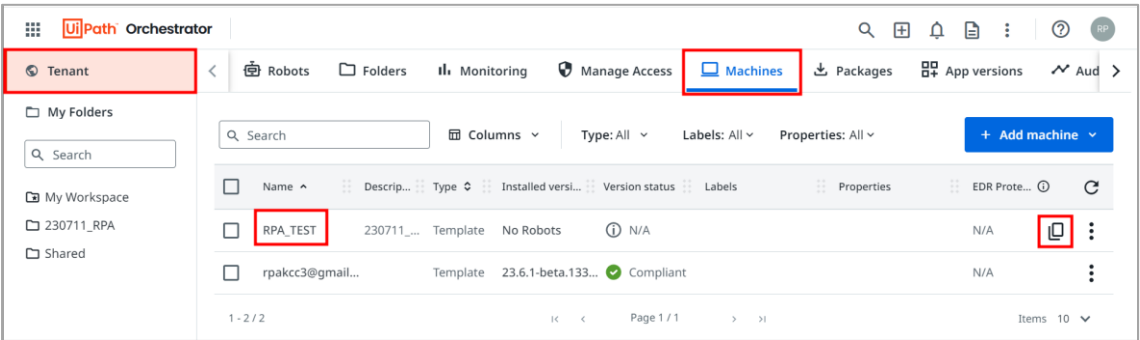


Step3. Orchestrator URL 복사 후 입력

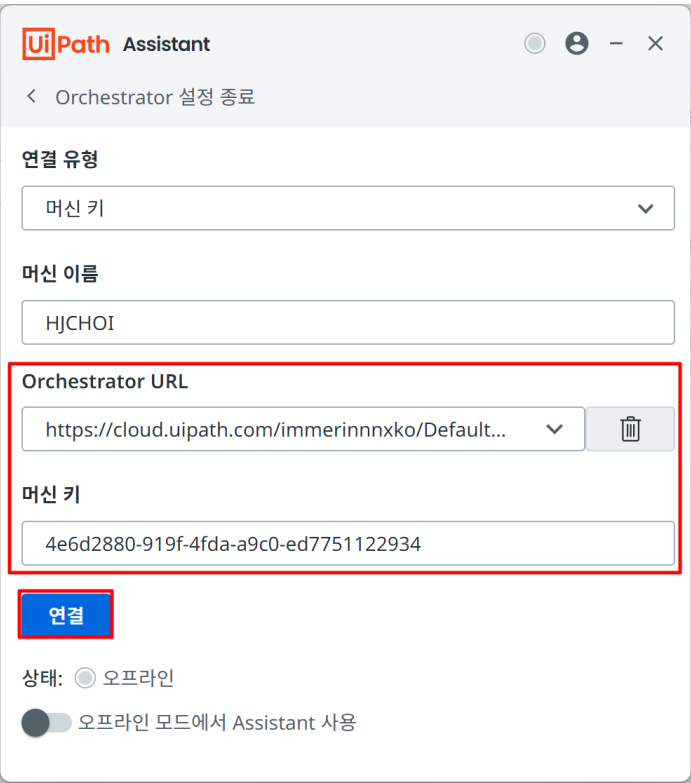


Step4. Machine Key 복사 후 입력

- Machines 메뉴에서 생성한 머신키 복사

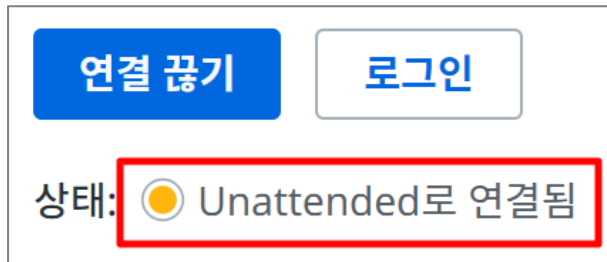


- 머신키 붙여 넣기 후 연결 버튼 클릭



Step5. 결과 확인

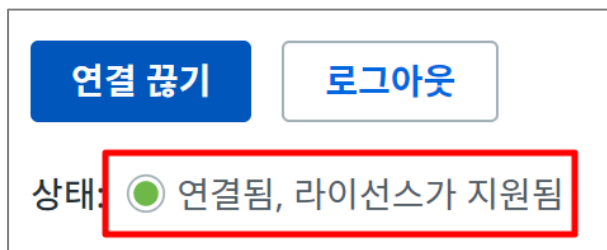
- 그림과 같이 나오는 경우는 현재 Unattended Robot 라이선스만 연결되었다는 의미이므로 UiPath Studio 를 열었을 때, 디버깅 모드만 지원이 된다는 메시지를 볼 수 있습니다. 만약, 해당 상태에서 Studio 개발까지 진행하고 싶을 때에는 ‘로그인’ 버튼을 눌러 Studio 라이선스도 연결시켜 주어야합니다.



- UiPath 열기 클릭

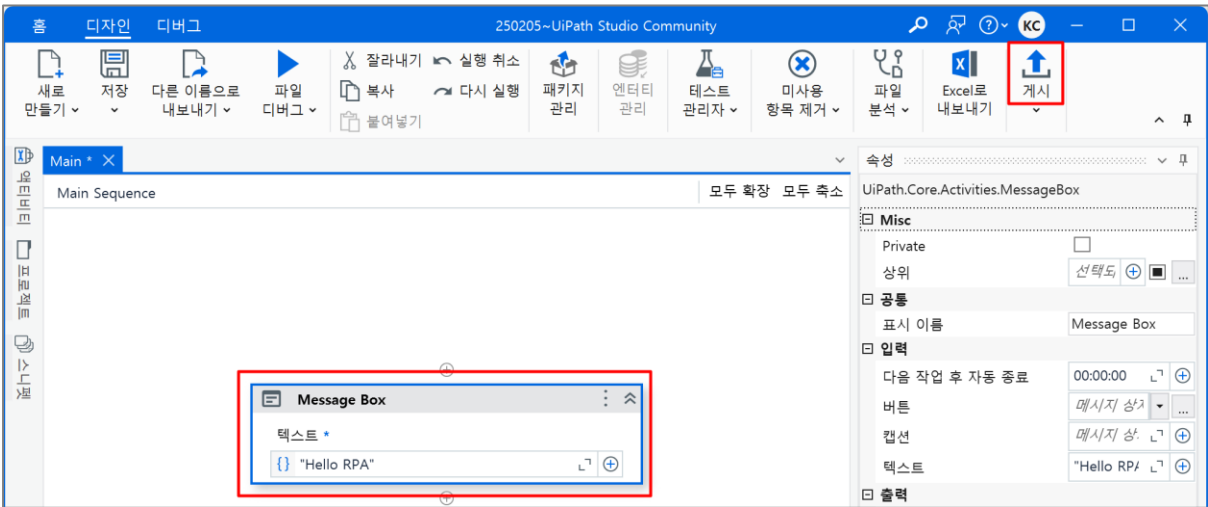


- 라이선스 정상 연결 확인

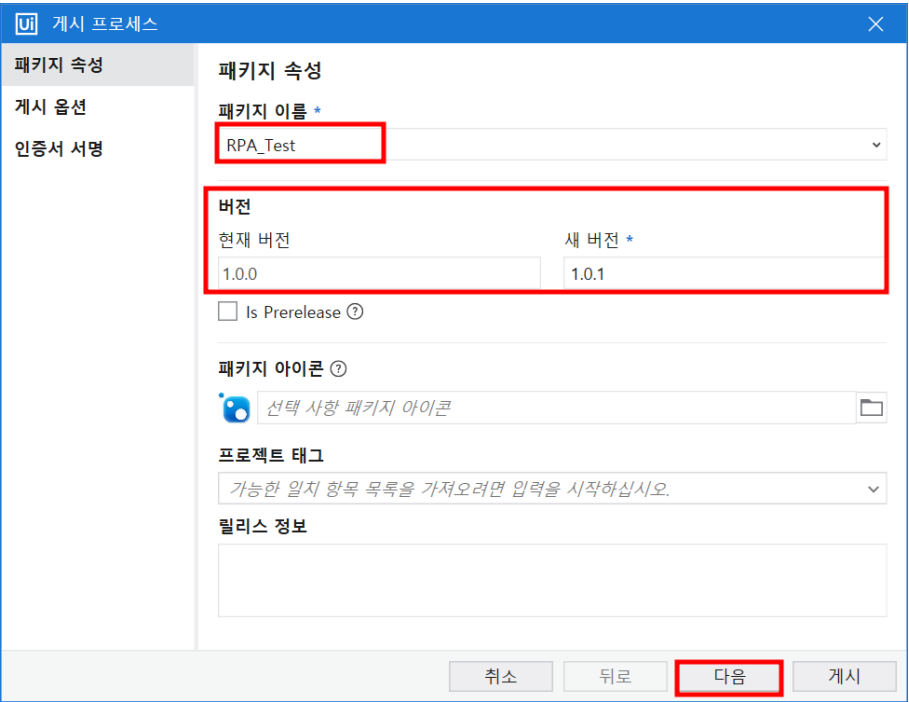


5.6. UiPath Studio 프로세스 배포

Step1. 배포할 프로세스 선택 후 게시 버튼 클릭



Step2. 패키지 이름 및 버전 확인 후 다음 버튼 클릭



Step3. 게시 옵션 – 다음에 게시: Orchestrator 테넌트 프로세스 피드 선택 후 게시 클릭

패키지 속성

게시 옵션

인증서 서명

게시 옵션

다음에 게시

Orchestrator 테넌트 프로세스 피드

사용자 지정 URL

NuGet 피드 URL 또는 로컬 폴더

API 키

선택적 API 키

컴파일 설정

☐ 사용되지 않은 종속성 제거 ?

취소

뒤로

다음

게시

– 게시 확인

정보

프로젝트(가) 게시되었습니다.

이름: RPA_Test

버전: 1.0.1

세부 정보

클립보드에 복사

확인

Step4. Orchestrator 테넌트 → 패키지 메뉴에 업로드 된 패키지 확인

Orchestrator

테넌트

로봇

폴더

모니터링

액세스 관리

머신

솔루션

패키지

검사

자격 증명

웹훅

라이선스

설정

자동화

라이브러리

앱 버전

피드

검색

레이블: 모두

속성: 모두

업로드

☐	이름	설명	실행 형식	User interaction	호환성	게시됨
☐	RPA_Test	빈 프로세스	무인	Required	Windows(.NET 5....	4분 전

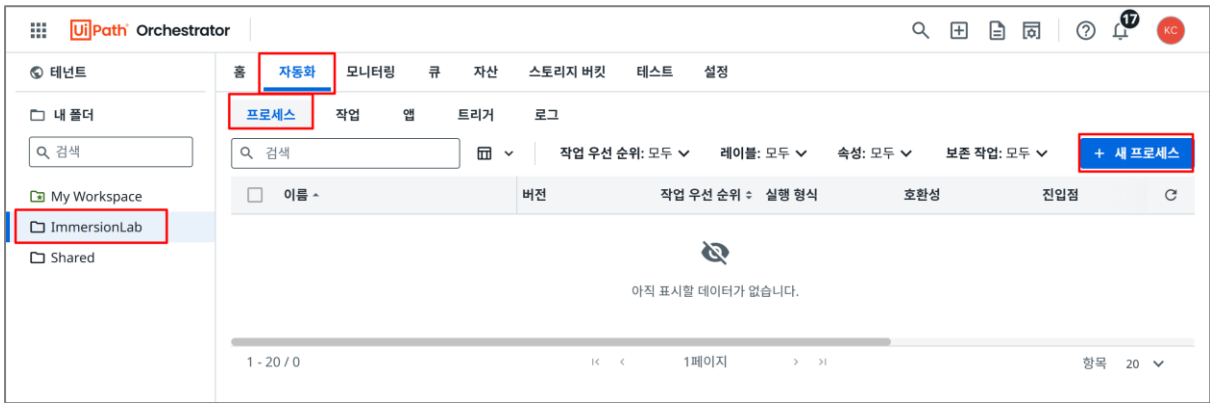
1 - 1 / 1

1/1페이지

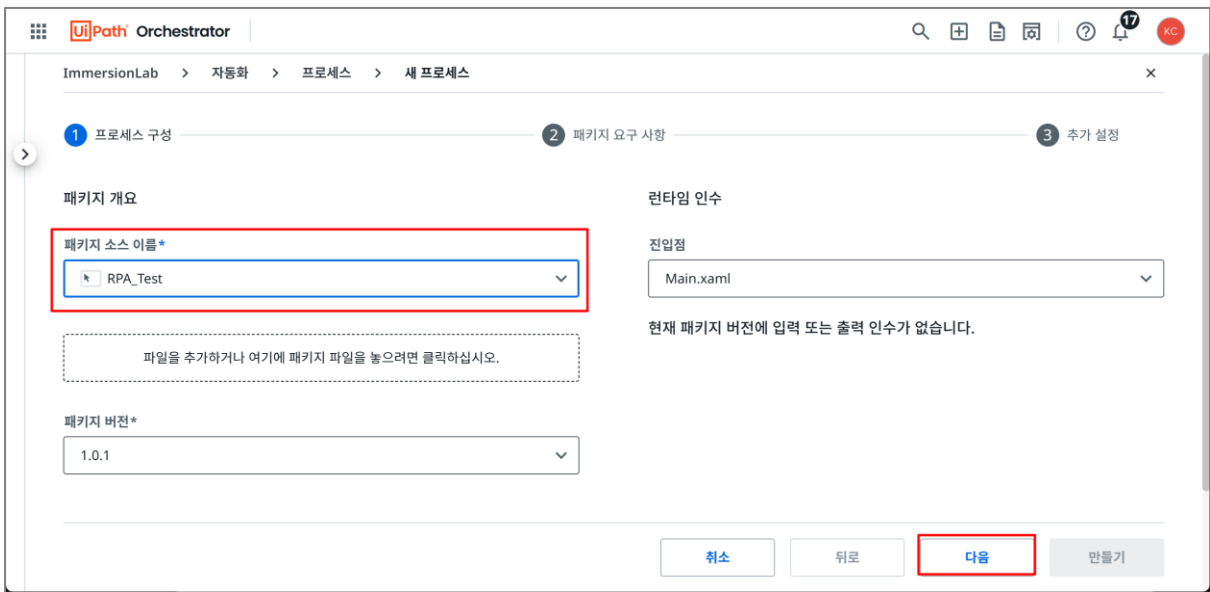
항목 20

Step5. 프로세스 배포

- 생성한 폴더 → 자동화 - 프로세스 메뉴 클릭 후 + ‘새 프로세스’ 클릭



Step6. 업로드한 패키지 선택



Step7. Process 세부사항 설정

- 프로세스 이름 및 설명, 우선순위 등 프로세스에 맞는 설정 선택 후 만들기 클릭

ImmersionLab

>

자동화

>

프로세스

>

새 프로세스

✕

프로세스 구성

페이지 요구 사항

추가 설정

프로세스 세부 정보

우선 순위 및 실행 옵션

표시 이름

250205_RPA_Test

설명

빈 프로세스

프로세스를 몇 단어로 설명하십시오. 처음에는 페이지 바전을 기준으로 상속되었습니다.

태그

레이블

가능한 결과 목록을 가져오려면 입력을 시작하십시오.

속성(가-값 쌍)

이 개체에는 속성이 없습니다.

고급 설정

작업 우선 순위*

중간

경쟁 작업에서 작업의 우선순위를 컨트롤합니다.

복구 에이전트 활성화

마러 보기

복구 에이전트를 활성화하여 향후 실행 중인 작업의 문제를 해결하는 방법에 대한 권장 사항과 제안을 제공합니다. 자가 복구 옵션을 사용할 수 있습니다.

복구 에이전트 자가 복구 활성화

실행 중인 작업에서 문제가 감지되면 복구 에이전트가 자가 복구되도록 활성화합니다.

Attended User에게 프로세스를 숨기기

이 옵션을 활성화하면 프로세스가 UIPath Assistant에서 숨겨집니다.

UIPath Assistant에서 프로세스를 중지할 수 없습니다.

Assistant를 사용하여 프로세스가 중지 또는 일시 중지되지 않도록 합니다.

프로세스를 자동으로 시작

로봇 에이전트가 시작되면 프로세스를 자동으로 시작합니다. UIPath Assistant에서 중지할 수 없는 프로세스와 함께 활성화한 경우, Assistant가 사용자 머신에서 우선 시작되면 프로세스가 시작됩니다.

보존 정책

보존 정책은 이전 작업이 제거되는 시점과 방식을 제어합니다. [문서 읽기](#)

액션*

삭제

보존 기간*

30

일

작업 기록

작업에 대한 스크린샷 또는 비디오 아티팩트를 생성하고 검색하여 장애가 발생한 작업의 문제를 해결하십시오. Unattended Robot에서만 사용할 수 있습니다.

라이브 스트림 허용

작업의 라이브 스트리밍을 확인합니다. 원격으로 제어하여 실행 중인 작업의 문제를 쉽게 해결하거나 중단할 수 있습니다. Unattended Robots에서만 사용할 수 있습니다.

취소

뒤로

다음

만들기

Step8. 배포된 프로세스 확인 후 ‘+ 트리거 추가’ - ‘시간 트리거 만들기’ 클릭

5.7. 시간 트리거 생성

Step1. 트리거 내용 입력

- 이름 지정 → 실행할 사용자와 머신 선택 → 반복 수행 주기 선택 후 Add 클릭

이름*

250205_RPA_Test

프로세스 이름*

250205_RPA_Test

작업 우선 순위*

상속됨

Runtime 유형*

Production (Unattended)

1 런타임 사용 가능, 1 연결됨

실행 탭

인수

동적으로 할당

프로세스 실행 1 횟수

계정

모든 사용자/로봇 계정

이 폴더에 Unattended Robot이 할당된 사용자 계정 또는 로봇을 검색하거나, 사용자 자격으로 자동화를 실행하십시오.

머신

모든 머신

구성된 머신 또는 템플릿 검색

표준 시간대*

(UTC+09:00) 서울

빈도

매분

매

1

분마다 반복

1

반복 예약은 작업의 표준 시간대에 적용됩니다. 표준 시간대가 변경되는 경우 예약 파라미터가 자동으로 업데이트되지 않습니다.

휴무일 제한 사항

휴무일

작업 실행 종료 예약

자동 트리거 비활성화 예약

보류 중이거나 재개된 상태로 작업이 중단된 경우 경고 생성

작업이 시작되었지만 완료되지 않은 경우 경고 생성

실행 기반 트리거 비활성화 설정

작업을 다시 시작할 때 계정/머신 할당 유지

취소

추가

Step2. 프로세스 및 트리거 생성 확인

홈

자동화

모니터링

큐

자산

스토리지 버킷

테스트

설정

프로세스

작업

앱

트리거

로그

작업 우선 순위: 모두

레이블: 모두

속성: 모두

보존 작업: 모두

+ 새 프로세스

☐	이름 <	버전	작업 우선 순위 : 실행 형식	호환성	진입점	
☐	* Z50205_RPA_Test	1.0.1	= 중간 무인	Windows(.NET 5...	Main.xaml	빈

1 - 1 / 1

1/1페이지

항목 20

홈

자동화

모니터링

큐

자산

스토리지 버킷

테스트

설정

프로세스

작업

앱

트리거

로그

시간 트리거

큐 트리거

이벤트 트리거

API 트리거

상태: 모두

작업 우선 순위: 모두

+ 새 트리거 추가

☐	이름	프로세스	트리거 세부 정보	작업 우선 순위	다음 런타임	다음 이후 중지
☐	250205_RPA_Test	250205_RPA_Test	매분	○ 상속됨	🕒 44초 후	

1 - 1 / 1

1/1 페이지

항목 20

5.8. Orchestrator 프로세스 실행

Step1. 자동화 - 프로세스 메뉴에서 바로 시작

- 작업 시작 버튼 클릭

홈	자동화	모니터링	큐	자산	스토리지 버킷	테스트	설정
프로세스	작업	앱	트리거	로그			
🔍 검색	📄	작업 우선 순위: 모두	레이블: 모두	속성: 모두	보존 작업: 모두	+ 새 프로세스	
☐ 이름	패키지 이름	버전	작업 우선 순위	실행 형식	진입점		
☐ 250205_RPA_Test	RPA_Test	1.0.1	중간	무인	Main.xaml		
1 - 1 / 1		1/1 페이지			항목 20		

- 어디에서 실행할 건지 선택한 다음 Start 클릭

ImmersionLab

자동화

프로세스

작업 시작

프로세스 이름*

250205_RPA_Test

작업 우선 순위*

상속됨

Runtime 유형*

Production (Unattended)

1 런타임 사용 가능, 1 연결됨

실행 타겟

인수

동적으로 할당

프로세스 실행 1 횟수

복구 에이전트 활성화

복구 에이전트를 활성화하여 향후 실행 중인 작업의 문제를 해결하는 방법에 대한 권장 사항과 제안을 제공합니다. 자가 복구 옵션을 사용할 수 있습니다.

복구 에이전트 자가 복구 활성화

실행 중인 작업에서 문제가 감지되면 복구 에이전트가 자가 복구되도록 활성화합니다.

작업 실행 종료 예약

보류 중이거나 재개된 상태로 작업이 중단된 경우 경고 생성

작업이 시작되었지만 완료되지 않은 경우 경고 생성

계정

모든 사용자/로봇 계정

이 폴더에 Unattended Robot이 할당된 사용자 계정 또는 로봇을 검색하거나, 사용자 자격으로 자동화를 실행하십시오.

머신

모든 머신

구성된 머신 또는 템플릿 검색

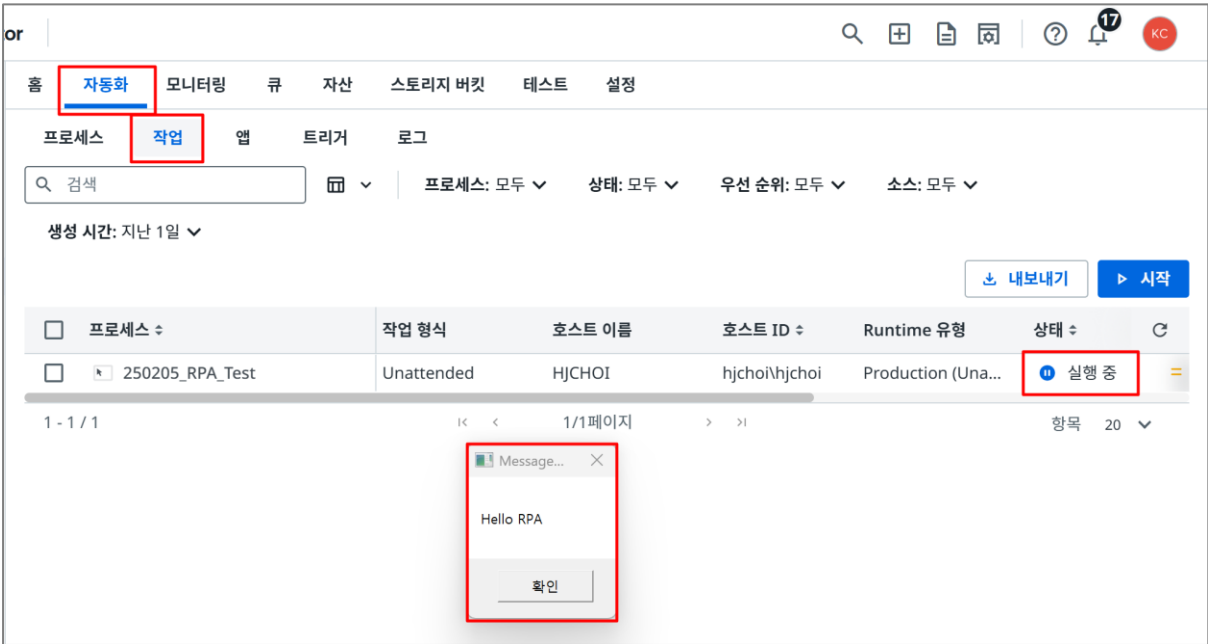
작업을 다시 시작할 때 계정/머신 할당 유지

취소

시작

115

- 결과 확인



Step2. 자동화 – 작업 메뉴에서 시작

- 시작 버튼 클릭



- 이후, 프로세스에서 시작하는 것과 동일

5.9. Orchestrator 프로세스 업데이트

- 기존에 배포된 프로세스를 업데이트하여 재배포한 경우, 다른 추가 작업 없이 Processes 메뉴에서 처리할 수 있습니다.

Step1. 패키지 이름, 버전 확인 후 게시 옵션 이동

게시 프로세스

패키지 속성

패키지 속성

패키지 이름 *

RPA_Test

버전

현재 버전

1.0.1

새 버전 *

1.0.2

Step2. 다음에 게시 - 'Orchestrator 테넌트 프로세스 피드' 선택 후, 게시 클릭

게시 프로세스

게시 옵션

다음에 게시

Orchestrator 테넌트 프로세스 피드

취소 뒤로 다음 **게시**

Step3. 프로젝트 게시 팝업 확인

정보

프로젝트(가) 게시되었습니다.

이름: RPA_Test

버전: 1.0.2

세부 정보 ▼ 클립보드에 복사 확인

Step5. 자동화 – 프로세스에서 업데이트 준비가 된 프로세스 확인



Step6. 프로세스 업데이트 표시 확인 후 최신 항목 사용 클릭



6. Orchestrator 정보 관리 (Asset)

6.1. 자산(Asset) 정의

- Asset이란 로봇이 실행되는 과정에서 사용할 수 있는 다양한 구성 요소를 의미합니다. Asset은 로봇이 어떠한 특정 작업을 수행하게 할 때 필요한 설정, 구성, 자격증명 등을 쉽게 관리할 수 있게 해줍니다.

6.2. 자산(Asset) 종류

텍스트(Text)	사용자 이름, 암호와 같은 간단한 문자열을 저장할 수 있습니다.
부울(Boolean)	참 / 거짓 값을 저장할 수 있습니다.
정수(Integer)	정수 값을 저장할 수 있습니다.
자격증명 (Credential)	사용자 이름과 암호를 안전하게 저장하고 관리할 수 있습니다. 이러한 정보는 보통 로그인과 같은 작업에서 사용됩니다.

6.3. 자산(Asset) – Single Value 생성

- Asset의 이름, 형식, 설명, 텍스트를 입력한 다음 만들기를 클릭합니다.

ImmersionLab > 자산 > Asset 추가

일반 세부 정보

Asset 이름*
과제명_담당자이메일

설명
부서_프로세스코드_이메일주소

태그

레이블

형식

텍스트

부울

정수

자격 증명

자산 값

전역 값

아래 표에 특별히 재정의된 경우를 제외하고, 전역 값을 사용하도록 설정되면 모든 사용자가 해당 값을 수신합니다.

텍스트*
hjchoi@kcc.co.kr

+ 로봇 자산 값 추가

사용자 머신 값

선택 사항으로 사용자별로 특정 값을 추가할 수도 있습니다.

취소

만들기

6.4. 자산(Asset) – Value Per Robot 생성

- Asset 의 이름, 형식, 설명, 텍스트를 입력한 다음 전역 값을 Off 합니다.
- ‘로봇 자산 값 추가’를 클릭한 다음 사용자, 머신, 사용자 이름, 암호를 작성 후 추가
- 만들기를 클릭하여 Asset 을 생성합니다.

ImmersionLab > 자산 > Asset 추가

일반 세부 정보

Asset 이름*

사이트이름_로그인계정

형식

자격 증명

설명

사이트이름_담당자이름_로그인계정

자격 증명 저장소*

Orchestrator Database

태그

레이블

가능한 결과 목록을 가져오려면 입력을 시작하십시오.

속성(카-값 쌍)

고급 설정

이 개체에는 속성이 없습니다.

자산 값

전역 값

아래 표에 특별히 지정된 경우를 제외하고, 전역 값을 사용하도록 설정되면 모든 사용자가 해당 값을 수신합니다.

사용자 이름*

사용자 이름

암호*

암호

+ 로봇 자산 값 추가

사용자

머신

값

사용자*

kccrpaedu2@gmail.com

머신

HJCHOI

사용자 이름*

hjchoi@kcc.co.kr

암호*

취소

추가

취소

만들기

+ 로봇 자산 값 추가

사용자	머신	값
kccrpaedu2@gmail....	HJCHOI	hjchoi@kcc.co.kr:****

- 생성된 Asset 확인

홈 자동화 모니터링 큐 자산 스토리지 버킷 테스트 설정

검색

레이블: 모두

속성: 모두

+ 자산 추가

☐	이름 ^	설명 ^	형식 ^	값	
☐	과제명_담당자이메일	부서_프로세스코드_이메일주소	텍스트	hjchoi@kcc.co.kr	:
☐	사이트이름_로그인계정	사이트이름_담당자이름_로그인계정	자격 증명	[자격 증명 저장소 내]	:

1 - 2 / 2

1/1페이지

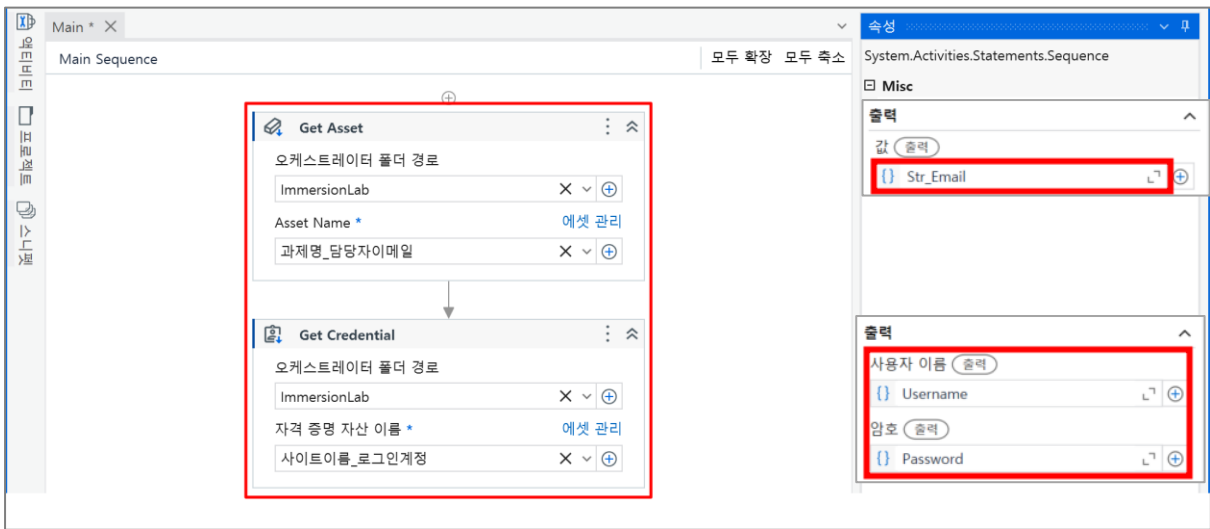
항목 20

6.5. UiPath Studio 에서 Asset 확인

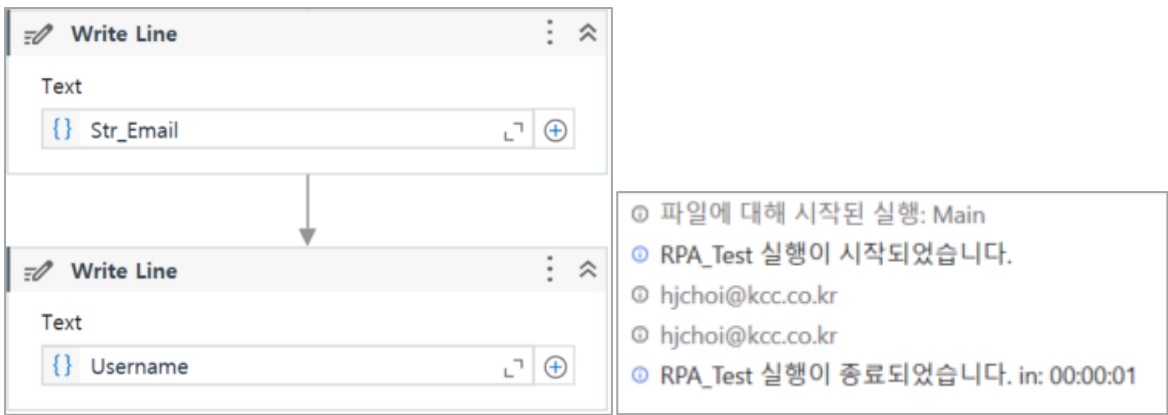
- Get Asset 및 Get Credential 을 사용해 Orchestrator Asset 을 불러올 수 있습니다.

Step1. UiPath Studio 프로세스 실행

Step2. Get Asset, Get Credential 액티비티 드래그 앤 드롭



Step3. 실행 및 결과 확인



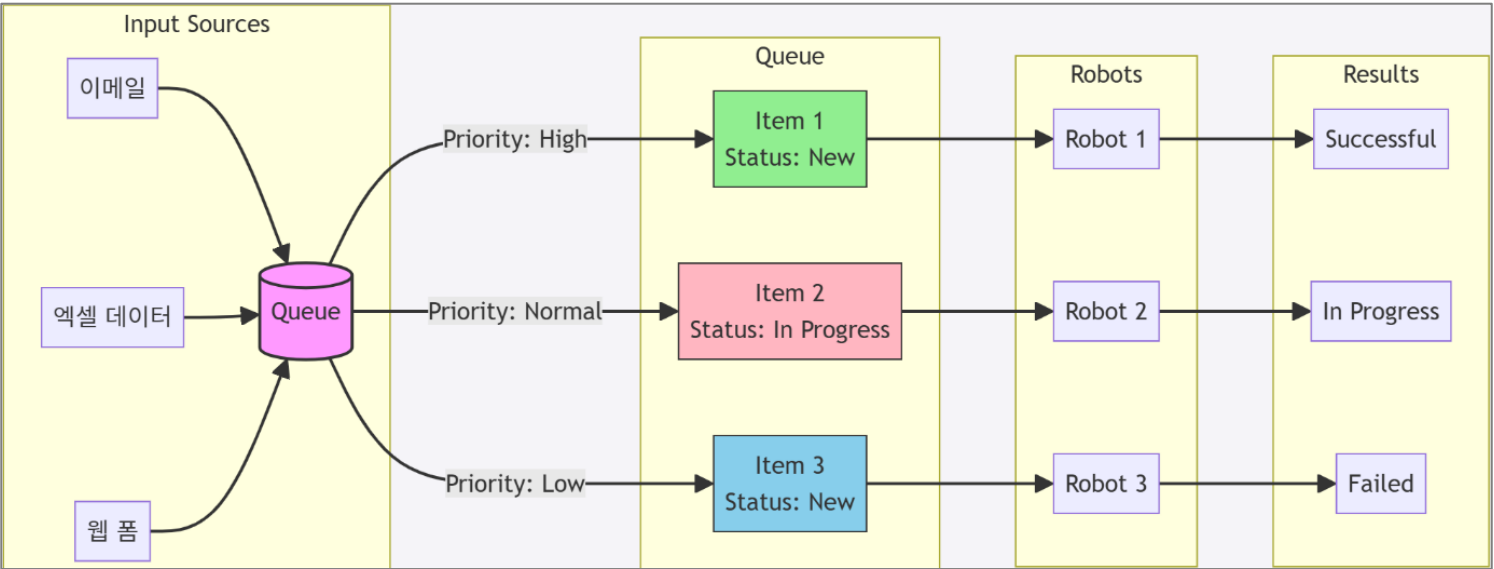
7. Orchestrator 데이터 관리 (Queue)

7.1. Queue 정의

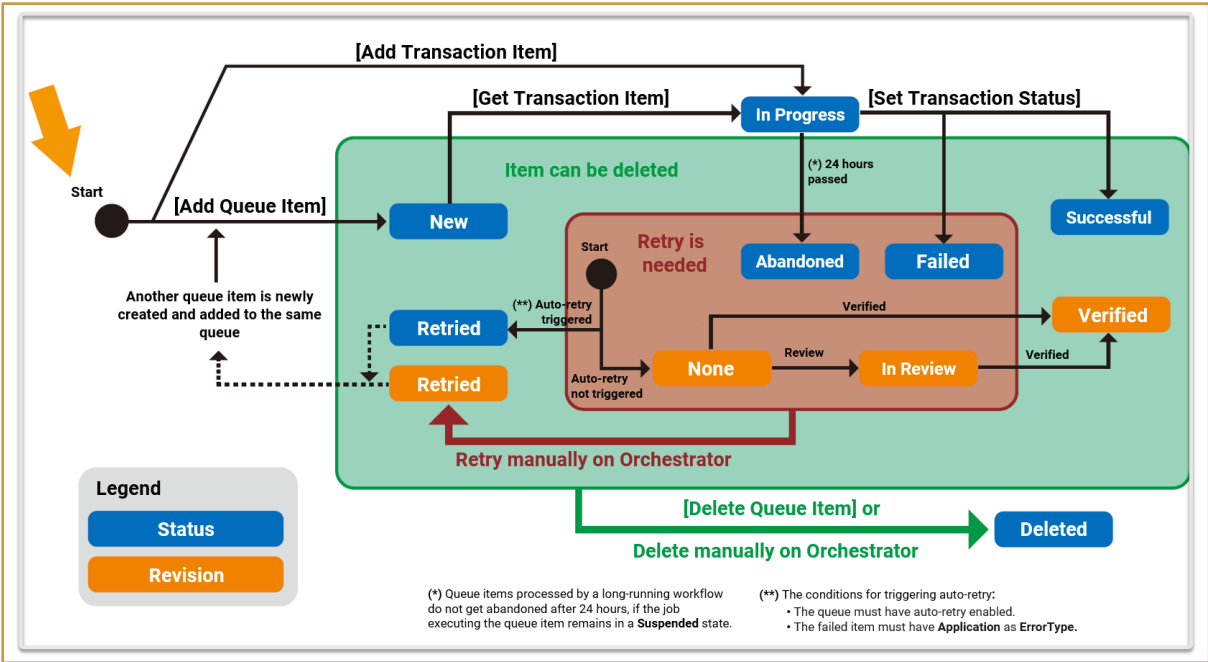
- UiPath Orchestrator의 Queue는 쉽게 말해서 '작업 대기열' 또는 '할 일 목록'이라고 생각하시면 됩니다. 은행의 번호표 시스템과 매우 유사한데, 손님이 오면 번호표를 뽑고 순서대로 창구에서 처리하는 것처럼 작업들이 Queue에 쌓이고 로봇들이 순서대로 처리하는 방식입니다.

Queue의 가장 큰 장점은 대량의 작업을 효율적으로 처리할 수 있다는 점입니다. 예를 들어, 엑셀에 1만 건의 데이터를 처리해야 한다면, 이를 Queue에 등록하고 여러 로봇이 나눠서 처리할 수 있습니다. 또한 모든 작업의 처리 상태를 실시간으로 확인할 수 있어 작업 추적이 매우 용이합니다.

실제 사용 사례를 보면, 인보이스 처리나 고객 주문 처리 같은 업무에서 많이 활용됩니다. 이메일로 받은 인보이스를 자동으로 Queue에 등록하고 여러 로봇이 동시에 처리할 수 있으며, 웹사이트로 들어온 고객 주문을 우선순위에 따라 처리하고 처리 상태를 고객에게 자동으로 알려줄 수 있습니다.



7.2. Queue Item 상태 종류



Step1. Item Status

상태	발생 조건	특징
New	• Add Queue Item 액티비티로 추가 • Item이 연기된 경우 • 마감시간이 추가된 경우	• 처리 대기 중인 초기 상태 • 로봇이 처리할 수 있는 상태
In Progress	• Get Transaction Item 사용 • Add Transaction Item 사용	• 현재 로봇이 처리 중인 상태 • 다른 로봇이 처리할 수 없음
Failed	• Set Transaction Status로 Failed 설정 • 비즈니스 룰 위반 • 응용 프로그램 요구사항 미충족	• 처리 실패 상태 • 자동 재시도 가능(설정된 경우)
Successful	• Set Transaction Status로 Successful 설정 • 모든 처리가 정상적으로 완료	• 최종 완료 상태 • 더 이상의 처리 불필요
Abandoned	• 24시간 동안 In Progress 상태 유지 • 처리되지 않은 경우	• 자동으로 변경됨 • 장기 실행 워크플로우의 경우 예외
Retried	• 응용 프로그램 예외 발생 • 자동 재시도 설정이 활성화된 경우	• 임시 상태 • Failed나 Successful로 최종 변경됨
Deleted	• Orchestrator에서 수동 삭제 • Delete Queue Item 사용	• 영구 삭제 상태 • 복구 불가능

Step2. Revision Statuses

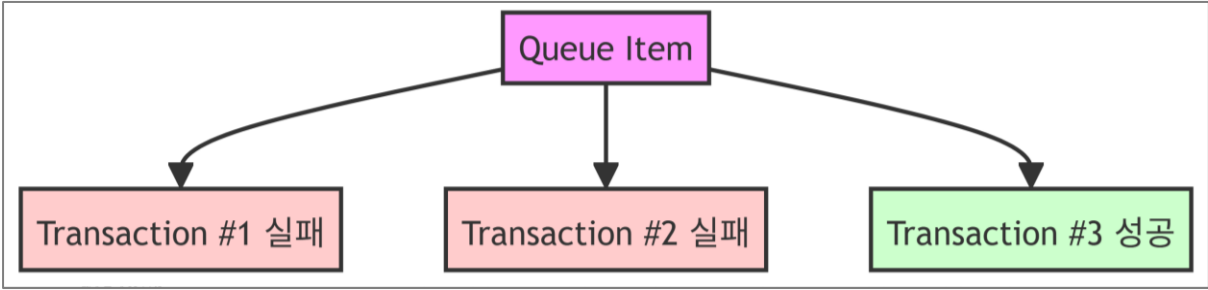
- Revision Status 는 Queue Item 중 실패(Failed)하거나 중단(Abandoned)된 항목에 대해서만 적용되는 특별한 검토 체계입니다. 지정된 검토자가 해당 항목을 수동으로 분석하고 상태를 변경하며, 이 모든 변경 사항은 Audit Details 의 History 에서 추적됩니다. 이를 통해 문제가 발생한 프로세스에 대한 품질 관리와 감사가 체계적으로 이루어지며, 불필요한 리소스 낭비를 방지할 수 있습니다.

Revision 상태	설명	특징	후속 처리
None	- 모든 Queue Item의 기본 상태 - 성공/실패 여부와 무관하게 설정	- 초기값 - 검토 프로세스 시작 전 상태	다른 Revision 상태로 변경 가능
In Review	- 응용 프로그램 예외로 실패한 항목 검토 중 - 검토자가 분석 진행 중	- 검토 진행 상태 표시 - 임시 상태	Verified 또는 Retried로 변경 가능
Verified	- 검토자가 확인 완료 - 추가 처리가 불필요한 상태	- 최종 검토 완료 상태 - 재시도 불가능	- 상태 변경 불가 - 프로세스 종료
Retried	- 수동으로 재시도 표시 - 새로운 Queue Item 생성	- 원본 항목의 재처리 요청 - 새 Queue Item 생성됨	- 새 Item은 'New' 상태로 생성 - 원본 항목은 이력으로 보관

7.3. 트랜잭션(Transaction) 정의

- 트랜잭션은 로봇이 Queue Item 을 처리하려는 시도를 의미합니다. 로봇이 Queue 에서 특정 항목을 가져와 처리를 시작하는 시점을 "트랜잭션 시작"이라고 합니다.

처리가 완료되면 해당 트랜잭션은 성공(Successful), 실패(Failed) 또는 중단(Abandoned) 등의 상태로 표시됩니다. Orchestrator 는 이러한 트랜잭션들을 실시간으로 모니터링하며, 대시보드를 통해 현재 상태와 처리 현황을 시각적으로 확인할 수 있습니다.



7.4. 트랜잭션(Transaction)의 예외사항 처리

- 트랜잭션 처리 중 발생하는 예외는 크게 "Business Exception"과 "Application Exception" 두 가지로 구분됩니다. 이러한 구분은 Queue Item 의 재처리 여부를 결정하는 중요한 기준이 됩니다.

7.4.1. 비즈니스 예외(Business Exception)

비즈니스 예외는 프로세스의 업무 규칙이나 데이터 검증 과정에서 발생하는 예외입니다.

로봇은 정상적으로 작동하지만, 처리하려는 데이터나 조건이 비즈니스 규칙에 부합하지 않을 때 발생합니다.

예시:

- 필수 입력 필드가 비어있는 경우
- 데이터 형식이 올바르지 않은 경우
- 비즈니스 로직에 위배되는 경우

중요한 특징은 이러한 예외는 자동 재시도 대상이 아니며, 담당자의 검토와 수정이 필요합니다.

7.4.2. 애플리케이션 예외(Application Exception)

애플리케이션 예외는 기술적인 문제로 인해 발생하는 예외입니다. 주로 시스템 환경이나 외부 요인에 의해 발생하며, 일시적인 문제일 가능성이 높습니다.

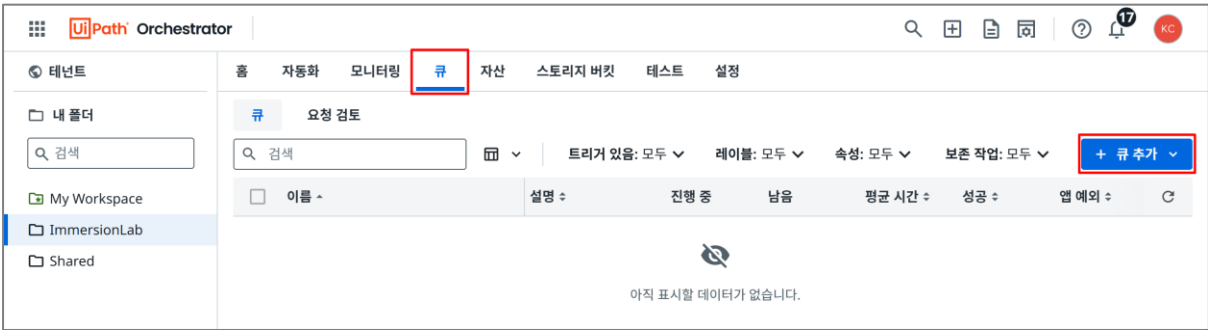
예시:

- 네트워크 연결 오류
- 시스템 응답 지연
- 애플리케이션 실행 오류
- 타임아웃 발생

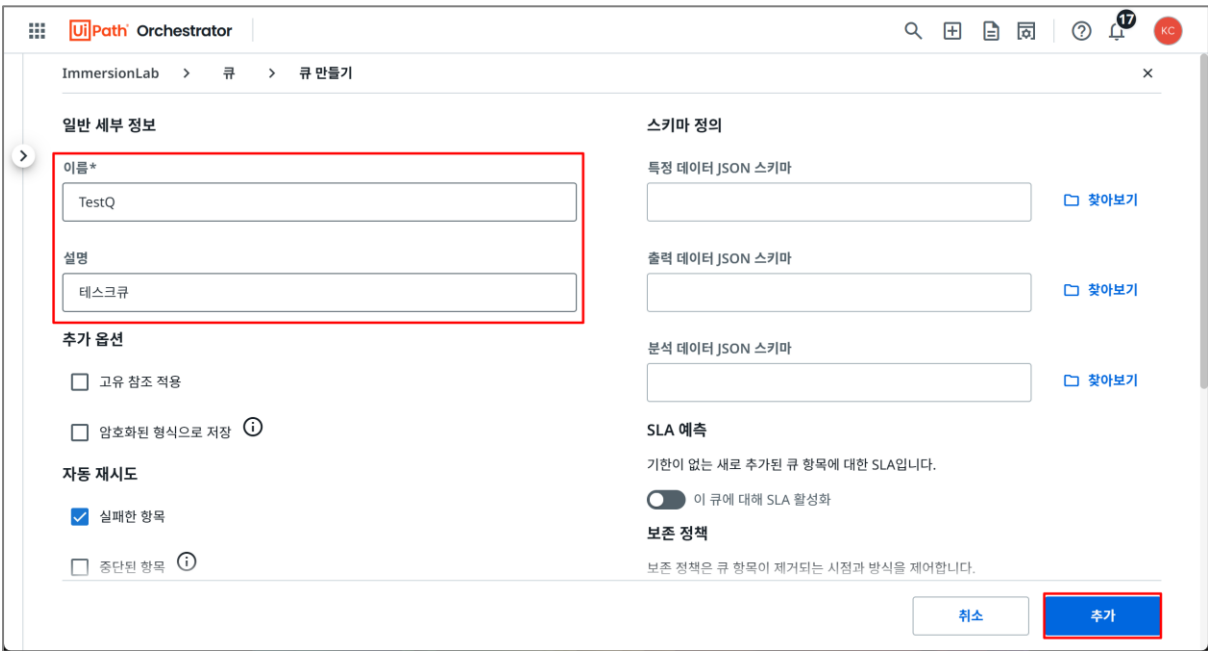
이러한 예외는 설정된 재시도 정책에 따라 자동으로 재처리될 수 있습니다. Orchestrator의 Retry Logic 설정에 따라 재시도 횟수와 간격을 조정할 수 있습니다.

7.5. Orchestrator Queue 생성

Step1. 생성한 폴더 선택 후 큐 메뉴 접근 → “+ 큐 추가” – ‘새 큐 만들기’ 클릭



Step2. 이름, 설명을 작성한 다음 추가 버튼을 클릭합니다.



Step3. 생성한 Queue 확인



7.6. UiPath 에서 Orchestrator 로 Queue 큐 전달

Step1. Add Queue Item 액티비티 내용 입력 후 실행

Add Queue Item

오케스트레이터 폴더 경로

ImmersionLab

×

+

큐 이름 *

TestQ

×

+

Item Information

사전의 1개 항목

Refresh arguments

Orchestrator 폴더 경로

: Orchestrator 서버에 있는

폴더 경로를 선택합니다.

큐 이름

: Orchestrator 서버에 있는

큐의 이름을 선택합니다.

Item Information

: Queue에 추가할 내용을 적습니다. String일 경우 ""안에 내용을 넣어주면 됩니다.

사전

Add Queue Item > Item Information (Dictionary<String,InArgument>)

추가할 특정 QueueItem에 대한 추가 정보의 컬렉션입니다. 정보는 항목에 저장되고 트랜잭션 처리 중에 사용됩니

다.

키

형식

값

이름

String

{ } "최효준"

항목 추가

저장

취소

Step2. 큐 메뉴 접근 → 추가 액션 – 트랜잭션 보기 클릭

홈

자동화

모니터링

큐

자산

스토리지 버킷

테스트

설정

큐

요청 검토

검색

트리거 있음: 모두

레이블: 모두

속성: 모두

보존 작업: 모두

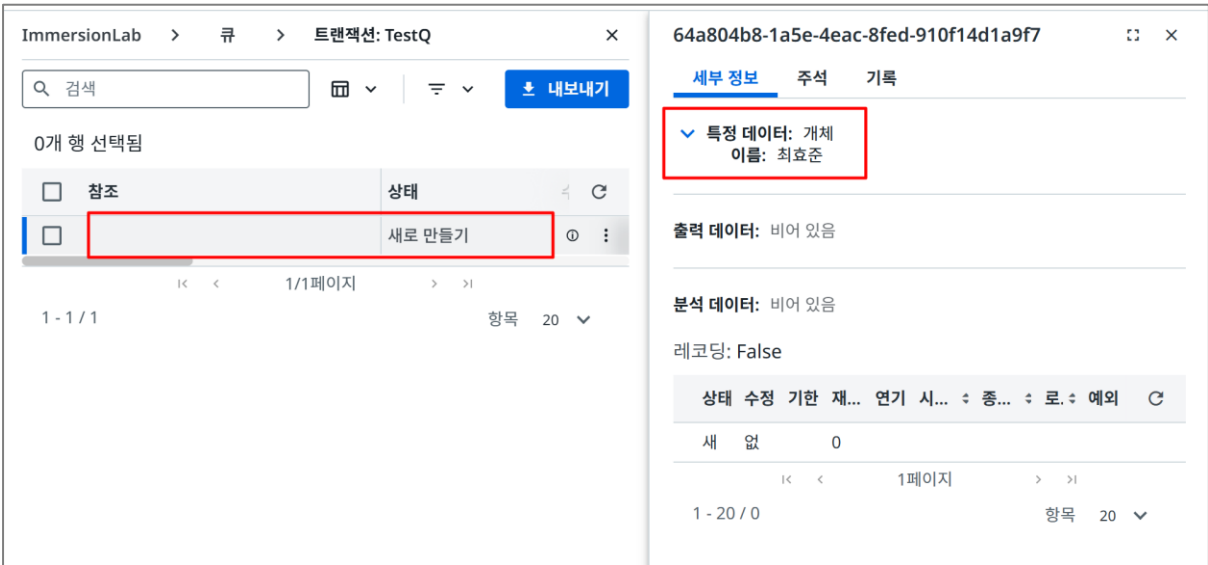
+ 큐 추가

☐	이름 ^	설명	진행 중	남음	평균 시간	성공	앱 예외	
☐	TestQ	테스크큐	0	1	0	0	0	⋮ 편집 트리거 추가 트랜잭션 보기 큐 모니터링 차트 보기 항목 업로드

1 - 1 / 1

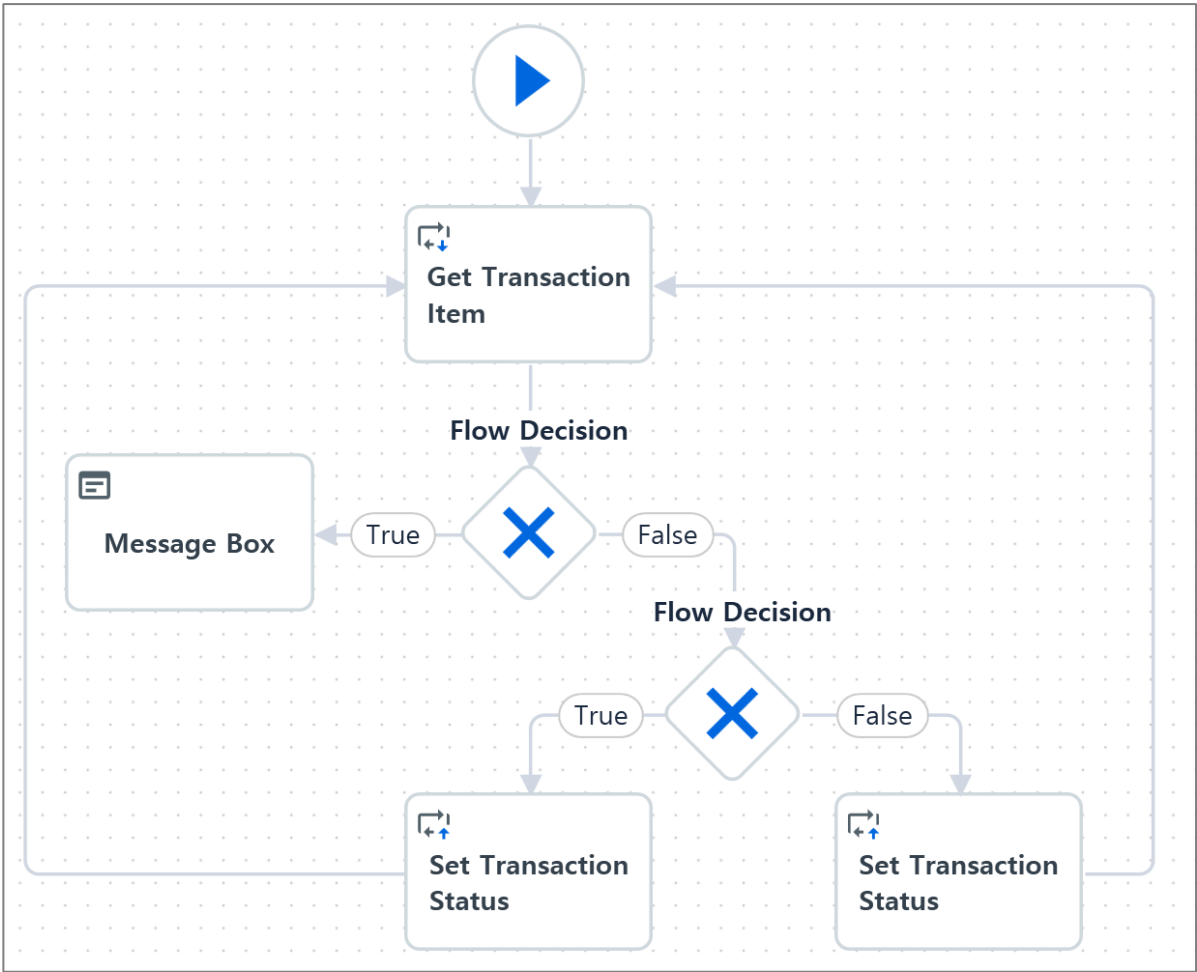
1/1페이지

Step3. 트랜잭션 화면에서 참조 내 빈 공간 클릭 → 세부 정보에 저장된 값 확인



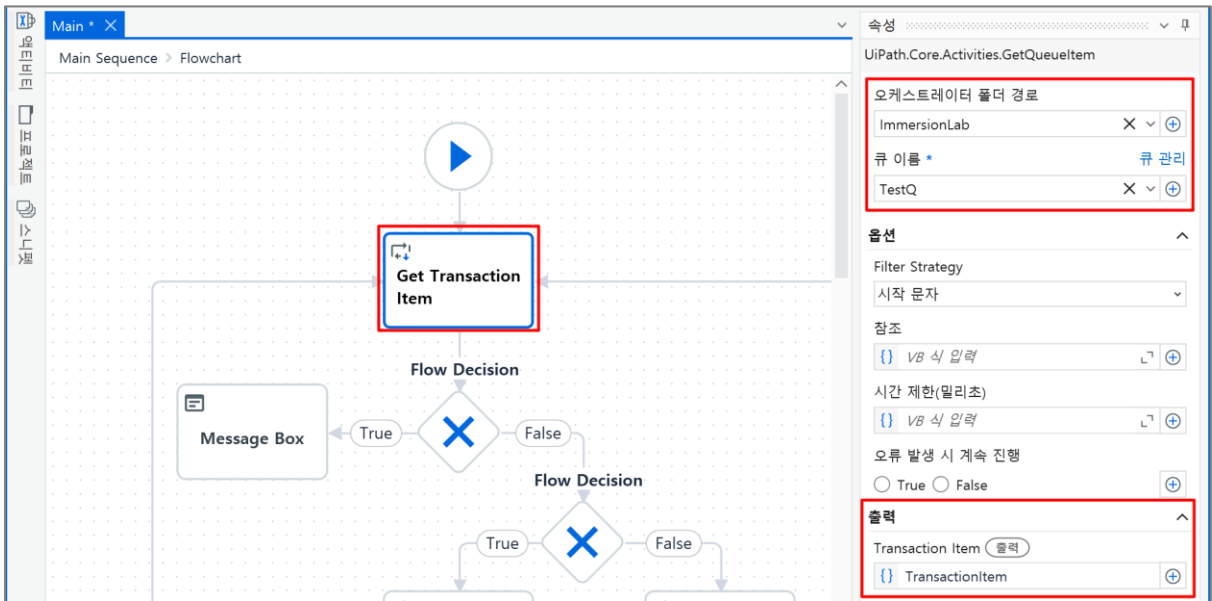
7.7. UiPath 에서 Transaction Item 호출

Step1. Get Transaction Item, Set Transaction Status 액티비티 드래그 앤 드롭



Step2. Get Transaction Item 내용 입력

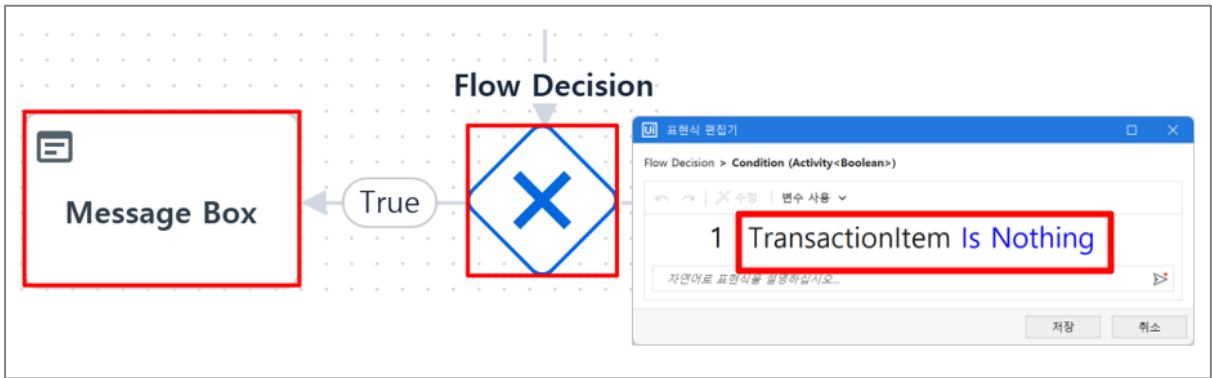
- 오케스트레이터 폴더 경로 및 큐 이름 작성
- 출력 - Transaction Item 변수 입력



Step3. 첫 번째 Flow Decision 조건 입력

:Transaction Item 이 비어있는 경우, 프로세스를 종료시키기 위한 작업

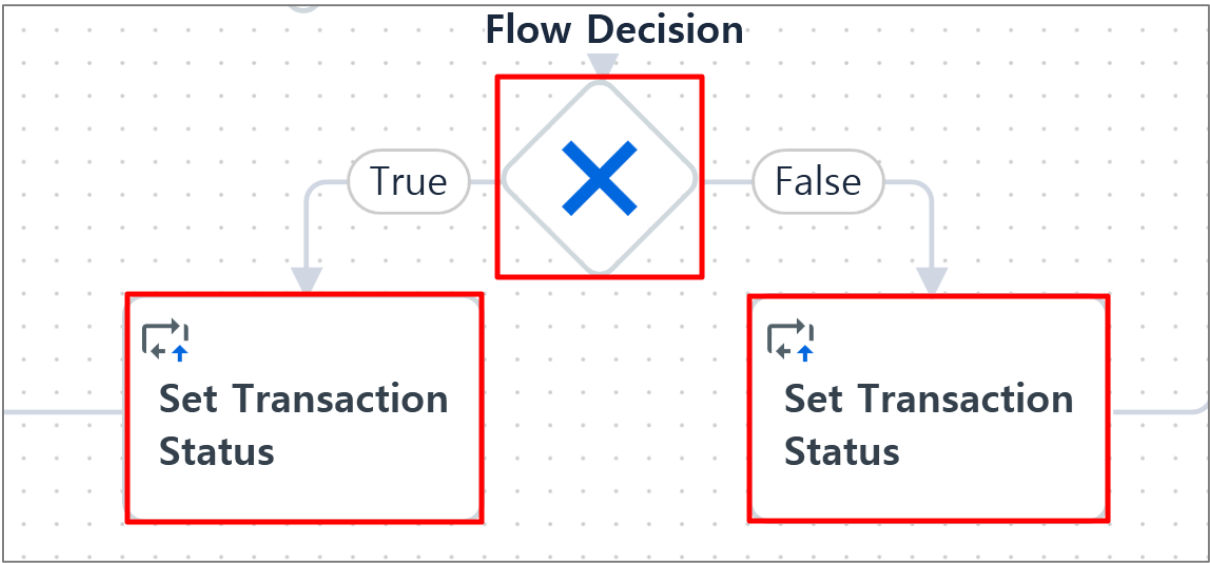
- Flow Decision: TransactionItem Is Nothing
- True 방향에 Message Box: TransactionItem Is Nothing 입력



Step4. 두 번째 Flow Decision 조건 입력

:Transaction Item 의 이름이 지정한 이름과 동일한 지 확인

- Flow Decision: TransactionItem.SpecificContent("이름").ToString = "이름"
- True 방향과 False 방향에 모두 Set Transaction Status 액티비티 위치
- 다음 Transaction Item 도 수행해야하기 때문에 화살표를 Get Transaction Item 으로 연결



- True 일 경우

Set Transaction Status

오케스트레이터 폴더 경로

ImmersionLab

X

+

Transaction Item *

{ TransactionItem

L

+

상태 *

성공

▼

- False 일 경우

Set Transaction Status

오케스트레이터 폴더 경로

ImmersionLab

X

+

Transaction Item *

{ TransactionItem

L

+

상태 *

실패함

▼

이유 *

{ "TransactionItem 이름이 다릅니다."

L

+

Step5. Orchestrator 서버에서 값 확인

홈

자동화

모니터링

큐

자산

스토리지 버킷

테스트

설정

큐

요청 검토

Q 검색

트리거 있음: 모두

레이블: 모두

속성: 모두

보존 작업: 모두

+ 큐 추가

☐	이름	설명	진행 중	남음	평균 시간	성공	앱 예외	비즈니스 ...	프로세스
☐	TestQ	테스크큐	0	0	1초	7	0	0	

ImmersionLab > 큐 > 트랜잭션: TestQ

Q 검색

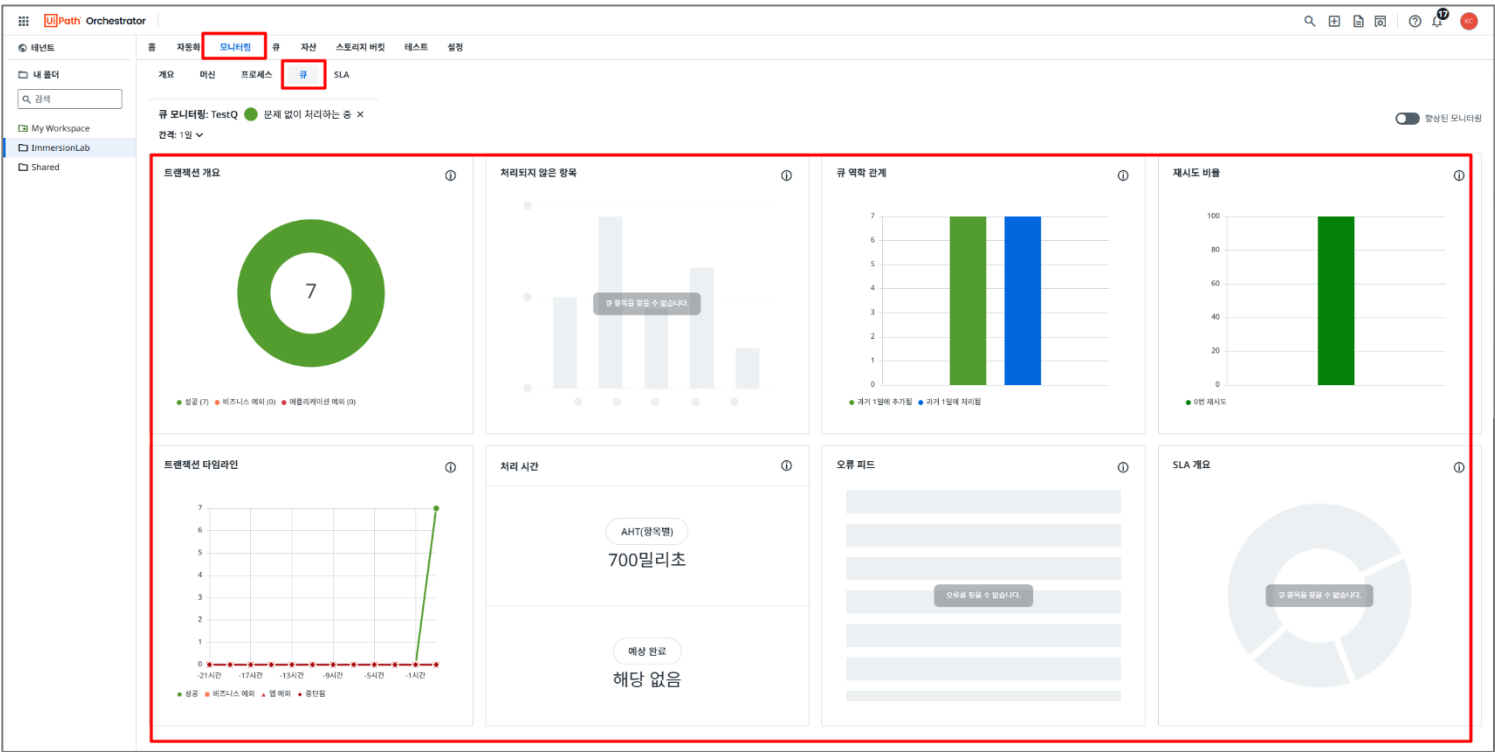
내보내기

0개 행 선택됨

☐ 참조	상태	수정	우선 순위	기한	연기	시작됨	
☐	성공	없음	일반			25분 전	
☐	성공	없음	일반			25분 전	
☐	성공	없음	일반			27분 전	
☐	성공	없음	일반			28분 전	
☐	성공	없음	일반			37분 전	
☐	성공	없음	일반			37분	ⓘ ⋮
☐	성공	없음	일반			38분 전	

Step6. 대시보드 확인

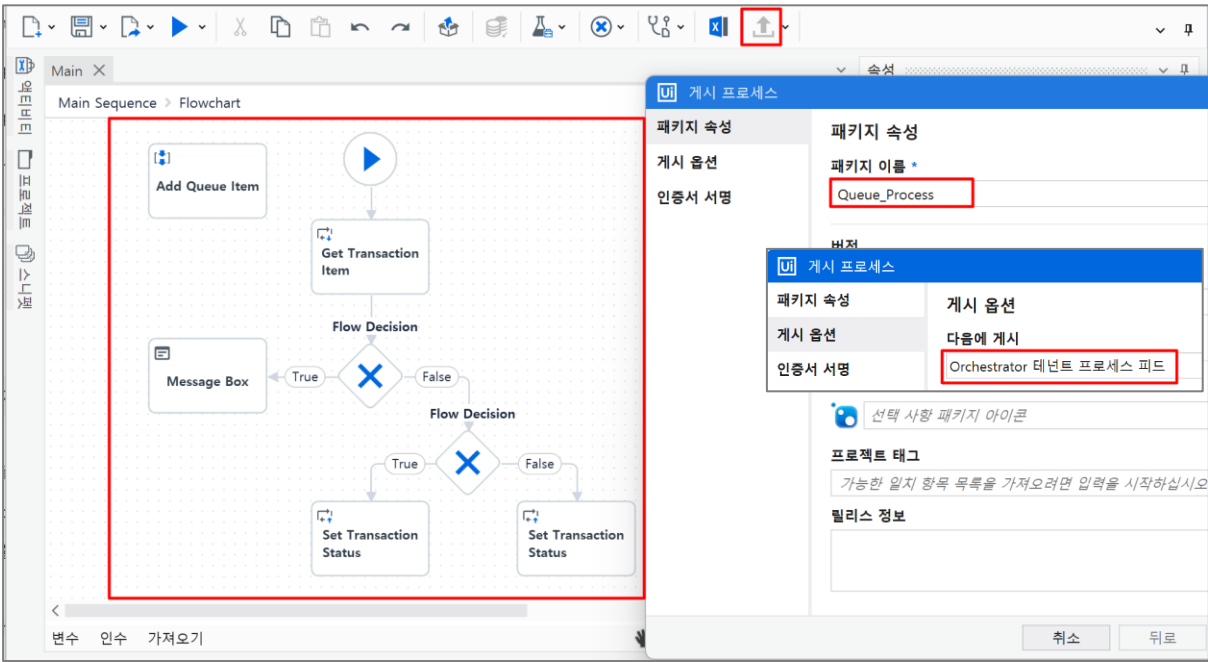
- 모니터링 - 큐에서 확인 가능



7.8. 큐 트리거 만들기

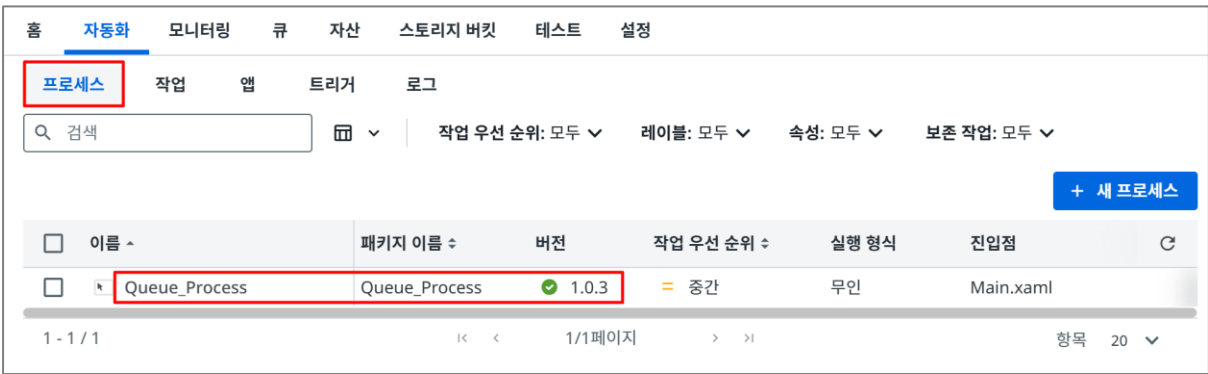
Step1. 프로세스 게시

- 기존 프로세스에서 Set Transaction Status 에서 나가는 선 삭제 후, 테넌트에 게시



Step2. Queue_Process 프로세스 게시

- 폴더 - 자동화 - 프로세스에서 게시한 패키지 프로세스로 만들기



Step3. 큐 트리거 생성

- 자동화 - 트리거 - 큐 트리거 - 새 트리거 추가 버튼 클릭

홈

자동화

모니터링큐자산스토리지 버킷테스트설정

프로세스작업앱트리거로그

시간 트리거

큐 트리거

이벤트 트리거API 트리거

Q 검색

상태: 모두작업 우선 순위: 모두

+ 새 트리거 추가

- 큐 트리거 이름과 수행할 프로세스 선택

- 작업할 큐 선택

- 첫 번째 작업을 트리거할 최소 항목 수: 트리거가 작동하기 위한 "최소" 큐 항목 수 -
동시에 적용되는 최대 보류 및 실행 작업 수: 동시에 실행될 수 있는 "최대" 작업 수

이름*

Queue_Process

프로세스 이름*

Queue_Process

작업 우선 순위*

상속됨

Runtime 유형*

Production (Unattended)

실행 타겟

모든 사용자/로봇 계정

머신

큐*

TestQ

첫 번째 작업을 트리거할 최소 항목 수 5 항목

동시에 허용되는 최대 보류 및 실행 작업 수입니다. 3 작업

각각에 대해 다른 작업이 트리거됩니다. 1 새 항목

☐ 작업 완료 후 조건을 재평가하고 가능한 경우 새 작업을 시작합니다.

표준 시간대*

(UTC+09:00) 서울

휴무일 제한 사항

일정이 선택되지 않았습니다.

취소

업데이트

- 모두 설정 후 업데이트 버튼 클릭

시간 트리거

큐 트리거

이벤트 트리거

API 트리거

Q

검색

▼

상태: 모두

▼

작업 우선 순위: 모두

▼

+ 새 트리거 추가

☐	이름	프로세스	트리거 세부 정보	작업 우선 순위	다음 런타임	다음 이후 중지	
☐	Queue_Process	Queue_Process	TestQ	상속됨	해당 없음	해당 없음	

1 - 1 / 1

1/1 페이지

항목 20

▼

SESSION5. REFRAMEWORK

학습 목표

- 각 워크플로우 형태들을 설명할 수 있다.
- REFramework 의 구성 단계를 이해할 수 있다.
- Transaction 또는 DataRow 를 통해 REFramework 에서 데이터 처리를 할 수 있다.

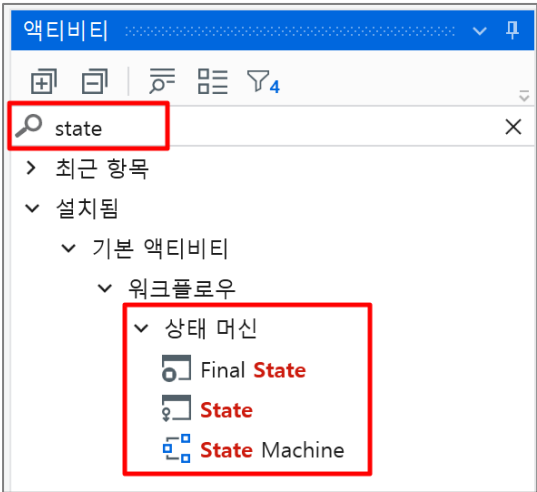
1. 비즈니스 프로세스 형태

UiPath 의 워크플로우는 단순한 순차 실행이 특징인 **Sequence**, 순서도 형태로 분기 처리가 쉬운 **Flowchart**, 그리고 상태 기반의 복잡한 프로세스를 다루는 **State Machine** 으로 나뉘며, 프로젝트의 규모와 복잡도에 따라 적절한 워크플로우를 선택하여 사용하면 됩니다.

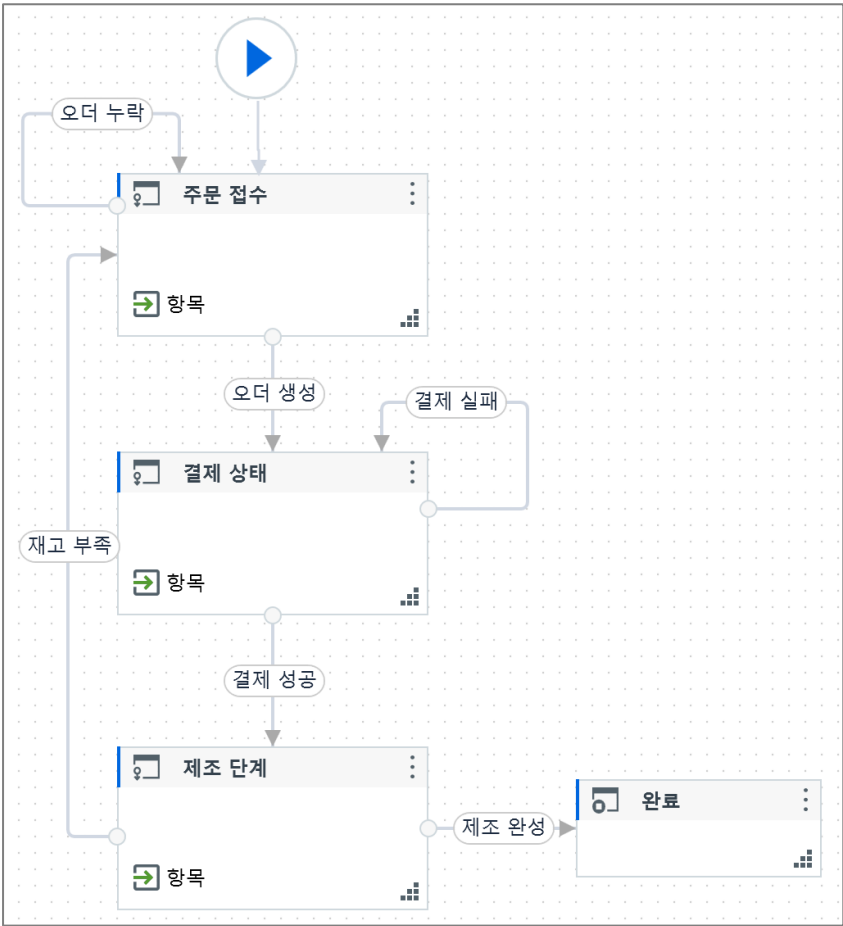
항목	Sequence	Flowchart	State Machine
정의	순차적으로 실행되는 선형 워크플로우	순서도 형태의 시각적 워크플로우	상태 기반의 복잡한 워크플로우
주요 특징	- 위에서 아래로 실행 - 단순한 구조 - 직관적인 개발 - 적은 메모리 사용	- 시각적 프로그래밍 - 유연한 흐름 제어 - 분기 처리 용이 - 디버깅 편리	- 상태 기반 로직 - 복잡한 프로세스 관리 - 이벤트 기반 전환 - 글로벌 예외 처리
사용 사례	- 단순한 자동화 작업 - 작은 규모의 프로젝트 - 재사용 가능한 컴포넌트	- 다중 분기 처리 - 반복적인 작업 - 중간 규모 프로젝트	- 대규모 프로젝트 - 트랜잭션 처리 - 복잡한 비즈니스 로직
장점	- 빠른 개발 - 쉬운 유지보수 - 명확한 실행 흐름	- 시각적 명확성 - 유연한 흐름 제어 - 직관적인 디버깅	- 강력한 예외 처리 - 상태 관리 용이 - 확장성
단점	- 복잡한 로직 구현 어려움 - 제한된 흐름 제어 - 대규모 프로젝트에 부적합	- 큰 프로세스인 경우 복잡 - 메모리 사용량 증가 - 코드 재사용 어려움	- 높은 학습 곡선 - 복잡한 설정 - 작은 프로젝트에 과도

1.1. State Machine 액티비티 살펴보기

- 기존 워크플로우를 만들 때는 액티비티를 Sequence 나 Flowchart 에 직접 생성했지만, State Machine 은 전용 워크플로우가 있습니다. 각 상태(State)는 자신만의 워크플로우를 가지고 있어 복잡한 비즈니스 로직을 모듈화하고 관리하기 용이한 구조를 제공합니다.

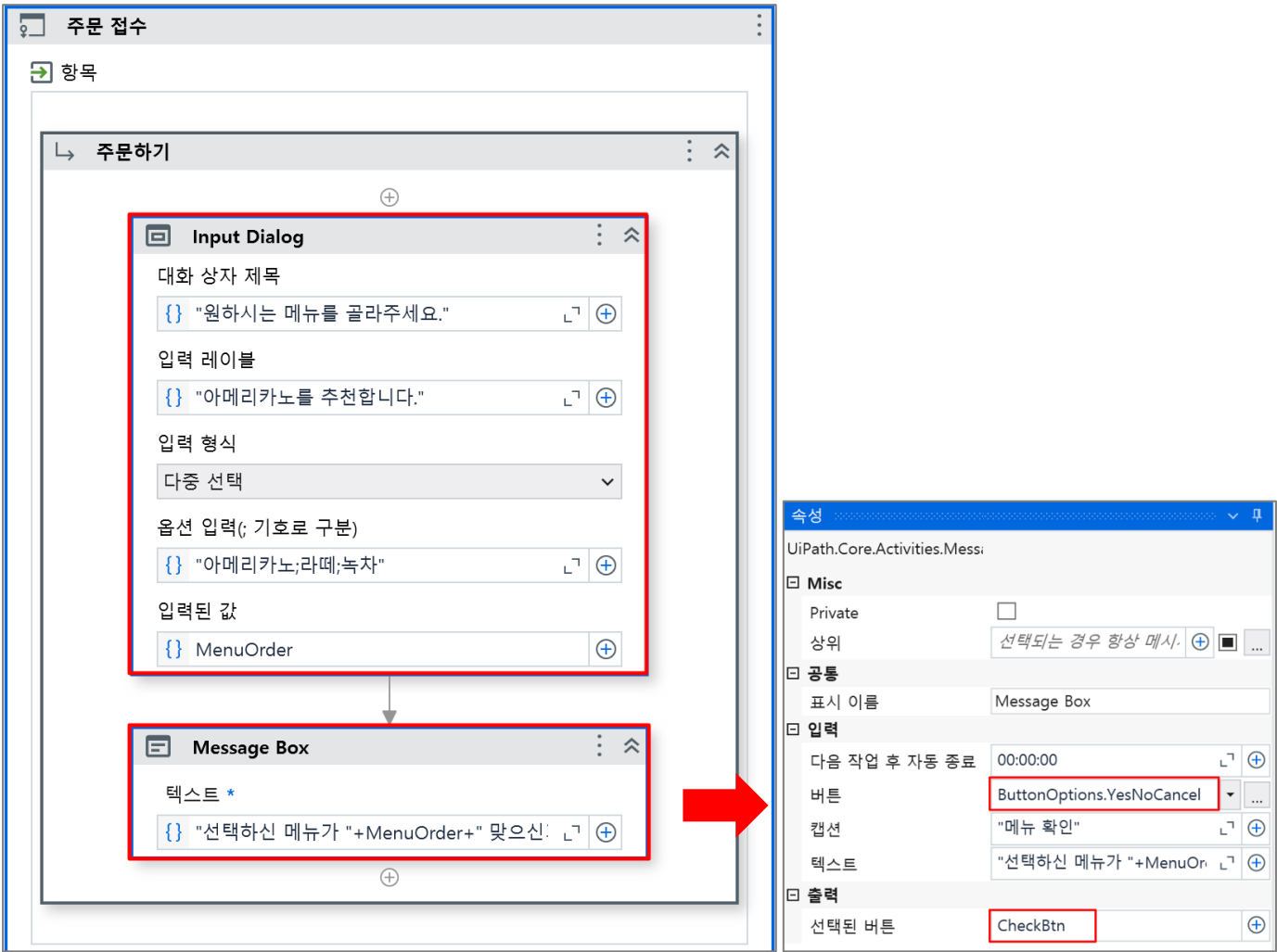


1.2. State Machine 프로세스 만들어보기



Step1. 주문 접수 State 만들기

- Input Dialog 와 Message Box 액티비티 드래그 앤 드롭 후, 양식 채우기
- Message Box 속성: 버튼(YesNoCancel), 출력 변수: CheckBtn 생성



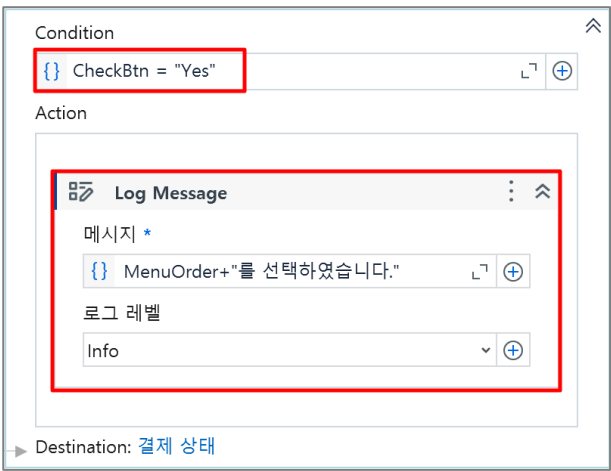
Step2. CheckBtn 출력 값에 따라 State 가 바뀔 수 있도록 수정

- CheckBtn = “Yes”인 경우, 주문이 올바르게 접수되어 결제 상태 State 로 이동



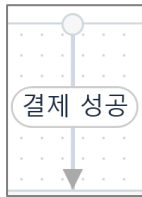
오더 생성 연결 선 더블 클릭 후,
Condition 과 Action 내용 입력

- CheckBtn = “No”, “Cancel”인 경우,
주문이 잘못 접수되어 다시 주문 접수
State 로 이동



Step4. CheckBtn 출력 값에 따라 State 가 바뀔 수 있도록 수정

- CheckBtn = "Yes"인 경우, 결제가 완료되어 제조 단계 State 로 이동



결제 성공 연결 선 더블 클릭 후,
Condition 과 Action 내용 입력

The screenshot shows the configuration window for a BPMN event. The 'Condition' field is set to `{ } CheckBtn = "Yes"`. The 'Action' section contains a 'Log Message' task with the message `{ } PayStatus+"를 선택하였습니다."` and the log level set to 'Info'. The 'Destination' is set to '제조 단계' (Manufacturing Stage).

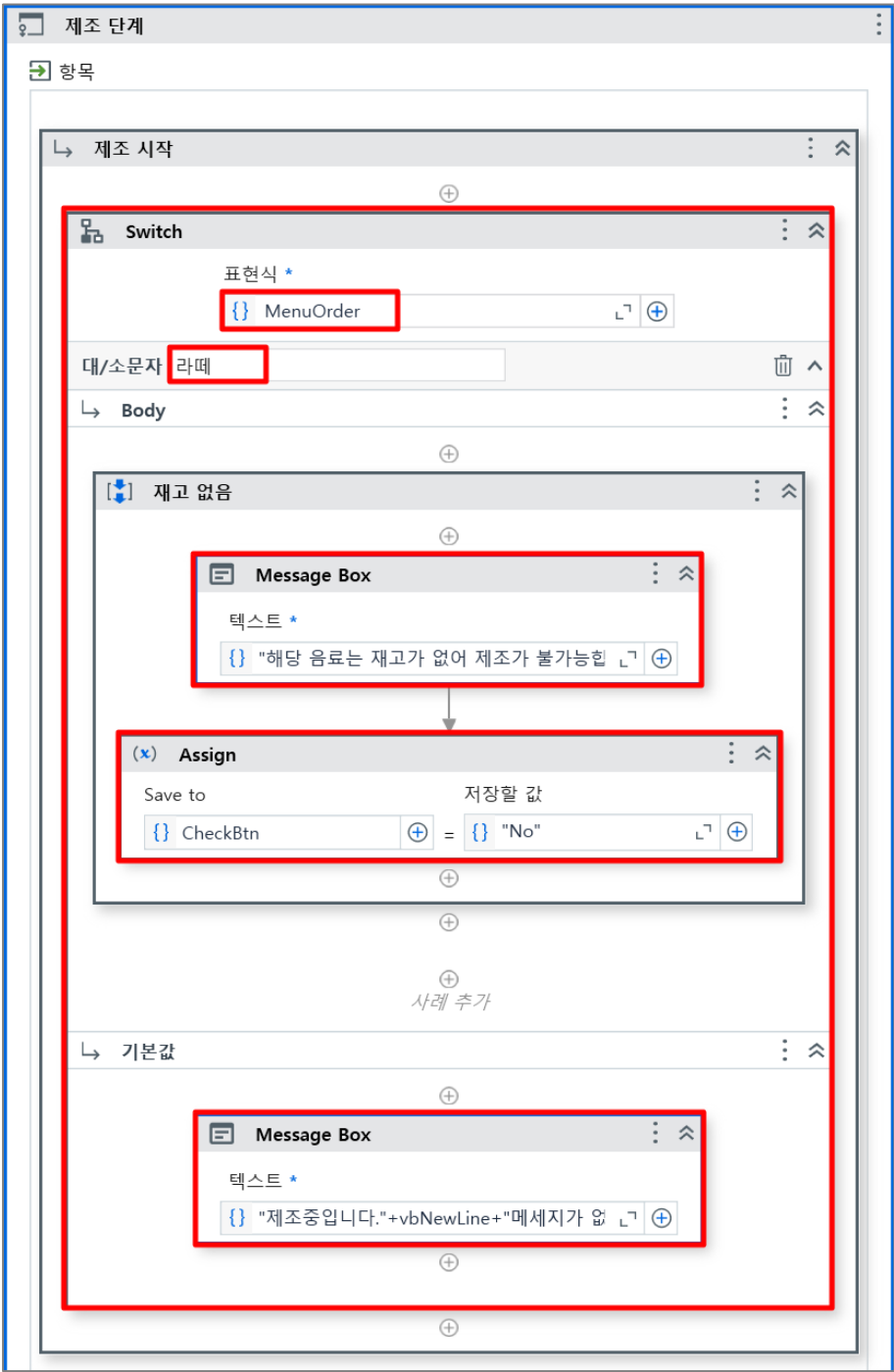
- CheckBtn = "No", "Cancel"인 경우, 결제가 실패해 다시 결제 상태 State 로 이동



결제 실패 연결 선 더블 클릭 후,
Condition 과 Action 내용 입력

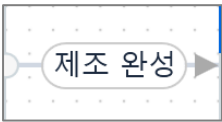
The screenshot shows the configuration window for a BPMN event. The 'Condition' field is set to `{ } CheckBtn = "No" or CheckBtn = "Cancel"`. The 'Action' section contains a 'Log Message' task with the message `{ } "선택을 취소합니다. 다시 주문창으로 넘"` and the log level set to 'Info'. The 'Destination' is set to '결제 상태' (Payment State).

Step5. 제조 단계 State 만들기



Step6. CheckBtn 출력 값에 따라 State 가 바뀔 수 있도록 수정

- CheckBtn = “Yes”인 경우, 제조가 완료되어 완료 Final State 로 이동



결제 성공 연결 선 더블 클릭 후,
Condition 과 Action 내용 입력

Condition

{ } CheckBtn = "Ok"

Action

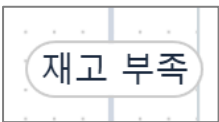
Message Box

텍스트 *

{ } "음료 제조가 완료되었습니다."

Destination: 완료

- CheckBtn = “No”, “Cancel”인 경우, 제조가 실패해 다시 주문 접수 State 로 이동



결제 실패 연결 선 더블 클릭 후,
Condition 과 Action 내용 입력

Condition

{ } CheckBtn = "No" or CheckBtn = "Cancel"

Action

Log Message

메시지 *

{ } "재고가 부족합니다. 다시 주문창으로 넘(

로그 레벨

Info

Destination: 주문 접수

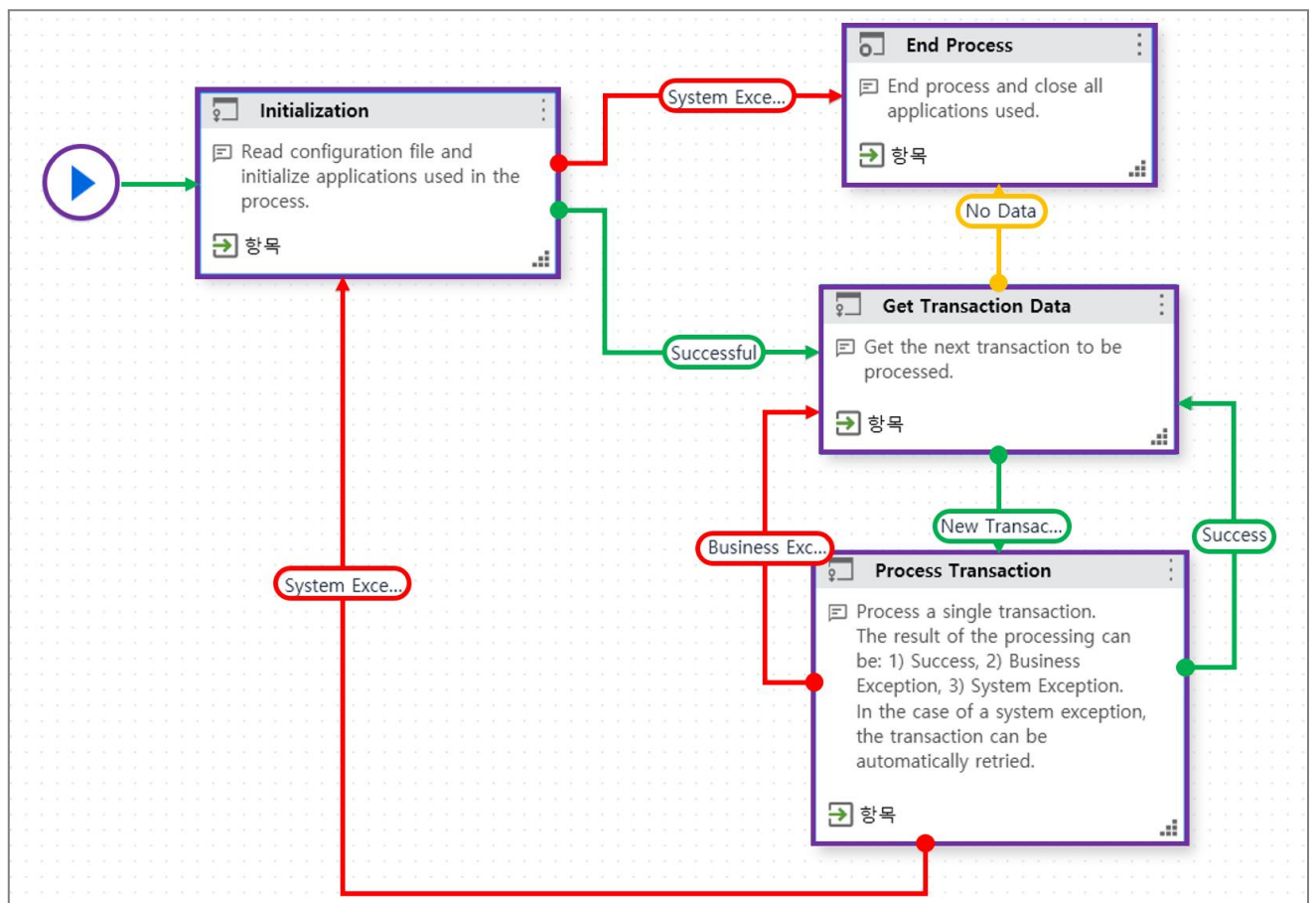
2. Dispatcher 와 Performer

- UiPath 내에서 Queue 기반 자동화를 구현함에 있어 중요한 개념으로 Dispatcher(디스패처)는 작업을 수집하고 Queue 에 등록하는 프로세스, Performer(퍼포머)는 Queue 에서 항목을 가져와 실제 처리를 수행하는 프로세스입니다. 또한 UiPath 에서는 Producer/Consumer 패턴이라는 용어도 사용하는데, 이는 Dispatcher/Performer 와 동일한 개념을 설명하는 또 다른 용어로 일반적인 소프트웨어 개발 패턴에서 사용되는 용어이며, 더 넓은 의미로 사용됩니다.

이제부터 설명할 Robotic Enterprise Framework(REFramework)는 UiPath 에서 Queue 기반의 업무처리를 위해 제공하는 표준 프레임워크입니다.

3. REFramework 의 정의

- Robotic Enterprise Framework (REFramework)는 UiPath 에서 제공하는 고급 템플릿입니다. 이 프레임워크는 대규모 업무 프로세스 자동화(Robotic Process Automation, RPA) 프로젝트를 구성하는 데 도움이 되도록 설계되었습니다.



4. REFramework 의 주요 이점

- REFramework 는 다양한 종류의 RPA 프로젝트에 사용할 수 있습니다. 복잡한 로직, 여러 시스템 간의 상호 작용, 큰 규모의 데이터 처리 등의 경우에 특히 유용합니다.

효율성	REFramework 는 프로세스의 각 부분을 세분화하여 프로세스의 실행을 효율적으로 만들어줍니다.
재사용성	재사용 가능한 컴포넌트를 통해 빠른 프로토타이핑과 쉬운 유지 관리를 가능하게 합니다.
표준화	공통의 프로세스 구조를 제공함으로써, 코드의 이해도를 높입니다.
예러 처리	내장된 예러 처리 메커니즘을 통해 프로세스에서 발생할 수 있는 예외 상황을 관리합니다.
로깅	프로세스의 각 단계에서 로깅을 수행함으로써, 프로세스의 문제점을 식별하고 개선합니다.

5. REFramework 의 구조 및 작동 원리

5.1. REFramework State 구성

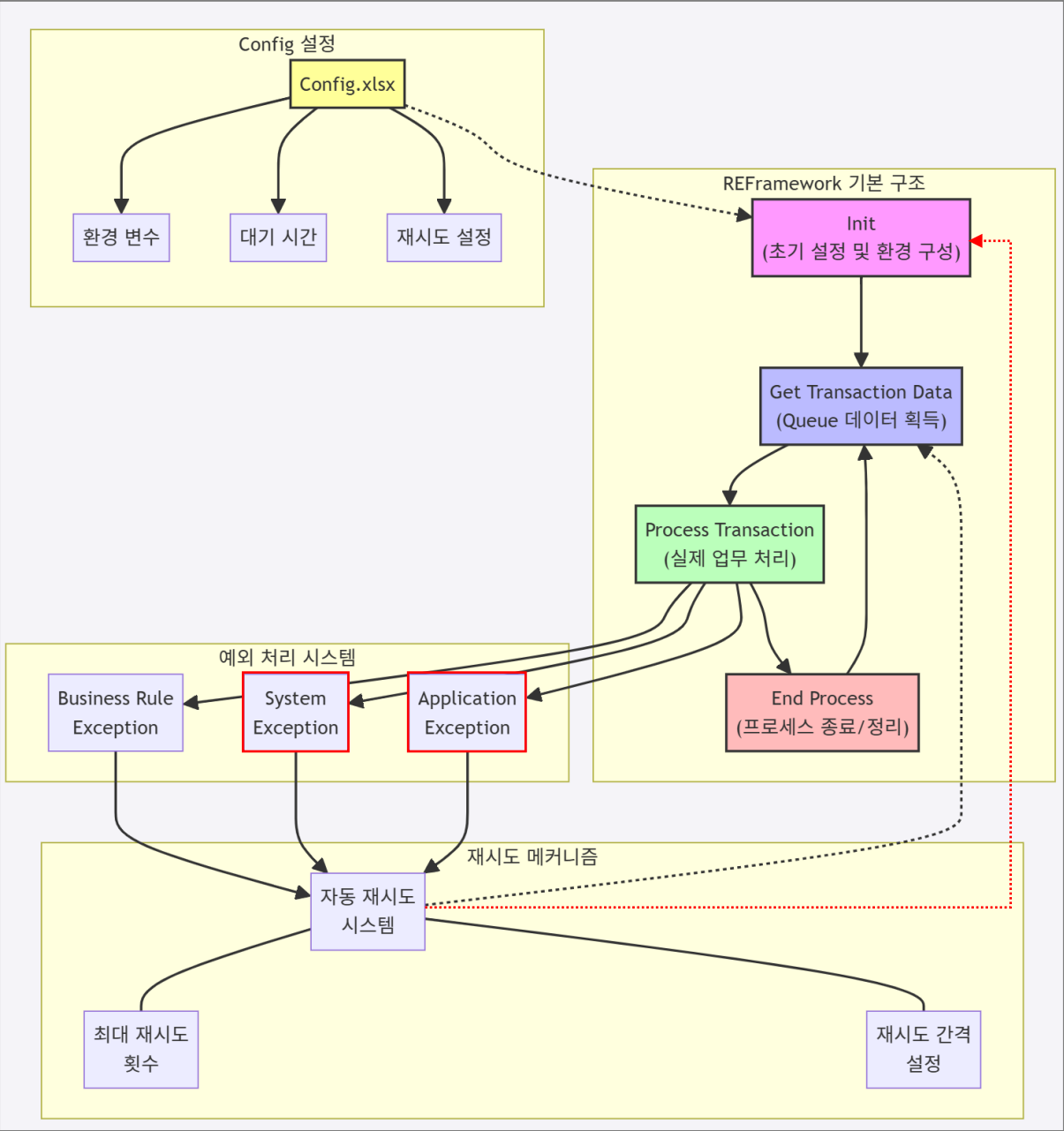
- REFramework 는 여러 개의 State'로 구성되며, 각 State'는 특정 작업을 수행합니다.

주요 State'는 다음과 같습니다:

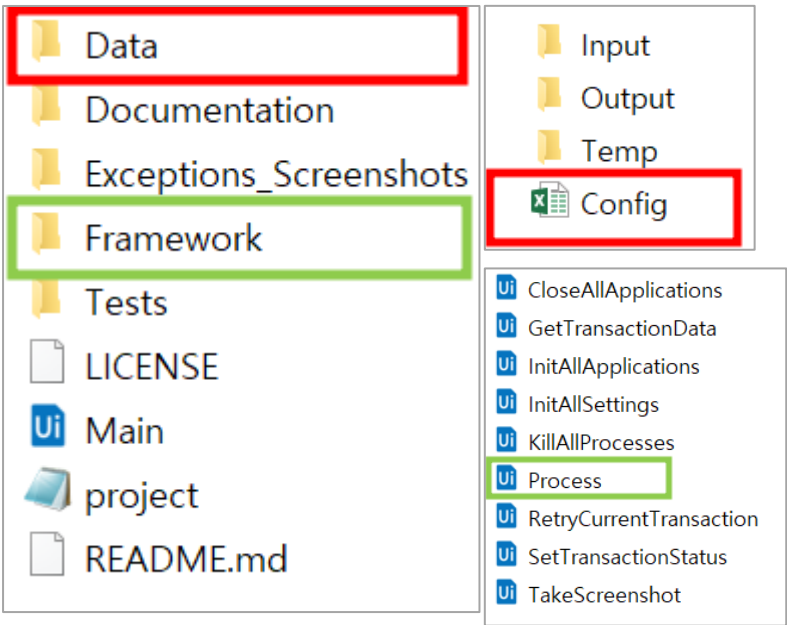
Initialization (Init)	프로세스 초기화를 위한 작업을 수행합니다. 이는 애플리케이션의 시작, 데이터 로드 등을 포함합니다.
Get Transaction Data	- 처리할 데이터(트랜잭션)를 가져온 다음, 데이터의 적합성을 확인합니다. - 트랜잭션 처리시 Queue/Non-Queue 방식 선택가능
Process Transaction	가져온 데이터를 처리합니다.
End Process	프로세스를 종료하고, 필요한 경우 정리 작업을 수행합니다.

이들 State'는 특정 조건에 따라 서로 전환됩니다. 예를 들어, 'Init' 상태에서 모든 초기화 작업이 성공적으로 완료되면 'Get Transaction Data' 상태로 전환되며, 처리할 데이터가 있는 경우에는 Process Transaction, 처리할 데이터가 없는 경우 'End Process' 상태로 전환됩니다.

5.2. REFramework Structure Diagram



5.2. REFramework 폴더 트리

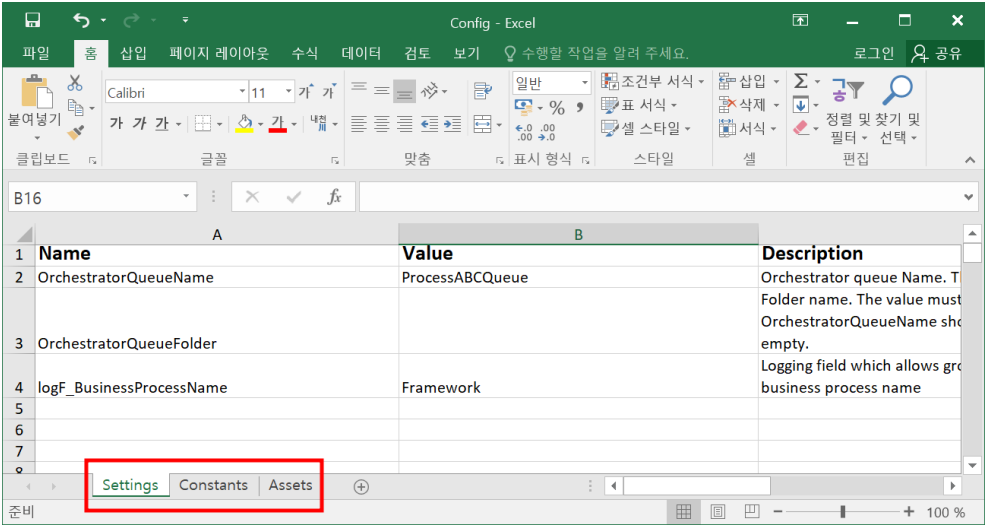


- Data 폴더: Config.xlsx 및 산출물을 관리
- Framework 폴더: 설계 목적에 따라 구성된 xaml 파일

Initialization(Init)	InitAllSettings	프로세스 실행에 필요한 설정을 초기화합니다.
	KillAllProcesses	입력된 모든 프로세스를 종료합니다.
	InitAllApplications	프로세스에 필요한 모든 애플리케이션을 초기화합니다.
Get Transaction Data	GetTransactionData	처리할 데이터 항목을 가져옵니다.
Process Transaction	Process	개별 데이터 항목을 처리하는 로직을 구현합니다.
	SetTransactionStatus	각 트랜잭션의 상태를 업데이트하고 오류를 기록합니다.
	RetryCurrentTransaction	현재 트랜잭션에 대한 재시도 로직을 구현합니다.
	TakeScreenshot	예외 발생 시 스크린샷을 찍어 오류 분석에 사용합니다.
	CloseAllApplications	프로세스가 끝날 때 모든 애플리케이션을 닫습니다.
	KillAllProcesses	입력된 모든 프로세스를 종료합니다.
End Process	CloseAllApplications	프로세스가 끝날 때 모든 애플리케이션을 닫습니다.

5.3. Config.xlsx 구성

- 프로젝트에서 자주 변경되는 정보를 관리하기 위해 사용(유지보수)
- Dictionary 변수 유형으로 관리



Setting	환경설정 내용들을 적는다. (URL, 폴더 경로, Queue 이름 등)
Constants	고정값을 적는다. (변하지 않는 코드, 회사 이름 등)
Assets	Orchestrator 에서 정의한 Asset Name 을 적는다.

5.4. 공통변수

이름	변수타입	설명
TransactionItem	QueueItem	현재 처리 중인 단일 트랜잭션 항목을 나타냅니다.
SystemException	Exception	시스템 수준의 예외가 발생했을 때 해당 예외 저장
BusinessException	BusinessRuleException	비즈니스 규칙 예외가 발생했을 때 해당 예외 저장
TransactionNumber	Int32	트랜잭션 항목의 개수
Config	Dictionary(Of String, Object)	프로세스 환경설정 데이터
RetryNumber	Int32	시스템 예외가 발생한 경우 트랜잭션 프로세스를 다시 시도하는 횟수
TransactionField1, 2	String	트랜잭션 항목에 추가적인 정보
TransactionID	String	정보 및 로깅(Logging) 목적으로 사용
dt_TransactionData	DataTable	트랜잭션이 데이터테이블에 저장될 때 사용

6. REFramework 의 세부 구조

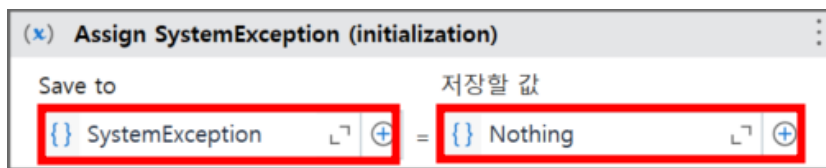
6.1. Initialization(Init)

- 프로세스를 실행하기 전 Config 정보 추출 및 데이터 전처리, 웹사이트 접근 등 프로세스 초기화를 진행하는 State 입니다.

Step1. System Exception 초기화

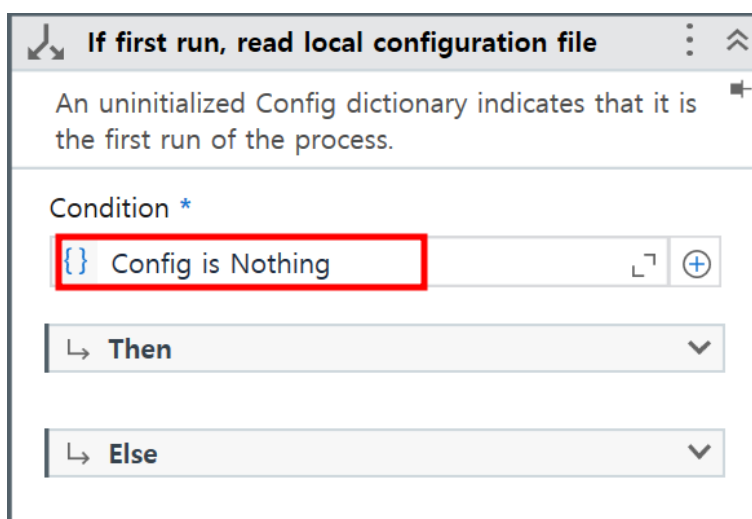
- System Exception 은 응용 프로그램이 정상적으로 작동하지 않거나, 파일이 없는 경우에 시스템 예외가 발생할 수 있습니다.

- 'Init' State 는 'Get Transaction Data'로 가기 전 System Exception 이 발생했는지 확인하는 작업을 거치며, 만약 SystemException' 변수에 다른 값이 들어가 있다면 재수행을 하지 않고 'End' State 로 이동하기 때문에, 처음 프로세스 실행 시 항상 초기화 하는 작업을 진행해야 합니다.



Step2. Config 변수에 데이터가 있는지 확인

- 재수행을 하게 된다면 현재 작업은 불필요하기에 먼저 Config 값이 있는지 확인합니다.



Step3. 만약 첫 수행일 경우, 현재 RPA가 진행 중인 모니터 해상도 출력

Log Message screen resolution

메시지 *

{ }

"The primary screen resolution is: "+Screen.PrimaryScreenResolution.ToString()

+

로그 레벨

Info

+

Step4. Config.xlsx 파일 데이터를 추출해 Config 변수 생성

Invoke InitAllSettings workflow

WorkflowFileName *

{ }

"Framework\InitAllSettings.xaml"

+

...

인수 가져오기

3

워크플로우 열기

- Config.xlsx 경로가 변경되었으면 해당 부분 수정

호출된 워크플로우의 인수

Name	Direction	Type	Value
in_ConfigFile	입력	String	"Data\Config.xlsx"
in_ConfigSheets	입력	String[]	{"Settings", "Constants"}
out_Config	출력	Dictionary<String,Object>	Config

OKCancel

Step5. 삭제해도 되는 부분(2018.3 이전의 오케스트레이터와 연결하고 싶을 때 사용)

- 액티비티 삭제
- 인수 삭제

If in_Orchestrator...
Overwrite queue name from in configuration file in case the argument in_OrchestratorQueueName is specified.
This provides backward compatibility with < 2018.3.
두 번 클릭하여 보기

If in_Orchestrator...
Overwrite folder name from in configuration file in case the argument in_OrchestratorQueueFolder is specified.
This provides backward compatibility with < 2018.3.
두 번 클릭하여 보기

이름	방향	인수 형식	기본값
in_OrchestratorQueueFolder	입력	String	VB 식 입력
in_OrchestratorQueueName	입력	String	VB 식 입력

인수 만들기
변수 인수 가져오기 126.09%

Step6. 프로세스 초기화

- 현재 열려있는 App 및 Web 모두 닫기 작업 추가

Invoke KillAllProcesses workflow (first run)

Kills all Windows processes representing applications used in this business process to assure that the execution starts in a clean state.
Since the applications are assumed to be already closed, CloseAllApplications is skipped and just KillAllProcess is invoked.

WorkflowFileName *
"Framework\KillAllProcesses.xar"
인수 가져오기 0 워크플로우 열기

Step7. 현재 진행 중인 프로세스 이름

Add Log Fields (BusinessProcessName)

Add the process name to the logs generated after this point.
This log field can be used to create reports and visualizations about the process.

필드

사전의 1개 항목

- 만약 프로세스 이름을 변경하고 싶다면, Config.xlsx - Setting 시트 - logF_BusinessProcessName 의 Value 값 변경

Name	Value
OrchestratorQueueName	ProcessABCQueue
OrchestratorQueueFolder	
logF_BusinessProcessName	Framework

Step8. 최대 재수행 횟수 제한

- Config.xlsx 파일에 있는 최대 재수행 횟수에 도달할 경우 End Process 로 이동하도록 예외 사항을 Throw 하는 작업

If maxConsecutiveSystemExceptions exceeded

If MaxConsecutiveSystemExceptions number was reached, throw Exception at initialization and go to End Process state, thus finalizing the execution.
If MaxConsecutiveSystemExceptions is 0, then any number of consecutive System Exceptions is allowed.

Condition *

{}

Cint(Config("MaxConsecutiveSystemExce

Then

Throw Consecutive Exceptions exceeded

Exception *

{}

new Exception(Config("ExceptionMessag

Else

Step9. 초기 프로세스(App 열기, Web 로그인 등) 작성

Invoke InitAllApplications workflow

WorkflowFileName *

{ } "FrameworkWInitAllApplications. ㄴ"

+

...

인수 가져오기

1

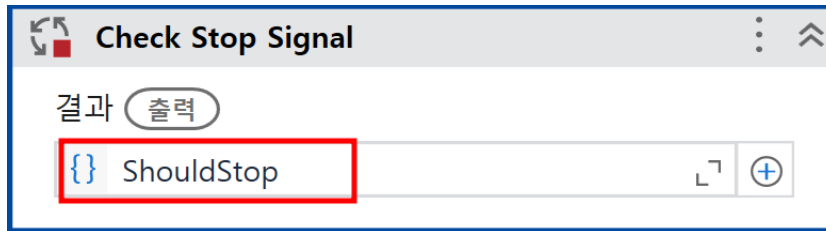
워크플로우 열기

6.2. Get Transaction Data

- Init State 에서 발생한 Transaction Data 를 가지고 적합성을 판단하는 State 입니다.

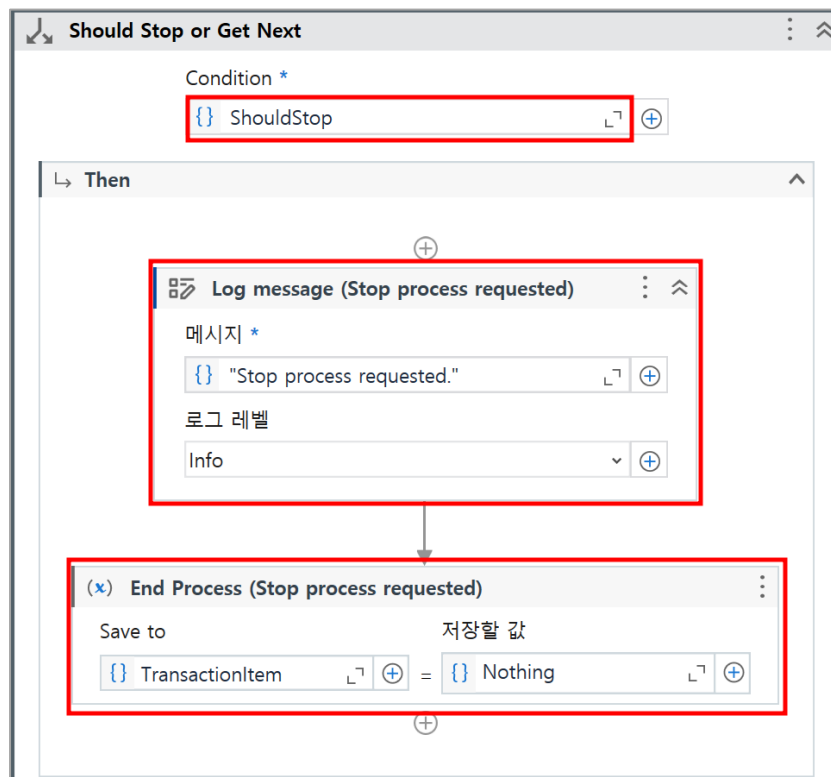
Step1. 프로세스 중단 여부 확인

- 일반적으로 Orchestrator 에서 중지 신호를 수신하여 프로세스 실행을 중단해야 할 때 사용됩니다. Check Stop Signal 액티비티는 중지 신호를 확인하고, 중지 신호가 수신된 경우 프로세스를 안전하게 중지시킵니다.



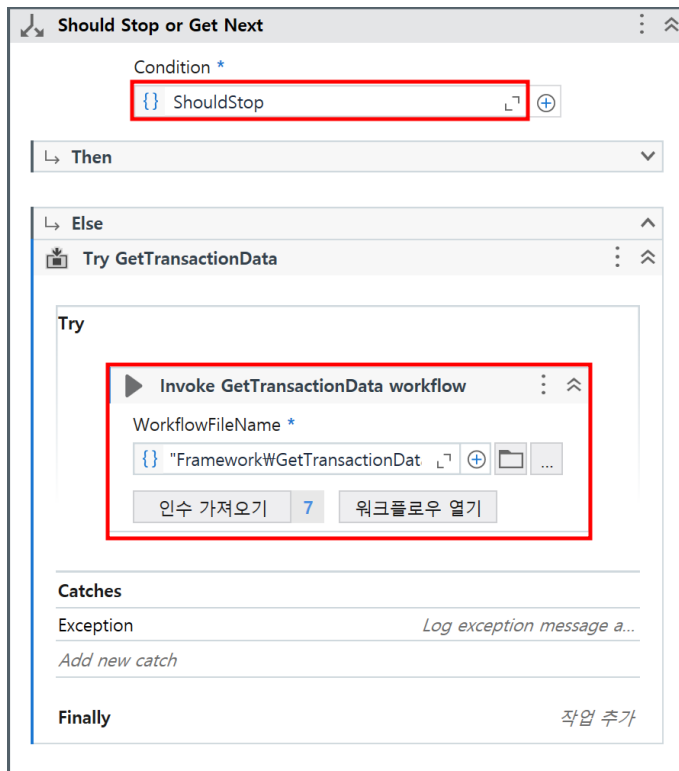
Step2. ShouldStop 이 True 인 경우 (Then)

- 프로세스 중지 로그 출력
- 현재 진행 중인 TransactionItem 을 비우고 End Process 로 이동



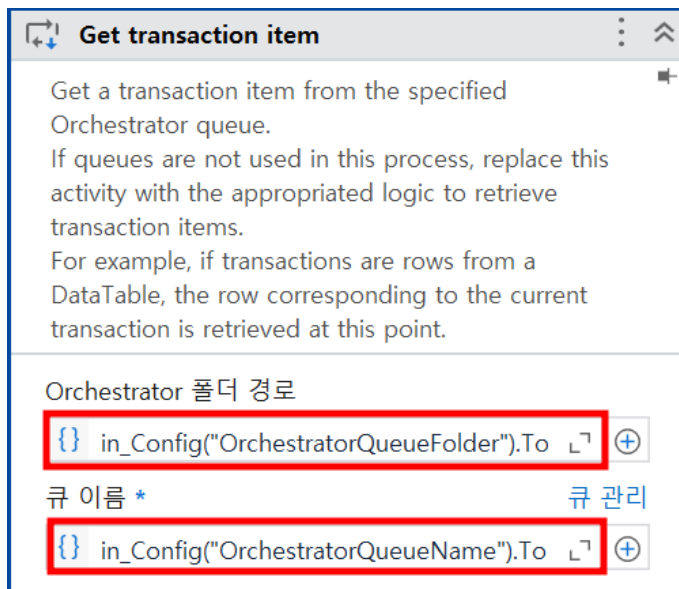
Step3. ShouldStop 이 False 인 경우 (Else)

- TransactionItem 추출 후 'Process Transaction' State 로 이동



Step4. 트랜잭션 아이템 추출

- GetQueueItem 액티비티로 Orchestrator 에 저장된 Queue Item 을 호출



6.3. Process Transaction

- Transaction Item 을 가지고 프로세스를 처리하는 곳으로 처리 결과는 1) Success, 2) Business Exception, 3) System Exception 으로 구분됩니다. 해당 State 에서는 'End Process' Transaction 으로 이동을 못합니다.

Success	Get Transaction Item 에서 새로운 Item 호출 후 진행
Business Exception	오류 건 로그 작성 후, Get Transaction Item 에서 새로운 Item 호출 후 진행
System Exception	Init 에서 프로세스 재수행

Step1. Business Exception 초기화

- Business Exception 이란 프로세스 실행 중에 발생하는 예외로, 대체로 예상된 오류입니다. 예를 들어, 사용자가 잘못된 데이터를 입력하거나, 필요한 필드를 비워 둔 경우 등이 있습니다.
- 만약, 이미 재수행이 된 상황에서 BusinessException 에 값이 있을 경우 프로세스 진행이 어렵기 때문에 초기화 하는 작업이 필요합니다.

(x) Assign BusinessException

Save to: {} BusinessException

저장할 값: {} Nothing

Step2. Transaction Item 처리 프로세스 진행

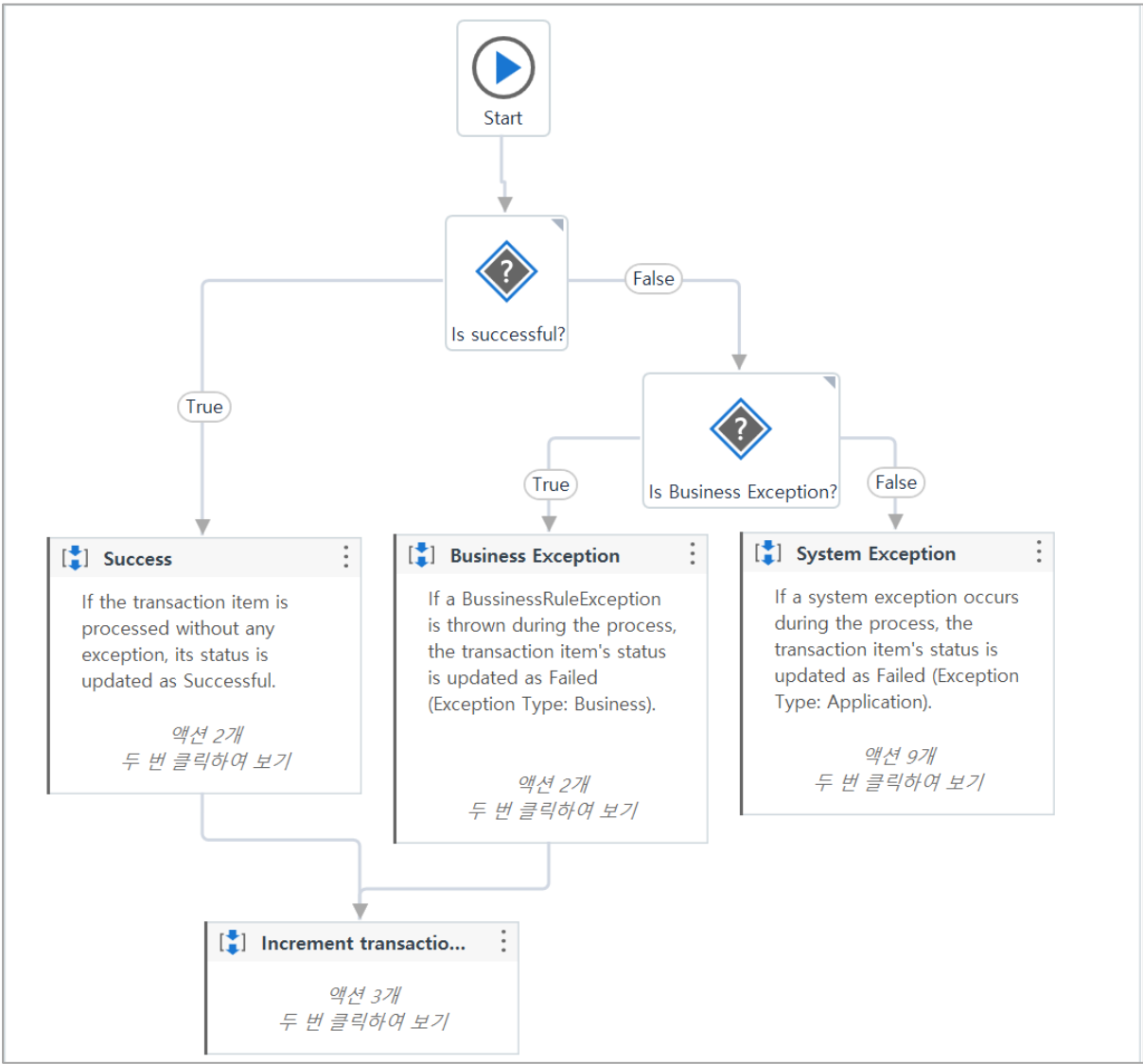
Invoke Process workflow

WorkflowFileName *

{} "FrameworkWProcess.xml"

인수 가져오기 2 워크플로우 열기

Step3. Process 종료 후 예외처리 상황에 따른 Transaction Item 상태 변경



Step3.1. Success

Set transaction status (Successful)

Orchestrator 폴더 경로

`in_Config("OrchestratorQueueFolder").To`

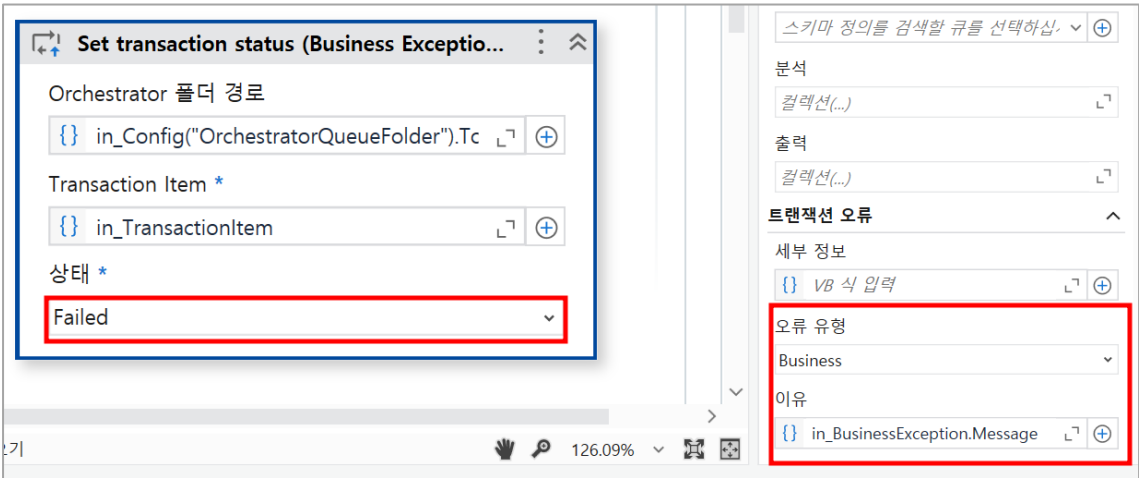
Transaction Item *

`in_TransactionItem`

상태 *

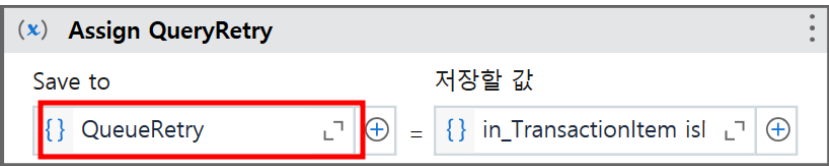
Successful

Step3.2. Business Exception

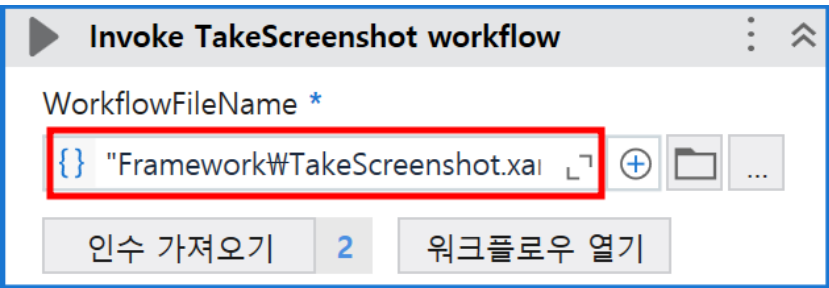


Step3.3. System Exception

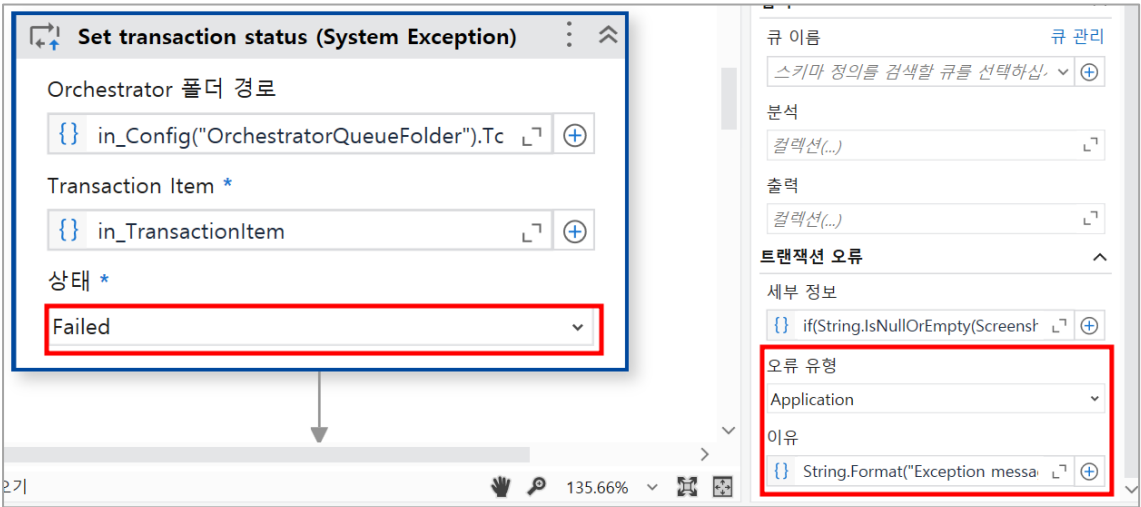
- QueueRetry 라는 변수는 트랜잭션을 재시도할지 여부를 결정하는 데 사용됩니다. 특정 트랜잭션에서 SystemException 이 발생하면, 그 예외는 일시적인 문제 때문일 수 있으므로 해당 트랜잭션을 재시도하려는 의도를 나타냅니다.



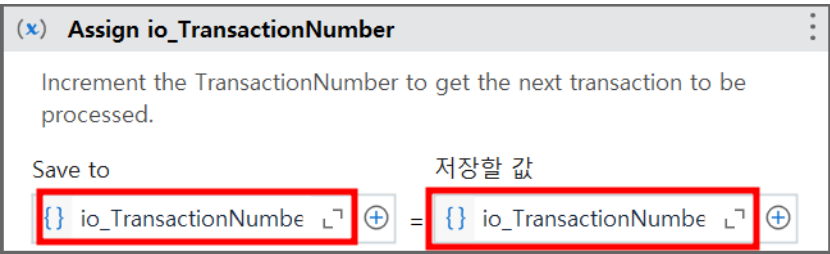
- 오류가 발생한 화면 캡처



- 오류 유형 확인



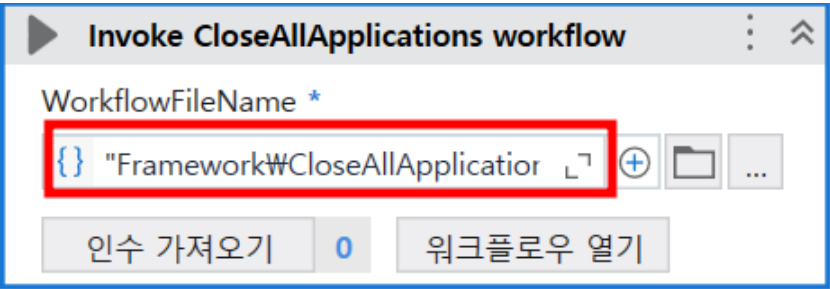
Step4. 트랜잭션 숫자 증가



6.4. End Process

- 프로세스를 종료하기 전 사용했던 App 이나 브라우저를 모두 닫고, 성공/실패 여부 이메일을 발송하고, 산출물 백업 관리를 하며 프로세스를 잘 마무리할 수 있도록 하는 State 입니다.

Step1. 모든 App, 브라우저 종료



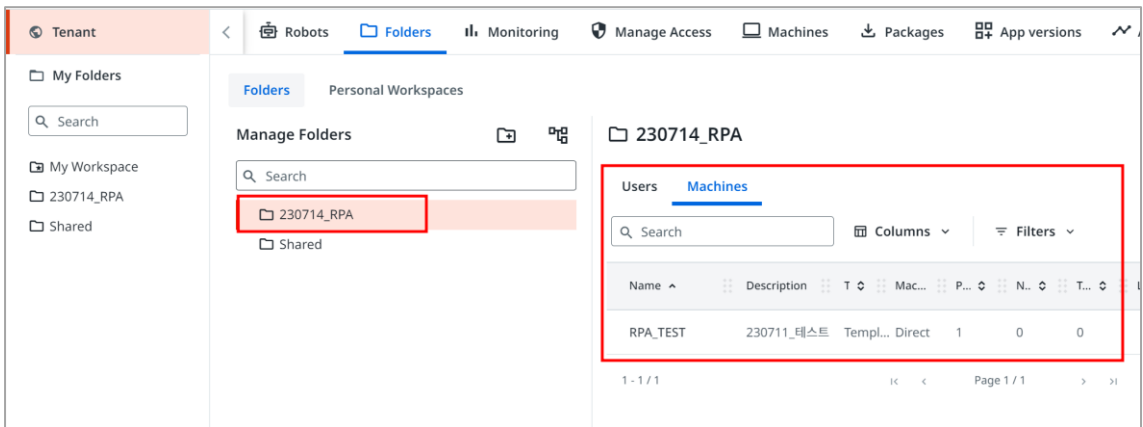
7. Orchestrator 와 데이터 주고받기

- Orchestrator 에 저장된 Asset 을 바탕으로 ACME 로그인을 진행하고, 전체 Work Item 중 Type 이 “WI5”인 항목의 ‘WIID’ 값을 Queue 에 저장한 다음, 차례대로 Work Item Detail 에 접근하는 프로세스를 만들어 보시다.

7.1. 초기 세팅

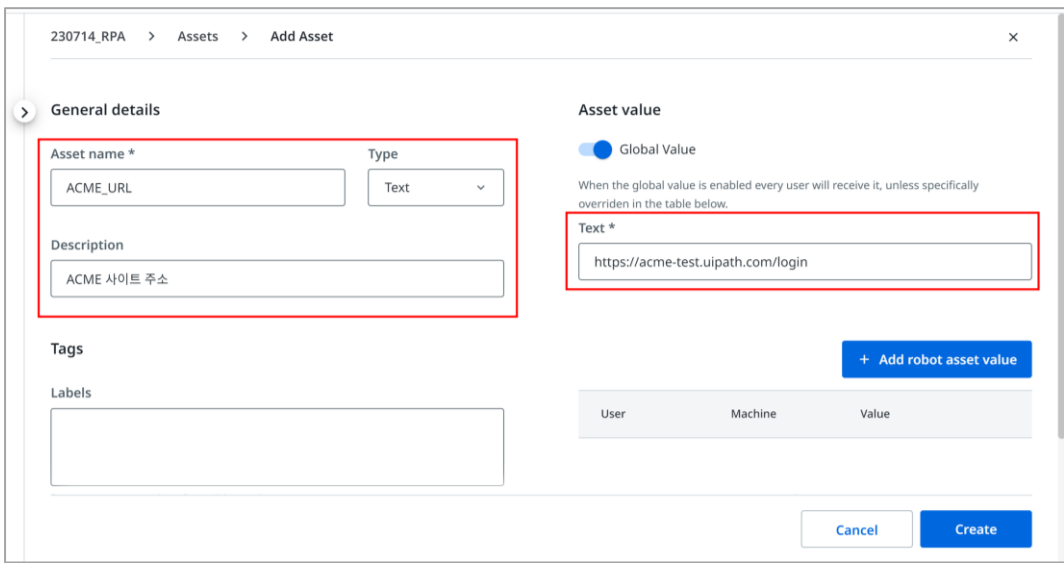
Step1. 폴더 생성

- 오늘 날짜의 폴더 생성 후 User 와 Machine 연결



Step2. Asset 생성

- ACME 사이트 URL Asset 생성



- ACME 로그인 계정 Asset 생성

General details

Asset name *

ACME_Account

Type

Credential

Description

ACME 로그인 계정 정보

Credential Store

Orchestrator Database

Tags

Labels

Properties (Key-value pairs)

Advanced settings

This object contains no properties.

Asset value

Global Value

When the global value is enabled every user will receive it, unless specifically overridden in the table below.

Username

Password

+ Add robot asset value

User	Machine	Value
User * rpakcc3@gmail.com	Machine RPA_TEST	Username * rpakcc3@gmail.com Password *

Cancel

Save

☐	Name ^	Description ^	Type ^	Value
☐	ACME_Account	ACME 로그인 계정 정보	Credential	[In credential store]
☐	ACME_URL	ACME 사이트 주소	Text	https://acme-test.uipath.com/login

Step3. Queue 생성

Tenant

My Folders

Search

My Workspace

230714_RPA

Shared

Home

Automations

Monitoring

Queues

Assets

Storage Buckets

Te...

Queues

Review Requests

Search

Columns

Filters

+ Add Queue

☐	Name ^	D ^	I...	R...	Ave...	...	...	...	...	Pr...	Labels	Properties
☐	ACME_WIID	0	0	0	0	0	0	0	0	0		

1 - 1 / 1

Page 1 / 1

Items 10

Step4. REFramework 생성

- Name: 오늘 날짜_REFramework

새 Robotic Enterprise Framework

새 Robotic Enterprise Framework

대규모 배포를 위한 모범 사례를 따르는 트랜잭션 비즈니스 프로세스를 만듭니다.

이름 *

230714_REFramework

위치 *

C:\Users\Whjchoi\Documents\UiPath

설명

Robotic Enterprise Framework

호환성 ?

Windows

언어

☒ VB

☐ C#

만들기

Step5. Config.xlsx 에 Asset 과 Queue 이름 입력

- Setting 시트

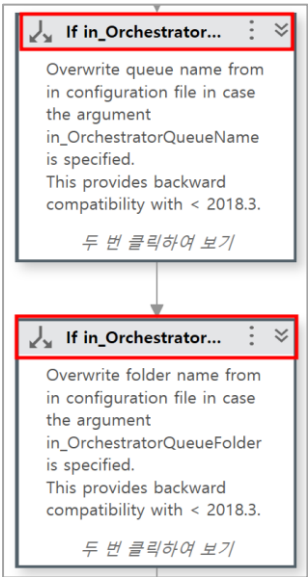
A		B
	Name	Value
	OrchestratorQueueName	ACME_WIID
	OrchestratorAssetName	ACME_Account
	OrchestratorFolder	230714_RPA
5		
6	logF_BusinessProcessName	Framework
7		
Settings Constants Assets +		

- Asset 정보

A		B
1	Name	Asset
2	URL	ACME_URL
3		OrchestratorAssetFolder
		230714_RPA
Settings Constants Assets +		

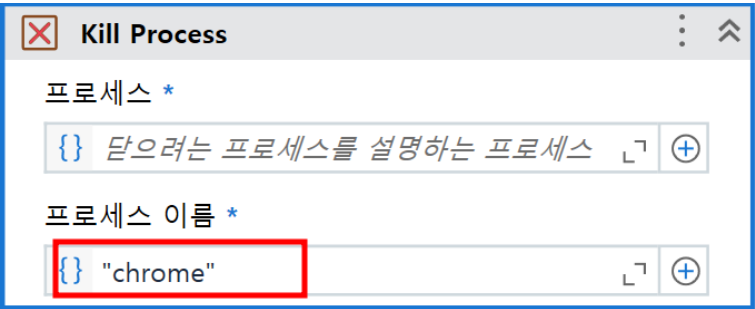
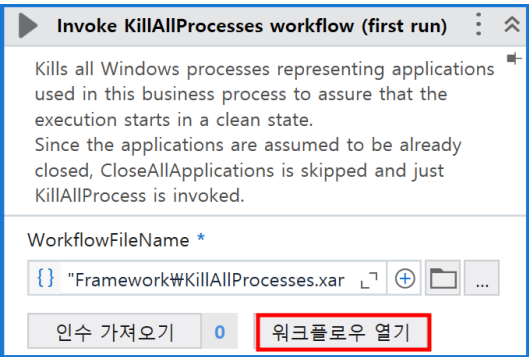
7.2. “Init” State

Step1. Init 프로세스 불필요 항목 삭제

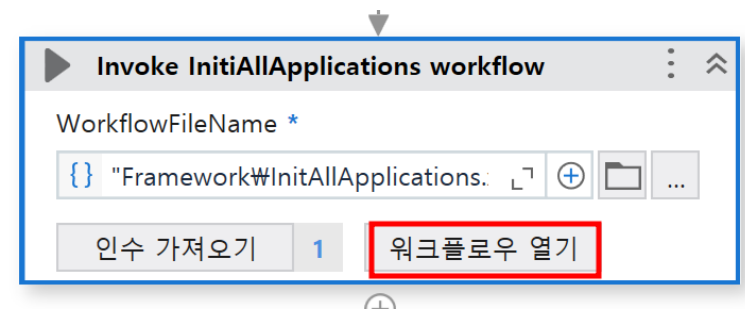


Step2. Kill Process 작성

– 강제 종료할 프로세스 이름을 액티비티에 입력

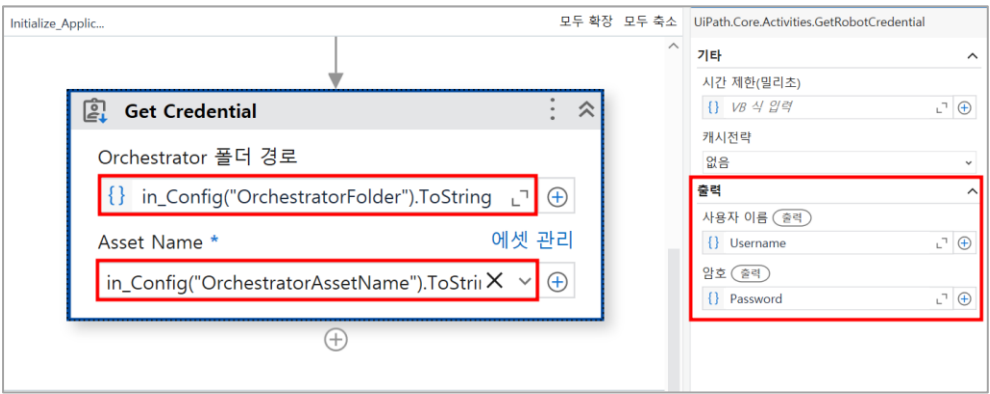


Step3. InitAllApplication 프로세스 내부로 이동



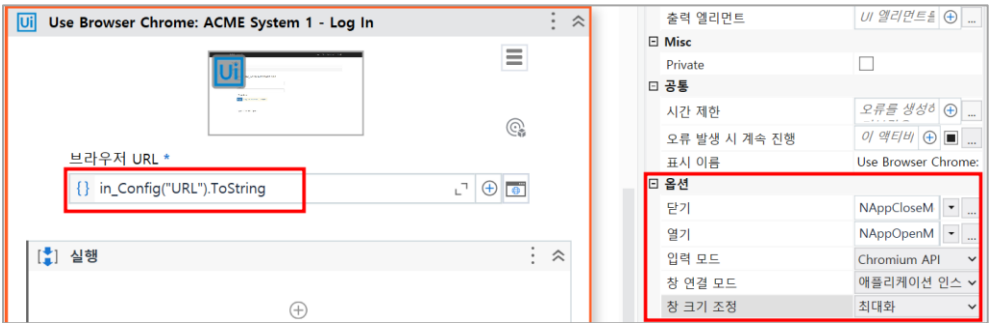
Step4. Orchestrator 에서 ACME 로그인 계정 값 추출하기

- Get Credential 액티비티로 ACME 로그인 정보 추출하기
- 폴더 경로: in_Config("OrchestratorFolder").ToString
- Asset Name: in_Config("OrchestratorAssetName").ToString
- 출력 변수에 각각 Username, Password 생성

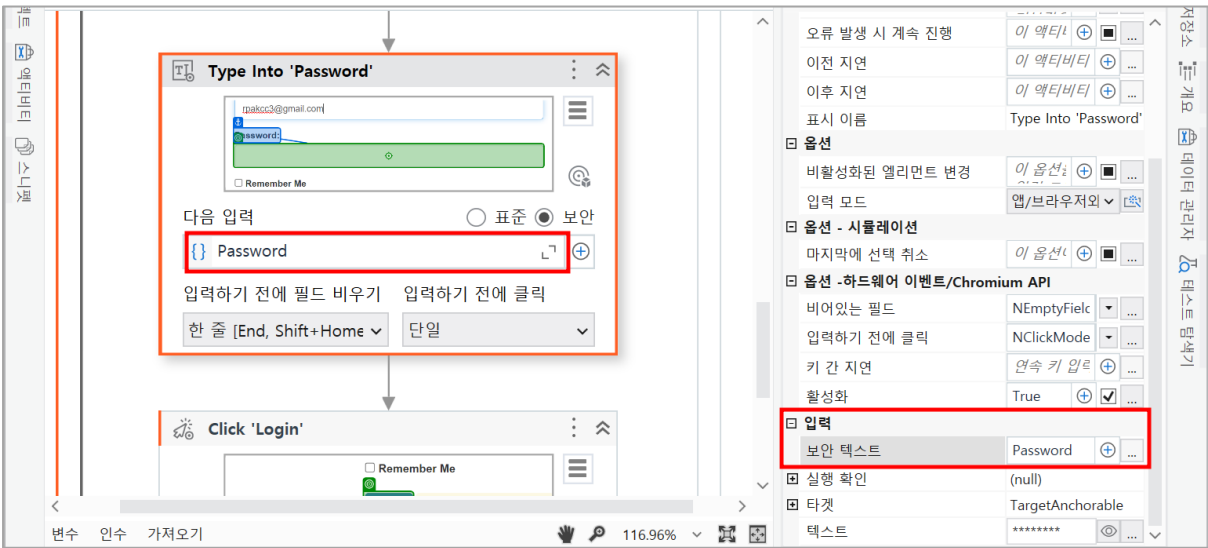


Step5. 로그인 프로세스 작성

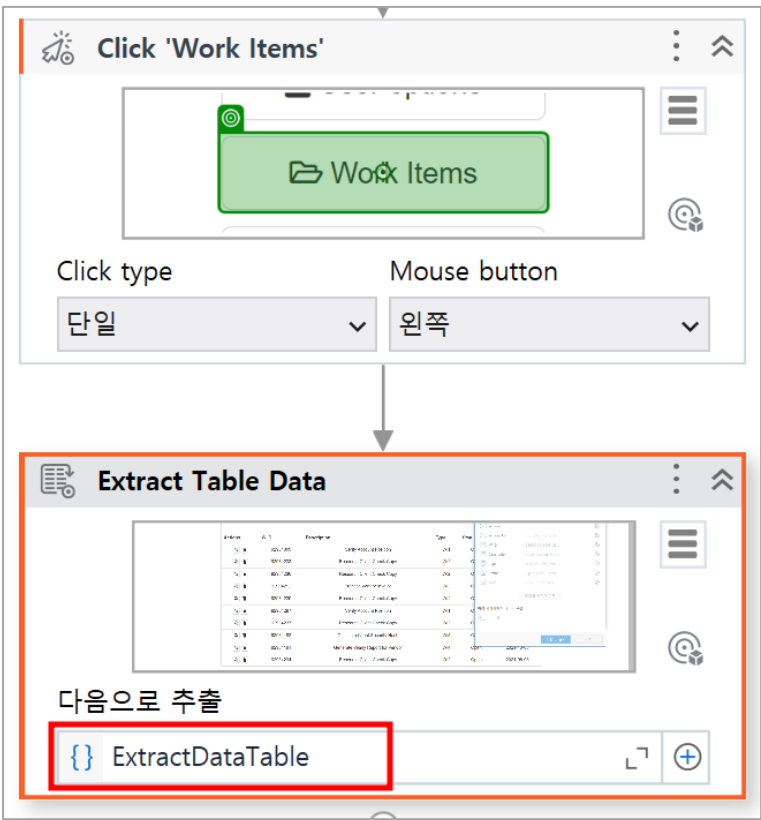
- Config 에서 브라우저 URL 호출 및 속성 변경



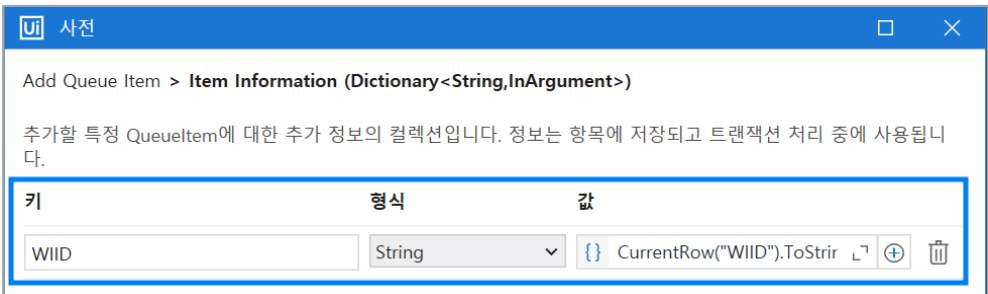
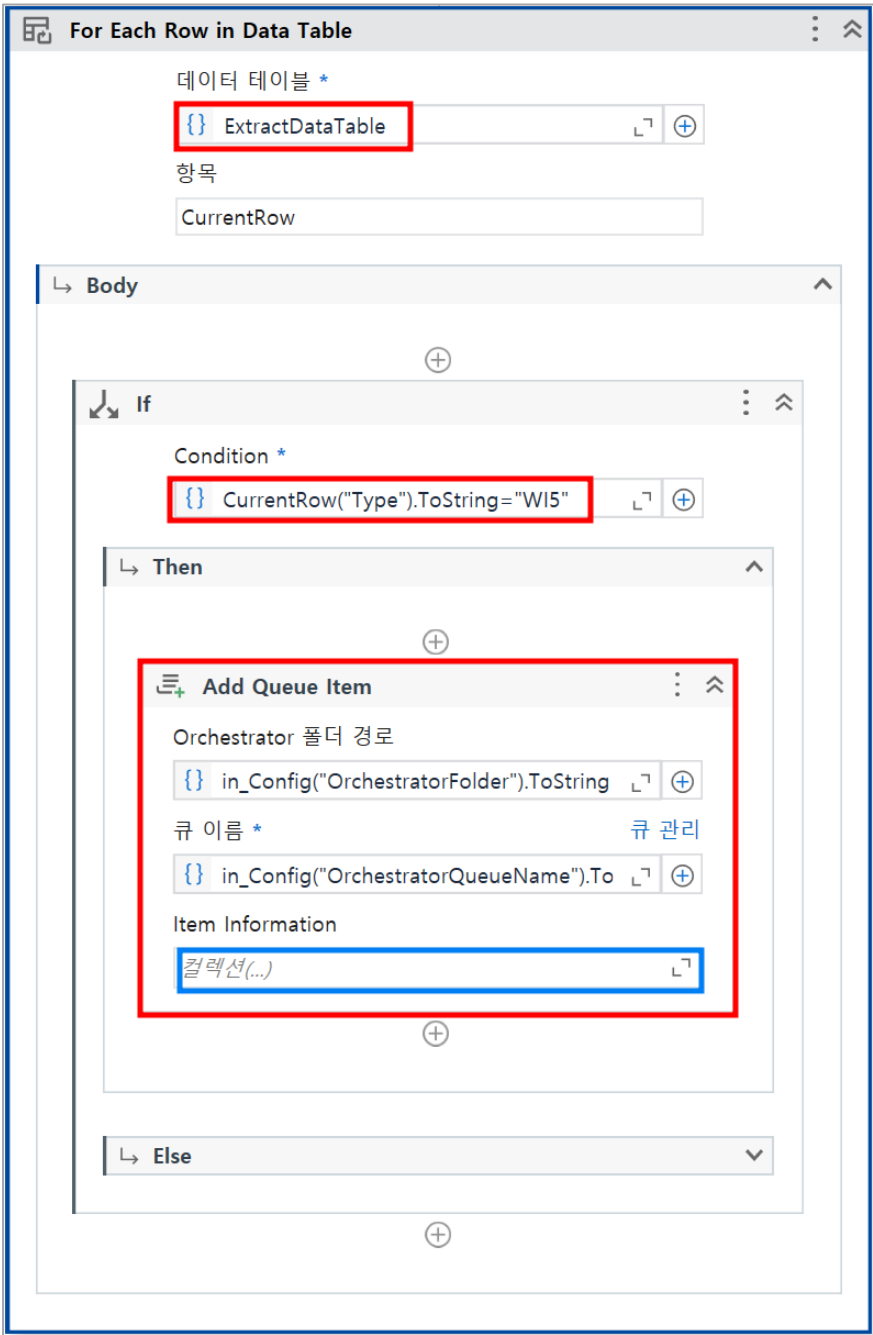
- Type Into 'Password' 속성 - 입력 - 보안 텍스트에 Password 입력



Step6. 전체 Work Item 데이터테이블 추출

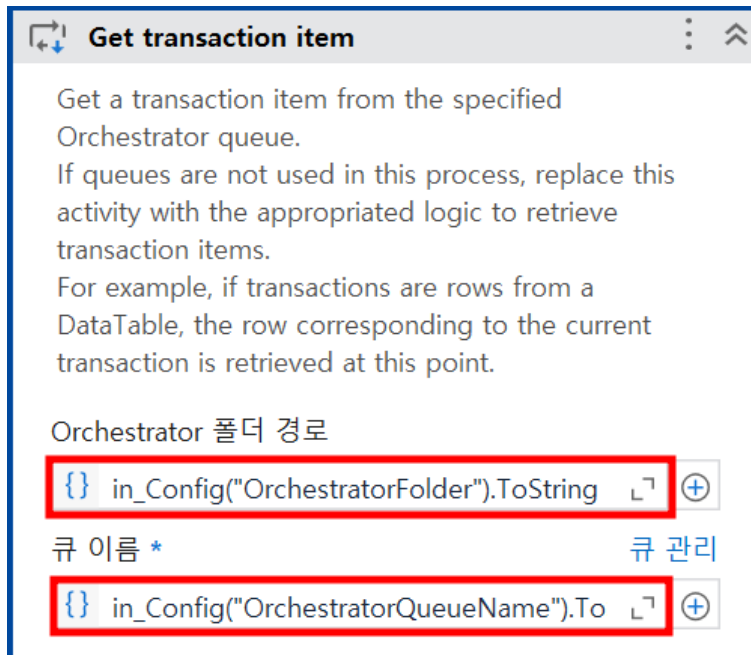


Step7. 전체 데이터 중 Type 이 W15 인 항목만 추출해 Queue 에 저장



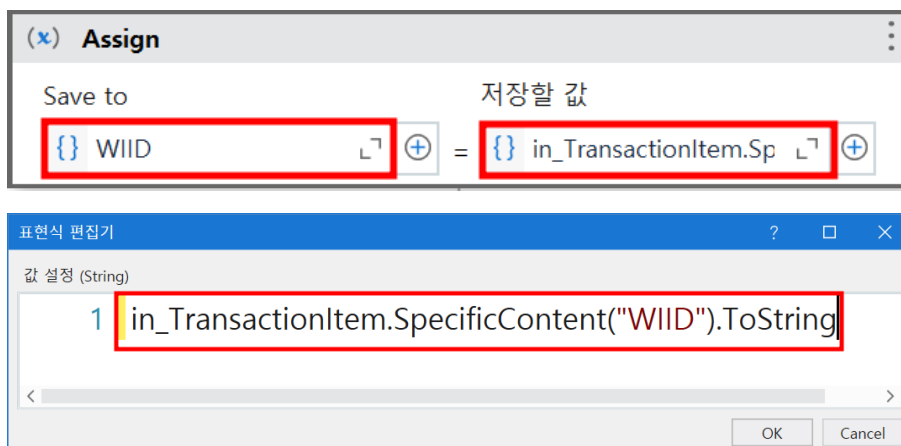
7.3. “Get Transaction Data” State

Step1. Get Transaction Item 액티비티 인수 확인

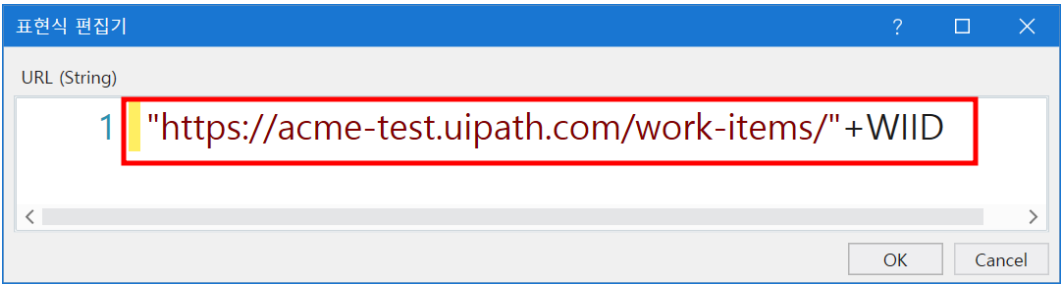
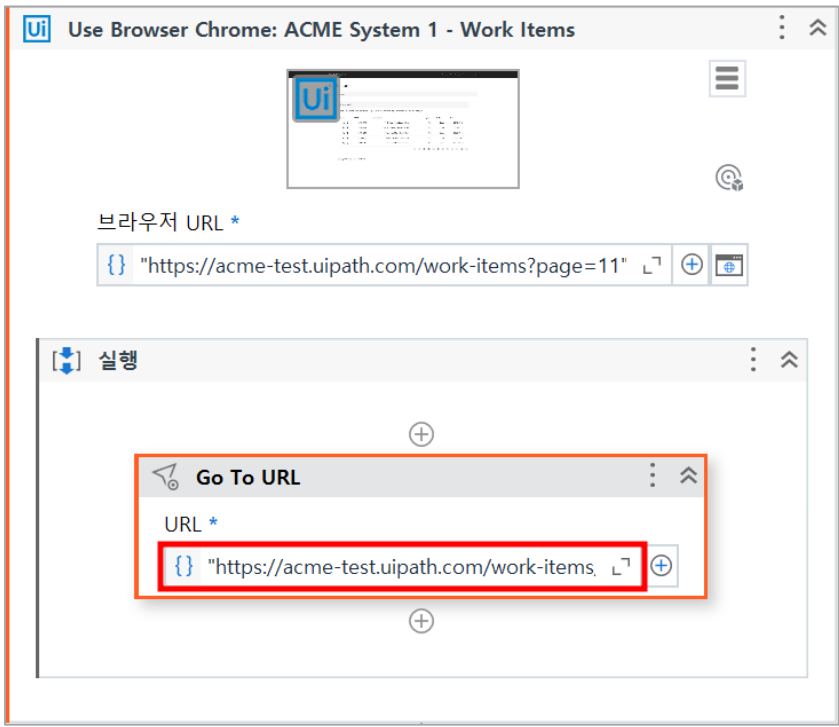


7.4. “Process Transaction” State

Step1. WIID 변수 생성



Step2. 사이트 이동 프로세스 작성



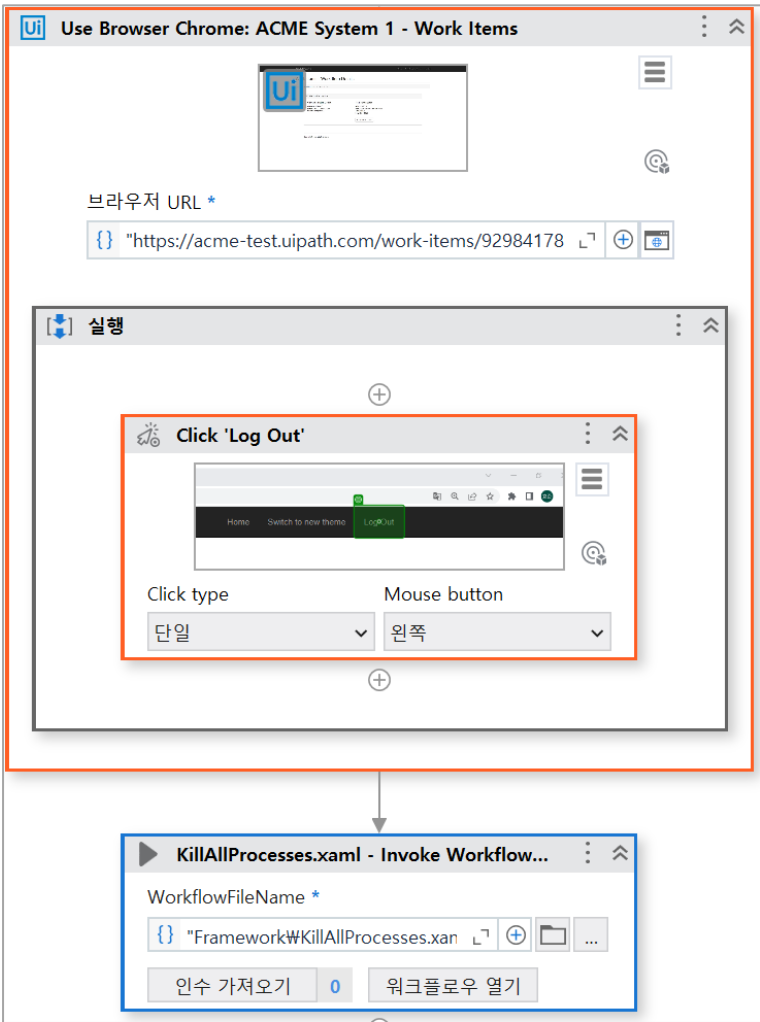
Step3. 사이트 이동 후 딜레이 2~3 초 추가



7.5. “End Process” State

– 현재까지 열린 APP 및 브라우저를 종료합니다.

Step1. ACME 사이트 로그아웃 후 브라우저 종료



8. Orchestrator 없이 사용하기

- Window Credential 에 저장된 정보를 바탕으로 ACME 로그인을 진행하고, 전체 Work Item 중 Type 이 “WI5”인 항목의 ‘WIID’ 값을 DataRow 에 저장한 다음, 차례대로 Work Item Detail 에 접근하는 프로세스를 만들어 보시다.

8.1. 초기 세팅

Step1. REFramework 생성

새 Robotic Enterprise Framework

대규모 배포를 위한 모범 사례를 따르는 트랜잭션 비즈니스 프로세스를 만듭니다.

이름 * 230714_REFramework

위치 * C:\Users\Whjchoi\Documents\UiPath

설명 Robotic Enterprise Framework

호환성 ? Windows

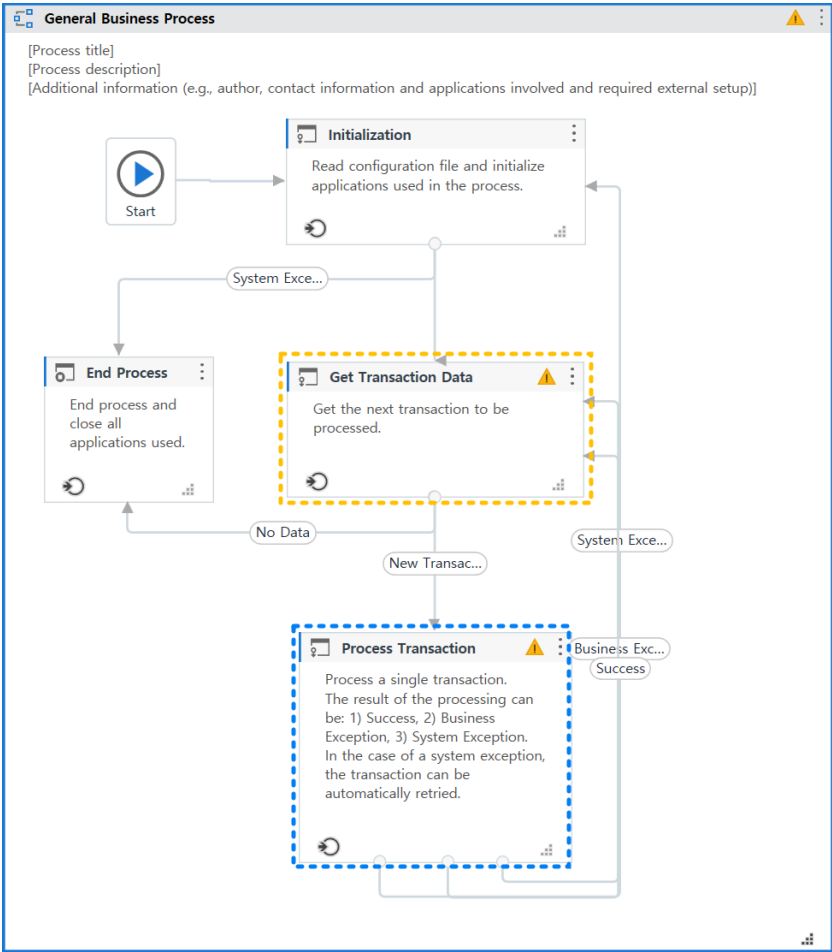
언어 ☒ VB ☐ C#

만들기

Step2. “TransactionItem” 변수 형식을 QueueItem → DataRow 로 변경해준다. (Non-Queue)

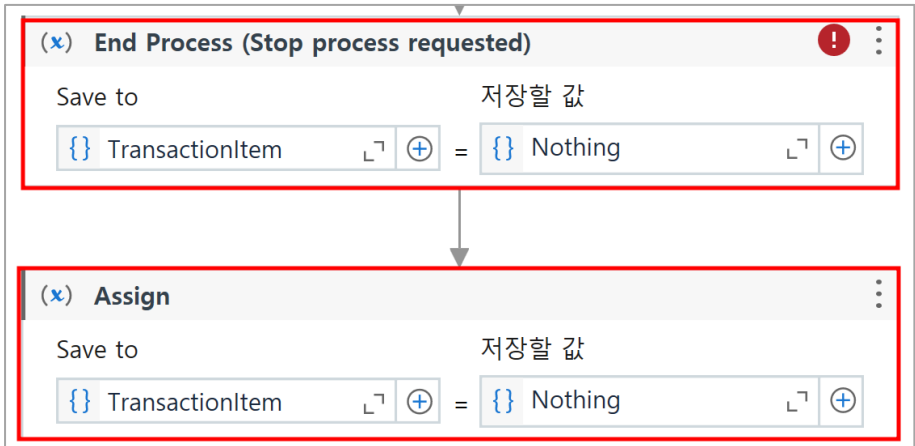
이름	변수 형식	범위
TransactionItem	DataRow	General Business Process
SystemException	Exception	General Business Process
BusinessException	BusinessRuleException	General Business Process
TransactionNumber	Int32	General Business Process
Config	Dictionary<String,Object>	General Business Process
RetryNumber	Int32	General Business Process
TransactionField2	String	General Business Process
TransactionField1	String	General Business Process
TransactionID	String	General Business Process
dt_TransactionData	DataTable	General Business Process
ConsecutiveSystemExceptions	Int32	General Business Process

Step3. 변수 형식이 바뀌고 난 다음 오류가 난 부분을 모두 수정



Step3.1. “Get Transaction Data” State 수정

– 해당 액티비티에 오류 표시가 있지만, 이 부분은 변수 유형이 바뀔 때 자주 발생하는 UiPath 버그 중 하나입니다. Assign 액티비티를 새로 만들면 해결됩니다.



Step3.2. “GetTransactionData” Workflow 수정

– out_TransactionItem 인수 유형 변경

Invoke GetTransactionData workflow

WorkflowFileName *

{}

"Framework\GetTransactionDat

↵

+

📁

...

인수 가져오기 7 워크플로우 열기

이름	방향	인수 형식
in_TransactionNumber	입력	Int32
in_Config	입력	Dictionary<String,Object>
out_TransactionItem	출력	DataRow
out_TransactionField1	출력	String
out_TransactionField2	출력	String
out_TransactionID	출력	String
io_dt_TransactionData	입력/출력	DataTable

- Retry Get ~ 삭제

Retry Get transaction item

Try Catch Get transaction item

Try

Get transaction item

Get a transaction item from the specified Orchestrator queue.
If queues are not used in this process, replace this activity with the appropriated logic to retrieve transaction items.
For example, if transactions are rows from a DataTable, the row corresponding to the current transaction is retrieved at this point.

Orchestrator 폴더 경로

{}

in_Config("OrchestratorQueueFolder").To

↵

+

큐 이름 *

{}

in_Config("OrchestratorQueueName").To

↵

+

큐 관리

Catches

Exception

Catch Get transaction item

Add new catch

Finally

작업 추가

174

Step3.3. “Process” Workflow 수정

- in_TransactionItem 인수 유형 변경

Invoke Process workflow

WorkflowFileName *

{}

"Framework\Process.xml"

인수 가져오기 2 워크플로우 열기

이름	방향	인수 형식	기본값
in_TransactionItem	입력	DataRow	VB 식 입력
in_Config	입력	Dictionary<String,Object>	VB 식 입력

Step3.4. “SetTransactionStatus” Workflow 수정

- in_TransactionItem 인수 유형 변경

Invoke SetTransactionStatus (Success)

WorkflowFileName *

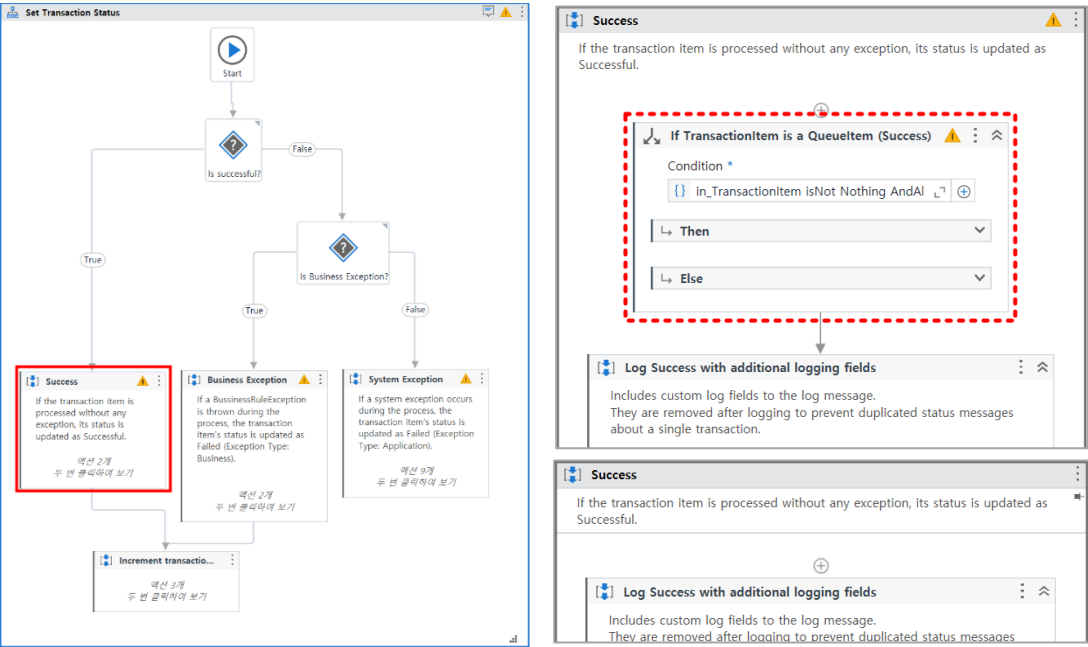
{}

"Framework\SetTransactionStat

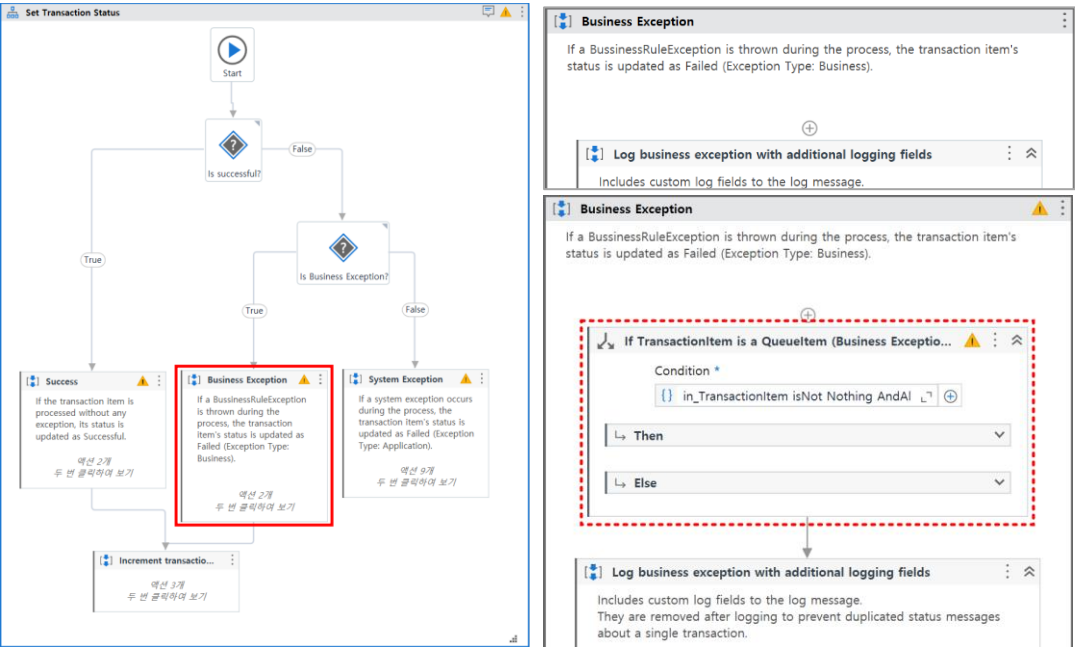
인수 가져오기 10 워크플로우 열기

이름	방향	인수 형식
in_BusinessException	입력	BusinessRuleException
in_TransactionField1	입력	String
in_TransactionField2	입력	String
in_TransactionID	입력	String
in_SystemException	입력	Exception
in_Config	입력	Dictionary<String,Object>
in_TransactionItem	입력	DataRow
io_RetryNumber	입력/출력	Int32
io_TransactionNumber	입력/출력	Int32
io_ConsecutiveSystemExceptions	입력/출력	Int32

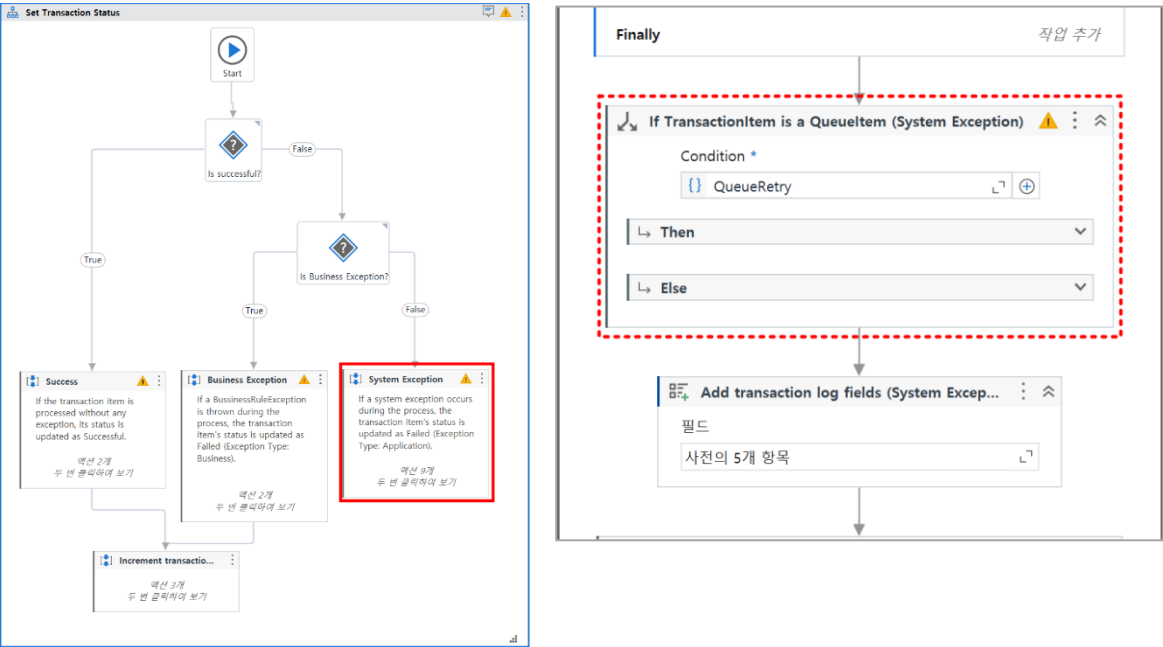
Step3.5. “Success” Sequence 수정 (If 액티비티 삭제)



Step3.6. “Business Exception” 수정 (If 액티비티 삭제)

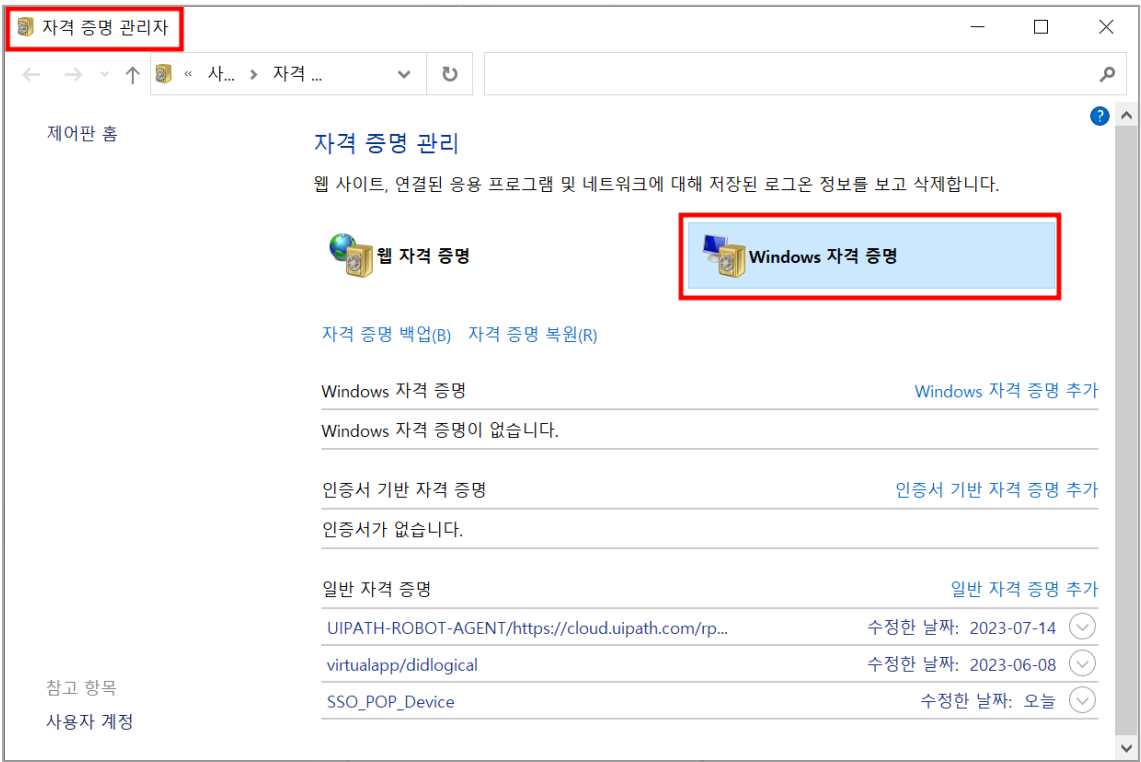


Step3.7. “System Exception” 수정 (If 액티비티 삭제)



Step4. Window Credential 설정

- 자격 증명 관리자 - Windows 자격 증명



- 일반 자격 증명 추가 클릭 후 내용 작성

일반 자격 증명 추가

← → ↶ ↷ ↵ ↶

« 자... » 일반 ...

제어판 검색

웹 사이트 주소 및 자격 증명 정보를 입력하십시오.

입력한 사용자 이름과 암호를 사용하여 해당 위치에 액세스할 수 있는지 확인하십시오.

인터넷 또는 네트워크 주소: System1_Accounts

사용자 이름: rpakcc3@gmail.com

암호: ●●●●●●●●

확인(O)

취소(N)

Step5. Config.xlsx 파일 작성

- URL 과 계정 정보를 읽어올 수 있도록 작성

Config - Excel

	A	B
1	Name	Value
2	OrchestratorQueueName	ProcessABCQueue
3	OrchestratorQueueFolder	
4		
5	logF_BusinessProcessName	Framework
6		
7	URL	https://acme-test.uipath.com/login
8	ACME_Login	System1_Accounts
9		

Settings Constants Assets

Step6. Window Credential 패키지 설치

Ui

UiPath.Credentials.Activities

기준 UiPath

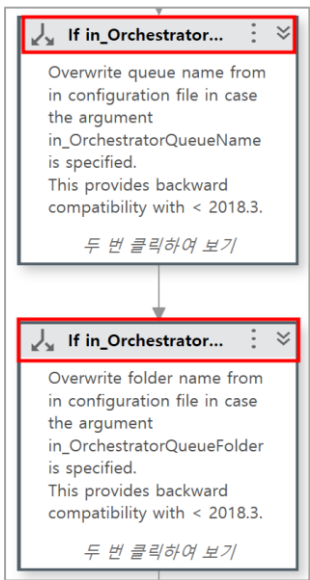
v2.0.0

UiPath Credentials Activities Pack

178

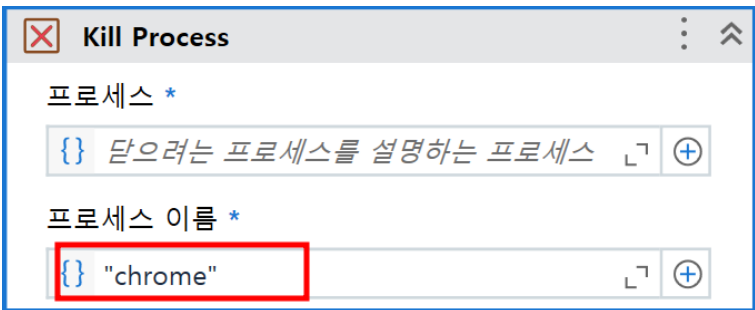
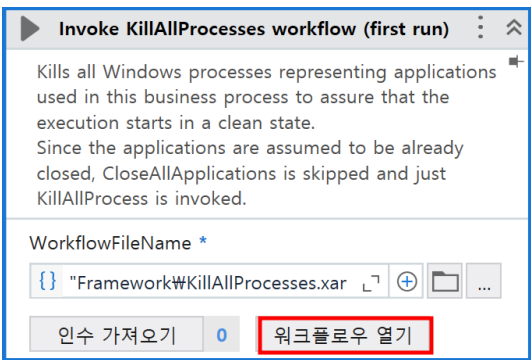
8.2. “Init” State

Step1. Init 프로세스 불필요 항목 삭제

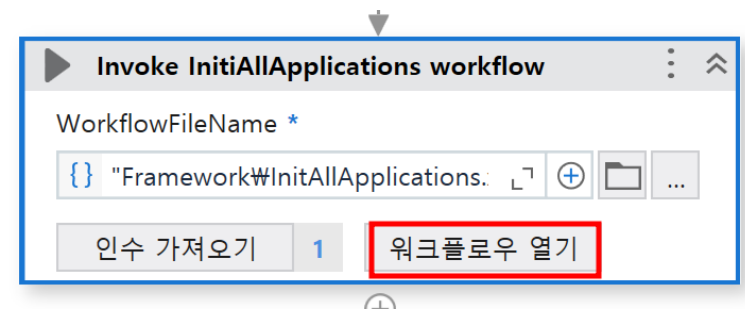


Step2. Kill Process 작성

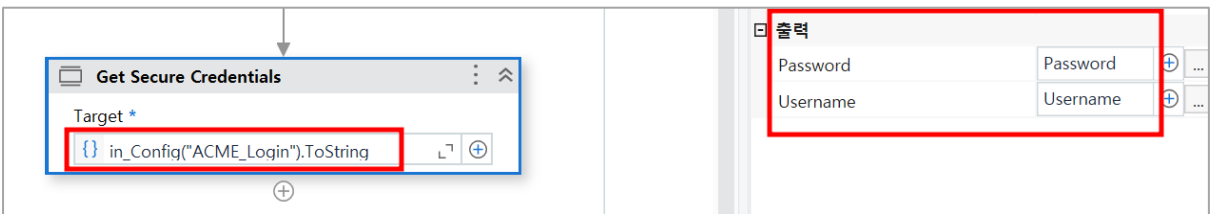
– 강제 종료할 프로세스 이름을 액티비티에 입력



Step3. InitAllApplication 프로세스 내부로 이동

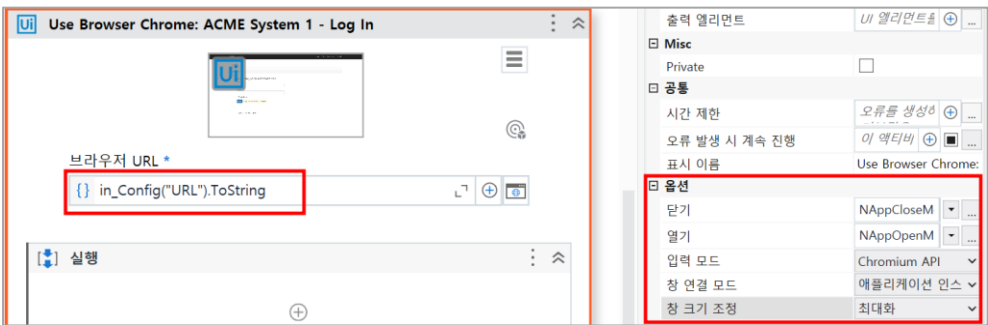


- Get Secure Credentials 액티비티로 ACME 로그인 정보 추출하기
- 타겟: in_Config("ACME_Login").ToString
- 출력 변수에 각각 Username, Password 생성

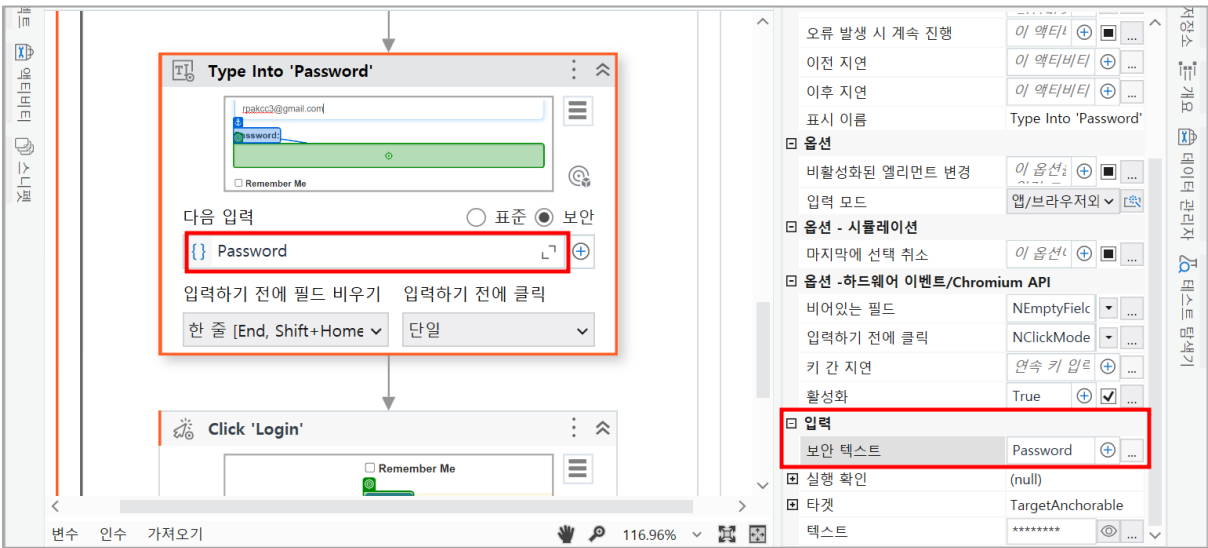


Step5. 로그인 프로세스 작성

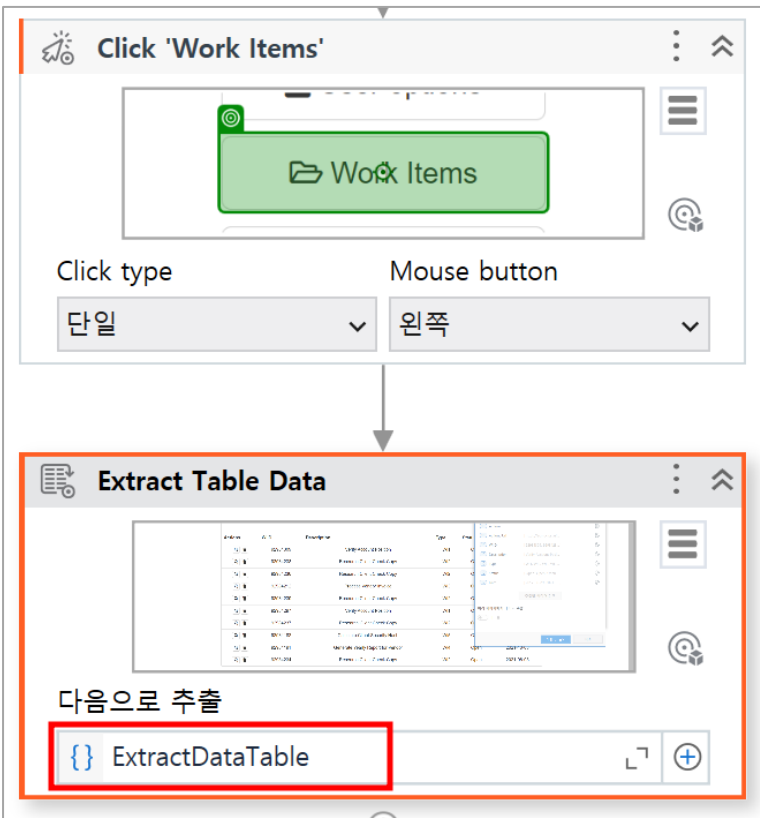
- Config 에서 브라우저 URL 호출 및 속성 변경



- Type Into 'Password' 속성 - 입력 - 보안 텍스트에 Password 입력

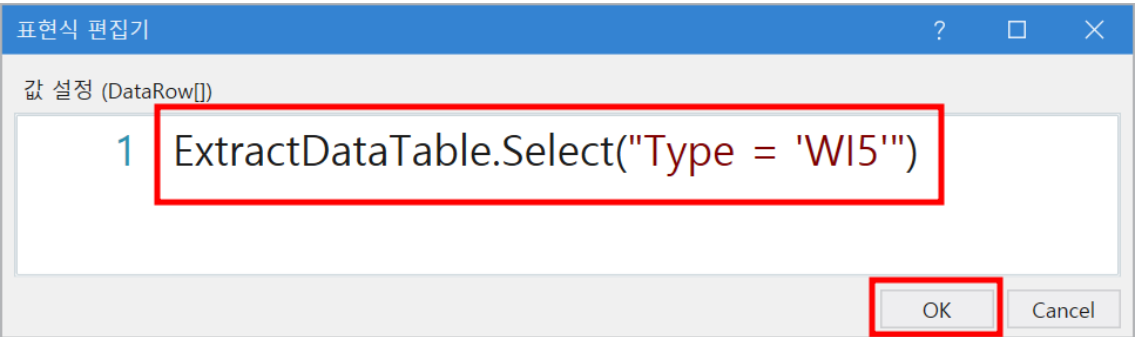
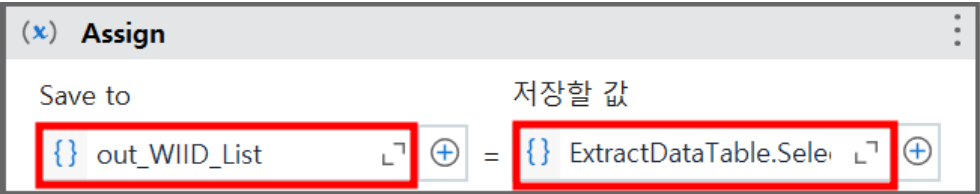


Step6. 전체 Work Item 데이터테이블 추출



Step7. 전체 데이터 중 Type 이 W15 인 항목만 추출해 “DataRow[]” 인수에 저장

- DataTable 메서드인 Select 를 사용



8.3. “Get Transaction Data” State

Step1. WIID_List 인수 입력방향으로 생성

이름	방향	인수 형식	기본값
in_TransactionNumber	입력	Int32	VB 식 입력
in_Config	입력	Dictionary<String,	VB 식 입력
out_TransactionItem	출력	DataRow	지원되지 않는 기본값
out_TransactionField1	출력	String	지원되지 않는 기본값
out_TransactionField2	출력	String	지원되지 않는 기본값
out_TransactionID	출력	String	지원되지 않는 기본값
io_dt_TransactionData	입력/출력	DataTable	지원되지 않는 기본값
in_WIID_List	입력	DataRow[]	VB 식 입력

Step2. If 조건문 변경

- 트랜잭션 개수에 따라 해당 트랜잭션이 존재하면 항목을 할당함

If a new transaction item is retrieved, get additional information about it

This optional step can be used to include more information about a transaction item and it is used mainly for logging and visualization purposes.
For example, if transaction items are invoices, then out_TransactionID can be the invoice number, out_TransactionField1 can be the invoice date and out_TransactionField2 can be the invoice amount.

Condition *

{}

 in_TransactionNumber < in_WIID_List.Co

+

표현식 편집기

If a new transaction item is retrieved, get additional information about it > Condition (InArgument<Boolean>)

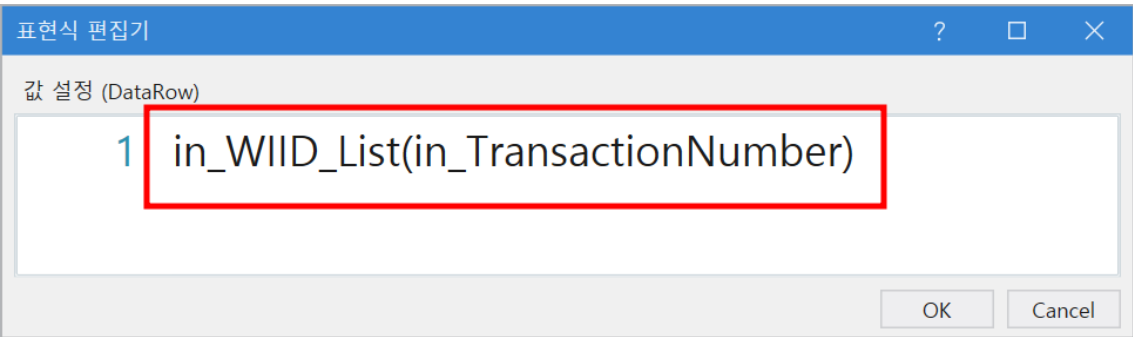
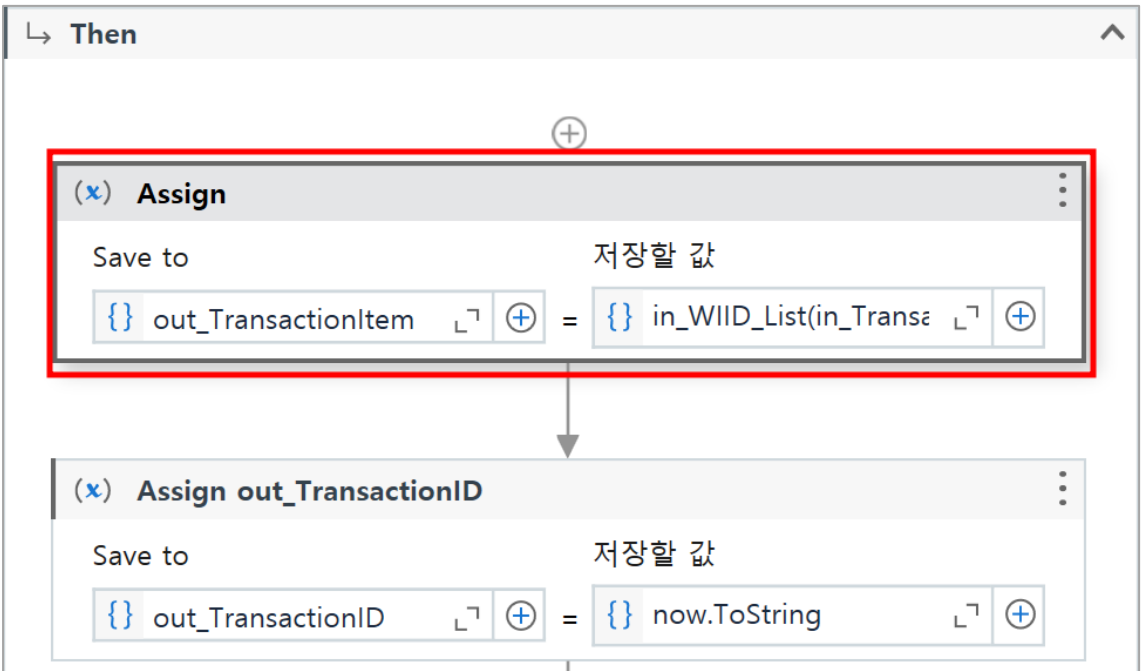
The condition to evaluate when determining which activities to execute.

변수 사용 ▾

1 in_TransactionNumber < in_WIID_List.Count

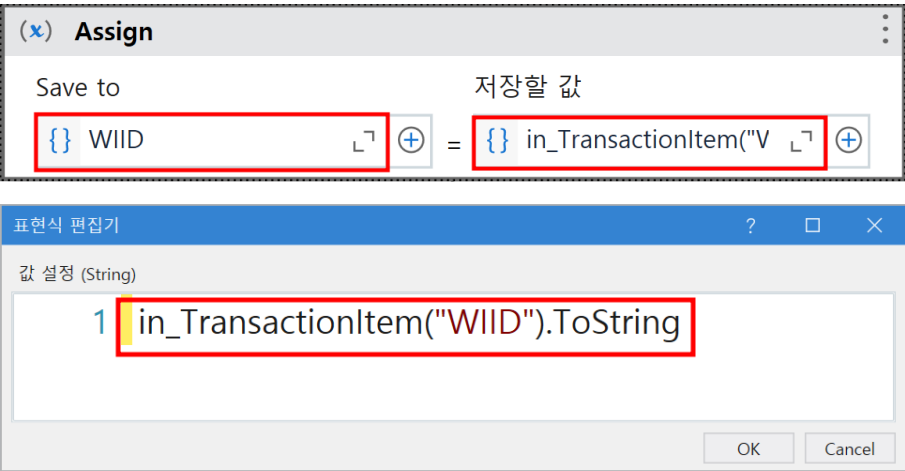
Step3. TransactionItem 할당

- in_TransactionNumber: 트랜잭션 항목을 카운팅 하는 인수
- out_TransactionItem: 트랜잭션 항목을 담을 Datarow 인수

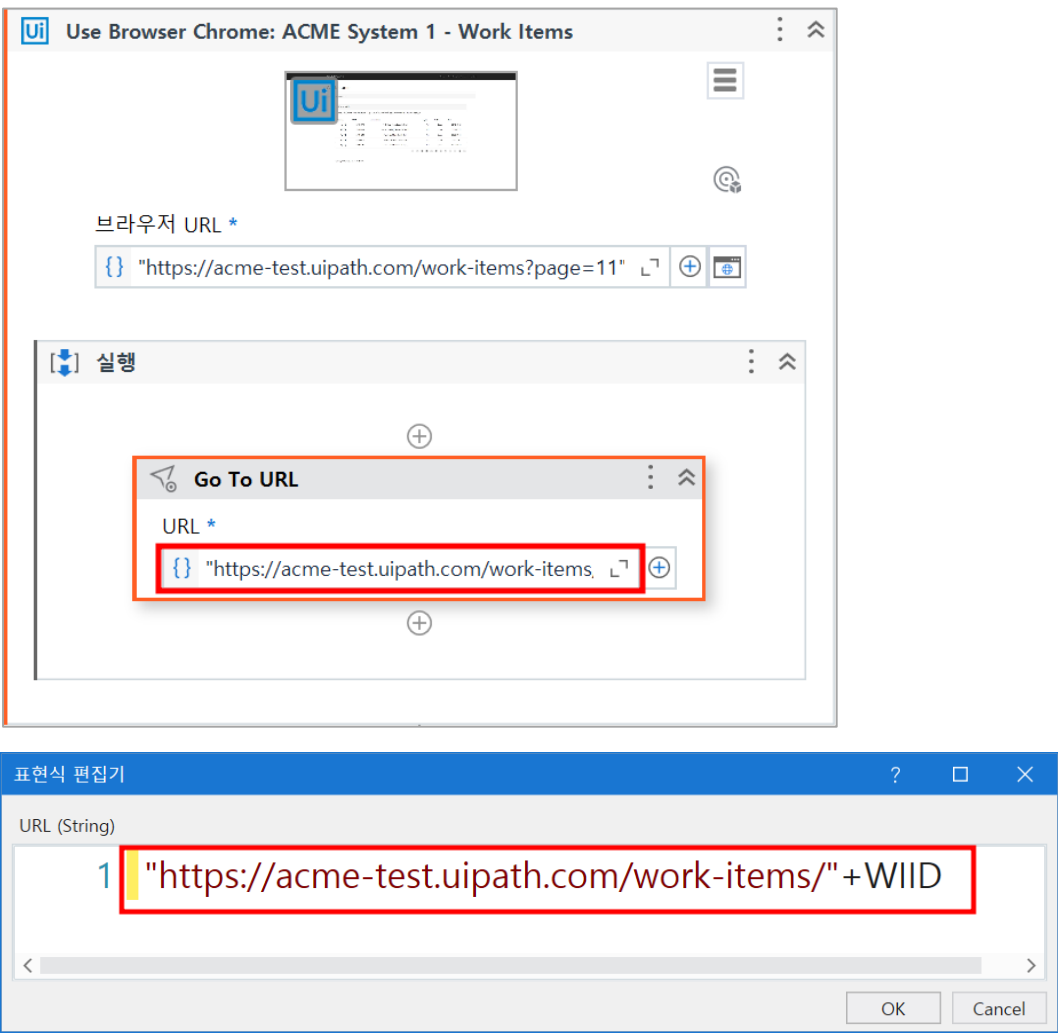


8.4. “Process Transaction” State

Step1. WIID 변수 생성



Step2. 사이트 이동 프로세스 작성



Step3. 사이트 이동 후 딜레이 2~3 초 추가



8.5. “End Process” State

– 현재까지 열린 APP 및 브라우저를 종료합니다.

Step1. ACME 사이트 로그아웃 후 브라우저 종료

